

Instituto Federal Catarinense
Curso de Bacharelado em Engenharia Elétrica
Campus Blumenau

Arthur Pedro Ferreira

Arquitetura de um Sistema Supervisório Orientada a Serviços para a Indústria 4.0

Blumenau
2026

Arthur Pedro Ferreira

Arquitetura de um Sistema Supervisório Orientada a Serviços para a Indústria 4.0

Trabalho de Conclusão de Curso submetido ao Curso Superior de Bacharelado em Engenharia Elétrica do Instituto Federal Catarinense - Campus Blumenau, para a obtenção do título de Bacharel em Engenharia Elétrica.
Orientador: Prof. Dr. Vitor Mateus Moraes

Blumenau
2026

No meio da dificuldade, encontra-se a oportunidade.
Albert Einstein

Resumo

Os sistemas supervisórios (*Supervisory Control and Data Acquisition*, SCADA) monitoram, supervisionam e controlam processos industriais em tempo real, reunindo em uma única interface a leitura das variáveis de campo, o registro histórico, a sinalização de alarmes e o envio de comandos aos equipamentos. Os sistemas convencionais, porém, exigem licenças proprietárias, hardware dedicado e instalação local especializada, o que os mantém fora do alcance de pequenas e médias empresas. Diante disso, este trabalho apresenta o desenvolvimento e a validação de um sistema supervisório web, multi-cliente e orientado a serviços, executado em contêineres Docker, acessado pelo navegador sem instalação no cliente. A arquitetura separa a interface web, a interface de programação de aplicações, os *workers* de processamento em segundo plano, o serviço de aquisição Modbus TCP e um motor isolado para a execução de código do usuário em JavaScript. O banco de dados relacional PostgreSQL guarda o cadastro e o histórico, com um banco próprio para cada cliente, e o Redis mantém o valor atual de cada ponto e as filas de trabalho, enquanto os serviços se comunicam por um barramento de mensagens (Moleculer). O acesso é controlado por cargo e por grupo de permissões, e o comando de equipamentos segue quatro fases, de seleção, verificação, confirmação e execução, com trava por ponto, idempotência e registro de auditoria. São revisados os conceitos de sistemas supervisórios, Indústria 4.0, contêineres e orquestração que fundamentam a proposta, e os requisitos do sistema são validados contra a arquitetura. A validação combinou testes automatizados, campanhas de carga e uma bancada com controlador lógico programável TM221 e inversor de frequência ATV320. Os resultados medidos mostram a criação automática do banco de um cliente em até 3,4 s, a gravação do histórico de cerca de 9.400 linhas por segundo para um cliente e a aquisição de até 9.750 pontos a cada 30 s por réplica do serviço de aquisição, além de 8 ms para aplicar um ciclo de valores no sinótico. Os requisitos de comando, de alarmes e de replicação ainda dependem de ensaios na bancada e de medidas com mais de uma réplica, e a implantação em provedor de nuvem e em orquestrador de contêineres fica como trabalho futuro.

Palavras-chave: SCADA; supervisão de processos industriais; monitoramento remoto; contêineres; arquitetura orientada a serviços; Indústria 4.0; Modbus.

Abstract

Supervisory control and data acquisition (SCADA) systems monitor, supervise and control industrial processes in real time, bringing together in a single interface the reading of field variables, historical logging, alarm signaling and the dispatch of commands to the equipment. Conventional systems, however, require proprietary licenses, dedicated hardware and specialized local installation, which keeps them out of reach for small and medium-sized enterprises. In view of this, this work presents the development and validation of a multi-client, service-oriented web supervisory system, executed in Docker containers and accessed through the browser with no client-side installation. The architecture separates the web interface, the application programming interface, the background processing workers, the Modbus TCP acquisition service and an isolated engine for running user code written in JavaScript. The PostgreSQL relational database stores the registry and the history, with a dedicated database for each client, and Redis keeps the current value of each point and the work queues, while the services communicate through a message bus (Moleculer). Access is controlled by role and by permission group, and the commanding of equipment follows four phases, namely selection, verification, confirmation and execution, with a lock per point, idempotency and an audit record. The concepts of supervisory systems, Industry 4.0, containers and orchestration that underpin the proposal are reviewed, and the system requirements are validated against the architecture. The validation combined automated tests, load campaigns and a test bench with a TM221 programmable logic controller and an ATV320 variable frequency drive. The measured results show the automatic creation of a client database in up to 3.4 s, history writing of about 9,400 rows per second for one client and the acquisition of up to 9,750 points every 30 s per replica of the acquisition service, besides 8 ms to apply one cycle of values in the synoptic screen. The requirements for commanding, alarms and replication still depend on bench trials and on measurements with more than one replica, and deployment on a cloud provider and on a container orchestrator is left as future work.

Keywords: SCADA; industrial process supervision; remote monitoring; containers; service-oriented architecture; Industry 4.0; Modbus.

Lista de Figuras

1 Linha do Tempo das Revoluções Industriais . . . 12
2 Pirâmide de Automação . . . 13
3 Tela de Supervisão . . . 18
4 Conversão linear de 4 a 20 mA para 0 a 10 bar . . . 21
5 Estrutura de um Sistema Ciber-Físico . . . 23
6 Interseções de IoT, CPS, IIoT e Indústria 4.0 . . . 24
7 Máquinas virtuais e contêineres . . . 25
8 Operação normal do protocolo Raft . . . 27
9 Fluxo de uma fila de mensagens com tentativas e fila de mensagens mortas . . . 29
10 Arquitetura do Sistema . . . 41
11 Modelo de Dados . . . 50
12 Mapa dos blocos usados nos fluxos . . . 52
13 Fluxo do cadastro de uma tag . . . 53
14 Fluxo da leitura do equipamento pela aquisição . . . 55
15 Fluxo da gravação do valor atual, do histórico e dos resumos . . . 56
16 Fluxo da exibição do valor atual por *polling* . . . 58
17 Fluxo da consulta de tendência . . . 59
18 Fluxo das fases de verificação e confirmação do comando . . . 61
19 Fluxo da fase de execução do comando . . . 62
20 Fluxo da detecção, da sinalização e do reconhecimento de um alarme . . . 64
21 Fluxo do cálculo de uma variável meta . . . 66
22 Fluxo da criação de um cliente . . . 68
23 Fluxo do login e da renovação da sessão . . . 70
24 Casco da aplicação com menu lateral e indicador de alarmes . . . 76
25 Tela de entrada . . . 77
26 Lista de clientes da administração da Movi . . . 78
27 Formulário de criação de cliente . . . 78
28 Tela de usuários com cargos e situações . . . 79
29 Formulário de grupo com o catálogo de permissões . . . 80
30 Formulário de uma tag analógica Modbus . . . 81
31 Grade de tags com valor ao vivo . . . 81
32 Lista de acompanhamento com valores ao vivo e gráfico . . . 82
33 Dashboard em execução com painéis de linha, barra e medidor . . . 83
34 Editor do sinótico com paleta, área de desenho e inspetor . . . 84
35 Sinótico em execução com valores ao vivo e objeto em alarme . . . 85
36 Mapa das plantas com o estado de alarme de cada uma . . . 85
37 Formulário de configuração de um alarme . . . 87
38 Tela de alarmes ativos com reconhecimento . . . 87
39 Linha do tempo de alarmes, comandos e eventos de dispositivo . . . 88
40 Tendência de uma tag em janela de sete dias . . . 89
41 Editor de código da variável meta e do script . . . 90
42 Diálogo de comando nas fases de confirmação e resultado . . . 91
43 Bancada de teste montada. . . . 93
44 Topologia de comunicação da bancada. . . . 95
45 Ligação serial RS-485 entre o CLP e o inversor. . . . 96
46 Configuração da comunicação serial no EcoStruxure Machine Expert Basic. . . . 98

47 Parametrização do inversor ATV320. 98
48 Blocos funcionais do programa do CLP. 99
49 Nível simulado em um ciclo de liga e desliga da bomba a 40 Hz. 102
50 Telas de supervisão da bancada no Movi. 104
51 Vazão do motor de scripts em função do tamanho do lote, um hospedeiro de 0,25 *vCPU*. 113
52 Vazão do motor de scripts com três hospedeiros em função do número de clientes ativos. 114
53 Teto de tags por réplica de aquisição, com uma e com cinco requisições por CLP. 116
54 Nível simulado e frequência do inversor no ensaio E1. 119
55 Nível simulado e frequência do inversor no ensaio E2. 120
56 Nível simulado e frequência do inversor no ensaio E3. 121
57 Nível simulado, limite do alarme e instantes de alarme, reconhecimento e retorno ao normal no ensaio E4. 122
58 Valor, qualidade e estado de comunicação durante a perda e o retorno de comunicação no ensaio E5. 123
59 Frequência de referência e frequência medida no ensaio E6. 124

Lista de Siglas

Sigla	Significado
ACL	Lista de Controle de Acesso (*Access Control List*)
API	*Application Programming Interface*
AWS	*Amazon Web Services*
CLP	Controlador Lógico Programável
CNC	Controlador Numérico Computadorizado
CPS	Sistemas Ciber-físicos (*Cyber-Physical Systems*)
CPU	Unidade Central de Processamento (*Central Processing Unit*)
ECS	*Elastic Container Service*
ERP	Sistemas de Gestão Empresarial (*Enterprise Resource Planning*)
HTTP	*Hypertext Transfer Protocol*
IHM	Interface Homem-Máquina
IIoT	Internet Industrial das Coisas (*Industrial Internet of Things*)
I/O	Entrada/Saída (*Input/Output*)
IoT	Internet das Coisas (*Internet of Things*)
IP	*Internet Protocol*
ISA	*International Society of Automation*
JSON	*JavaScript Object Notation*
JWT	*JSON Web Token*
LAN	Rede Local (*Local Area Network*)
M2M	Comunicação Máquina a Máquina (*Machine-to-Machine*)
MES	Sistemas de Execução da Manufatura (*Manufacturing Execution Systems*)
MVP	Produto Mínimo Viável (*Minimum Viable Product*)
ORM	Mapeamento Objeto-relacional (*Object-Relational Mapping*)
RAM	*Random Access Memory*
RBAC	*Role-Based Access Control*
RDS	*Relational Database Service*
REST	*Representational State Transfer*
RF	Radiofrequência
RPC	Chamada de Procedimento Remoto (*Remote Procedure Call*)
RS-485	
RTU	*Remote Terminal Unit*
SCADA	Supervisão, Controle e Aquisição de Dados (*Supervisory Control and Data Acquisition*)
SGBD	Sistema Gerenciador de Banco de Dados
SMR	Replicação de Máquinas de Estados (*State Machine Replication*)
SOA	Arquitetura Orientada a Serviços (*Service-Oriented Architecture*)
SOAP	*Simple Object Access Protocol*
SQL	*Structured Query Language*
TA	Tecnologia da Automação
TCP	*Transmission Control Protocol*
TI	Tecnologia da Informação
TLS	*Transport Layer Security*
TTL	*time-to-live*
URL	*Uniform Resource Locator*
UUID	*Universally Unique Identifier*
VM	Máquina Virtual (*Virtual Machine*)
Wasm	*WebAssembly*
WCAG	*Web Content Accessibility Guidelines*
WS	*Web Services*
XML	*eXtensible Markup Language*

Lista de Tabelas

1 Requisitos Funcionais. . . . 38
2 Requisitos Não Funcionais. . . . 39
3 Tecnologias adotadas em cada serviço. . . . 42
4 Contêineres, responsabilidades e acessos. . . . 45
5 Garantias comuns aos fluxos do sistema. . . . 72
6 Efeito da falha de cada bloco nos fluxos do sistema. . . . 72
7 Resumo dos fluxos do sistema. . . . 73
8 Telas, cargos que acessam e requisitos atendidos. . . . 91
9 Equipamentos da bancada de teste. . . . 94
10 Parâmetros propostos para o ATV320. . . . 96
11 Registradores Modbus do ATV320 utilizados (endereços a confirmar). . . . 97
12 Mapa de memória proposto para o CLP. . . . 99
13 Cadastro da bancada no Movi. . . . 102
14 Tags cadastradas no Movi para a bancada (endereços propostos). . . . 103
15 Resumo dos ensaios da bancada. . . . 105
16 Ambiente de ensaio. . . . 109
17 Limites de recursos dos contêineres. . . . 110
18 Medidas de criação de cliente e de autenticação. . . . 111
19 Resultados do *Lighthouse* (celular), mediana de três execuções. Des.: desempenho, Ace.: acessibilidade, BP: boas práticas, LCP e TBT em ms. . . . 112
20 Teto sustentado de variáveis derivadas e de tags internas geradas por contêiner de 0,25 *vCPU*. . . . 114
21 Teto de leitura Modbus por réplica de aquisição (0,25 *vCPU*, 25 tags por CLP, cinco requisições por ciclo). . . . 116
22 Ensaio de carga de leitura contra o ProtoProbe. . . . 117
23 Ensaios da bancada. . . . 118
24 Tempos do ensaio E1. . . . 119
25 Tempos do ensaio E2. . . . 120
26 Tempos do ensaio E3. . . . 121
27 Tempos do ensaio E4. . . . 122
28 Tempos do ensaio E5. . . . 123
29 Tempos do ensaio E6. . . . 124
30 Validação dos requisitos. . . . 125

Índice

1 Introdução **12**
1.1 Contextualização . . . 12
1.2 Justificativa . . . 14
1.3 Objetivos . . . 14
1.3.1 Objetivo Geral . . . 15
1.3.2 Objetivos Específicos . . . 15
1.4 Metodologia . . . 15

2 Conceitos e Tecnologias Utilizadas **17**
2.1 Sistema SCADA . . . 17
2.1.1 Aquisição e Registro de Dados . . . 18
2.1.2 Sincronismo de Tempo . . . 18
2.1.3 Níveis de Acesso . . . 19
2.1.4 Comando e Controle . . . 19
2.1.5 Escalabilidade . . . 19
2.1.6 Redundância . . . 19
2.2 Ajuste para Escala de Engenharia e Linearização de Sensores . . . 20
2.3 Automação e Tecnologias da Indústria 4.0 . . . 22
2.4 Computação em Nuvem . . . 24
2.5 Contêineres e Orquestração . . . 25
2.6 Microsserviços . . . 28
2.7 Moleculer . . . 28
2.8 Filas de Mensagens e Processamento Assíncrono . . . 29
2.9 Banco de Dados . . . 30
2.10 Redis . . . 31
2.11 Node.js, TypeScript e Fastify . . . 31
2.12 React e Next.js . . . 32
2.13 HTTP, API REST e JSON . . . 33
2.14 Autenticação e Controle de Acesso . . . 34
2.15 Execução Isolada de Código . . . 35
2.16 Interfaces de Alto Desempenho para Operação . . . 36
2.17 Protocolo Modbus TCP e Modbus RTU . . . 36

3 Arquitetura e Requisitos do Sistema **38**
3.1 Requisitos . . . 38
3.1.1 Requisitos Funcionais . . . 38
3.1.2 Requisitos Não Funcionais . . . 39
3.2 Arquitetura . . . 40
3.3 Validações dos Requisitos . . . 42
3.3.1 Aquisição e Registro de Dados . . . 42
3.3.2 Sincronismo de Tempo . . . 43
3.3.3 Níveis de Acesso . . . 43
3.3.4 Comando e Controle . . . 44
3.3.5 Escalabilidade . . . 44
3.3.6 Redundância . . . 45
3.4 Serviços . . . 45

3.4.1 Web . . . 46
3.4.2 API . . . 46
3.4.3 Worker . . . 47
3.4.4 Aquisição . . . 47
3.4.5 Runtime de Scripts . . . 48
3.4.6 PostgreSQL . . . 48
3.4.7 Redis . . . 48
3.4.8 Comunicação Moleculer e Filas . . . 49
3.4.9 Segurança entre os Serviços . . . 49
3.5 Modelo de Dados . . . 49
3.6 Implantação . . . 50

4 Fluxos do Sistema **52**
4.1 Cadastro de uma Tag . . . 53
4.2 Do Campo ao Banco . . . 55
4.3 Do Banco à Tela em Tempo Real . . . 57
4.4 Histórico e Tendência . . . 59
4.5 Comando em Quatro Fases . . . 60
4.6 Alarme . . . 63
4.7 Variável Meta . . . 65
4.8 Criação de um Cliente . . . 67
4.9 Login e Sessão . . . 69
4.10 Garantias Comuns aos Fluxos . . . 71
4.11 Resumo dos Fluxos . . . 73

5 Interface do Sistema **75**
5.1 Princípios de interface adotados . . . 75
5.2 Estrutura geral da aplicação . . . 76
5.3 Entrar e trocar a senha . . . 77
5.4 Administração da Movi: clientes . . . 77
5.5 Usuários, cargos e grupos . . . 79
5.6 Cadastro: plantas, dispositivos de E/S, tags e pastas . . . 80
5.7 Lista de acompanhamento . . . 82
5.8 Dashboard . . . 83
5.9 Sinótico . . . 83
5.10 Mapa das plantas . . . 85
5.11 Alarmes e eventos . . . 86
5.12 Linha do tempo . . . 88
5.13 Histórico e tendência . . . 88
5.14 Variável meta e scripts . . . 89
5.15 Diálogo de comando de quatro fases . . . 90
5.16 Síntese: telas, cargos e requisitos . . . 91

6 Bancada de Teste **93**
6.1 Objetivo e Visão Geral . . . 93
6.2 Equipamentos Utilizados . . . 94
6.3 Arquitetura de Comunicação . . . 94
6.4 Configuração do Inversor ATV320 . . . 96
6.5 Programa do Controlador . . . 98

6.6 Modelo do Tanque Simulado . . . 100
6.7 Configuração no Movi . . . 102
6.8 Roteiro de Ensaios . . . 105
6.8.1 E1: Leitura Contínua com a Bomba Desligada . . . 105
6.8.2 E2: Partida pelo Supervisório e Enchimento . . . 106
6.8.3 E3: Parada e Esvaziamento . . . 106
6.8.4 E4: Alarme de Nível Alto e Reconhecimento . . . 107
6.8.5 E5: Perda de Comunicação e Retorno . . . 107
6.8.6 E6: Troca de Referência de Frequência . . . 108

7 Resultados **109**
7.1 Ambiente de ensaio . . . 109
7.2 Medidas de desempenho já realizadas . . . 110
7.2.1 Cadastro de clientes e custo da autenticação . . . 110
7.2.2 Desempenho das telas . . . 111
7.2.3 Motor de scripts isolado . . . 112
7.2.4 Gravação do histórico e isolamento entre clientes . . . 115
7.3 Aquisição e ensaio com o simulador . . . 115
7.3.1 Capacidade de leitura Modbus . . . 115
7.3.2 Ensaio ponta a ponta com o servidor de teste . . . 116
7.4 Resultados da bancada . . . 117
7.4.1 E1: leitura contínua com a bomba desligada . . . 118
7.4.2 E2: partida da bomba e enchimento . . . 119
7.4.3 E3: parada da bomba e esvaziamento . . . 120
7.4.4 E4: alarme de nível alto e reconhecimento . . . 121
7.4.5 E5: perda e retorno de comunicação . . . 122
7.4.6 E6: troca da referência de frequência . . . 123
7.5 Validação dos requisitos . . . 124
7.6 Qualidade de software . . . 126
7.7 Limitações e comparação com a literatura . . . 126

8 Conclusão **128**
8.1 Retomada dos objetivos . . . 128
8.2 Contribuições . . . 129
8.3 Limitações . . . 129
8.4 Trabalhos futuros . . . 130

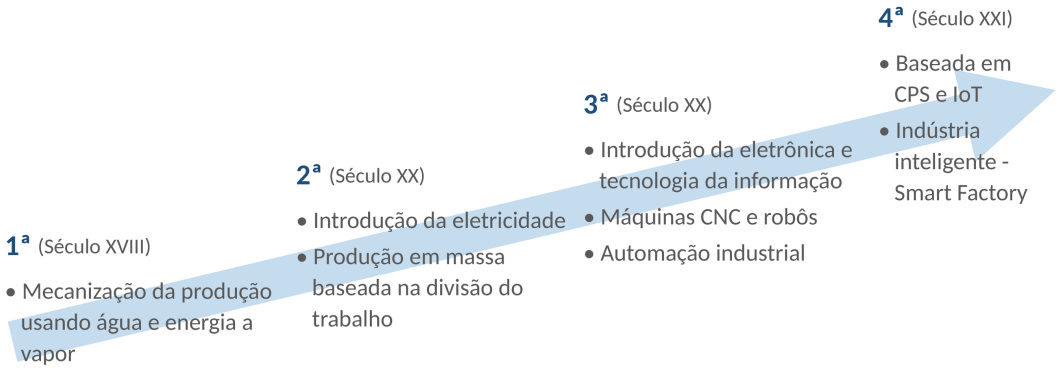
1 Introdução

1.1 Contextualização

A indústria é o setor da economia dedicado à produção de bens materiais, apoiado em trabalho humano, energia e equipamentos de alto valor agregado. Ao longo de sua evolução, sucessivos saltos tecnológicos redefiniram processos, produtos e formas de organização do trabalho, as chamadas "revoluções industriais"(Lasi *et al.*, 2014).

A Figura 1 resume essa linha do tempo. Na primeira fase, destaca-se a mecanização da produção, na segunda, a difusão da energia elétrica e a produção em massa baseada no modelo Fordista e na terceira fase, a digitalização (Lasi *et al.*, 2014). Foi nessa fase que a introdução da tecnologia da informação, da automação e da eletrônica impulsionou o início da automação dos processos produtivos. Surgiram então os controladores lógicos programáveis (CLPs), os Controladores Numéricos Computadorizados (CNCs) e os robôs (Pisching, 2017). Essas tecnologias tornaram-se essenciais para as indústrias que buscam maior eficiência e qualidade com menores custos e prazos. Sua adoção contribui diretamente para o aumento da produtividade (Bigheti, 2020).

Figura 1: Linha do Tempo das Revoluções Industriais



Fonte: Adaptado de Pisching (2017).

Atualmente, vive-se a quarta fase, a chamada Indústria 4.0, baseada na indústria inteligente, na Internet das Coisas (*Internet of Things*, IoT) e nos sistemas ciber-físicos (*Cyber-Physical Systems*, CPS) (Bigheti, 2020). Nesses sistemas, subsistemas computacionais cooperam entre si e permanecem integrados ao mundo físico e a seus processos, o que permite medir, controlar e analisar dados em tempo real por meio da rede, inclusive a Internet (Monostori *et al.*, 2016).

A Figura 2 organiza a automação na Indústria 4.0 em forma de pirâmide, distribuída em cinco níveis. No nível 0 está o processo físico de produção, com elementos como motores, bombas e tanques. No nível 1, o sensoriamento e a atuação, realizados por sensores, transmissores e válvulas. O nível 2 reúne o monitoramento, a supervisão e o controle, camada em que se encontram os controladores lógicos programáveis (CLPs), os sistemas supervisórios (SCADA) e as interfaces homem-máquina (IHM), responsáveis por acompanhar e comandar o processo em tempo real e por se comunicar com o nível 1 por protocolos seriais ou diretamente pelas entradas e saídas (I/O). O nível 3 corresponde às operações de manufatura e ao controle da produção, em que atuam os sistemas de execução da manufatura (*Manufacturing Execution Systems*, MES). No topo, o nível 4 abrange o planejamento de negócios e a logística, em que

se destacam os sistemas de gestão empresarial (*Enterprise Resource Planning*, ERP) (Bigheti, 2020; Bangemann *et al.*, 2014).

Figura 2: Pirâmide de Automação



Fonte: Adaptado de Bangemann *et al.* (2014).

Recentemente, as camadas de supervisão e de gestão, do nível 2 ao nível 4, ganharam mais notoriedade: ao digitalizar serviços antes restritos ao ambiente físico da planta, passou-se a ofertá-los sobre infraestrutura em nuvem e a compartilhá-los entre diferentes sistemas e usuários. Em geral, esses serviços seguem a arquitetura orientada a serviços (*Service-Oriented Architecture*, SOA), materializada por serviços em nuvem (Bangemann *et al.*, 2014; Abbas, 2011). Esse movimento é sustentado pela própria natureza da Indústria 4.0, que se caracteriza pelo grande avanço dos protocolos de comunicação e de telemetria, pela popularização do desenvolvimento de software e pela interconexão inteligente de máquinas e sistemas (Lasi *et al.*, 2014; Pisching, 2017; Bigheti, 2020).

A materialização desses serviços apoia-se hoje em contêineres e em sua orquestração. Um contêiner empacota uma aplicação com suas dependências e a executa de forma isolada, o que permite dividir um sistema em serviços independentes, replicá-los e distribuí-los em diferentes servidores. Ferramentas de orquestração, como o Kubernetes, coordenam esses contêineres, mantêm o estado desejado dos serviços e sustentam a replicação com tolerância a falhas (Netto, Hylson Vescovi *et al.*, 2017; Netto, Hylson V. *et al.*, 2016). Essa base é a mesma que sustenta os serviços modernos oferecidos em nuvem, e seus conceitos são detalhados no Capítulo 2.

Tais avanços permitiram o monitoramento e a supervisão centralizada de processos complexos em tempo real, elevando o papel dos sistemas supervisórios de simples interfaces de operação para centrais estratégicas de análise de dados. Nesse cenário, este trabalho propõe e implementa um sistema supervisório web orientado a serviços, multi-cliente, que disponibiliza pelo navegador funcionalidades antes restritas ao ambiente físico da planta (Bangemann *et al.*, 2014). O sistema é executado em contêineres num servidor, e essa mesma arquitetura pode ser levada a um provedor de nuvem ou a um orquestrador, o que mantém a nuvem como horizonte de evolução. Alinhada às tendências da Indústria 4.0 (Bigheti, 2020), a proposta torna o monitoramento e a supervisão acessíveis e compartilháveis entre diferentes sistemas e usuários.

1.2 Justificativa

Os processos industriais geram grande volume de dados, utilizados para acompanhar variáveis e interpretar a tendência do processo. Em geral, essa informação é organizada e apresentada ao operador por um software de supervisão, por meio de uma interface homem-máquina (IHM), que converte leituras de campo em telas, tendências e alarmes de forma legível e acionável (Zanghi, 2019). Com o aumento da capacidade de processamento dos computadores, o aprimoramento dos protocolos de comunicação e a maior integração com redes externas, inclusive a Internet, consolidou-se a supervisão remota e cresceu a exigência de visibilidade em tempo real sobre o que ocorre na planta industrial. Em setores como saneamento, transporte e energia, nos quais instalações costumam estar geograficamente espalhadas, ganham peso a telemetria e o controle supervisionado à distância. De modo esquemático, uma arquitetura típica de sistema SCADA integra os equipamentos de campo, como CLPs, aos módulos de monitoramento e controle e centraliza a visualização dos dados para o operador (Abbas, 2011).

Apesar de consolidados, os sistemas supervisórios convencionais costumam exigir investimento elevado em licenças de software proprietário, hardware dedicado e infraestrutura local, o que os mantém fora do alcance de pequenas e médias empresas, para as quais a aquisição de uma licença raramente se justifica diante do porte da operação (Phuyal *et al.*, 2020). Nesse cenário, oferecer a supervisão como um serviço web, acessado pelo navegador e compartilhado por vários clientes sobre uma mesma instalação, reduz esse investimento inicial e democratiza o acesso a uma tecnologia antes economicamente inviável para esse público. Além de reduzir custos, a supervisão via web amplia o alcance do monitoramento e do controle a instalações geograficamente dispersas e favorece a colaboração entre equipes em locais distintos, reduzindo deslocamentos presenciais e agilizando a resposta a ocorrências (Abbas, 2011).

A computação em nuvem é um modelo em que recursos computacionais, como armazenamento, processamento, bancos de dados e aplicações, são ofertados como serviços remotos, flexíveis e escaláveis (Pedrosa; Nogueira, 2011). Em geral, o usuário não precisa conhecer a localização física nem os detalhes da infraestrutura que sustenta esses serviços, e a capacidade é provisionada sob demanda, o que reduz a necessidade de investimento direto em hardware e software no ambiente local (Henriques da Silva, 2010). Por sua vez, a conectividade, em especial por meio de redes sem fio, é essencial para que dispositivos e sistemas ciber-físicos mantenham troca de dados com a nuvem e com demais elementos da arquitetura distribuída (Stankovic, 2014; Monostori *et al.*, 2016).

Neste trabalho, a adoção da arquitetura em nuvem se dá pelo seu princípio e não por um provedor específico. O sistema é empacotado em contêineres executados num servidor, o que dispensa contratos de hospedagem gerenciada e mantém o custo de operação baixo, requisito central para o público de pequenas e médias empresas. Como cada serviço é um contêiner independente, a mesma solução pode ser replicada e coordenada por um orquestrador, ou levada a um provedor de nuvem, sem alteração de código (Netto, Hylson Vescovi *et al.*, 2017; Netto, Hylson V. *et al.*, 2016). Por fim, o acesso pelo navegador e o isolamento dos dados de cada cliente permitem que uma única instalação atenda a várias empresas, cada uma com seus usuários, suas plantas e suas permissões, o que reduz o custo por cliente e simplifica a implantação.

1.3 Objetivos

A seguir são apresentados os objetivos gerais e específicos deste Trabalho de Conclusão de Curso.

1.3.1 Objetivo Geral

Desenvolver e validar um sistema supervisório web multi-cliente com arquitetura orientada a serviços em contêineres, capaz de apoiar o monitoramento, o comando e a supervisão de processos industriais, com serviços bem definidos, reutilizáveis e acessíveis pelo navegador.

1.3.2 Objetivos Específicos

- Revisar os conceitos de Indústria 4.0, sistemas supervisórios, arquitetura orientada a serviços (SOA), contêineres e computação em nuvem, de modo a embasar a solução proposta.
- Especificar os requisitos funcionais e não funcionais e projetar a arquitetura de serviços em contêineres, com a separação entre interface web, API, serviços de aquisição, filas de trabalho e armazenamento de dados.
- Implementar a aquisição de dados via Modbus TCP, com agendamento das leituras e escritas, e o registro do valor atual e do histórico das variáveis, com banco de dados próprio para cada cliente.
- Implementar a interface web de supervisão, com telas de tempo real, alarmes, gráficos de tendência e sinótico.
- Implementar o comando de equipamentos em quatro fases, com seleção, verificação, confirmação e execução.
- Implementar o controle de acesso por cargo e por grupo de permissões.
- Validar o sistema em uma bancada de teste com um controlador lógico programável (CLP) e um inversor de frequência, e medir seu desempenho frente ao volume de dados esperado.

1.4 Metodologia

O desenvolvimento deste trabalho é dividido em etapas, com o objetivo de estruturar, implementar e validar um sistema supervisório web com arquitetura orientada a serviços, voltado ao monitoramento e à supervisão de processos industriais. Cada etapa parte dos resultados da anterior, de modo que a fundamentação teórica orienta a especificação, que por sua vez direciona a implementação e, por fim, a validação da solução proposta. Além deste capítulo de introdução, o trabalho está organizado em outros sete capítulos, associados às etapas descritas a seguir.

Inicialmente, é realizada uma revisão bibliográfica sobre os conceitos que sustentam o trabalho, abrangendo automação e informática industrial, sistemas supervisórios (SCADA), Indústria 4.0 e suas tecnologias correlatas, IoT e CPS. Nessa etapa também são estudados a SOA, os contêineres e sua orquestração, a computação em nuvem, os bancos de dados, as filas de trabalho e o protocolo Modbus em suas variantes TCP e RTU. Esses conceitos e tecnologias são apresentados no Capítulo 2.

Na etapa seguinte, são especificados os requisitos funcionais e não funcionais e a arquitetura do sistema, com destaque para a divisão em serviços executados em contêineres, para a separação entre interface, API, aquisição e armazenamento e para o isolamento dos dados de cada cliente. A definição dessa arquitetura estabelece as fronteiras entre os componentes e orienta as decisões de implementação, sendo detalhada no Capítulo 3.

Posteriormente, são descritos os fluxos do sistema, isto é, os caminhos percorridos pelo dado e pelas ações do usuário entre os componentes. São apresentados a aquisição Modbus TCP, o agendamento das leituras, o registro do valor atual e do histórico, a geração de alarmes, a variável meta, o comando em quatro fases, a criação de um cliente e o login com a sessão, com as garantias comuns a cada fluxo, conforme o Capítulo 4.

Em seguida, é apresentada a interface web do sistema, que consome os serviços disponibilizados e evidencia a interoperabilidade entre apresentação e serviços. São descritas as telas de cadastro, de usuários e grupos, de acompanhamento em tempo real, de alarmes, de linha do tempo, de histórico e tendência, o dashboard, o sinótico e o mapa das plantas, além do controle de acesso por cargo e grupo, no Capítulo 5.

Na etapa de validação, descreve-se a bancada de teste, composta por um CLP e um inversor de frequência, e a forma como o sistema é conectado a esses equipamentos, no Capítulo 6. Os resultados obtidos na bancada, as medições de desempenho e a qualidade do software são apresentados no Capítulo 7 e confrontados com os requisitos definidos na etapa de especificação, permitindo avaliar as contribuições, as limitações e as potencialidades da solução proposta. Por fim, o Capítulo 8 reúne as conclusões do trabalho e as perspectivas de evolução, entre elas a execução em um provedor de nuvem ou sob um orquestrador de contêineres.

2 Conceitos e Tecnologias Utilizadas

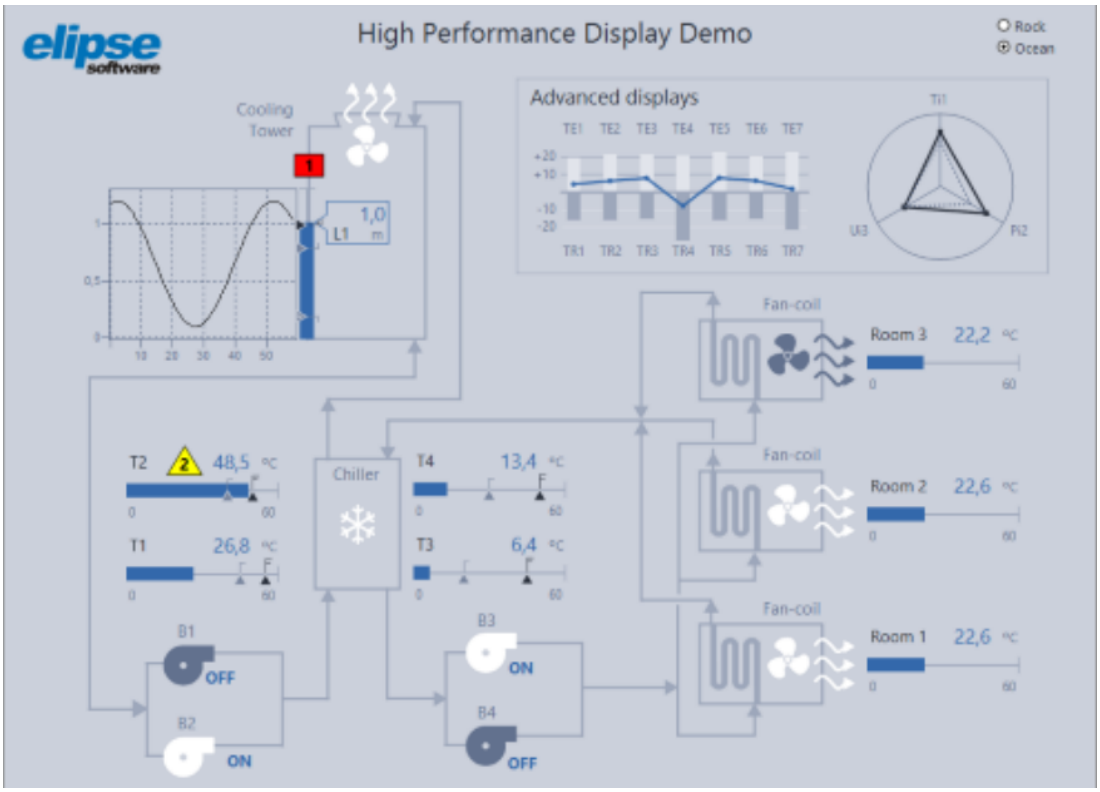
Este capítulo aborda os conceitos e as tecnologias que sustentam o trabalho: automação industrial, tecnologia da informação aplicada ao contexto industrial e desenvolvimento de software. O objetivo é reunir as definições, os enquadramentos e as ferramentas necessários para compreender um sistema supervisório web orientado a serviços. Os temas partem da fundamentação do supervisório e da automação, passam pela infraestrutura de execução, pelo armazenamento de dados, pelas tecnologias de aplicação e de segurança e chegam ao protocolo de comunicação com os equipamentos de campo.

2.1 Sistema SCADA

A indústria evoluiu muito na busca por mais monitoramento e controle dos processos. Com o tempo, os custos de computadores, hubs e conversores de rede caíram bastante, ao passo que o desenvolvimento de software ganhava relevância. (Abbas, 2011) Nesse contexto, as empresas de software industrial passaram a oferecer soluções personalizadas para cada tipo de processo e indústria, com ampla gama de protocolos de comunicação. A esse tipo de sistema deu-se o nome de SCADA, sigla em inglês para *Supervisory Control and Data Acquisition* (Zanghi, 2019). De forma geral, define-se o SCADA como um pacote de software posicionado sobre o hardware ao qual se interliga, que viabiliza a supervisão e o controle de processos industriais de forma local ou remota (Phuyal *et al.*, 2020).

A aplicação dos sistemas SCADA estende-se a praticamente todos os segmentos que operam processos distribuídos e de funcionamento contínuo, muitos deles classificados como infraestruturas críticas, cuja interrupção compromete serviços essenciais (Yadav; Paul, 2020). Entre os usos típicos estão a geração, a transmissão e a distribuição de energia elétrica, o tratamento e a distribuição de água e o saneamento, a produção e o transporte de óleo e gás por dutos, os sistemas de transporte, as indústrias química e farmacêutica, a produção de papel e celulose, o setor de alimentos e bebidas e a manufatura discreta, como a automotiva e a aeroespacial (Stouffer *et al.*, 2015). A Figura 3 ilustra uma tela de supervisão.

Figura 3: Tela de Supervisão



Fonte: Elipse Software (2026).

Um sistema SCADA deve contemplar seis módulos, descritos nas subseções a seguir: aquisição e registro de dados, sincronismo de tempo, níveis de acesso, comando e controle, escalabilidade e redundância (Zanghi, 2019).

2.1.1 Aquisição e Registro de Dados

A aquisição das informações de campo é o primeiro grande desafio em um sistema SCADA (Zanghi, 2019). Atualmente, predominam dispositivos genéricos inteligentes que disponibilizam algum protocolo de comunicação, responsáveis por adquirir grandezas analógicas e digitais e disponibilizá-las ao supervisório em formato binário por meio de uma rede de dados (Yadav; Paul, 2020). O meio de comunicação varia conforme a distância e a infraestrutura disponível, indo das redes locais cabeadas, tipicamente Ethernet, a enlaces sem fio como rádio frequência (RF) e telefonia móvel (4G) (Stouffer *et al.*, 2015), até soluções orientadas à Internet das Coisas, que trafegam os dados pela internet (Khalid *et al.*, 2024).

Os pontos adquiridos em um sistema SCADA são denominados *tags* e armazenados no banco de dados do supervisório. Cada tag assume um tipo de dado conforme a natureza da grandeza que representa, podendo ser booleano, inteiro, real ou texto. O registro contínuo desses valores pelo supervisório permite análises posteriores, na forma de gráficos históricos de tendência e de sequenciais de eventos (Zanghi, 2019).

2.1.2 Sincronismo de Tempo

Outro requisito fundamental dos sistemas SCADA é o sincronismo de tempo. Todo dado coletado deve possuir um registro de tempo de origem, a chamada estampa de tempo (*time*

stamp), de modo que cada variação de estado fique associada a um instante de referência. Essa exigência decorre da necessidade de análises posteriores de ocorrências e faltas, que dependem da cronologia precisa dos acontecimentos para permitir a identificação das causas (Zanghi, 2019).

Na prática, a aquisição é realizada de forma cíclica: o supervisório consulta periodicamente os dispositivos de campo, processo conhecido como *polling* (Stouffer *et al.*, 2015), em ciclos de leitura sucessivos cuja frequência é definida conforme a dinâmica do processo (Yadav; Paul, 2020). Como consequência, a estampa de tempo atribuída a cada ponto corresponde ao ciclo em que o valor foi lido, e não ao instante físico exato da transição, de modo que a resolução temporal do registro fica limitada ao período do ciclo de aquisição.

2.1.3 Níveis de Acesso

Os níveis de acesso de um sistema SCADA definem a separação das permissões de supervisão e controle entre os usuários. Classicamente organizada em níveis por instalação (Zanghi, 2019), essa separação materializa-se como acessos definidos por planta, em que cada usuário recebe permissões apenas sobre os pontos e comandos da instalação à qual está vinculado, segundo o controle de acesso baseado em cargos e o princípio do menor privilégio, apoiado em mecanismos adequados de autenticação e autorização (Stouffer *et al.*, 2015).

2.1.4 Comando e Controle

As ações executadas remotamente pelo supervisório consistem no envio de comandos aos dispositivos de campo, isto é, na escrita de uma informação no dispositivo (Yadav; Paul, 2020). Quanto ao momento de execução, o comando pode ser instantâneo, escrevendo de imediato o valor desejado ou agendado, programado para ações conforme cenários previamente definidos, com o instante de execução registrado (Zanghi, 2019).

O processamento de um comando segue quatro fases. Na seleção, o operador escolhe na interface o equipamento. Na verificação, conferem-se os intertravamentos e as condições de segurança. Na confirmação, o operador autoriza explicitamente a ação. Na execução, a ordem é enviada ao elemento final de controle (Zanghi, 2019). Após a execução, o supervisório verifica a confirmação para garantir que a solicitação foi aceita pelo dispositivo e, em caso de falha, gera um alarme, pois comandos indevidos podem danificar equipamentos ou comprometer a segurança do processo (Stouffer *et al.*, 2015).

2.1.5 Escalabilidade

A expansão das instalações é frequente em ambientes industriais, com a inclusão de novos equipamentos, pontos de medição e circuitos. Para acompanhá-la, o software SCADA deve ser escalável, permitindo acrescentar pontos à base de dados e criar novas telas de operação sem comprometer o que já está em produção (Zanghi, 2019).

A análise da escalabilidade deve abranger não apenas o supervisório, mas também os elementos de aquisição, controle e rede, que precisam permitir o acréscimo de novos dispositivos ao barramento de comunicação, integração a ser prevista já na fase inicial do projeto (Zanghi, 2019).

2.1.6 Redundância

A redundância consiste no acréscimo de um elemento idêntico ao original, de modo que, em caso de falha do primeiro, o segundo assuma suas funções da forma mais transparente possível

para a operação (Zanghi, 2019). A prática aplica-se a diversos componentes do sistema, como softwares supervisórios, dispositivos de aquisição e controle e elementos de rede, apesar do custo adicional, eleva a confiabilidade a graus compatíveis com a criticidade de aplicações industriais sensíveis.

Em uma arquitetura redundante típica, múltiplas estações de operação comunicam-se simultaneamente com os dispositivos de campo e as unidades de aquisição e controle operam em pares espelhados (Zanghi, 2019). De modo geral, recomenda-se que cada componente crítico tenha um par redundante, sobre redes igualmente redundantes, e que o sistema seja projetado para degradação suave, preservando a supervisão e o controle mesmo diante de falhas pontuais (Stouffer *et al.*, 2015).

2.2 Ajuste para Escala de Engenharia e Linearização de Sensores

Os sensores e transmissores de campo convertem uma grandeza física, como pressão, nível, temperatura ou vazão, em um sinal elétrico padronizado. Os padrões mais difundidos são a corrente de 4 a 20 mA e a tensão de 0 a 10 V. A faixa de 4 a 20 mA é preferida em instalações industriais porque a corrente é menos sensível a ruído e à queda de tensão em cabos longos, e porque o início da faixa em 4 mA, e não em 0 mA, permite distinguir uma medição no valor mínimo de uma falha do circuito, como o rompimento do cabo, que leva a corrente a zero.

O sinal elétrico, no entanto, não chega ao supervisório como uma grandeza física. Ele é digitalizado por um conversor analógico-digital (A/D), que entrega um número inteiro, o valor bruto, cuja faixa depende da resolução em bits do conversor. Um conversor de 12 bits, por exemplo, representa o fundo de escala com 4096 níveis, de 0 a 4095 contagens. O ajuste para escala de engenharia é a operação que transforma esse valor bruto na grandeza que o operador reconhece, expressa na unidade de engenharia (u.e.) correspondente, como bar, metro ou litro por segundo.

Quando a relação entre o sinal e a grandeza é linear, a conversão é uma reta, como mostra a Equação 1, em que x é o valor bruto, y é o valor em unidade de engenharia, a é o ganho e b é o deslocamento, também chamado de *offset*.

$$y = a \cdot x + b \quad [\text{u.e.}] \tag{1}$$

Os dois coeficientes são determinados por dois pontos de calibração conhecidos, o limite inferior e o limite superior da faixa, em que $x_{\min}$ e $x_{\max}$ são os valores brutos nos extremos e $y_{\min}$ e $y_{\max}$ são as grandezas correspondentes. Impondo que a reta passe pelos dois pontos, obtém-se o sistema da Equação 2, cuja solução fornece o ganho, na Equação 3, e o deslocamento, na Equação 4.

$$\begin{cases} y_{\min} = a \cdot x_{\min} + b \\ y_{\max} = a \cdot x_{\max} + b \end{cases} \tag{2}$$

$$a = \frac{y_{\max} - y_{\min}}{x_{\max} - x_{\min}} \quad [\text{u.e./contagem}] \tag{3}$$

$$b = y_{\min} - a \cdot x_{\min} \quad [\text{u.e.}] \tag{4}$$

Um exemplo numérico completo ilustra o procedimento. Considere um transmissor de pressão calibrado de 0 a 10 bar, com saída de 4 a 20 mA, ligado a uma entrada analógica de 12 bits que associa 0 mA a 0 contagem e 20 mA a 4095 contagens. A corrente de 4 mA corresponde, portanto, a $\frac{4}{20} \cdot 4095 = 819$ contagens. Os pontos de calibração são então $x_{\min} = 819$ para $y_{\min} = 0$ bar e $x_{\max} = 4095$ para $y_{\max} = 10$ bar. O ganho resulta na Equação 5.

$$a = \frac{10 - 0}{4095 - 819} = \frac{10}{3276} \approx 3{,}0525 \times 10^{-3} \ \frac{\text{bar}}{\text{contagem}} \tag{5}$$

O deslocamento é obtido pela Equação 6.

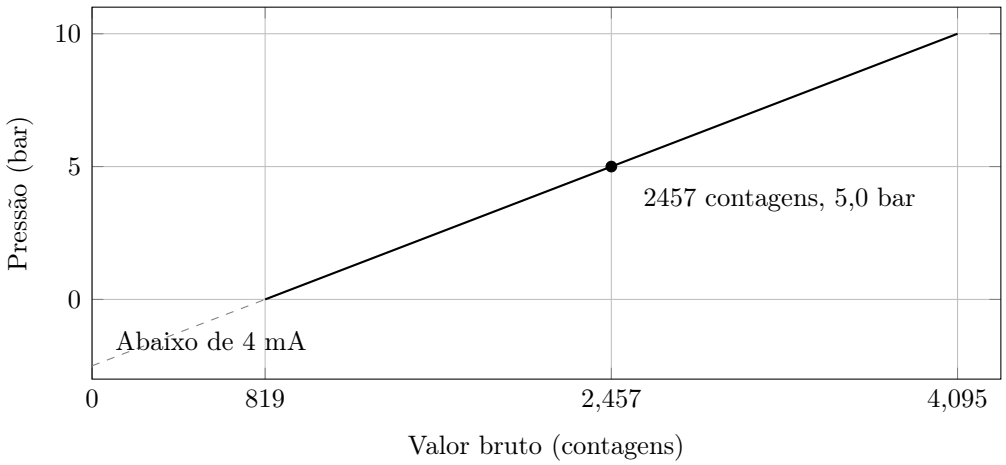
$$b = 0 - \frac{10}{3276} \cdot 819 = -2{,}5 \text{ bar} \tag{6}$$

Supondo agora que o conversor entregue o valor bruto $x = 2457$, a pressão correspondente é calculada pela Equação 7.

$$y = \frac{10}{3276} \cdot 2457 - 2{,}5 = 7{,}5 - 2{,}5 = 5{,}0 \text{ bar} \tag{7}$$

O resultado pode ser conferido pela corrente. O valor bruto 2457 equivale a $\frac{2457}{4095} \cdot 20 = 12$ mA, que está exatamente no meio da faixa de 4 a 20 mA, e o meio da faixa de 0 a 10 bar é 5 bar. A Figura 4 mostra a reta obtida. Valores brutos abaixo de 819 contagens indicam corrente inferior a 4 mA, condição que não corresponde a uma medição válida e deve ser tratada como falha do sensor ou do circuito, e não como pressão negativa.

Figura 4: Conversão linear de 4 a 20 mA para 0 a 10 bar



Fonte: Autor.

Nem todo sensor ou medidor tem resposta linear. Nos medidores de vazão por pressão diferencial, por exemplo, a vazão é proporcional à raiz quadrada da pressão diferencial medida, de modo que o sinal de 4 a 20 mA não é proporcional à vazão. A conversão exige a extração da raiz, como na Equação 8, em que $Q_{\max}$ é a vazão de fundo de escala. Com o sinal em 25 % da faixa, a vazão calculada é $\sqrt{0{,}25} = 50\,\%$ do fundo de escala, o que mostra por que tratar essa medição como linear levaria a um erro grosseiro.

$$Q = Q_{\max} \cdot \sqrt{\frac{x - x_{\min}}{x_{\max} - x_{\min}}} \quad \text{[u.e.]} \tag{8}$$

Para relações não lineares sem expressão simples, como a conversão do nível de um tanque de geometria irregular em volume, ou a curva de um sensor de temperatura, usa-se a linearização por partes. A curva é descrita por uma tabela de pontos de calibração (x_i, y_i) e, para um valor bruto x situado entre dois pontos consecutivos, a grandeza é obtida por interpolação linear, como na Equação 9, válida para $x_i \leq x \leq x_{i+1}$. Quanto maior o número de pontos, melhor a aproximação da curva real.

$$y = y_i + \frac{y_{i+1} - y_i}{x_{i+1} - x_i} \cdot (x - x_i) \quad \text{[u.e.]} \tag{9}$$

Em resumo, o ajuste de escala é a ponte entre o mundo físico, em que a grandeza é medida, e o mundo digital, em que ela é armazenada, exibida e comparada com limites de alarme. Por isso a escala deve ser configurada para cada ponto de forma explícita, e o valor bruto e o valor em unidade de engenharia devem ser distinguidos com clareza, a fim de que um erro de calibração não passe despercebido na operação.

2.3 Automação e Tecnologias da Indústria 4.0

A automação industrial é o conjunto de tecnologias e práticas que substituem a intervenção humana direta na execução de processos produtivos por sistemas eletrônicos, mecânicos e computacionais capazes de medir, controlar, comandar e supervisionar variáveis físicas. A informática industrial, por sua vez, designa a aplicação das tecnologias da informação e da comunicação a esse contexto, articulando o mundo da Tecnologia da Automação (TA), voltado a tempo real determinístico, robustez e segurança operacional, com o da Tecnologia da Informação (TI), orientado a interoperabilidade, escala e segurança da informação. A convergência entre TA e TI é um dos principais vetores de transformação da automação contemporânea (Bangemann *et al.*, 2014), frequentemente descrita como o desfecho que conduz à Indústria 4.0 (Pisching, 2017; Bigheti, 2020).

A IoT é uma rede global que liga à internet objetos físicos e virtuais, cada um com identidade digital própria, capazes de trocar informações entre si sem intervenção humana (Pisching, 2017; Bigheti, 2020). Ela reúne em um mesmo ambiente tecnologias como a computação móvel, as redes de sensores sem fio e os sistemas ciber-físicos, apontando para um cenário em que medir e monitorar o mundo se torna tão corriqueiro quanto o uso da eletricidade (Stankovic, 2014).

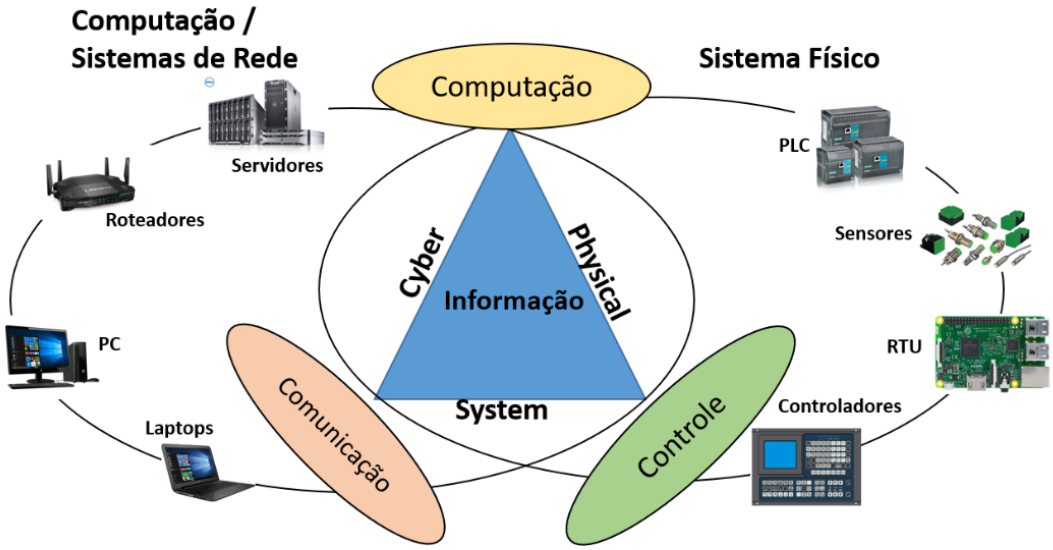
Quando esse paradigma é levado para o ambiente industrial, sensoriamento de processos, controle de máquinas e otimização da produção, recebe o nome de Internet Industrial das Coisas (*Industrial Internet of Things*, IIoT), um ramo da IoT voltado a aumentar a eficiência e a agilidade das fábricas (Bigheti, 2020). Embora partam da mesma base, a IoT de consumo e a IIoT diferem no grau de criticidade: a de consumo atende a usos casuais e com exigências leves, enquanto a industrial opera com equipamentos fixos, grande volume de dados e requisitos rígidos de resposta em tempo real, confiabilidade e segurança (Bigheti, 2020). A expansão da IoT na indústria leva os fabricantes a incorporar tecnologias que sustentam a convergência TA/TI (Pisching, 2017). À medida que os dispositivos de campo passam a oferecer suas funções como serviços de rede, a pirâmide ISA-95 (International Society of Automation, 2000) tende a achatar-se em uma arquitetura plana baseada em serviços (Bangemann *et al.*, 2014).

A base de comunicação que sustenta esses conceitos é a troca direta de informações entre dispositivos, denominada comunicação Máquina a Máquina (*Machine-to-Machine*, M2M). Originada na telemetria industrial e nos primeiros sistemas SCADA, a M2M precede e fundamenta a IoT e viabiliza a noção de máquina inteligente: dispositivos que coordenam ações, ajustam

parâmetros e reportam estados sem intervenção manual (Bigheti, 2020; Pisching, 2017). Tecnologicamente, apoia-se em uma família de protocolos de comunicação que conectam os equipamentos de campo aos serviços em nuvem, mesmo em redes de longa distância, por meio de modems celulares e *gateways* que concentram as sessões, permitindo supervisionar centenas de pontos remotos sem hardware proprietário em cada instalação (Bangemann *et al.*, 2014).

Os CPS, termo proposto em 2006 nos Estados Unidos, descrevem o acoplamento intensivo entre entidades computacionais e o mundo físico (Pisching, 2017; Monostori *et al.*, 2016). A Figura 5 ilustra esse acoplamento. Mais do que a soma das partes, é a interseção entre os mundos físico e cibernético que caracteriza o CPS (Monostori *et al.*, 2016). Herdeiros dos sistemas embarcados, os CPS acrescentam conectividade, autonomia e cooperação, e no domínio da manufatura recebem o nome de sistemas ciber-físicos de produção. Três características são essenciais aos CPS (Monostori *et al.*, 2016). A inteligência é a capacidade de adquirir informações do entorno e agir de forma autônoma. A conectividade traduz-se no uso de conexões com outros elementos, pessoas e serviços. A responsividade é a reação em tempo apropriado a mudanças. Os CPS oferecem a base física e computacional sobre a qual a IoT viabiliza a Indústria 4.0 (Pisching, 2017).

Figura 5: Estrutura de um Sistema Ciber-Físico

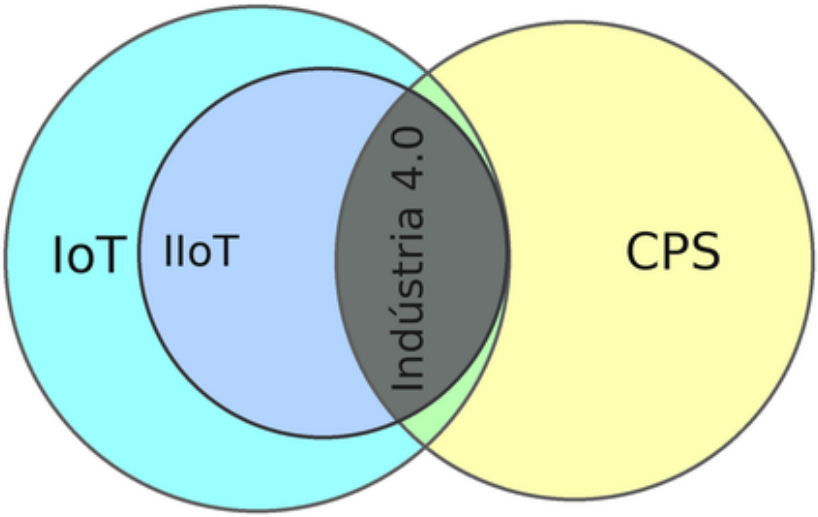


Fonte: Adaptado de Bigheti (2020).

A Indústria 4.0 nasce do encontro de duas forças. De um lado, a demanda do mercado por produtos personalizados, flexíveis e de produção descentralizada. De outro, o avanço da tecnologia, com mais automação, digitalização e equipamentos cada vez menores (Lasi *et al.*, 2014). Pode ser entendida como um conjunto de tecnologias coordenadas para conferir competitividade e customização, habilitadas por CPS, IIoT, computação em nuvem, redes sem fio e uso massivo de dados, cujo conceito-síntese é a fábrica inteligente (Bigheti, 2020). Seus princípios de projeto são a interoperabilidade, a virtualização, a descentralização, a operação em tempo real, a orientação a serviços e a modularidade, padronizados pelo modelo de referência RAMI 4.0 (Pisching, 2017). Nesse contexto, o SCADA deixa de ser um sistema fechado de supervisão local e passa a um nó de uma rede de serviços que integra produção, manutenção, logística e clientes em uma cadeia de valor digital (Bigheti, 2020; Bangemann *et al.*, 2014). A Figura 6 sintetiza as relações entre esses conceitos: a IoT como universo mais amplo, a IIoT em sua interseção com o domínio industrial, os CPS como categoria de forte acoplamento computação-

físico, e a Indústria 4.0 emergindo na interseção entre IIoT e CPS (Bigheti, 2020; Pisching, 2017).

Figura 6: Interseções de IoT, CPS, IIoT e Indústria 4.0



Fonte: Bigheti (2020, p. 27).

2.4 Computação em Nuvem

A Computação em Nuvem é um modelo no qual os dados e as aplicações deixam de residir no computador do usuário e passam a ser executados em grandes centros de processamento e armazenamento, disponibilizados na forma de serviços acessados pela internet (Henriques da Silva, 2010). Em vez de adquirir e manter a própria infraestrutura, o usuário contrata de um provedor a capacidade de processamento, o armazenamento e os programas de que necessita, bastando um dispositivo conectado à rede para utilizá-los (Henriques da Silva, 2010). A "nuvem" é a camada conceitual que oculta os equipamentos físicos e os recursos por trás de uma interface padronizada, de modo que a capacidade computacional se apresenta ao usuário como praticamente ilimitada e é provisionada conforme a necessidade (Pedrosa; Nogueira, 2011; Henriques da Silva, 2010). Para uma plataforma ser de fato uma nuvem, ela precisa reunir cinco características: contratar recursos sozinho e na hora em que precisa, acessar tudo pela rede de qualquer lugar, compartilhar os mesmos recursos entre vários clientes, aumentar ou reduzir a capacidade com rapidez e medir o consumo dos recursos (Henriques da Silva, 2010; Pedrosa; Nogueira, 2011). Quanto à forma de implantação, a nuvem pode ser pública, privada, comunitária ou híbrida, sendo esta última capaz de recorrer a recursos públicos para absorver picos de demanda (Henriques da Silva, 2010).

Os serviços de nuvem costumam ser classificados em três modelos conforme o nível de abstração oferecido. Na infraestrutura como serviço (*Infrastructure as a Service*, IaaS), o usuário contrata máquinas virtuais, armazenamento e redes e administra o que roda sobre eles. Na plataforma como serviço (*Platform as a Service*, PaaS), o provedor também administra o ambiente de execução e o usuário só fornece a aplicação. No *software* como serviço (*Software as a Service*, SaaS), o usuário consome uma aplicação pronta pela internet (Henriques da Silva, 2010; Pedrosa; Nogueira, 2011). Grandes empresas de tecnologia atuam como provedores desses serviços, como a Amazon, a Microsoft e o Google, e a escolha entre eles é um tema relevante porque a migração de uma aplicação entre provedores diferentes é trabalhosa quando ela depende

de serviços proprietários (Henriques da Silva, 2010).

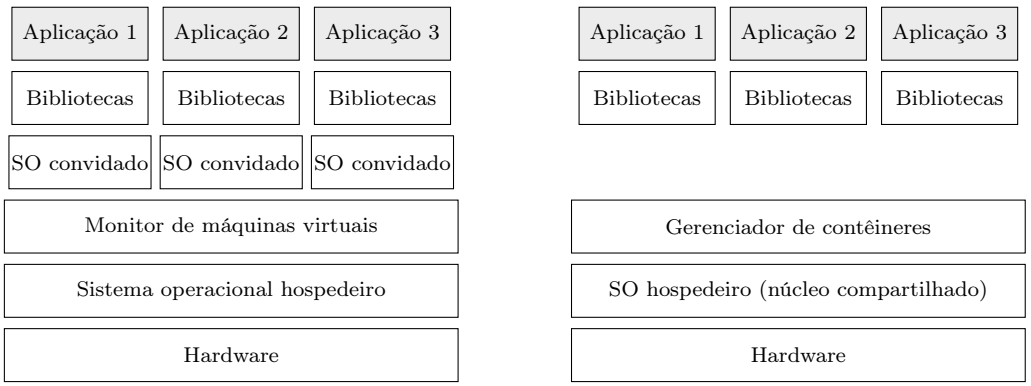
Esse modelo traz como principais vantagens a mobilidade de acesso, a dispensa de manter infraestrutura própria, a facilidade de aumentar ou reduzir a capacidade e a confiabilidade dos grandes provedores. Em contrapartida, ainda apresenta desafios, como a segurança dos dados, a dificuldade de migrar entre provedores diferentes e a garantia de disponibilidade (Henriques da Silva, 2010; Pedrosa; Nogueira, 2011). No contexto da Indústria 4.0, a nuvem funciona como a infraestrutura que conecta produtos, máquinas, sensores e serviços, e passa a tratar o SCADA, antes restrito a salas de controle locais, como mais um ponto de uma rede distribuída de serviços (Bigheti, 2020; Pisching, 2017).

2.5 Contêineres e Orquestração

A virtualização é a tecnologia que abstrai os recursos físicos de um sistema de computação para oferecer um ambiente computacional virtual, de modo a melhorar o aproveitamento dos recursos disponíveis (Netto, Hylson Vescovi *et al.*, 2017). Na forma tradicional, a virtualização ocorre por meio de máquinas virtuais, ambientes de software construídos sobre as interfaces dos recursos físicos, em que cada máquina executa um sistema operacional completo, chamado de convidado, sob o controle de um monitor de máquinas virtuais, também denominado hipervisor (Netto, Hylson Vescovi *et al.*, 2017). Esse modelo, predominante nos centros de dados que sustentam a computação em nuvem, oferece forte isolamento entre as cargas de trabalho, mas cada máquina carrega o custo de um sistema operacional inteiro (Netto, Hylson V. *et al.*, 2016).

A virtualização baseada em contêineres propõe uma alternativa mais leve. Em vez de virtualizar o hardware, o contêiner virtualiza o próprio sistema operacional: ele compartilha o núcleo do sistema hospedeiro e isola os seus processos dos demais processos em execução, cabendo a um gerenciador de contêineres mediar o uso dos recursos (Netto, Hylson Vescovi *et al.*, 2017). A ideia tem origem em mecanismos como o comando *jail*, do sistema FreeBSD, e as *zones*, do sistema Solaris, e se difundiu com o Linux, cujo núcleo oferece os recursos de isolamento e de limitação de consumo de processador e memória sobre os quais os contêineres atuais se apoiam (Netto, Hylson Vescovi *et al.*, 2017; Docker, Inc., 2026). A Figura 7 compara os dois modelos.

Figura 7: Máquinas virtuais e contêineres



(a) Máquinas virtuais (b) Contêineres

Fonte: Elaborado pelo autor com base em Hylson Vescovi Netto *et al.* (2017).

Como o contêiner não precisa inicializar um sistema operacional próprio, ele é instanciado e recebe seus recursos de forma muito mais rápida que uma máquina virtual, e a aplicação só

consome recursos enquanto o contêiner está em execução (Netto, Hylson Vescovi *et al.*, 2017). O contêiner é criado a partir de uma imagem, um pacote estático e imutável que reúne a aplicação, suas bibliotecas e dependências, normalmente construído em camadas, de modo que apenas os arquivos que ainda não existem no sistema hospedeiro ocupam espaço adicional (Netto, Hylson V. *et al.*, 2016). Essa característica garante portabilidade, pois a mesma imagem executa de forma idêntica no computador do desenvolvedor, em um servidor local ou em um provedor de nuvem. Em contrapartida, como as imagens são estáticas, o contêiner é essencialmente um ambiente sem estado, e tanto o estado do contêiner quanto o das aplicações que nele executavam se perdem quando ele é encerrado, a menos que os dados sejam gravados em volumes externos (Netto, Hylson Vescovi *et al.*, 2017).

O Docker é a ferramenta de contêineres mais difundida. Ele oferece o formato de imagem, a descrição declarativa da construção da imagem em um arquivo de receita, o registro onde as imagens são armazenadas e distribuídas e o motor que executa os contêineres (Docker, Inc., 2026). Para aplicações formadas por vários contêineres que precisam trabalhar juntos, uma aplicação com servidor web, banco de dados e serviços de apoio, por exemplo, a ferramenta Docker Compose descreve em um único arquivo os contêineres, as redes que os conectam e os volumes que guardam seus dados, o que permite subir todo o conjunto com um comando (Docker, Inc., 2026). Contudo, a implementação nativa do Docker não oferece mecanismos de gerenciamento, orquestração e integração de contêineres em um agrupamento de máquinas (*cluster*), requisito desejável para a tolerância a faltas (Netto, Hylson Vescovi *et al.*, 2017).

A orquestração de contêineres é a automação do ciclo de vida de muitos contêineres distribuídos em várias máquinas. O orquestrador mais difundido é o Kubernetes, criado pelo Google a partir da experiência com o sistema interno Borg e entregue à *Cloud Native Computing Foundation*, fundação criada para orientar a adoção de contêineres em nuvem de maneira padronizada (Netto, Hylson V. *et al.*, 2016; The Kubernetes Authors, 2026). Suas principais funcionalidades são o controle de admissão dos contêineres, o balanceamento de recursos entre as máquinas, a descoberta de serviços, a publicação de um serviço para acessos externos ao agrupamento e o balanceamento da carga entre as réplicas de um contêiner (Netto, Hylson Vescovi *et al.*, 2017).

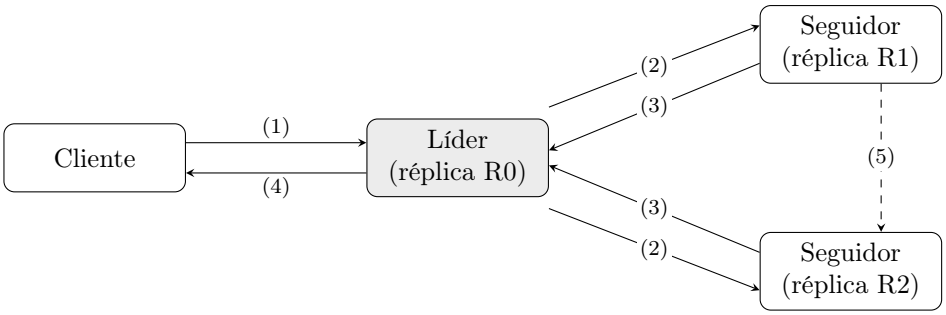
A arquitetura do Kubernetes é composta por máquinas, físicas ou virtuais, denominadas nós. A unidade básica de gerenciamento é o *pod*, que abriga um ou mais contêineres fortemente acoplados, os quais compartilham recursos como volumes e endereço de rede. Cada nó executa um agente, o *kubelet*, que gerencia os *pods* e informa o estado dos contêineres ao nó principal, e um *proxy*, que recebe as requisições e as encaminha aos *pods*, distribuindo a carga entre as réplicas, por exemplo pelo critério *round robin*. O nó principal reúne os componentes de gerenciamento, o servidor de API, pelo qual os demais componentes e o operador interagem com o agrupamento, e o armazenamento distribuído *etcd*, onde as informações do agrupamento são persistidas (Netto, Hylson Vescovi *et al.*, 2017; Netto, Hylson V. *et al.*, 2016). O Kubernetes replica contêineres para aumentar a disponibilidade, e quando um contêiner falha ele o recria a partir da imagem (Netto, Hylson V. *et al.*, 2016). Esses recursos de escalonamento, réplicas, descoberta de serviços e recuperação de falhas permitem tratar um conjunto de máquinas como uma única plataforma de execução, na qual a aplicação declara quantas réplicas deseja e o orquestrador trabalha para manter esse estado.

O Kubernetes, porém, replica contêineres e não o estado das aplicações neles hospedadas, pois o contêiner recriado volta ao estado da imagem. Usar volumes externos mitiga o problema, mas exige que o volume seja protegido contra falhas e que a aplicação controle os acessos concorrentes a ele (Netto, Hylson V. *et al.*, 2016). Para aplicações que alteram um estado e precisam tolerar a falha de réplicas, uma abordagem consolidada é a replicação de máquinas de estados (*State Machine Replication*, replicação de máquinas de estados (*State Machine Re-*

plication, SMR)), técnica que mantém um conjunto de computadores sincronizados no mesmo estado apesar das faltas que ocorram no sistema (Netto, H. *et al.*, 2019). Para tolerar até f faltas por parada, o sistema precisa de pelo menos $n \geq 2f + 1$ réplicas. A técnica se apoia em três propriedades: todas as réplicas corretas partem do mesmo estado inicial, as operações são determinísticas, de modo que a mesma entrada sobre o mesmo estado produz a mesma saída e o mesmo novo estado, e todas as réplicas processam a mesma sequência de comandos (Netto, H. *et al.*, 2019). A terceira propriedade, a coordenação, exige que todo pedido seja entregue às réplicas na mesma ordem, o que se obtém com um protocolo de difusão atômica ou de consenso distribuído, equivalentes entre si (Netto, H. *et al.*, 2019; Netto, Hylson Vescovi *et al.*, 2017).

O Raft é um algoritmo de consenso que pode ser usado para implementar a replicação de máquinas de estados, concebido para ser mais fácil de compreender que o Paxos, algoritmo de consenso mais conhecido (Ongaro; Ousterhout, 2014; Netto, H. *et al.*, 2019). No Raft, uma réplica atua como líder e as demais como seguidoras. As atualizações do estado replicado fluem apenas do líder para os seguidores, o que simplifica o gerenciamento do registro replicado, e por isso somente o líder aceita pedidos dos clientes. A operação normal ocorre em fases, ilustradas na Figura 8. O cliente envia o pedido ao líder (1), que define a ordem do pedido e o propõe aos seguidores (2). Os seguidores aceitam a proposta e respondem ao líder (3). Quando a maioria aceitou, o líder confirma o pedido e responde ao cliente (4) e, na mensagem seguinte, informa a confirmação aos seguidores, que também a efetivam (5) (Netto, H. *et al.*, 2019).

Figura 8: Operação normal do protocolo Raft



Fonte: Elaborado pelo autor com base em Hylson Netto *et al.* (2019).

Se uma réplica não recebe comunicação do líder, nem pelas mensagens periódicas de batimento (*heartbeat*) nem por novos pedidos ordenados, ela se torna candidata e inicia uma eleição, solicitando votos às demais. O novo líder é aquele aceito pela maioria das réplicas. O uso de temporizadores aleatórios nas eleições resolve conflitos de forma simples e rápida e combina com a estratégia do Kubernetes, em que os contêineres não são diferenciados entre si e qualquer um pode se tornar líder. O Raft também usa o conceito de quórum, a maioria das réplicas, para manter o funcionamento durante as trocas de configuração (Netto, H. *et al.*, 2019). O próprio *etcd* do Kubernetes é uma implementação desse protocolo (Netto, H. *et al.*, 2019).

Há ao menos três maneiras de incorporar a coordenação das réplicas a um ambiente existente, a integração, em que um componente do ambiente é construído ou modificado para oferecer a funcionalidade, a interceptação, em que as mensagens destinadas à aplicação são capturadas e mapeadas em um sistema de comunicação de grupo, e o serviço, em que uma camada de software entre o cliente e a aplicação se encarrega de coordenar os pedidos (Netto, Hylson Vescovi *et al.*, 2017). Cada uma dessas abordagens foi explorada para o Kubernetes. A integração, usando o *etcd* como memória compartilhada para guiar a coordenação, foi avaliada experimentalmente

e mostrou-se viável (Netto, Hylson V. *et al.*, 2016). A abordagem baseada em serviço acopla a coordenação à aplicação de maneira leve e simples, e foi avaliada com aplicações de consistência forte e de consistência eventual (Netto, Hylson Vescovi *et al.*, 2017). A incorporação do Raft em contêineres gerenciados pelo Kubernetes, por sua vez, adaptou o algoritmo para a descoberta automática das réplicas e para o encaminhamento ao líder dos pedidos recebidos por seguidores, e a avaliação mostrou desempenho próximo ao do Raft executado em máquinas físicas, com maior consumo de rede na versão em contêineres e maior consumo de processamento e memória na versão em máquinas físicas (Netto, H. *et al.*, 2019).

Em síntese, os contêineres tornam a implantação leve, portátil e repetível, e os orquestradores adicionam escalonamento, réplicas, descoberta de serviços e recuperação automática de falhas. Quando a tolerância a falhas deve alcançar também o estado das aplicações, a replicação de máquinas de estados com protocolos de consenso como o Raft complementa essa base. Essa decomposição do software em pequenas unidades independentes, executadas e replicadas em contêineres, é o terreno sobre o qual se desenvolve o estilo arquitetural discutido a seguir.

2.6 Microsserviços

O paradigma de microsserviços é um estilo arquitetural em que a aplicação é estruturada como um conjunto de serviços pequenos, autônomos e independentemente implantáveis, cada um responsável por um domínio bem delimitado e comunicando-se por protocolos leves, tipicamente APIs HTTP/REST ou mensageria assíncrona. O estilo é herdeiro da SOA, que organiza as funcionalidades como serviços passíveis de serem publicados, descobertos e consumidos sob demanda (Pisching, 2017), mas distingue-se da SOA clássica pela granularidade e pela autonomia, ao adotar serviços pequenos, com bancos próprios e ciclos de vida independentes, em vez de serviços grandes orquestrados por um barramento central (Bangemann *et al.*, 2014).

Esse modelo supera limitações do monolito, em que todas as responsabilidades vivem em um único processo e a escalabilidade aplica-se ao todo, não à parte realmente sobrecarregada. Suas vantagens centrais são a implantação independente, a escalabilidade granular, na qual se replica apenas o gargalo, e a heterogeneidade tecnológica, em que cada serviço usa a linguagem e o banco mais adequados (Bigheti, 2020). No contexto da Indústria 4.0, esses serviços alinham-se aos princípios de modularidade, descentralização e orientação a serviços (Pisching, 2017; Bigheti, 2020).

2.7 Moleculer

O Moleculer é uma ferramenta para a construção de microsserviços na plataforma Node.js, voltada a sistemas distribuídos formados por serviços pequenos e independentes (MoleculerJS, 2026). Em uma única biblioteca leve, reúne os recursos necessários para que esses serviços se localizem na rede, dividam a carga de trabalho entre si e continuem operando diante de falhas, o que a torna uma alternativa mais simples a soluções mais complexas (MoleculerJS, 2026).

Cada serviço expõe um conjunto de operações que podem ser chamadas remotamente por outros serviços, e também envia avisos quando algo relevante acontece, para que os demais serviços reajam. Cada serviço registra-se em um componente central de coordenação, responsável por localizar automaticamente os serviços disponíveis e encaminhar cada chamada à cópia mais adequada, distribuindo o trabalho entre as cópias em execução (MoleculerJS, 2026). Quando uma dessas cópias falha, esse coordenador redireciona a chamada para outra ou devolve uma resposta alternativa, evitando que a falha de um único serviço se propague pelo restante do sistema. Essa capacidade de manter o funcionamento mesmo diante de falhas alinha-se às

arquiteturas de microsserviços empregadas na Indústria 4.0 (Bigheti, 2020).

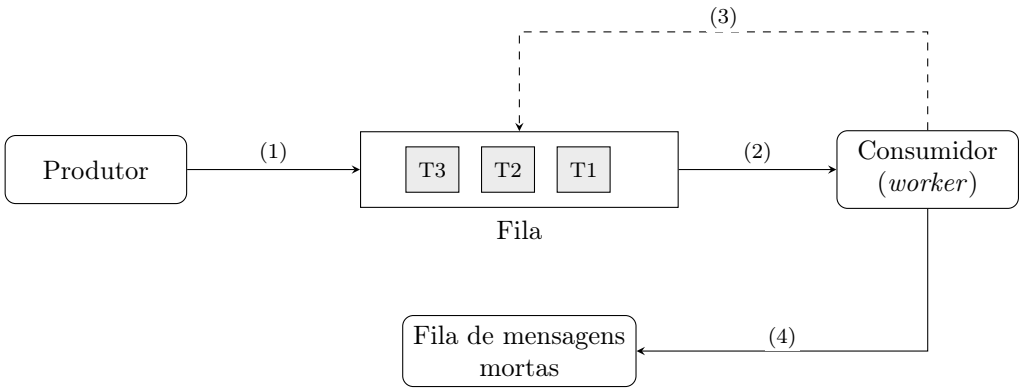
2.8 Filas de Mensagens e Processamento Assíncrono

Uma fila de mensagens é uma estrutura intermediária que desacopla quem solicita um trabalho de quem o executa. O componente que cria a mensagem, chamada de tarefa ou *job*, e a insere na fila é o produtor. O componente que retira as mensagens e as processa é o consumidor, frequentemente chamado de *worker*. Como produtor e consumidor não precisam estar disponíveis ao mesmo tempo nem operar no mesmo ritmo, o processamento passa a ser assíncrono: o produtor registra o pedido e segue adiante sem esperar o resultado, enquanto o consumidor o atende quando houver capacidade (Taskforce.sh Inc., 2026).

Esse arranjo traz benefícios. A fila absorve picos de demanda, pois as tarefas excedentes esperam em vez de sobrecarregar o consumidor. Permite escalar o processamento de forma independente, bastando acrescentar consumidores para drenar a fila mais depressa. Isola falhas, já que a indisponibilidade momentânea do consumidor não impede o produtor de registrar novas tarefas. E retira do caminho de uma requisição interativa os trabalhos demorados, de modo que o usuário recebe a resposta sem aguardar a conclusão do trabalho (Taskforce.sh Inc., 2026).

A confiabilidade do processamento assíncrono depende de algumas práticas. As filas costumam entregar cada tarefa ao menos uma vez, o que significa que uma mesma tarefa pode ser executada mais de uma vez, por exemplo quando um consumidor falha antes de confirmar a conclusão. Por isso o tratamento da tarefa deve ser idempotente, isto é, executá-lo duas vezes deve produzir o mesmo resultado que executá-lo uma vez. Gravar um registro com uma chave única, ou atualizar um valor para um estado final em vez de somá-lo a um acumulado, são formas usuais de obter idempotência. Quando o tratamento falha, a fila repete a tarefa por um número limitado de tentativas, com espera crescente entre elas (*backoff*), para não sobrecarregar um recurso que ainda se recupera. Há falhas que não se resolvem repetindo, como um dado inválido, e para elas a tarefa deve ser encerrada de imediato e movida para uma fila de mensagens mortas (*dead-letter*), onde permanece disponível para análise. Convém também limitar a concorrência de cada consumidor, atribuir prioridades, manter o conteúdo da mensagem pequeno, com identificadores em vez de dados completos, e limpar periodicamente as tarefas concluídas para que a fila não cresça sem limite (Taskforce.sh Inc., 2026). A Figura 9 resume o fluxo.

Figura 9: Fluxo de uma fila de mensagens com tentativas e fila de mensagens mortas



Fonte: Autor.

A figura mostra o caminho de uma tarefa. O produtor insere a tarefa na fila (1) e o

consumidor a retira e a processa (2). Se o processamento falha e ainda restam tentativas, a tarefa retorna à fila após a espera definida (3). Esgotadas as tentativas, ou diante de uma falha sem solução, a tarefa é movida para a fila de mensagens mortas (4).

O BullMQ é uma biblioteca de filas para a plataforma Node.js que guarda o estado das filas no Redis. Ela oferece tentativas com *backoff*, tarefas com atraso, tarefas repetíveis agendadas por expressões de tempo, prioridades, limite de concorrência e de taxa, e a remoção automática de tarefas concluídas e falhadas segundo critérios de idade e de quantidade (Taskforce.sh Inc., 2026). Como todo o estado da fila reside no Redis, a configuração desse banco influi na confiabilidade do conjunto. Em particular, uma política de memória que descarte chaves quando o limite é atingido pode eliminar tarefas em silêncio, razão pela qual recomenda-se a política que recusa novas escritas, em vez de remover dados existentes, quando a memória se esgota (Redis Ltd., 2026; Taskforce.sh Inc., 2026).

2.9 Banco de Dados

O banco de dados é o componente de software responsável por armazenar, organizar e recuperar informações de forma persistente. Quanto à organização dos dados, destacam-se dois grandes paradigmas: o relacional, que estrutura os dados em tabelas com esquema definido e relações explícitas entre as entidades, e o NoSQL, voltado a cenários de alta variabilidade de esquema e escala horizontal (Mohamed; Altrafi; Ismail, 2014). Por organizar os dados de forma estruturada e preservar a integridade entre entidades relacionadas, o modelo relacional é o adequado a sistemas que exigem consistência e consultas expressivas, sendo o detalhado a seguir.

O modelo relacional, materializado em bancos como PostgreSQL, MySQL e Oracle, organiza os dados em tabelas de linhas e colunas, cada uma definida por um esquema, uma chave primária que identifica unicamente cada registro e, opcionalmente, chaves estrangeiras que estabelecem relações entre tabelas, garantindo integridade referencial (Mohamed; Altrafi; Ismail, 2014). A linguagem de consulta padrão é o SQL (*Structured Query Language*), com operações de seleção, inserção, atualização e remoção, além de junções, agregações, transações e índices.

Em ambientes de produção, boas práticas incluem indexar os campos mais consultados, particionar as tabelas históricas por intervalo de tempo para evitar tabelas monolíticas e empregar réplicas que assumem o serviço automaticamente quando o nó principal falha e cópias do estado do banco em pontos definidos no tempo para alta disponibilidade.

O particionamento de tabelas divide uma tabela lógica muito grande em partes físicas menores, as partições, segundo um critério como o intervalo de valores de uma coluna. Para dados históricos que crescem continuamente, como medições coletadas ao longo do tempo, o critério natural é o intervalo de tempo, com uma partição por mês, por exemplo. O planejador de consultas ignora as partições que não podem conter os registros pedidos, técnica denominada poda de partições, e por isso uma consulta sobre um período recente varre apenas as partições correspondentes. A remoção de dados antigos também fica mais barata, pois descartar uma partição inteira é muito mais rápido que apagar milhões de linhas uma a uma e evita a fragmentação do armazenamento (PostgreSQL Global Development Group, 2026). Associado ao particionamento, é usual manter tabelas de agregados pré-calculados, como as médias por minuto ou por hora, que permitem exibir longos períodos sem percorrer todos os registros brutos.

Em sistemas que atendem várias organizações clientes, chamados multi-inquilino (*multi-tenant*), existem três estratégias principais de organização dos dados. Na primeira, todos os clientes compartilham o mesmo banco e as mesmas tabelas, e uma coluna identifica o inquilino

dono de cada linha. Na segunda, cada cliente tem seu próprio esquema dentro de um banco compartilhado. Na terceira, cada cliente tem um banco de dados dedicado (Chong; Carraro, 2006). As duas primeiras favorecem o custo e a densidade de clientes por servidor, mas dependem de filtros corretos em toda consulta para evitar a exposição de dados entre clientes. O banco dedicado oferece o isolamento mais forte, permite fazer cópias de segurança, restaurar e excluir os dados de um cliente de modo independente e até hospedá-los em um servidor escolhido por ele, ao custo de mais conexões abertas, de mais bancos a administrar e de migrações de esquema que precisam ser aplicadas a cada banco (Chong; Carraro, 2006).

2.10 Redis

O *Redis* é um armazenamento de dados em memória, de código aberto, que opera como banco chave-valor de altíssimo desempenho. Diferentemente dos bancos relacionais, em que a persistência em disco é a prioridade, o Redis mantém todo o conjunto de trabalho em memória RAM, oferecendo latências sub-milissegundo para operações de leitura e escrita (Redis Ltd., 2026).

O Redis oferece persistência opcional por snapshots periódicos e log de operações de escrita, expiração automática de chaves por *time-to-live* (TTL) e um mecanismo publicador-assinante (Pub/Sub) para troca de mensagens entre processos. Por essas características, é comumente empregado como cache de acesso rápido, como barramento de eventos e como armazenamento de dados temporários (Redis Ltd., 2026).

Como o Redis guarda os dados em memória, ele usa cópias e recuperação automática para não parar diante de falhas. A replicação mantém uma ou mais réplicas sempre atualizadas da instância principal. Sobre isso, o *Redis Sentinel* cuida da disponibilidade: vigia a instância principal e, se ela cai, promove sozinho uma réplica ao seu lugar, sem intervenção manual. Já o *Redis Cluster* cuida do crescimento: reparte as chaves entre vários nós, de modo que basta acrescentar nós para aumentar a capacidade, e cada nó pode ter réplicas para o serviço continuar mesmo se um deles falhar (Redis Ltd., 2026).

Além de simples pares de texto, o Redis oferece estruturas de dados como listas, conjuntos, conjuntos ordenados e *hashes*, com operações atômicas sobre cada uma, e permite executar pequenos programas na linguagem Lua no próprio servidor, o que garante que uma sequência de comandos seja executada sem interferência de outros clientes (Redis Ltd., 2026). Essas características o tornam adequado para guardar o valor mais recente de uma informação que muda com frequência, aplicar limites de taxa de requisições, implementar travas distribuídas entre processos e servir de base para filas, como visto anteriormente.

2.11 Node.js, TypeScript e Fastify

O Node.js é um ambiente de execução que permite rodar JavaScript fora do navegador, construído sobre o motor V8. Seu modelo de execução é orientado a eventos, com operações de entrada e saída não bloqueantes: um único fluxo de execução atende muitas conexões simultâneas, pois, enquanto uma operação de rede ou de disco aguarda, o laço de eventos processa outras requisições (OpenJS Foundation, 2026b). Esse modelo é adequado a servidores com grande volume de operações de entrada e saída, como APIs e serviços que se comunicam com bancos de dados e com a rede. Em contrapartida, cálculos longos que ocupam o processador bloqueiam o laço de eventos e atrasam todas as demais requisições, razão pela qual esse tipo de trabalho deve ser transferido para processos ou fluxos de execução separados (OpenJS Foundation, 2026b).

O TypeScript é um superconjunto do JavaScript que acrescenta um sistema de tipos estáticos. O compilador verifica a coerência dos tipos antes da execução e, em seguida, gera código JavaScript comum, de modo que erros como o uso de um campo inexistente ou a passagem de um argumento de tipo errado são detectados no desenvolvimento, e não em produção (Microsoft, 2026). Os tipos, contudo, existem apenas em tempo de compilação e são apagados na execução. Dados que chegam de fora do programa, como o corpo de uma requisição, não têm garantia de corresponder ao tipo declarado, e por isso precisam ser validados nas fronteiras do sistema.

O Fastify é um *framework* web para Node.js voltado a baixo custo computacional por requisição e boa experiência de desenvolvimento. Sua estrutura se baseia em plugins encapsulados, em que cada conjunto de rotas, de configurações e de dependências é registrado em um contexto próprio, o que favorece a organização do código em módulos. O ciclo de vida de cada requisição é dividido em etapas nas quais se podem inserir ganchos (*hooks*), como a autenticação, a validação e o tratamento de erros, e o *framework* permite descrever o formato esperado da requisição e da resposta por meio de esquemas, usados tanto para validar a entrada quanto para serializar a saída com eficiência (OpenJS Foundation, 2026a).

A validação de contratos por esquema é a técnica que descreve, de forma declarativa, a estrutura que os dados devem ter, e rejeita o que não a respeita. A biblioteca Zod, desenvolvida para o TypeScript, permite declarar esse esquema em código e dele inferir automaticamente o tipo estático correspondente, de modo que um único artefato serve como validação em tempo de execução e como tipo em tempo de compilação (Zod, 2026). Quando o esquema é compartilhado entre o servidor e o cliente, ambos concordam sobre o formato das mensagens e uma divergência, como a remoção de um campo, é detectada na compilação. Os esquemas também podem ser usados para gerar a documentação da API, mantendo-a sincronizada com o comportamento real (Zod, 2026; OpenJS Foundation, 2026a).

2.12 React e Next.js

O React é uma biblioteca JavaScript, desenvolvida pela Meta, para montar interfaces de usuário a partir de componentes, isto é, pequenos blocos reutilizáveis de tela, como botões, formulários e gráficos, que se combinam para formar páginas inteiras. Sua abordagem é declarativa: o desenvolvedor descreve como a interface deve ser para um dado estado da aplicação, e o React se encarrega de atualizar apenas as partes da tela que mudaram quando o estado muda, comparando a nova descrição com a anterior em uma representação intermediária em memória (Meta Platforms, Inc., 2026). Os componentes recebem dados por propriedades e mantêm estado interno por meio de ganchos (*hooks*), e a composição de componentes pequenos facilita o reaproveitamento e o teste (Meta Platforms, Inc., 2026).

O Next.js é um *framework* para a construção de aplicações web criado sobre o React. O React, sozinho, monta a interface diretamente no navegador do usuário: envia um arquivo quase vazio acompanhado de um programa em JavaScript que desenha a tela depois que a página carrega. Esse modelo, chamado de renderização no cliente, funciona bem, mas tem desvantagens, a primeira carga pode ser lenta e os mecanismos de busca têm dificuldade para ler o conteúdo (Vercel, Inc., 2026).

O Next.js surge justamente para contornar essas limitações, oferecendo outras formas de gerar as páginas. Na renderização no servidor, a página já é montada por completo antes de ser enviada ao navegador, o que acelera a exibição e facilita a leitura pelos buscadores. Na geração estática, a página é montada uma única vez, no momento em que o sistema é publicado, e a mesma versão pronta é entregue a todos os usuários, garantindo o melhor desempenho para

conteúdos que mudam pouco. Uma vantagem importante é que o Next.js permite escolher a estratégia mais adequada para cada tela, em vez de impor uma só a toda a aplicação (Vercel, Inc., 2026).

Além das opções de renderização, o Next.js simplifica tarefas comuns do desenvolvimento. O roteamento, isto é, a associação entre os endereços e as páginas, baseia-se na própria organização dos arquivos do projeto: cada arquivo criado em uma pasta específica vira automaticamente um endereço da aplicação, sem necessidade de configuração manual. O *framework* ainda traz, de fábrica, recursos que melhoram o desempenho, como carregar apenas o código necessário a cada página, preparar de antemão as páginas que o usuário provavelmente acessará, otimizar imagens automaticamente e oferecer suporte nativo ao TypeScript (Vercel, Inc., 2026).

O *framework* também permite distinguir os componentes que executam apenas no servidor, que podem acessar recursos privados e não enviam seu código ao navegador, dos componentes que executam no cliente, necessários quando há interação, estado ou eventos (Vercel, Inc., 2026). Essa distinção viabiliza usar o Next.js unicamente como camada de apresentação, responsável pelas telas, enquanto a regra de negócio e o acesso aos dados ficam em uma API separada, que o navegador consulta por requisições HTTP. Nessa divisão, o *framework* entrega as páginas e o código de interface, e a API concentra a validação, a autenticação e a persistência.

2.13 HTTP, API REST e JSON

O protocolo HTTP (*Hypertext Transfer Protocol*) é a forma como os programas trocam informações na web. Funciona no modelo cliente-servidor: o cliente faz um pedido a um endereço (URL) e o servidor devolve uma resposta com os dados e um código que indica se a operação deu certo. Sobre o HTTP apoia-se o padrão REST (*Representational State Transfer*), um conjunto de regras para organizar APIs web de forma simples e previsível, em que cada operação usa um método do próprio HTTP, como GET para consultar, POST para criar, PUT para atualizar e DELETE para remover. O REST é uma das maneiras de implementar serviços web, recomendados para concretizar a SOA por integrarem sistemas de modo independente de plataforma (Pisching, 2017).

Os dados trocados nessas APIs costumam vir no formato JSON (*JavaScript Object Notation*), um texto leve e fácil de ler, tanto por pessoas quanto por máquinas. Em sistemas industriais, expor os dispositivos de campo por meio de APIs REST em nuvem é uma alternativa cada vez mais comum ao uso direto do Modbus e ajuda a aproximar o chão de fábrica dos sistemas de gestão (Bangemann *et al.*, 2014).

Algumas características do HTTP são importantes para o projeto de APIs. O protocolo não guarda estado entre requisições, de modo que cada pedido deve conter todas as informações necessárias para ser atendido, como a credencial do usuário. A resposta traz um código de estado agrupado por categoria, em que os da família 2xx indicam sucesso, os da 4xx erro do cliente, como a falta de autenticação (401) ou de permissão (403), e os da 5xx erro do servidor. Para que a API evolua sem quebrar os clientes existentes, adota-se o versionamento, em que a versão compõe o endereço, como em um prefixo `/v1`, e uma mudança incompatível só é publicada em uma nova versão.

Quando o cliente precisa acompanhar um dado que muda com o tempo, há duas formas de obtê-lo. Na primeira, o servidor envia as atualizações ao cliente assim que ocorrem, por meio de conexões persistentes como as *WebSockets* ou os eventos enviados pelo servidor. Na segunda, o cliente consulta o servidor repetidamente em intervalos curtos, técnica denominada *polling*. O *polling* é mais simples e funciona sobre qualquer infraestrutura HTTP comum, como balanceadores de carga e servidores intermediários, ao custo de um volume maior de requisições

e de uma latência que não pode ser menor que o intervalo de consulta.

2.14 Autenticação e Controle de Acesso

A autenticação e a autorização são etapas distintas da proteção de um sistema. A autenticação verifica a identidade de quem acessa, confirmando que o usuário é quem afirma ser. A autorização decide, uma vez conhecida a identidade, quais recursos e operações aquele usuário pode utilizar. Em sistemas de supervisão e controle, em que uma ação indevida pode danificar equipamentos ou comprometer a segurança do processo, as duas devem ser aplicadas de forma rigorosa e verificadas no servidor, e não apenas na interface, pois o controle feito no cliente serve somente à usabilidade (Stouffer *et al.*, 2015).

O primeiro cuidado na autenticação por senha é nunca armazená-la em texto legível. O sistema guarda apenas o resultado de uma função de *hash* unidirecional, de modo que a senha original não pode ser recuperada nem mesmo por quem acessa o banco. Para dificultar ataques de força bruta, a função deve ser deliberadamente custosa e usar um valor aleatório próprio de cada senha, o sal (*salt*). O Argon2 é uma função desse tipo, vencedora de uma competição internacional de funções de *hash* de senhas, cujo custo se ajusta por parâmetros de uso de memória, de número de iterações e de paralelismo, o que encarece o ataque com placas gráficas e circuitos dedicados. A variante Argon2id combina as características das outras variantes e é a recomendada para o armazenamento de senhas (Biryukov *et al.*, 2021).

Após a autenticação, o sistema precisa reconhecer as requisições seguintes do mesmo usuário sem pedir a senha novamente. Uma solução comum são os tokens. O JWT (*JSON Web Token*) é um formato compacto de token que carrega declarações (*claims*) sobre o usuário, como o identificador, o emissor e o instante de expiração, e é assinado digitalmente, de forma que qualquer alteração em seu conteúdo é detectada (Jones; Bradley; Sakimura, 2015). A assinatura garante a integridade e a autenticidade do token, mas não o torna secreto, e por isso ele não deve conter dados sensíveis (Jones; Bradley; Sakimura, 2015). Como um token válido concede acesso a quem o possuir, adota-se um token de acesso de curta duração, de alguns minutos, o que limita o dano de um eventual vazamento. Para que o usuário não precise se autenticar a cada expiração, emite-se junto um token de renovação (*refresh token*), de vida mais longa, usado apenas para obter um novo token de acesso. O token de renovação é guardado em um *cookie* com os atributos *HttpOnly*, que o torna inacessível ao código JavaScript da página, *Secure*, que restringe seu envio a conexões cifradas, e *SameSite*, que limita seu envio em requisições originadas de outros sítios (OWASP Foundation, 2026). A cada renovação o token é rotacionado, isto é, substituído por um novo, e o anterior se torna inválido. O reaparecimento de um token já rotacionado indica que ele foi copiado e usado por terceiros, e a resposta adequada é encerrar toda a sessão (OWASP Foundation, 2026).

Outras medidas complementam o conjunto. A limitação de tentativas de acesso, com bloqueio temporário após falhas consecutivas, reduz a eficácia de ataques de adivinhação de senhas. Mensagens de erro idênticas para usuário inexistente e senha incorreta evitam revelar quais contas existem. E a revogação de sessões permite encerrar de imediato o acesso de um usuário desligado ou de um dispositivo perdido (OWASP Foundation, 2026; Stouffer *et al.*, 2015).

Quanto à autorização, o modelo de controle de acesso baseado em cargos (*Role-Based Access Control*, RBAC) organiza as permissões em cargos, como administrador, supervisor e operador, e concede a cada usuário os cargos correspondentes às suas funções, em vez de atribuir permissões individualmente. Esse modelo simplifica a administração em organizações com muitos usuários e reduz erros, pois as pessoas mudam de função com mais frequência do que as funções mudam de atribuições (Stouffer *et al.*, 2015). A configuração de cada cargo deve seguir

o princípio do menor privilégio, segundo o qual cada usuário recebe somente as permissões necessárias ao seu trabalho, e nenhuma além disso (Stouffer *et al.*, 2015). Uma extensão usual é o agrupamento de permissões em conjuntos nomeados, que podem ser combinados, e a separação de funções, em que operações críticas exigem cargos distintos. Em plataformas que atendem várias organizações, a autorização inclui ainda o escopo da organização, de modo que um usuário jamais alcance dados de outra.

2.15 Execução Isolada de Código

Há situações em que um sistema precisa executar código escrito por seus próprios usuários, como fórmulas, regras de cálculo ou pequenos programas de automação. Executar esse código no mesmo processo e com os mesmos privilégios da aplicação é arriscado, porque um programa defeituoso ou malicioso pode consumir todo o processador ou a memória, entrar em laço infinito, ler arquivos, acessar a rede ou obter segredos da aplicação. A resposta a esse risco é a execução isolada, também chamada de *sandbox*, um ambiente restrito no qual o código roda com capacidades e recursos deliberadamente limitados, em aplicação do princípio do menor privilégio. Entre as técnicas disponíveis estão o uso de processos com permissões reduzidas do sistema operacional, de contêineres, de máquinas virtuais e de interpretadores embutidos com isolamento próprio, cada uma com um compromisso diferente entre a força do isolamento, o custo de inicialização e a facilidade de uso.

O WebAssembly é um formato de instruções binárias, portável e compacto, executado por uma máquina virtual baseada em pilha. Embora tenha nascido para executar código de alto desempenho nos navegadores, é independente de plataforma e de linguagem e hoje também é usado fora deles (WebAssembly Community Group, 2026; Haas *et al.*, 2017). O modelo foi projetado com a segurança como requisito. Um módulo é validado antes de executar e só acessa a própria memória linear, que é isolada da memória do hospedeiro. Ele não tem acesso direto a arquivos, rede ou relógio, e só alcança o mundo externo pelas funções que o hospedeiro explicitamente decide importar, o que configura um modelo de capacidades, em que o que não foi concedido não existe para o código (WebAssembly Community Group, 2026; Haas *et al.*, 2017).

O QuickJS é um interpretador JavaScript pequeno e embutível, escrito em C, que implementa uma versão recente da especificação da linguagem. Por ser compacto, inicializar rapidamente e interpretar o código em vez de compilá-lo dinamicamente para instruções de máquina, ele tem uma superfície de ataque reduzida, e oferece mecanismos para que o hospedeiro limite a memória utilizada e interrompa a execução, por exemplo por tempo (Bellard; Gordon, 2026). Por ser escrito em C, pode ser compilado para WebAssembly, o que permite executar a linguagem JavaScript dentro do ambiente isolado do WebAssembly.

O Wasmtime é um ambiente de execução independente para WebAssembly, mantido pela *Bytecode Alliance*, que compila e executa módulos de forma segura e eficiente. Cada instância de um módulo tem sua memória própria e o hospedeiro controla o que ela pode acessar. O ambiente oferece limites de recursos, como o teto de memória por instância e mecanismos para interromper a execução depois de certo consumo de instruções ou de tempo, de modo que um código em laço infinito não trava o sistema (Bytecode Alliance, 2026). A combinação desses elementos produz um isolamento em camadas: o código do usuário é interpretado pelo QuickJS, que roda dentro do módulo WebAssembly, que por sua vez é executado sob as restrições do Wasmtime, em um processo que pode ainda ter privilégios reduzidos pelo sistema operacional. Instanciar um ambiente novo a cada execução evita que o estado de uma execução vaze para a seguinte, e a comunicação com o restante do sistema pode se limitar a uma interface estreita,

em que os dados entram e saem serializados e o código não tem como alcançar bancos de dados, rede ou segredos (Bytecode Alliance, 2026; WebAssembly Community Group, 2026).

2.16 Interfaces de Alto Desempenho para Operação

A interface homem-máquina (IHM) é o meio pelo qual o operador enxerga o processo e age sobre ele. Telas mal projetadas, com excesso de cores, de animações e de elementos decorativos, dificultam a percepção de situações anormais e aumentam a fadiga e a chance de erro. A norma ISA-101, que trata de interfaces homem-máquina para sistemas de automação de processos, estabelece diretrizes para o projeto, a implementação e a manutenção dessas interfaces, e fundamenta a chamada IHM de alto desempenho (International Society of Automation, 2015).

As diretrizes dessa abordagem dizem respeito, em primeiro lugar, ao uso da cor. As telas devem usar cores discretas, sem gradientes, sobre fundos de tons intermediários de cinza, evitando o branco e o preto puros, que impedem representar objetos mais claros ou mais escuros. Cores chamativas ficam reservadas para as condições anormais, de modo que um equipamento em funcionamento normal não deve ser destacado com cor viva, e assim o que foge da normalidade salta aos olhos do operador. Os alarmes devem ter indicadores próprios, diferenciados não só pela cor mas também pela forma, para atender pessoas com daltonismo, e um equipamento deve usar poucas cores, com estados adicionais indicados por ícones (Elipse Software, 2026; International Society of Automation, 2015).

Em segundo lugar, a abordagem trata da apresentação da informação. Valores de processo devem ser exibidos em contexto, com indicadores analógicos, como barras, que mostram a posição do valor em relação à faixa normal e aos limites de alarme, preferindo-se o comprimento das linhas às áreas para expressar quantidades, pois o cérebro as avalia com mais facilidade. A representação digital se reserva aos casos em que a precisão do valor é mais importante que o contexto, e os gráficos de tendência permitem projetar o comportamento da variável (Elipse Software, 2026). A organização das telas segue uma hierarquia, que vai de visões gerais de uma área inteira, com poucas informações essenciais, até telas de detalhe de um equipamento e de diagnóstico, de modo que o operador navegue do geral para o específico e perceba rapidamente onde está o problema (International Society of Automation, 2015). Os objetos devem ser organizados com layouts planos, sem perspectiva, e o tamanho dos objetos deve refletir sua importância (Elipse Software, 2026).

2.17 Protocolo Modbus TCP e Modbus RTU

O Modbus é um dos protocolos de comunicação mais difundidos na automação industrial. Segue o modelo de requisição e resposta entre um cliente, que faz os pedidos, e um ou mais servidores, que são os dispositivos de campo, tradicionalmente chamados de *master* e *slaves*. Em um sistema de supervisão, o cliente envia pedidos para ler ou escrever valores no dispositivo, e este responde com os dados solicitados ou com um código de erro quando a operação não é permitida (Modbus Organization, 2012a).

Os dados de um dispositivo Modbus ficam organizados em quatro áreas de memória: *coils* e *input coils*, para estados do tipo booleano, e *holding registers* e *input registers*, para valores numéricos de 16 bits. Cada operação é identificada por um código de função, havendo funções específicas para ler ou escrever em cada uma dessas áreas (Modbus Organization, 2012a).

Para que um valor de processo seja lido corretamente, é preciso configurar, para cada ponto, alguns parâmetros definidos pelo próprio protocolo, que indicam onde o dado se encontra no dispositivo: o endereço do dispositivo, que identifica o equipamento, o código de função, que

define a área de memória e a operação, o endereço inicial e a quantidade de registradores a serem lidos. Esses parâmetros formam a base da configuração de uma tag Modbus e permitem que um mesmo software de aquisição se comunique com medidores e controladores de fabricantes diferentes apenas alterando a configuração, sem mudar o código.

Além desses parâmetros do protocolo, é preciso atenção especial à forma como o dado chega, pois é nesse ponto que surgem as maiores diferenças entre fabricantes. Cada registrador Modbus tem apenas 16 bits, mas muitos valores ocupam mais espaço: um inteiro de 32 bits ou um número de ponto flutuante usam dois registradores, e um valor de 64 bits, quatro. É necessário, portanto, configurar corretamente o tipo de dado bruto e a ordem em que esses registradores e seus bytes são combinados, a chamada ordem de bytes e de palavras, que varia de um fabricante para outro e, se interpretada de forma errada, produz um valor incorreto. Por fim, quando aplicável, configuram-se os fatores de escala que convertem o valor bruto em uma grandeza de engenharia.

O Modbus possui duas variantes principais, que se diferenciam pelo meio de transporte. O *Modbus TCP* trafega sobre redes TCP/IP, identificando o dispositivo por um endereço IP e um *Unit ID*, e é comum em equipamentos com interface Ethernet. O *Modbus RTU* é a forma usada em redes seriais, normalmente sobre o padrão RS-485, em que os dispositivos são identificados pelo endereço de escravo (Modbus Organization, 2012b).

O desempenho de uma aquisição Modbus é limitado pelo tempo de resposta e pela latência do ciclo de *polling*. Como o cliente envia um pedido e aguarda a resposta antes do pedido seguinte, o tempo total de um ciclo sobre vários dispositivos é aproximadamente a soma dos tempos de cada transação, e uma leitura só reflete o estado do processo no instante em que foi realizada, de modo que a informação exibida ao operador tem idade de até um ciclo completo. Cada transação soma o tempo de transmissão do pedido e da resposta, o tempo de processamento no dispositivo e, no caso do *Modbus TCP*, a latência da rede e da conexão. No *Modbus RTU*, o tempo de transmissão domina e é calculado pela taxa de transmissão, pois cada caractere ocupa 11 bits na linha, contando o bit de início, os oito de dados, o de paridade e o de parada. A uma taxa de 9600 bit/s, um caractere leva cerca de 1,15 ms. Em uma leitura de 10 registradores, o pedido tem 8 bytes e a resposta tem 25, somando 33 bytes, ou seja, $33 \cdot 11 = 363$ bits, que ocupam $\frac{363}{9600} \approx 37{,}8$ ms na linha. Acrescentando os dois intervalos de silêncio de 3,5 caracteres que delimitam os quadros, cerca de 4 ms cada, a transação leva em torno de 46 ms, sem contar o processamento do escravo. Com dez dispositivos consultados em sequência, o ciclo passa de 0,46 s. Por isso o período de leitura de cada ponto deve ser compatível com a capacidade do meio de comunicação, e agrupar registradores contíguos em uma única requisição reduz a sobrecarga de cabeçalho e de silêncios por valor lido.

No *Modbus RTU*, o meio físico mais comum é o RS-485, um padrão de transmissão diferencial por par trançado, que tolera ruído e permite cabos longos em ambientes industriais, com vários dispositivos ligados ao mesmo barramento. A comunicação é *half-duplex*, isto é, em um dado instante só um lado transmite, e há um único mestre no barramento, que fala com um escravo de cada vez. Cada escravo tem um endereço de 1 a 247, o quadro termina com uma verificação de redundância cíclica de 16 bits para detectar erros, e o fim de um quadro é reconhecido por um intervalo de silêncio na linha (Modbus Organization, 2012b). Esse arranjo é frequentemente usado para ligar um controlador a um dispositivo de campo, como um inversor de frequência, por um simples par de fios.

3 Arquitetura e Requisitos do Sistema

Este capítulo especifica os requisitos da solução e apresenta a arquitetura que os atende. Primeiro, são descritos os requisitos funcionais e não funcionais, derivados dos seis pilares de um sistema SCADA discutidos anteriormente e das demais funcionalidades da aplicação. Em seguida, mostra-se a arquitetura em contêineres, com a divisão de responsabilidades entre os serviços. Depois, valida-se cada pilar frente a essa arquitetura, detalha-se cada serviço, descreve-se o modelo de dados e, por fim, a implantação. O capítulo responde à pergunta "quem é quem" no sistema. O caminho percorrido por cada dado e por cada comando entre esses serviços é tratado no capítulo seguinte.

3.1 Requisitos

Os requisitos do sistema foram organizados em funcionais, que descrevem o que a aplicação deve fazer, e não funcionais, que estabelecem as qualidades e as restrições sob as quais essas funções devem operar. Boa parte deles deriva diretamente dos seis pilares de um sistema SCADA: aquisição e registro de dados, sincronismo de tempo, níveis de acesso, comando e controle, escalabilidade e redundância (Zanghi, 2019). A esses somam-se requisitos próprios de uma solução web orientada a serviços, como o cadastro hierárquico das instalações, o isolamento entre clientes, os valores calculados e os scripts, a visualização das informações e a sinalização de alarmes e eventos (Abbas, 2011).

3.1.1 Requisitos Funcionais

A Tabela 1 apresenta os requisitos funcionais do sistema, com a indicação do pilar do SCADA ou da funcionalidade complementar que os origina.

Tabela 1: Requisitos Funcionais.

Id	Requisito	Origem
RF01	Adquirir periodicamente, por *polling*, grandezas analógicas e digitais dos dispositivos de campo via Modbus TCP, inclusive com quadro RTU encapsulado, com período de varredura configurável por tag	Aquisição
RF02	Registrar continuamente as leituras no histórico, conforme o tipo de dado da tag, mantendo também médias por minuto e por hora para consultas longas	Aquisição
RF03	Manter o valor atual de cada tag, com indicação de qualidade, disponível para as telas com baixa latência	Aquisição
RF04	Associar a cada leitura uma estampa de tempo referente ao ciclo de aquisição	Sincronismo
RF05	Autenticar os usuários, gerir suas sessões e permitir a troca da própria senha	Níveis de acesso
RF06	Controlar o acesso por cliente, por cargo e por grupo de permissões, segundo o princípio do menor privilégio	Níveis de acesso
RF07	Executar o comando em quatro fases (seleção, verificação, confirmação e execução) como escrita imediata no dispositivo	Comando

Id	Requisito	Origem
RF08	Executar escrita em hora marcada por meio de evento agendado associado a um script, registrando a origem da escrita	Comando
RF09	Concluir o comando pela resposta do dispositivo à escrita, com trava por tag, idempotência e registro de auditoria	Comando
RF10	Incluir novos pontos, dispositivos e telas com o sistema em produção, sem alterar o código	Escalabilidade
RF11	Manter o cadastro hierárquico de cliente, planta, dispositivo de E/S e tag, com pastas de tags e o mapeamento Modbus de cada ponto	Cadastro
RF12	Manter um banco de dados próprio para cada cliente, criado e removido de forma automática	Multicliente
RF13	Calcular valores derivados por variável meta e por tag interna, a partir de outras tags	Cálculo
RF14	Executar scripts do usuário em JavaScript, em eventos, variáveis meta e componentes de conteúdo, de forma isolada	Cálculo
RF15	Exibir a lista de acompanhamento de tags em tempo real e o dashboard com gráficos configuráveis	Visualização
RF16	Permitir desenhar telas sinóticas em um editor gráfico e executá-las com os valores das tags e com comandos	Visualização
RF17	Exibir as plantas em um mapa e consultar o histórico e a tendência das tags por janela de tempo	Visualização
RF18	Gerar alarmes em quatro níveis fixos a partir de eventos de tag e de eventos de sistema do dispositivo, com reconhecimento	Alarmes
RF19	Registrar eventos, alarmes e comandos e exibi-los em uma linha do tempo	Alarmes

Fonte: Autor.

3.1.2 Requisitos Não Funcionais

A Tabela 2 apresenta os requisitos não funcionais, agrupados por categoria de atributo de qualidade: desempenho, escalabilidade, disponibilidade, segurança, interoperabilidade, manutenibilidade, portabilidade e usabilidade.

Tabela 2: Requisitos Não Funcionais.

Id	Requisito	Categoria
RNF01	As telas de tempo real refletem as leituras com baixa latência, buscando o valor corrente mantido em memória, com atualização por *polling* curto	Desempenho
RNF02	Os serviços de aquisição e de processamento em segundo plano são replicáveis na horizontal, com distribuição do trabalho por afinidade ao dispositivo	Escalabilidade
RNF03	O trabalho de cada cliente é enfileirado de forma justa, de modo que um cliente não monopoliza o processamento dos demais	Escalabilidade

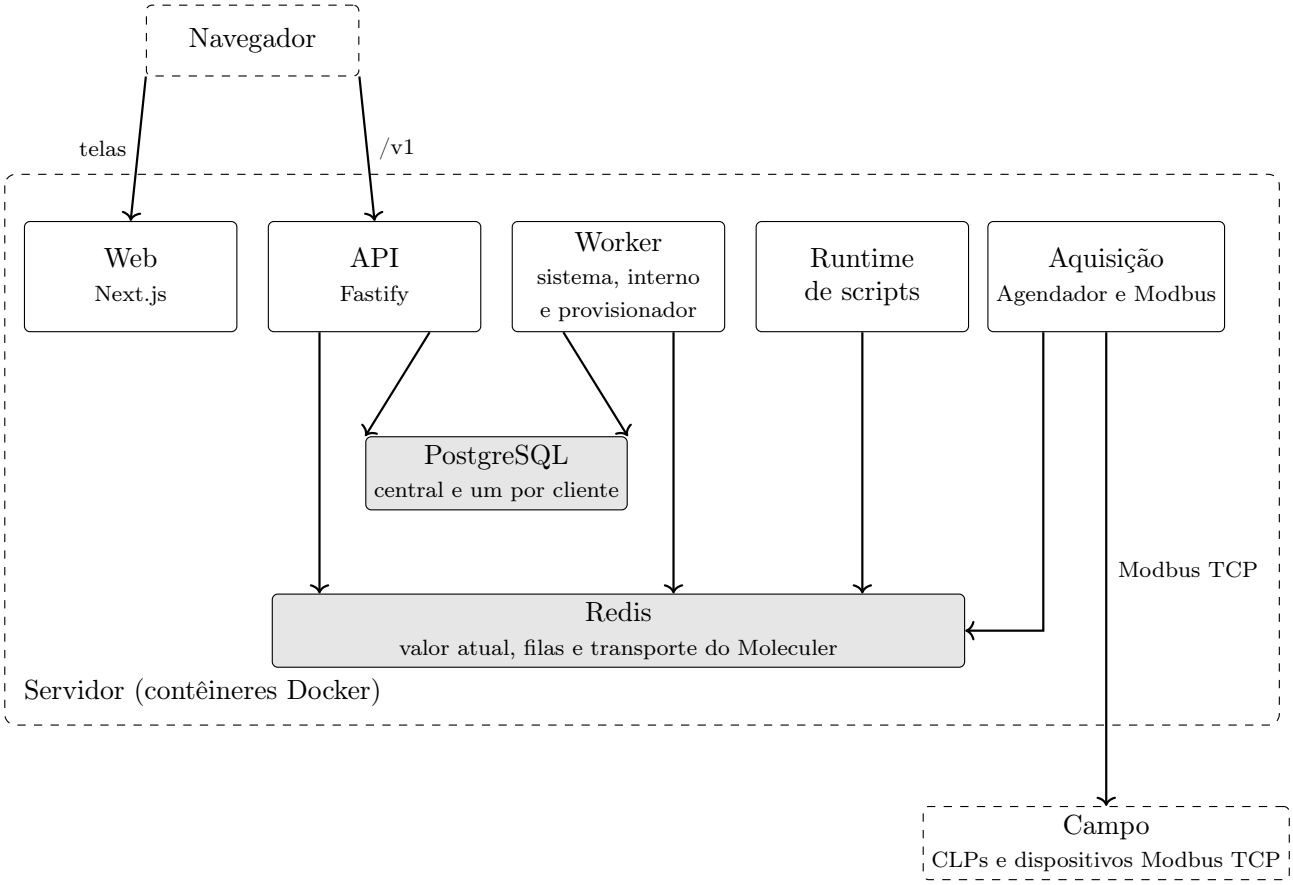
Id	Requisito	Categoria
RNF04	Os serviços possuem réplicas e filas duráveis, e o cache em memória é reconstruído a partir do banco quando volta vazio, com degradação suave em vez de dado incorreto	Disponibilidade
RNF05	O acesso é autenticado por credencial e autorizado por cargo e grupo, com senhas guardadas por Argon2id e isolamento entre clientes	Segurança
RNF06	O sistema limita as requisições por usuário e por endereço, protege comandos críticos com travas distribuídas e mantém registro de auditoria	Segurança
RNF07	O código escrito pelo usuário executa isolado, sem rede, sem arquivos e sem segredos, com limites de tempo, de memória e de saída	Segurança
RNF08	Os segredos que o sistema precisa ler de volta, como a senha do banco de um cliente, são guardados cifrados	Segurança
RNF09	A comunicação com medidores e CLPs de fabricantes distintos se dá apenas por configuração Modbus	Interoperabilidade
RNF10	A comunicação entre a interface e o servidor usa padrões abertos (REST/JSON), com contratos validados e versionados	Interoperabilidade
RNF11	A solução separa-se em serviços com implantação independente e baixo acoplamento, cada um com camadas bem delimitadas	Manutenibilidade
RNF12	Todos os serviços executam em contêineres, de modo que o mesmo conjunto pode ser levado a outro servidor sem alterar o código	Portabilidade
RNF13	O acesso ocorre por navegador, de qualquer lugar, sem instalação local, com a interface em português	Usabilidade

Fonte: Autor.

3.2 Arquitetura

O sistema organiza-se em serviços executados em contêineres *Docker* num servidor, conforme ilustrado na Figura 10. A divisão separa a interface de supervisão, os serviços que tratam as regras de negócio, o serviço que conversa com o campo, o serviço que executa o código do usuário e os dois armazenamentos de dados. O navegador do operador alcança o servidor por duas vias, a das telas, atendida pela aplicação web, e a da interface de programação, atendida pela API. Os dispositivos de campo ficam fora do servidor e só são alcançados pelo serviço de aquisição.

Figura 10: Arquitetura do Sistema



Fonte: Autor.

A solução reúne cinco serviços de aplicação, apoiados por dois serviços de dados. A aplicação web, em Next.js, entrega apenas as telas e não acessa banco de dados (Vercel, Inc., 2026). A API, em Fastify, concentra as regras de negócio, o cadastro, a autenticação e o acesso aos bancos. O *worker* executa, a partir de filas, as tarefas em segundo plano que precisam de banco, como a gravação do histórico, a avaliação de eventos, o cálculo de valores derivados e a criação do banco de cada cliente. O serviço de aquisição reúne o agendador, que decide o instante de cada leitura, e o cliente Modbus, que fala com o dispositivo de campo. O *runtime* de scripts executa o código JavaScript escrito pelos usuários, de forma isolada. O PostgreSQL guarda o que precisa durar, e o Redis guarda o que precisa ser rápido.

Os serviços não se chamam diretamente por endereço de rede. A comunicação entre eles usa o Moleculer, um *framework* de microsserviços que descobre os nós disponíveis e distribui as chamadas entre eles (MoleculerJS, 2026), tendo o Redis como meio de transporte. O trabalho assíncrono e repetível passa por filas do BullMQ, também guardadas no Redis. Essa combinação mantém cada serviço pequeno, com uma responsabilidade clara, e permite criar réplicas de um serviço sem que os demais precisem saber (Bigheti, 2020). A arquitetura segue, portanto, o paradigma de microsserviços e a orientação a serviços que a literatura aponta para os supervisórios modernos (Yadav; Paul, 2020), com a vantagem de empacotar cada serviço em um contêiner, o que facilita a implantação e a replicação (Netto, Hylson Vescovi *et al.*, 2017).

Duas decisões de projeto merecem destaque. A primeira é que cada cliente do sistema possui um banco de dados próprio, de modo que o isolamento entre empresas é físico e não apenas

lógico. A segunda é que o serviço de aquisição e o *runtime* de scripts não têm acesso a banco de dados nem a credenciais de armazenamento. O primeiro recebe a configuração pronta de cada ponto e devolve o valor lido. O segundo recebe o código e os valores de entrada e devolve o resultado. Se um desses serviços fosse comprometido, o invasor não alcançaria diretamente os dados dos clientes, o que reduz a superfície de ataque exatamente nos dois pontos que lidam com o mundo externo, o campo e o código do usuário (Stouffer *et al.*, 2015).

O código-fonte é organizado em um único repositório, com os serviços e os pacotes compartilhados lado a lado. Os contratos de dados, escritos como esquemas declarativos, e as funções puras comuns ficam em um pacote que todos os serviços importam, de modo que a interface, a API e os demais serviços falam sobre os mesmos tipos. A Tabela 3 resume as tecnologias adotadas em cada serviço. Todas são de código aberto e têm documentação pública, o que reduz o custo de licenciamento, tema recorrente na literatura sobre supervisórios acessíveis (Phuyal *et al.*, 2020).

Tabela 3: Tecnologias adotadas em cada serviço.

Serviço	**Tecnologias principais**
Web	Next.js, React, TypeScript, gráficos em ECharts, consultas ao servidor com cache no cliente
API	Node.js, TypeScript, Fastify, esquemas Zod, mapeamento objeto-relacional tipado, BullMQ, Moleculer
Worker	O mesmo código da API, em modo de processamento de filas
Aquisição	Node.js, TypeScript, cliente Modbus TCP, BullMQ, Moleculer
Runtime de scripts	Interpretador JavaScript compilado para WebAssembly, executado em um ambiente isolado, Moleculer
Dados	PostgreSQL e Redis

Fonte: Autor.

O isolamento entre os serviços é reforçado por regras automáticas de verificação do código. Cada serviço só pode importar o que a sua camada permite, e dependências sensíveis ficam confinadas a um único lugar, como o interpretador de scripts, que só pode ser usado dentro do próprio *runtime*, e a biblioteca de cifra, que só pode ser usada no adaptador responsável pelos segredos. Com isso, a separação desenhada na arquitetura não depende apenas da disciplina de quem escreve o código, pois a ferramenta de verificação reprova a violação.

3.3 Validações dos Requisitos

As subseções a seguir mostram, pilar a pilar, como a arquitetura da Figura 10 atende aos requisitos apresentados, derivados dos pilares clássicos de um sistema SCADA (Zanghi, 2019): aquisição e registro de dados, sincronismo de tempo, níveis de acesso, comando e controle, escalabilidade e redundância.

3.3.1 Aquisição e Registro de Dados

Na aquisição e no registro de dados (RF01 a RF03), as leituras são feitas pelo serviço de aquisição, que conversa diretamente com os dispositivos de campo por Modbus TCP e entrega os valores em formato digital (Modbus Organization, 2012a). Cada ponto é cadastrado como

tag, com um tipo de dado que pode ser booleano, inteiro de 16 ou 32 bits, ponto flutuante, decimal codificado em binário ou texto, e com um período de varredura próprio, de cinco segundos por padrão. A tag carrega a escala entre o valor bruto do registrador e a grandeza de engenharia, o que permite apresentar pressão, vazão, nível ou corrente em suas unidades usuais. O dispositivo pode ser acessado por Modbus TCP puro ou por um *gateway* que encapsula o quadro RTU, o que cobre redes seriais legadas sem exigir porta serial no servidor.

O resultado de cada leitura segue por dois caminhos. O valor atual de cada tag é guardado no Redis, junto com um indicador de qualidade, que distingue o valor bom, o desatualizado, a falha, o desconhecido e o implausível, e as telas o consultam sem sobrecarregar o banco. O histórico é gravado no PostgreSQL, em uma tabela de leituras particionada por mês, complementada por médias por minuto e por hora que aceleram os gráficos de períodos longos (PostgreSQL Global Development Group, 2026). A gravação do histórico passa por filas duráveis, de modo que um pico de leituras não derruba a gravação, e uma banda morta configurável por tag evita registrar variações sem significado para o processo.

3.3.2 Sincronismo de Tempo

O sincronismo de tempo (RF04) vem do carimbo de tempo que o serviço de aquisição aplica ao ciclo de leitura. Todas as tags lidas no mesmo ciclo de um dispositivo recebem a mesma estampa, o que reflete o instante da amostragem e não o instante exato da transição do processo, conforme a natureza da aquisição por *polling*. Esse carimbo acompanha o valor por toda a cadeia, do serviço de aquisição ao banco, ao cache e à tela do operador, sem ser alterado. Assim, leituras, alarmes e eventos de pontos diferentes recebem marcações coerentes, o que garante a cronologia necessária para a análise posterior de ocorrências (Zanghi, 2019). Como todos os serviços rodam em contêineres do mesmo servidor e compartilham o relógio da máquina hospedeira, a referência de tempo é comum a todos os componentes. Em implantações com mais de um servidor, o requisito passa a depender da sincronização de relógio entre as máquinas, premissa registrada para a evolução do sistema. Os instantes são guardados em UTC e convertidos para o fuso do cliente apenas na exibição.

3.3.3 Níveis de Acesso

Nos níveis de acesso (RF05 e RF06), o controle combina três elementos. O primeiro é o cliente, que é a fronteira rígida do sistema, pois cada cliente tem o seu banco e uma pessoa pertence a um único cliente. O segundo são os quatro cargos fixos, o administrador do sistema, que é a equipe do fornecedor, o administrador do cliente, que configura o sistema, o supervisor, que cuida dos usuários e dos operadores, e o operador, que supervisiona e comanda. O terceiro são os grupos de permissões, criados pelo próprio cliente a partir de um catálogo fixo de dez permissões, que somam seus efeitos quando a pessoa pertence a mais de um grupo. Assim, o princípio do menor privilégio é aplicado de forma granular, e quem não tem a permissão de comandar vê os valores, mas não consegue escrever no equipamento (Stouffer *et al.*, 2015).

A autenticação usa um token de acesso de curta duração, mantido em memória no navegador, e um token de renovação guardado em *cookie* protegido contra leitura por scripts. As senhas são guardadas apenas como resumo criptográfico Argon2id, e o cadastro de uma pessoa gera uma senha de uso único que só serve para entrar e trocar a senha. A API verifica a cada requisição o cargo e as permissões da pessoa, lidos de um cache que é invalidado a cada mudança, de modo que a alteração de um grupo vale já na próxima requisição. Como a solução atende vários clientes sobre a mesma infraestrutura, o isolamento por banco de dados, combinado com a resolução do cliente a partir da sessão e nunca de um parâmetro enviado pelo navegador,

impede que usuários de uma empresa acessem ou comandem pontos de outra (Wali; Alshehry, 2024).

3.3.4 Comando e Controle

O comando e o controle (RF07 a RF09) seguem as variantes previstas na literatura (Zanghi, 2019). O comando instantâneo, em que o operador muda na hora o estado de um equipamento, e o controle por valor de referência, em que define um alvo a ser mantido, tornam-se uma escrita Modbus executada pelo serviço de aquisição. As quatro fases, seleção, verificação, confirmação e execução, dividem-se entre a tela, que conduz o operador, a API, que verifica a permissão e as condições do comando e guarda o registro, e o serviço de aquisição, que realiza a escrita. Cada comando tem uma chave de idempotência, que impede a execução dupla em caso de reenvio, e uma trava por tag, que impede dois comandos simultâneos sobre o mesmo ponto (Stouffer *et al.*, 2015).

A conclusão do comando decorre da resposta do próprio dispositivo à escrita. O sistema não relê o ponto para confirmar, pois a resposta Modbus à escrita já espelha o que foi aceito. Quando o dispositivo não responde no prazo, o comando falha, e quando o sistema perde o rastro da execução, o estado é registrado como desconhecido, nunca como sucesso presumido. Todo comando e todo reconhecimento de alarme ficam registrados com o autor e a origem. O comando agendado de uma pessoa não existe no sistema. A escrita em hora marcada é obtida de outra forma, por um evento agendado que dispara um script capaz de escrever em uma tag, com a origem da escrita registrada como o nome do script.

3.3.5 Escalabilidade

A escalabilidade (RF10, RNF02 e RNF03) é atendida em três frentes. Na expansão do cadastro, novos pontos, dispositivos e telas entram com o sistema em produção, sem mexer no código, pois a aquisição é parametrizada pelo cadastro e uma recarga incremental leva cada alteração ao agendador sem reiniciar o serviço. Na capacidade de processamento, os serviços que carregam a maior parte do trabalho, a aquisição e o *worker*, são replicáveis na horizontal. Cada réplica da aquisição assume um subconjunto dos dispositivos, com afinidade por dispositivo, de modo que todas as leituras e escritas de um mesmo equipamento passam pela mesma instância e as ordens de comando e de leitura não se atropelam. A replicação de serviços em contêineres é um padrão consolidado para obter capacidade sob demanda (Netto, Hylson Vescovi *et al.*, 2017).

Na justiça entre clientes, as filas de trabalho aplicam um limite de concorrência por cliente, de forma que um cliente com muitas tags não monopoliza os *workers* nem o *runtime* de scripts. O *runtime* mantém uma fila justa por cliente e tetos globais de execução simultânea. Na infraestrutura de dados, o PostgreSQL cresce em volume por meio do particionamento do histórico e da retenção configurável, e o Redis é dimensionado com um limite de memória explícito e política que recusa novas escritas em vez de descartar dados em silêncio. O acréscimo de clientes, por sua vez, é feito pela criação automática de um novo banco, sem alterar o código nem os demais clientes. Esse arranjo evita o limite de pontos comum nos supervisórios comerciais, embora o teto de cada instalação dependa, na prática, da capacidade da máquina que hospeda os contêineres.

3.3.6 Redundância

A redundância (RNF04) apoia-se nas réplicas dos serviços e na durabilidade das filas (Zanghi, 2019). O serviço de aquisição e o *worker* funcionam como uma frota, em que cada instância retira trabalho das mesmas filas, de modo que a perda de uma é absorvida pelas demais. A aplicação web e a API também rodam em duas réplicas. As filas do BullMQ são persistidas no Redis, e os trabalhos que falham são repetidos com espera controlada, os que esgotam as tentativas ficam guardados para inspeção e a gravação do histórico tolera pausas curtas do banco sem perder leituras. O comando é protegido por travas com prazo de validade, de modo que a queda de uma instância no meio de uma operação libera a trava por expiração e nunca a deixa presa.

O ponto mais sensível é o Redis, que concentra o valor atual, as filas e o transporte entre serviços. Ele é configurado com um limite de memória e com política que não descarta chaves, e o sistema foi desenhado para que o seu conteúdo seja recuperável. Quando o Redis volta vazio, a ausência de uma chave sentinela, o *boot-marker*, aciona uma rotina de reconstrução que repovoa o valor atual de cada tag a partir da última leitura guardada no banco, reagenda as tarefas repetíveis e refaz o estado dos alarmes, e durante o processo as rotas que dependem do Redis respondem indisponibilidade em vez de dados incorretos. Os valores reabastecidos são marcados como desatualizados até a próxima leitura real.

Ficam como evolução os recursos que dependem de mais de um servidor. A replicação do PostgreSQL com comutação automática e a execução dos contêineres sob um orquestrador, que reinicia e redistribui as instâncias entre máquinas, ampliam a disponibilidade para além de um único hospedeiro. A literatura sobre contêineres gerenciados por orquestradores mostra que é possível manter o estado de serviços replicados coerente com protocolos de replicação de máquina de estados e de consenso, e avaliar o comportamento sob falhas (Netto, Hylson V. *et al.*, 2016; Netto, H. *et al.*, 2019). Como os serviços já são contêineres sem estado local relevante, essa evolução não exige mudança no código dos serviços, apenas na forma de implantá-los.

3.4 Serviços

O sistema divide-se em cinco serviços de aplicação e dois de dados, cada um em seu contêiner, com responsabilidades bem delimitadas e ciclos de vida próprios. Essa separação segue o paradigma de microsserviços para sistemas industriais orientados a serviços (Bigheti, 2020), em que cada serviço é implantado de forma independente, o que permite escalar de modo granular e isolar falhas. Dentro de cada serviço de aplicação, o código segue uma organização em quatro camadas, domínio, aplicação, infraestrutura e apresentação, em que a dependência aponta sempre para dentro e uma camada nunca acessa diretamente o que pertence a outra. A Tabela 4 resume o papel de cada contêiner e o que ele pode acessar.

Tabela 4: Contêineres, responsabilidades e acessos.

Contêiner	**Responsabilidade**	**Acessa banco?**	**Acessa Redis?**
Web	Entregar as telas ao navegador, sem regra de negócio	Não	Não
API	Atender toda rota `/v1`, autenticar, autorizar, cadastrar, consultar e enfileirar trabalho	Sim	Sim

Contêiner	Responsabilidade	Acessa banco?	Acessa Redis?
Worker	Executar as tarefas das filas: histórico, eventos, cálculos e criação do banco de cada cliente	Sim	Sim
Aquisição	Agendar e executar a leitura e a escrita Modbus nos dispositivos de campo	Não	Sim
Runtime de scripts	Executar o JavaScript do usuário de forma isolada	Não	Sim, restrito
PostgreSQL	Guardar o banco central e um banco por cliente	Próprio	Não
Redis	Guardar valor atual, filas, limites, travas e o transporte do Moleculer	Não	Próprio

Fonte: Autor.

3.4.1 Web

A aplicação web é desenvolvida em Next.js com React e entrega apenas as telas (Vercel, Inc., 2026). Ela não tem banco de dados, não executa regras de negócio e não enfileira trabalho. As telas oferecem a lista de acompanhamento, o dashboard, o editor e a execução do sinótico, o mapa das plantas, a consulta de histórico, os alarmes, a linha do tempo e os formulários de cadastro e de administração. O navegador, depois de carregar a aplicação, chama a API diretamente, com o token do usuário, por caminhos que começam em `/v1`, sem um intermediário na aplicação web, pois não há credencial a esconder.

As telas de tempo real funcionam por *polling* curto, em que o navegador pede periodicamente à API os valores atuais das tags que está exibindo. O período é único por cliente, de um a dez segundos, definido pelo fornecedor, e uma resposta de falha de infraestrutura é tratada com aviso de conexão e nova tentativa. A interface é montada a partir de um conjunto próprio de componentes, com tema claro, textos em português e verificação de acessibilidade, de modo que telas do mesmo tipo se comportam do mesmo modo.

3.4.2 API

A API é o centro do sistema e é desenvolvida em Node.js com o *framework* Fastify. Toda rota do sistema nasce sob o prefixo de versão `/v1`, e uma mudança incompatível exigiria uma nova versão, o que protege os clientes que a consomem. Cada rota valida a entrada contra um esquema declarativo compartilhado com a interface e devolve erros em um formato padronizado. A API realiza o *login*, a renovação de sessão e a verificação de permissões, e executa o cadastro de clientes, plantas, dispositivos, tags e pastas, as consultas ao histórico, a gestão de usuários, cargos e grupos, a edição de telas, scripts e eventos e o disparo dos comandos do operador.

A API é o único serviço, além do *worker*, que fala com o banco de dados, por meio de um mapeamento objeto-relacional tipado, e escolhe o banco do cliente a partir da sessão da pessoa. Ela também fala com o Redis, para ler o valor atual das tags, controlar limites de taxa e travas, e com o Moleculer, para chamar o serviço de aquisição e o *runtime* de scripts. O processo HTTP atende apenas requisições, e nenhuma tarefa de fundo roda dentro dele, o que evita que um trabalho pesado afete o tempo de resposta das telas. Quando uma operação precisa ser executada depois ou repetida, a API a coloca em uma fila e responde ao usuário, deixando a execução para o *worker*.

3.4.3 Worker

O *worker* usa o mesmo código da API, mas em um modo sem servidor HTTP, e consome as filas do BullMQ. Ele roda em um único contêiner no qual um supervisor mantém um subprocesso por família de fila, e cada subprocesso recebe apenas o ambiente e a credencial da sua família. A família *sistema* grava as leituras no histórico, atualiza o valor atual e as médias, avalia os eventos que geram alarmes, dispara os eventos agendados e executa as reconciliações e as varreduras de manutenção. A família *interno* calcula as tags internas e as variáveis meta, valores derivados de outras tags, e para isso solicita a avaliação de scripts ao *runtime*. A família *provisionador* cria e remove o banco de cada cliente, e é a única com a credencial de criação de estruturas no PostgreSQL.

Essa separação por família permite que o administrador veja e escale cada tipo de trabalho de forma independente, e que a falha ou a lentidão de uma família não atrase as demais. O *worker* não atende requisições externas. Ele também não fala com o campo, pois a leitura e a escrita nos dispositivos pertencem à aquisição. O isolamento entre as famílias é por processo e não por contêiner, uma escolha que simplifica a operação à custa de compartilhar o mesmo disco e as mesmas permissões do contêiner.

3.4.4 Aquisição

O serviço de aquisição é a fronteira do sistema com o campo e o único componente que conversa em Modbus com os dispositivos. Ele é um serviço Node sem servidor HTTP e sem banco de dados, e reúne dois papéis. O agendador decide o instante de cada leitura, a partir do período de varredura de cada tag, agrupa as tags de um mesmo dispositivo em uma única rodada para reduzir o tráfego e evita sobrepor ciclos quando a leitura anterior ainda não terminou. O cliente Modbus monta o quadro, envia-o ao dispositivo, valida o tamanho da resposta e os códigos de erro do protocolo, decodifica os registradores, combinando palavras e aplicando a ordem de bytes e a escala cadastradas, e produz o valor na unidade de engenharia (Modbus Organization, 2012a).

A aquisição recebe a configuração de cada dispositivo já pronta, de modo que não precisa conhecer o cadastro nem o banco, e se mantém por réplicas com afinidade por dispositivo, o que garante que as leituras e as escritas de um mesmo equipamento passem sempre pela mesma instância. A trava por dispositivo, guardada no Redis, estabelece a prioridade entre os usos da conexão, com o comando acima da leitura periódica. A conexão com cada dispositivo é mantida aberta, e as falhas de comunicação são contadas por dispositivo, com tempo limite, número de tentativas e de ciclos configuráveis. Quando uma leitura falha, o último valor válido é mantido e marcado como desatualizado, e a ocorrência alimenta os eventos de sistema do dispositivo, pois, em um SCADA, a ausência de um dado precisa ser tão visível quanto um limite de processo ultrapassado.

Os resultados saem da aquisição como mensagens, que o *worker* transforma em histórico, valor atual e eventos. O serviço também executa as escritas de comando, que chegam pelo Moleculer e viram uma escrita Modbus no dispositivo. O que a aquisição não faz é igualmente relevante. Ela não guarda dados, não decide permissões, não conhece usuários e não executa scripts. Essa pequenez facilita a replicação, o teste e a substituição do protocolo, pois outros protocolos industriais entrariam como novas famílias de trabalho sem alterar o restante.

3.4.5 Runtime de Scripts

O *runtime* de scripts executa o código JavaScript escrito pelos usuários, que aparece em quatro lugares, a variável meta, o script de um evento, o componente de conteúdo do sinótico e a tag interna gerada. Deixar o usuário escrever código em um sistema que supervisiona processos industriais exige isolamento forte. Por isso, cada avaliação roda em uma instância nova de um interpretador JavaScript compilado para *WebAssembly* e executado dentro de um *sandbox* de execução, num processo hospedeiro que é reciclado. O código não tem acesso à rede, ao sistema de arquivos nem a segredos, e cada execução tem tetos de tempo de processamento, de memória, de recursão e de tamanho de saída.

O serviço roda em contêiner próprio, sem credencial de banco e sem a chave de cifra do sistema. Quem lê o banco é a API ou o *worker*, que enviam os valores no pedido de avaliação. O serviço fala com o resto do sistema apenas pela ação de avaliação do Moleculer, em um espaço de nomes próprio, e a conta que ele usa no Redis tem permissões reduzidas ao mínimo. As avaliações entram em uma fila justa por cliente, com tetos globais e por cliente, de modo que um script lento de um cliente não impede o dos demais. Assim, mesmo que um script malicioso consiga escapar do interpretador, não alcança o banco, a rede interna nem os comandos de escrita do campo, o que atende ao requisito de segurança do código do usuário.

3.4.6 PostgreSQL

O PostgreSQL é um banco relacional de código aberto, com bom suporte a tipos complexos, particionamento e consultas em janelas de tempo (PostgreSQL Global Development Group, 2026), e guarda por longo prazo todo o estado relevante do sistema. Há um banco central, que guarda os clientes, os usuários, as sessões e o registro de qual banco pertence a cada cliente, e um banco para cada cliente, que guarda o cadastro, o histórico, os alarmes e eventos, as telas, os scripts e os grupos de permissões. Os dois níveis de banco são descritos adiante, no modelo de dados.

O acesso é feito por um mapeamento objeto-relacional tipado, em conjuntos de conexões separados por finalidade, de modo que a ingestão de leituras, as telas e as consultas pesadas de histórico não disputam as mesmas conexões. Há um teto global de conexões por processo, e o esquema de cada banco evolui por migrações compatíveis com a versão anterior do código, executadas sob trava e ensaiadas antes da aplicação real. O histórico de leituras é particionado por mês, e a retenção é configurável por cliente. Os índices acompanham as consultas, e cada consulta nova nasce com o índice que a atende, para que o custo não cresça com o tamanho da tabela inteira.

3.4.7 Redis

O Redis é um armazenamento de estruturas de dados em memória (Redis Ltd., 2026) que acumula vários papéis. Como cache, mantém o valor atual de cada tag, lido pelas telas sem onerar o banco. Como memória das filas, guarda os trabalhos do BullMQ. Como meio de transporte, sustenta a comunicação do Moleculer entre os serviços. Como apoio à segurança e à coordenação, guarda os contadores dos limites de taxa, as travas distribuídas dos comandos e dos dispositivos e a lista das sessões revogadas.

Há duas categorias de chaves. As de apoio têm prazo de validade e se renovam sozinhas, como os contadores de limite e as travas. As que sustentam o estado em tempo real, como o último valor de cada tag, não expiram, e por isso o sistema trata o Redis como recuperável, reconstruindo-o do banco quando volta vazio. Toda chave cujo identificador vem do banco

de um cliente leva o identificador do cliente, para que a limpeza na exclusão de um cliente apague apenas o que é dele. O Redis é acessível apenas pela rede interna dos contêineres, e as permissões de cada serviço são restritas por conta de acesso, de modo que o *runtime* de scripts, por exemplo, só alcança a ação de avaliação.

3.4.8 Comunicação Moleculer e Filas

A comunicação entre os serviços de aplicação é coordenada pelo Moleculer, que se apoia no Redis como meio de transporte e funciona como orquestrador lógico da solução (MoleculerJS, 2026). É por ele que a API pede ao serviço de aquisição uma escrita, que a aquisição entrega os resultados das leituras e que a API e o *worker* pedem a avaliação de scripts ao *runtime*. O Moleculer oferece descoberta automática dos serviços, distribui as chamadas entre as instâncias replicadas, aplica tempo limite e isola falhas para que não se propaguem. A afinidade por dispositivo da aquisição é obtida por uma estratégia de distribuição que direciona as chamadas de um mesmo dispositivo à mesma instância.

O trabalho que não precisa de resposta imediata passa por filas do BullMQ. Cada família de trabalho tem a sua fila geral, e uma fila a mais só existe quando há motivo, como uma credencial diferente, um limite de concorrência próprio ou um caminho quente de dado. As filas aplicam limites por cliente, descartam o histórico excedente quando a memória do Redis passa de um limiar, mantêm os trabalhos falhados por um período para inspeção e exigem que cada tarefa seja idempotente, de modo que repetir uma tarefa não produz efeito duplicado. Os dois mecanismos se complementam. O Moleculer cobre a chamada com resposta, e as filas cobrem o trabalho durável e repetível.

3.4.9 Segurança entre os Serviços

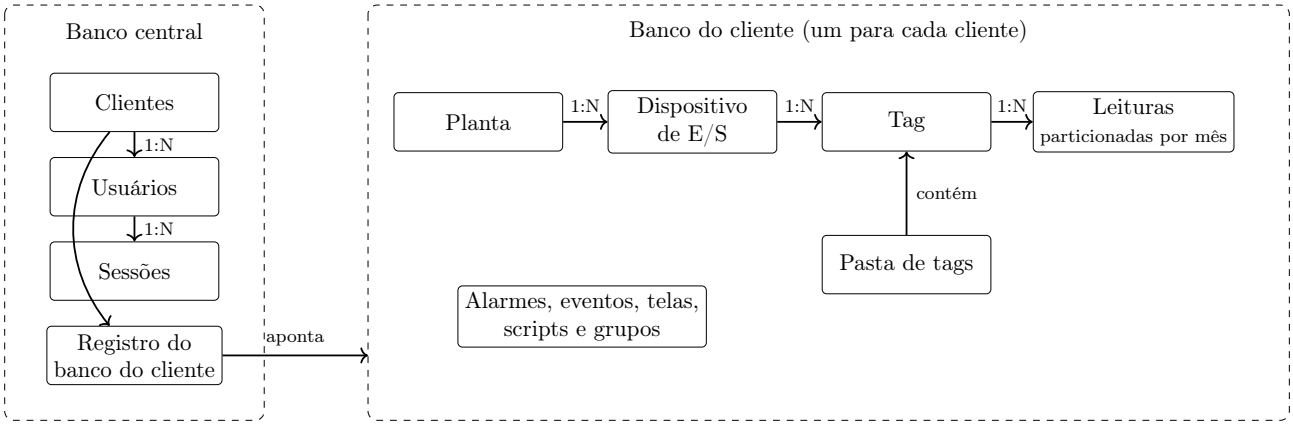
A segurança atravessa todos os serviços e se apoia em camadas. Na borda, a API aplica limites de taxa por usuário e por endereço de origem, com limites mais rígidos para o *login* e para as consultas pesadas de histórico, e bloqueia temporariamente o acesso após tentativas de entrada malsucedidas. Os segredos que o sistema precisa ler de volta, como a senha do banco de um cliente, são guardados com cifra simétrica autenticada, usando uma chave de dados por cliente protegida por uma chave mestra, enquanto as senhas de usuários e as chaves de acesso emitidas pelo sistema são guardadas apenas como resumo irreversível (Stouffer *et al.*, 2015).

Entre os serviços, cada contêiner recebe somente o ambiente de que precisa. A aquisição não tem as credenciais do banco, o *runtime* de scripts não tem a chave de cifra, e dentro do *worker* cada família de fila recebe apenas a sua credencial, de modo que só a família que cria bancos de clientes dispõe da permissão de criação de estruturas. A comunicação pelo Moleculer confere a identidade de quem chama, e as contas do Redis de cada serviço restringem quais ações ele pode executar. Esse desenho segue o princípio de que o comprometimento de uma peça não pode se transformar em acesso irrestrito ao conjunto, requisito central em sistemas de controle industrial conectados a redes corporativas e à internet (Wali; Alshehry, 2024).

3.5 Modelo de Dados

O modelo de dados separa o que é comum a todos os clientes do que pertence a cada um. A Figura 11 apresenta a visão geral. O banco central guarda os dados que o sistema precisa antes de saber quem é o cliente, que são os clientes, os usuários, as sessões e o registro que indica em qual banco estão os dados de cada cliente. O banco de cada cliente guarda tudo o que é operacional daquele cliente.

Figura 11: Modelo de Dados



Fonte: Autor.

O cadastro obedece a uma hierarquia de quatro níveis. O cliente é a empresa contratante. A planta é a instalação física, em um único nível, com nome, identificador textual e coordenadas geográficas, que a posicionam no mapa. O dispositivo de E/S é a conexão com a origem do dado, que pode ser um equipamento Modbus TCP ou um dispositivo interno, que gera valores calculados ou simulados, e pertence a uma única planta, em uma lista simples. A tag é a variável monitorada ou comandada, a menor unidade do cadastro, e carrega o endereço do registrador, o tipo de dado, a ordem de bytes, a escala, o período de varredura e a banda morta. Uma árvore de pastas, separada dos dispositivos e própria de cada planta, organiza as tags para a visualização. Uma pasta pode reunir tags de dispositivos diferentes da mesma planta, e a tag fica na raiz ou em uma única pasta.

As leituras ligam-se à tag e formam a maior tabela do sistema. Elas são particionadas por mês, e as partições futuras são criadas por tarefas de manutenção. Para as consultas de períodos longos, o sistema mantém ainda as médias por minuto, usadas quando o período de varredura da tag é inferior a sessenta segundos, e as médias por hora. O banco do cliente guarda também as ocorrências de alarme, os eventos e seus estados, as telas, as listas de acompanhamento, os scripts, as variáveis meta, os grupos de permissões e o registro de comandos, com a cópia do nome de quem os emitiu.

A separação entre o banco central e o banco do cliente tem consequências práticas. O cliente é resolvido pela sessão, e a conexão ao banco dele é aberta a partir do registro guardado no banco central, com a senha cifrada. A criação de um novo cliente é uma tarefa retomável do *worker*, que cria o banco, aplica o esquema e os dados iniciais e registra o resultado, e que desfaz tudo se falhar no meio. A exclusão do cliente apaga o banco inteiro e as chaves dele no Redis. A exclusão de registros do cadastro é real, sem marcar como apagado, e o que se pode fazer é desabilitar um item, que é um estado de negócio. As referências entre os dois bancos, como o vínculo do usuário com os grupos, não têm chave estrangeira, e uma tarefa de limpeza remove os vínculos que ficaram órfãos.

3.6 Implantação

A implantação reúne os contêineres em um servidor com Docker, descrito por uma composição de serviços em que cada contêiner tem a sua imagem, as suas variáveis de ambiente e os seus limites de memória e de processamento. São seis contêineres de aplicação e de cache, a aplicação

web, a API, o *worker*, a aquisição, o *runtime* de scripts e o Redis, além do PostgreSQL, que pode estar na mesma máquina ou em uma instância dedicada. Os contêineres se comunicam por uma rede interna, e apenas a porta de entrada do servidor é exposta ao navegador. Um *proxy* reverso encaminha, para o mesmo endereço, as rotas `/v1` e a verificação de saúde à API e todo o resto à aplicação web, de modo que o navegador conhece uma única origem e o sistema dispensa configuração de compartilhamento de recursos entre origens.

Confirmar com o autor qual servidor hospeda o sistema no trabalho (computador do autor ou servidor do instituto), a configuração da máquina (processador, memória) e o *proxy* reverso usado na bancada, para completar esta subseção.

O acesso aos dispositivos de campo é feito a partir do contêiner de aquisição, por conexões Modbus TCP de saída. Os dispositivos não precisam enxergar o restante do servidor, e a rede do campo precisa permitir apenas o endereço da máquina que hospeda a aquisição. O Redis e o PostgreSQL não são expostos fora da rede interna. Cada serviço recebe apenas as variáveis de ambiente de que precisa, organizadas por assunto, e os segredos de desenvolvimento são gerados localmente e nunca versionados. As mesmas imagens servem a qualquer ambiente, pois a configuração entra em tempo de execução.

A arquitetura foi desenhada para sair de um único servidor sem mudança de código. Como todos os serviços já são contêineres, a levada para um provedor de computação em nuvem, com banco gerenciado, cache gerenciado e balanceamento de carga, ou para um orquestrador como o Kubernetes, que distribui e reinicia os contêineres entre várias máquinas, é uma mudança na forma de implantar. A literatura mostra que a replicação de serviços e a coordenação de contêineres sob um orquestrador permitem obter tolerância a falhas e escala (Netto, Hylson Vescovi *et al.*, 2017; Netto, Hylson V. *et al.*, 2016), e é essa evolução que se aponta como trabalho futuro, junto da replicação do banco de dados com consenso (Netto, H. *et al.*, 2019).

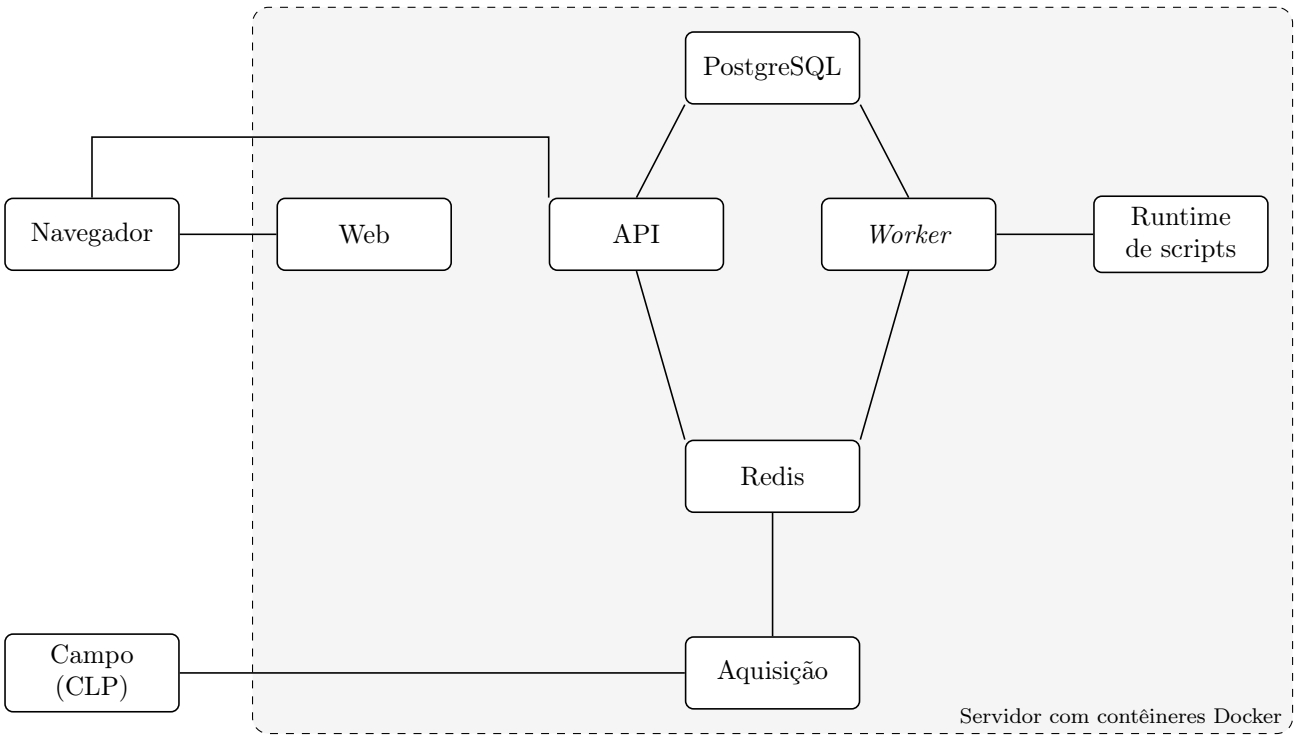
A implantação em servidor único traz uma limitação que deve ficar explícita. A máquina hospedeira é um ponto único de falha, e uma queda dela interrompe todo o sistema. A disponibilidade obtida pelas réplicas e pelas filas protege contra a falha de um processo ou de um contêiner, mas não contra a falha do servidor. Os requisitos de disponibilidade foram redigidos com essa premissa, e a evolução para a nuvem ou para um orquestrador é o caminho para eliminar essa limitação.

4 Fluxos do Sistema

Conforme a estrutura apresentada na introdução, este capítulo descreve os caminhos que o dado e as ações percorrem dentro do sistema, sem recorrer a trechos de programa. Enquanto o capítulo anterior apresentou os serviços e a responsabilidade de cada um, aqui o foco é o movimento: quem pede, quem responde, onde cada informação é guardada e o que acontece quando uma das peças falha. Para isso, cada fluxo é apresentado por um diagrama de blocos com setas numeradas, seguido de um texto que percorre os passos na ordem, no formato usado na descrição de protocolos executados em contêineres (Netto, Hylson Vescovi *et al.*, 2017; Netto, H. *et al.*, 2019).

Os blocos são sempre os mesmos e correspondem aos processos e aos serviços de dados da arquitetura, com as posições fixas da Figura 12. O navegador do usuário e o equipamento de campo ficam fora do servidor. Dentro dele, cada serviço de aplicação executa em um contêiner próprio: a aplicação Web, que entrega apenas as telas, a API, que concentra as regras de negócio, o *worker*, que consome as filas de trabalho e grava no banco, o serviço de aquisição, que conversa com o equipamento, e o *runtime* de scripts isolado, que executa o código escrito pelos clientes. O PostgreSQL guarda os dados duráveis e o Redis guarda o valor atual de cada tag, as filas, as travas e o meio de transporte da comunicação entre os serviços. Cada fluxo a seguir acende apenas um subconjunto desses blocos.

Figura 12: Mapa dos blocos usados nos fluxos



Fonte: Autor.

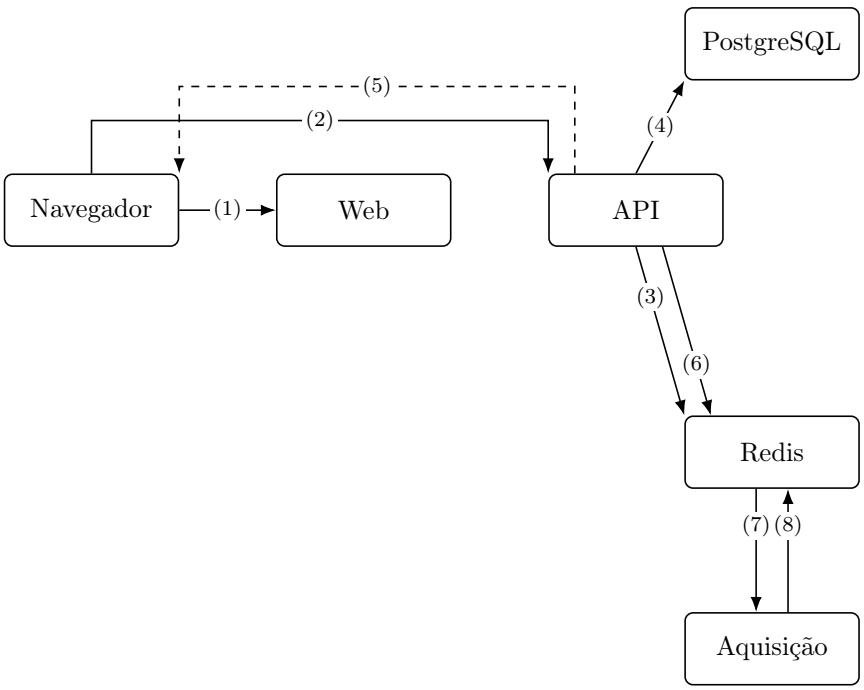
A legenda das setas é a mesma em todas as figuras. A linha cheia indica o caminho normal, a linha tracejada indica uma resposta que volta ao solicitante e a linha de traço e ponto indica um caminho alternativo, percorrido quando alguma peça falha. Os números entre parênteses seguem a ordem dos passos, e o mesmo número em dois trechos de linha indica um único passo

que atravessa o Redis, como ocorre nas chamadas entre a API e a aquisição, que o Moleculer transporta por esse banco (MoleculerJS, 2026; Redis Ltd., 2026). Os passos alternativos recebem o número do passo normal acompanhado de uma letra, como em (4b), e são descritos logo depois do caminho normal. Cada fluxo termina com um parágrafo sobre o que acontece quando uma das peças deixa de funcionar. O capítulo percorre, nesta ordem, o cadastro de uma tag, o caminho do dado do campo ao banco, a exibição em tempo real, a consulta ao histórico, o comando em quatro fases, o alarme, a variável meta, a criação de um cliente e o login.

4.1 Cadastro de uma Tag

O cadastro de uma tag é o ponto em que o sistema passa a conhecer uma nova variável do processo. A tag nasce dentro de um dispositivo de entrada e saída, que pertence a uma planta, que por sua vez pertence a um cliente, e carrega o mapeamento do ponto no equipamento, a escala de engenharia, o período de varredura e as regras de histórico. Como o cadastro é feito com o sistema em produção e sem alterar o código, a nova tag precisa chegar ao serviço de aquisição sem reiniciar nenhum processo. A Figura 13 mostra o caminho percorrido.

Figura 13: Fluxo do cadastro de uma tag



Fonte: Autor.

O administrador do cliente abre a tela de tags de um dispositivo, e a página é entregue pela aplicação Web (1), que contém apenas as telas e não acessa banco algum. Ao salvar o formulário, o próprio navegador envia o pedido de criação à API (2), levando o *token* de acesso da sessão. A API confere no Redis se a sessão continua válida e quais permissões o usuário possui (3) e só então lê o corpo do pedido. Em seguida, aplica as regras do cadastro: a combinação de tipo de registrador, tipo bruto e ordem de bytes precisa existir na matriz do protocolo Modbus, o tamanho da tag em registradores não pode passar do limite do dispositivo, a retenção do histórico não pode superar o teto contratual do cliente e a soma de linhas de histórico por dia não pode ultrapassar o limite do cliente. Um endereço que se sobrepõe ao de

outra tag apenas gera um aviso. Aprovado o pedido, a API grava (4), em uma única transação no banco do cliente, a tag, o mapeamento Modbus e uma linha no registro de mudanças do cadastro, que carrega a nova versão de configuração da planta. Essa técnica, conhecida como *outbox*, garante que nunca exista uma tag gravada sem o registro da mudança correspondente, nem o contrário.

Concluída a transação, a API responde ao navegador (5), com o registro criado e os avisos, e a tela passa a listar a nova tag. Ainda depois da gravação, a API publica o evento de mudança do cadastro (6), que informa a planta, a versão e os motivos da alteração, mas não o conteúdo da tag. O Moleculer entrega o evento, pelo Redis, ao Agendador, que roda dentro do serviço de aquisição (7). Como a aquisição não possui banco de dados, o Agendador pede à API a configuração pronta do dispositivo (8), e a API a monta lendo o cadastro. O Agendador guarda a configuração, com a versão, no Redis e na memória e passa a incluir a tag no ciclo do dispositivo. Se o dispositivo já possuía uma tarefa repetível com o mesmo período, a tag apenas entra no plano de leitura junto das demais tags que vencem no mesmo instante. Se o período é novo, a tarefa é criada, e somente a tarefa afetada é recriada. A primeira leitura da nova tag acontece no ciclo seguinte, e até lá a tela a mostra como tag sem dado.

O cadastro rápido por grade, em que o administrador cola várias linhas de uma planilha, percorre o mesmo caminho. Como uma rajada de alterações gera muitas linhas no registro de mudanças, o publicador as junta e emite poucos eventos, cada um com a lista dos motivos que o compõem, e o Agendador aplica a união dos efeitos com uma única consulta de configuração.

A tag também tem um lugar na organização do cadastro. Ela pertence para sempre ao dispositivo em que nasceu e, além disso, pode ser colocada em uma pasta da árvore de pastas da planta, que é independente da lista de dispositivos. Uma pasta pode reunir tags de dispositivos diferentes da mesma planta, o que permite montar agrupamentos que fazem sentido para a operação sem alterar a forma como a leitura é feita. O identificador textual da tag, composto a partir da planta, do dispositivo e do nome, é único dentro do cliente, e a exclusão exige que o administrador o digite para confirmar. A tag desaparece da lista na mesma hora, e as leituras dela saem do banco depois, em segundo plano, por uma tarefa de limpeza que consulta uma lista durável de tags a apagar, de modo que a exclusão não precise segurar uma transação sobre milhões de linhas.

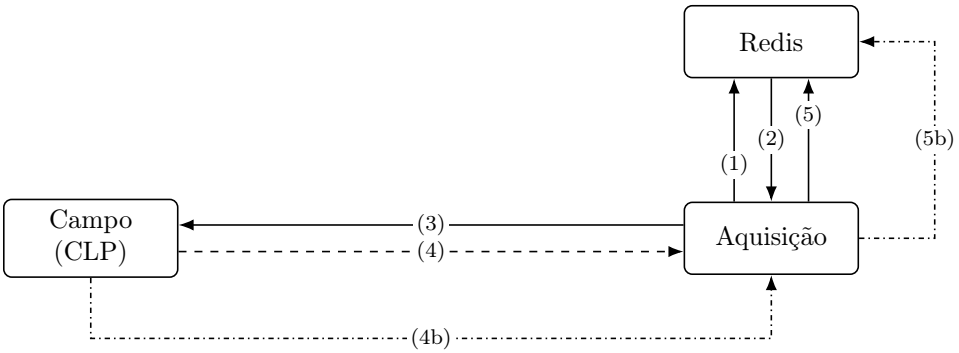
Quando a validação falha, o caminho se encerra no passo (4b): a API recusa o pedido com uma mensagem no idioma do produto, e nada é gravado. Se outro usuário alterou a mesma tag antes, o validador de versão enviado pelo navegador não confere e o pedido é recusado com conflito, e a tela oferece recarregar o formulário. Pode ocorrer ainda a queda do processo entre a gravação e a publicação do evento (6b). Nesse caso, a mudança continua registrada no *outbox*, e uma varredura periódica a republica. Além disso, a cada cinco minutos a réplica líder do Agendador compara a versão do cadastro de cada planta com a que possui e relê apenas as plantas que mudaram, de modo que, no pior caso, a nova tag entra no ciclo na conferência seguinte. Um evento que chega com versão anterior à já conhecida é ignorado, o que impede que uma resposta atrasada reintroduza uma configuração antiga.

Se uma peça falha, o efeito é limitado. Sem a API ou sem o Redis, o cadastro não é possível, e as rotas autenticadas respondem que o serviço está indisponível, mas a leitura do campo continua, porque a aquisição mantém a última configuração recebida e não depende da API para ler. Se o banco do cliente está fora, o cadastro daquele cliente falha e os demais clientes não são afetados. Se a aquisição está parada, o cadastro funciona normalmente, e ao voltar o Agendador reconcilia todas as plantas pelas versões, sem que nenhuma mudança se perca.

4.2 Do Campo ao Banco

Este é o fluxo principal de um sistema supervisório, o caminho percorrido por uma leitura desde o registrador do equipamento até o histórico no banco. Ele envolve três etapas encadeadas por filas e não por chamadas diretas: a aquisição lê o equipamento e publica o resultado, o *worker* atualiza o valor atual no Redis e grava o histórico, e rotinas periódicas calculam os resumos. Separar as etapas por filas duráveis permite que cada uma escale e falhe de forma independente (Taskforce.sh Inc., 2026). A Figura 14 mostra a primeira etapa, e a Figura 15 mostra a segunda.

Figura 14: Fluxo da leitura do equipamento pela aquisição



Fonte: Autor.

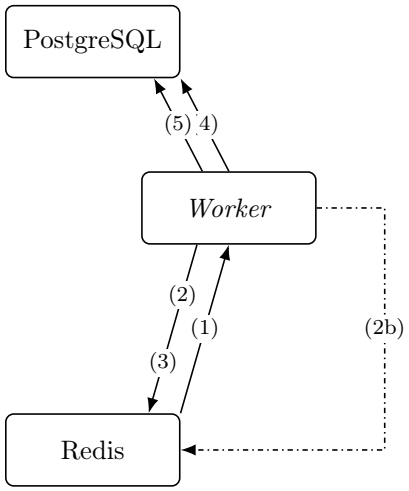
O ciclo começa no Agendador, que roda dentro do serviço de aquisição. Para cada dispositivo, uma tarefa repetível, com fase fixa para espalhar as leituras no tempo, vence no ritmo do período de varredura das tags e marca o dispositivo como devido no Redis (1). Quando a janela chega, o Agendador chama a ação de leitura da aquisição pelo Moleculer (2), entregando o plano de leitura, que reúne todas as tags vencidas do dispositivo. A afinidade por dispositivo faz com que a chamada chegue sempre à mesma réplica, e uma trava no Redis impede dois ciclos simultâneos no mesmo dispositivo. A aquisição agrupa os registradores contíguos em blocos e envia uma requisição Modbus TCP por bloco, sobre a conexão mantida com o CLP (3) (Modbus Organization, 2012a). O equipamento responde com os registradores brutos (4). Para cada bloco, a aquisição carimba o instante em que a resposta chegou, decodifica os bytes conforme o tipo e a ordem configurados, aplica a escala do valor bruto para a unidade de engenharia, atribui a qualidade e um número de sequência formado por época e contador, e publica o resultado em duas filas do Redis, uma de histórico e outra de ingestão (5). O carimbo de tempo acompanha o valor por todo o restante do caminho, sem ser alterado.

Se o equipamento não responde dentro do prazo, se a conexão não se estabelece ou se o equipamento devolve uma exceção Modbus (4b), a aquisição não descarta o ciclo. Ela publica o resultado com as tags marcadas com qualidade ruim e com o motivo da falha (5b), e o Agendador conta a falha no seu disjuntor. Depois de um número configurável de ciclos falhos seguidos, por padrão três, o estado do dispositivo passa a degradado. Esse estado é reemitido a cada sessenta segundos e é a origem do evento de falha de comunicação do dispositivo, tratado no fluxo dos alarmes. O primeiro ciclo bem-sucedido fecha o estado.

O Agendador merece alguns detalhes, porque é ele que decide quando cada equipamento é lido. Tags do mesmo dispositivo com períodos diferentes não geram requisições separadas: no instante em que várias vencem juntas, entram no mesmo plano e viram uma leitura só, de modo que doze tags com período de um segundo resultam em uma leitura por segundo. As

tarefas repetíveis têm fase fixa por dispositivo, o que espalha no tempo as leituras de muitos dispositivos, e uma tarefa só é recriada quando falta ou quando o período muda. O comando de um operador passa na frente da leitura, e quando o serviço volta de uma queda não há rajada de leituras acumuladas. O Agendador despacha o trabalho para filas separadas por faixa de tempo de varredura, para que uma leitura rápida não espere por uma lenta, e uma fila de despacho segura o dispositivo durante o ciclo. Como a aquisição escala por réplicas e a afinidade mantém cada dispositivo em uma réplica só, acrescentar capacidade significa acrescentar réplicas, sem alterar o cadastro. A cada cinco minutos, a réplica líder confere o cadastro de todos os clientes e corrige o que divergir, e um cliente suspenso ou em exclusão não é lido.

Figura 15: Fluxo da gravação do valor atual, do histórico e dos resumos



Fonte: Autor.

A segunda etapa é feita pelo *worker* da família de filas do sistema, que consome a fila de ingestão (1). O primeiro passo é gravar o valor atual da tag no Redis, junto com a qualidade e o carimbo (2). A gravação é atômica e usa o número de sequência, e nunca o relógio, para decidir se a leitura é mais nova que a guardada, de modo que uma leitura atrasada jamais sobrescreve um valor mais recente. O número de sequência é composto por uma época, semeada pelo relógio do Redis e que só cresce, e por um contador, e por isso a ordem entre as leituras não depende dos relógios das réplicas. Em seguida, o *worker* consome a fila de histórico e decide, ainda no Redis, se a leitura deve ser gravada (3), conforme o modo de histórico da tag: não gravar, gravar toda leitura, ou gravar quando o valor muda além da banda morta e, além disso, periodicamente. A decisão é atômica e usa o último valor gravado e o número de sequência, o que evita duplicar uma linha quando duas réplicas processam a mesma tag. Decidido o registro, o *worker* insere a linha (4) na tabela de leituras do banco do cliente, que é particionada por mês, com os limites calculados em UTC e partições criadas com antecedência de três meses (PostgreSQL Global Development Group, 2026).

Os resumos são calculados por tarefas periódicas, uma por cliente a cada minuto (5). Para cada tag analógica, a tarefa calcula em SQL, no próprio banco do cliente, a média ponderada no tempo, o mínimo, o máximo, a contagem e a cobertura de cada intervalo. O resumo de um minuto só existe para tags com período efetivo menor que sessenta segundos, e o resumo de uma hora existe para toda tag analógica com histórico. Como a média sozinha esconde picos curtos, o mínimo e o máximo acompanham cada balde. Uma leitura que chega atrasada dispara o recálculo apenas da faixa afetada. A cada trinta segundos, o último valor bom de

cada tag também é copiado para uma tabela do banco, que serve de espelho para reabastecer o Redis caso ele volte vazio. Por fim, uma rotina diária, na madrugada, apaga o histórico que ultrapassou a retenção de cada tag e o teto do cliente.

O histórico de longo prazo exige manutenção contínua, que também faz parte do fluxo. Uma tarefa diária cria as partições dos meses seguintes e confere as que existem, e o apagamento do histórico vencido só ocorre em uma janela noturna, entre uma e quatro horas da manhã no horário de São Paulo, com no máximo dois bancos de cliente de cada vez e em lotes com confirmação de transação a cada lote, para que o apagamento não concorra com a operação diurna. O corte de cada tag é o maior entre a retenção configurada nela e o teto do cliente, medido no calendário de São Paulo, e a retenção pode chegar a dez anos. Quando a Movi reduz o teto de um cliente, nenhuma consulta devolve dado mais antigo que o novo teto desde aquele instante, e o apagamento da noite seguinte remove fisicamente o excedente.

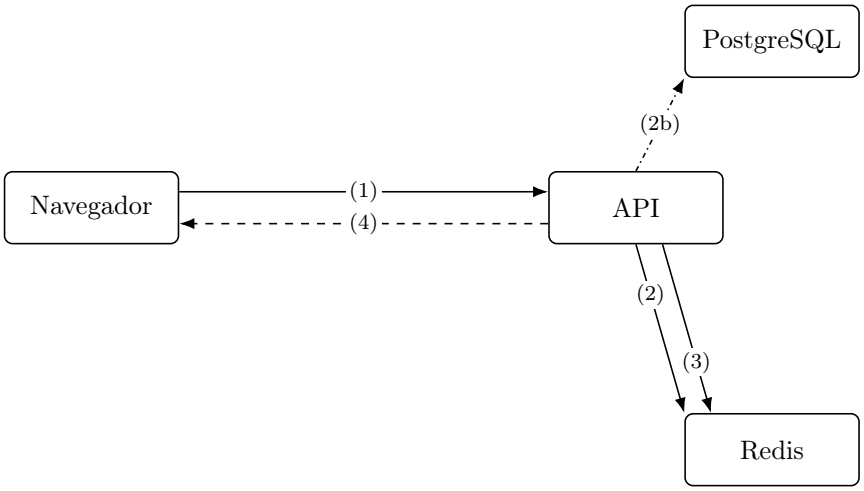
No caminho alternativo, a leitura com qualidade ruim também é consumida (1), mas o *worker* mantém o valor e o carimbo anteriores e apenas registra desde quando o problema dura e o motivo (2b). A tela continua a exibir o último valor e, depois de três períodos sem leitura nova, passa a identificá-lo como desatualizado. Uma leitura com dado inválido, como um carimbo fora dos meses com partição, é desviada para uma fila de mortos e registrada como erro do trabalho, para análise posterior. Já uma falha passageira do banco ou da rede apenas faz o trabalho ser repetido com espera crescente, e a fila durável garante que nenhuma leitura se perca nesse intervalo.

Cada peça tem um efeito próprio quando falha. Se o *worker* para, as leituras se acumulam nas filas do Redis, e ao voltar são consumidas na ordem, sem prejuízo, porque o valor atual depende do número de sequência. Se o Redis cai, a aquisição não consegue publicar nem obter a trava, e o ciclo falha sem recuperação: ao voltar, o Agendador retoma o ritmo normal, sem rajada de leituras atrasadas. Se o Redis volta vazio, uma rotina de reconstrução refaz o valor atual a partir do espelho, sempre marcado como desatualizado até a próxima leitura. Se o banco de um cliente fica indisponível, os trabalhos de histórico daquele cliente esperam e tentam de novo, o valor atual continua a ser atualizado no Redis, mas a tela de valores daquele cliente informa indisponibilidade, e os demais clientes seguem normalmente. Se a fila de histórico acumula além de metade da memória do Redis, o trabalho mais antigo que três minutos é descartado, uma proteção deliberada para que um cliente lento não derrube todos os outros.

4.3 Do Banco à Tela em Tempo Real

A exibição do valor atual é obtida por *polling* curto, e não por conexão permanente, o que simplifica a operação e a recuperação de falhas. A tela pergunta, em um período entre um e dez segundos definido para o cliente, quais são os valores atuais das tags da planta que está aberta, e a resposta vem apenas do Redis, sem consulta ao banco de dados nem ao equipamento. A Figura 16 resume o caminho.

Figura 16: Fluxo da exibição do valor atual por *polling*



Fonte: Autor.

A tela envia o pedido de valores da planta à API (1), com o *token* de acesso e o validador de versão da última resposta. A API verifica a assinatura do *token* e confere no Redis se a sessão não foi revogada e quais são as permissões do usuário (2). Essas permissões ficam em um *cache* no Redis, que é apagado sempre que o cargo, o grupo ou o vínculo do usuário muda, de modo que a alteração vale já na requisição seguinte. A API mantém em memória a lista de tags da planta, conferindo a sua versão a cada minuto, e lê do Redis, em lote, o valor atual de cada uma (3). Com o valor, o carimbo e o estado guardado, ela deriva a qualidade de cada tag, que pode ser boa, desatualizada, com falha, sem dado, implausível ou sem acesso. Se nada mudou desde o pedido anterior, a resposta é vazia e leve. Caso contrário, a API devolve os valores (4), e a tela atualiza o que mudou.

A qualidade é apresentada ao operador com texto e ícone. Um valor bom aparece sem marcação. Um valor parado há mais de três períodos aparece como desatualizado, com a data desde a qual não é renovado. Uma tag cuja leitura falha mostra o último valor bom com um símbolo de alerta e o texto de falha de comunicação. Como a idade do valor depende do relógio, a própria tela promove um valor bom a desatualizado quando o tempo passa, mesmo que o servidor tenha respondido que nada mudou.

O mesmo caminho serve a todas as telas que mostram valores. A lista de acompanhamento, o sinótico, o medidor do painel de acompanhamento e a grade de tags do cadastro usam a mesma consulta e o mesmo componente de exibição, de modo que a qualidade e o estado de falha aparecem da mesma forma em qualquer lugar. Para que a tela não abra vazia, a primeira resposta pode ser entregue já junto com a página, pelo servidor, e o *polling* continua a partir dela, sem repetir o pedido na abertura. A grade de tags mostra, além do valor, a qualidade e a idade dele, o que permite ao administrador conferir um mapeamento recém-cadastrado sem sair da tela de cadastro.

Se o *cache* de permissões estiver vazio, a API recalcula o acesso lendo os bancos central e do cliente (2b) e repovoa o *cache*. A tela também conta as próprias falhas de pedido. Depois de três falhas seguidas, mostra o aviso de conexão instável e passa a exibir os valores como desatualizados, porque sem resposta ela não sabe se ainda valem. Nesse período, o intervalo entre as tentativas cresce, para não sobrecarregar um servidor em recuperação.

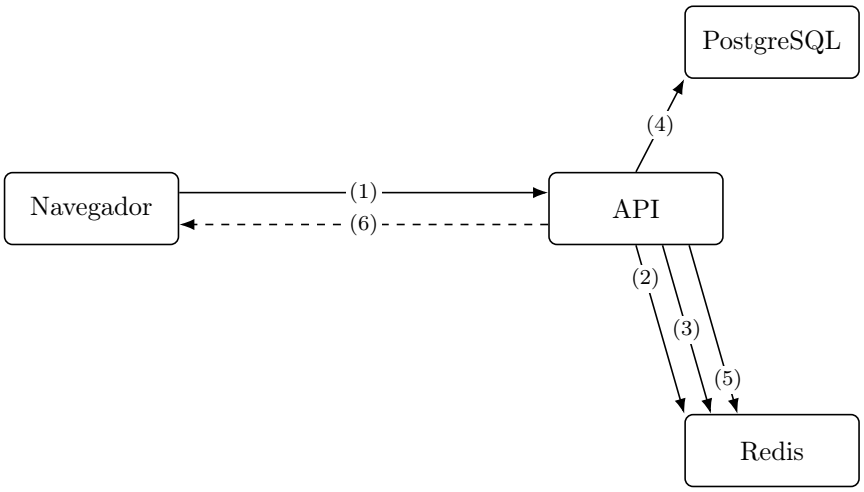
Quando uma peça falha, a tela deixa claro o que acontece. Com o Redis fora do ar, o login, a renovação e todas as rotas autenticadas respondem que o serviço está indisponível, e a tela

mostra esse aviso em vez de valores. Se o Redis volta vazio, a rota de valores de cada cliente responde indisponível, e não que a planta não existe, até que a rotina de reconstrução termine aquele cliente. Com o banco do cliente fora do ar, a tela de valores daquele cliente também deixa de responder, para qualquer cargo. A queda do equipamento de campo, por sua vez, não derruba a tela: ela continua a responder com o último valor e a qualidade correspondente.

4.4 Histórico e Tendência

A consulta ao histórico alimenta os gráficos de tendência do painel de acompanhamento. O desafio é entregar, em tempo curto, janelas que vão de alguns minutos a dez anos sem que uma consulta mal dimensionada prejudique as telas e os comandos dos demais usuários. O sistema resolve isso escolhendo, a partir do tamanho da janela, entre a leitura bruta e os resumos já calculados. A Figura 17 mostra o caminho.

Figura 17: Fluxo da consulta de tendência



Fonte: Autor.

A tela envia à API o pedido de tendência (1), com até doze tags e o intervalo desejado, e a API confere no Redis a sessão, as permissões e a cota de consultas pesadas, de trinta por minuto para cada usuário (2). O teto de retenção do cliente também é lido do estado guardado no Redis, e o que fica abaixo dele é tratado como se não existisse. Em seguida, a API escolhe o detalhe pelo tamanho da janela, com a mesma regra usada pela tela: até uma hora, todas as leituras brutas, até um dia, a média de um minuto, até sete dias, a média de dez minutos, até trinta e um dias, a média de trinta minutos, e acima disso a média de seis horas. Toda média acompanha o mínimo e o máximo do intervalo, para que um pico de poucos segundos não desapareça. A API procura a resposta no *cache* de tendência (3). O *cache* só guarda janelas já fechadas, com fim anterior a trinta minutos, séries pequenas, consultas que levaram mais de trezentos milissegundos e apenas enquanto o Redis tem folga de memória.

Se a resposta não está no *cache*, a API consulta o banco do cliente (4), em uma transação somente de leitura e em um conjunto de conexões reservado às leituras pesadas, de modo que uma consulta lenta não ocupe as conexões das telas e dos comandos. A consulta lê o resumo de um minuto ou de uma hora quando a janela é longa e completa a cabeça e a cauda do intervalo com as leituras brutas, para que a curva cubra exatamente o período pedido, mesmo que o último resumo ainda não tenha sido calculado. Em seguida, quando o resultado atende

às condições, a API o guarda no *cache* por dez minutos (5) e o devolve (6) junto com a hora do servidor e com o instante a partir do qual o resumo ainda pode ser refeito. A tela desenha a série com a biblioteca de gráficos ECharts (Apache Software Foundation, 2026).

Depois da primeira carga, a tela pede apenas o trecho novo, a partir do instante informado pelo servidor, e o junta ao que já tem, com recarga completa a cada quinze minutos e ao voltar para a aba. Só o painel visível atualiza, e a página inteira respeita um orçamento de pedidos por minuto, com prioridade para o clique do usuário. Para a leitura bruta de uma ou mais tags, página por página, a API devolve um cursor assinado, que só vale para a tag, o usuário e o cliente da consulta, de modo que um cursor alterado não leia a série de outra tag.

O painel de acompanhamento oferece três tipos de gráfico, o de linha, o de barra e o medidor. O de linha mostra a leitura bruta até uma hora e as médias acima disso, com a faixa entre o mínimo e o máximo, e a linha se interrompe onde falta dado, em vez de ligar pontos separados por uma falha. O de barra apresenta, por intervalo, a média, o mínimo, o máximo, o volume ou o tempo ligado, com o total da janela e a cobertura. O medidor não consulta o histórico, e sim o valor atual, pelo fluxo anterior. A cota de trinta consultas pesadas por minuto é o que obriga a página a ter um orçamento próprio de pedidos, de vinte por minuto e dividido em três faixas, para que um painel cheio não esgote a cota e deixe o usuário sem resposta no clique seguinte.

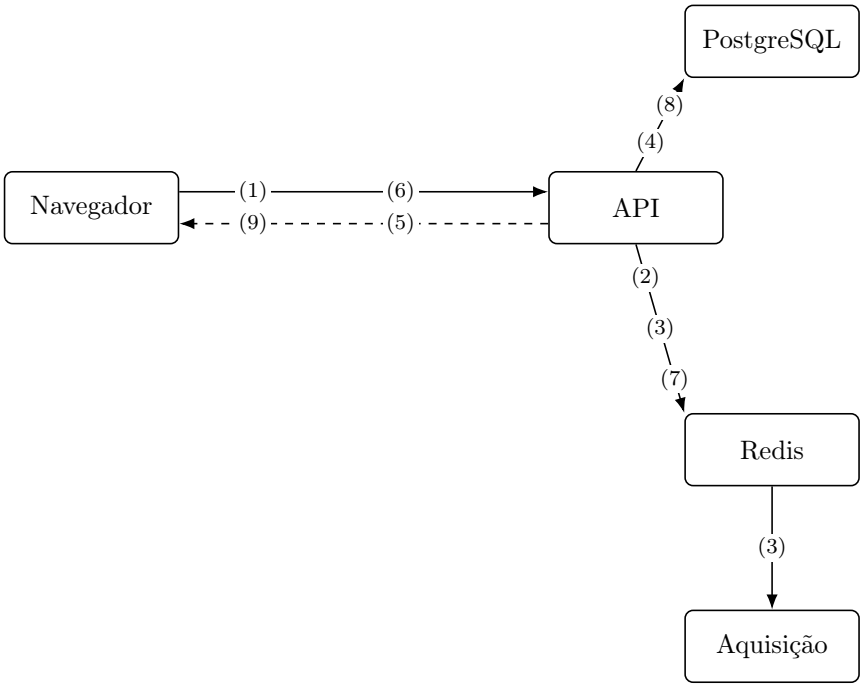
A retenção também limita o que a consulta devolve. A Movi define, para cada cliente, um teto de retenção de até dez anos, e a API o aplica como piso de leitura em toda consulta, usando o valor guardado no estado do cliente no Redis, sem ida ao banco central. Se o teto é reduzido, a mudança vale na hora para as consultas, antes mesmo de o apagamento noturno remover os dados, e a tela de administração mostra antes o impacto, com o número de tags e o intervalo de dados que serão perdidos, e pede que o administrador digite o identificador do cliente para confirmar.

Há caminhos alternativos de recusa. Um pedido grande demais é recusado, nunca cortado em silêncio, e a agregação que o tipo da tag não possui, como a média de uma tag digital, também é recusada. Se o conjunto de conexões pesadas está cheio, a resposta é de indisponibilidade, e não um erro interno. A falha na leitura do *cache* é tratada da mesma forma, enquanto a falha na escrita do *cache* apenas é registrada, porque o dado já foi obtido. Sem o Redis, a consulta não é autorizada e responde indisponibilidade. Sem o banco do cliente, não há histórico a devolver. Em nenhum dos casos a consulta altera dado, e a falha de uma não prejudica a exibição dos valores atuais, que têm caminho próprio.

4.5 Comando em Quatro Fases

O comando é a escrita de um valor em um ponto do equipamento, como ligar um motor ou alterar um ponto de ajuste. Por atuar sobre o processo físico, é o fluxo mais protegido do sistema, e seu caminho se divide em quatro fases: seleção, verificação, confirmação e execução. A seleção acontece na tela, quando o operador escolhe a tag em uma lista de acompanhamento ou em um sinótico e o diálogo de comando se abre, mostrando a tag, o valor atual, apenas como informação, e o campo do novo valor. As três fases seguintes estão na Figura 18, e a execução está na Figura 19.

Figura 18: Fluxo das fases de verificação e confirmação do comando

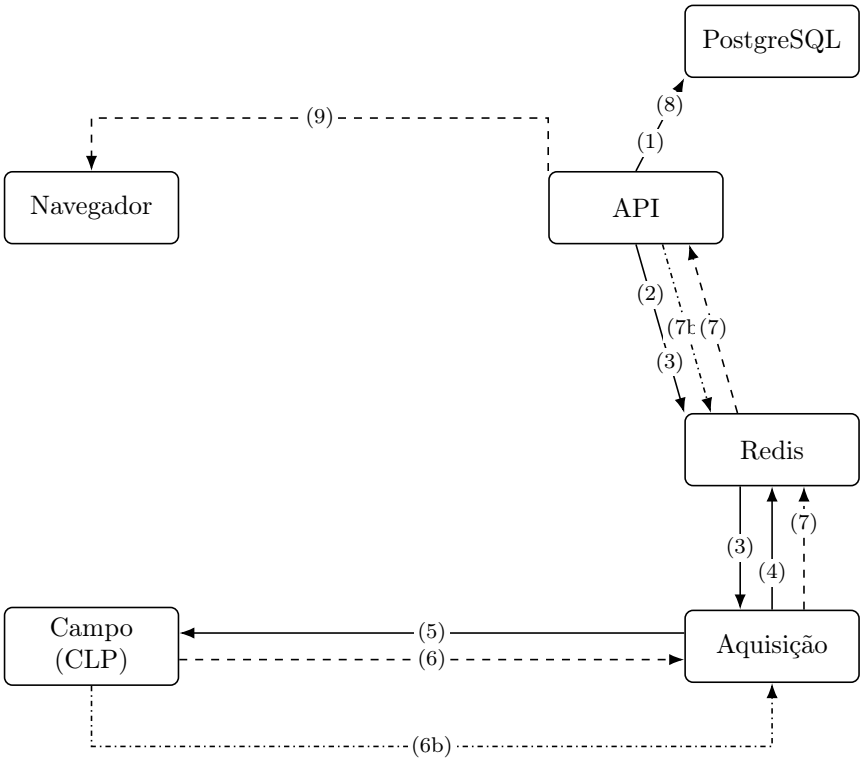


Fonte: Autor.

Na verificação, o navegador envia à API o pedido de comando, com a tag, o valor e uma chave de idempotência (1). A API aplica, na mesma requisição, uma série de conferências. A primeira lê no Redis o cargo e as permissões do usuário, confere se ele possui a permissão de comando do tipo da tag e verifica as três janelas de limite de comandos, por usuário, por usuário e tag e por cliente (2). A tag precisa existir no cliente e permitir escrita, o dispositivo precisa estar habilitado, e a API pergunta ao Agendador, pela aquisição, o estado de comunicação do dispositivo (3). Essa verificação falha de modo fechado: sem a certeza de que o equipamento está se comunicando, o comando é recusado. Por fim, o tipo e a faixa do valor são conferidos. Aprovado o pedido, a API grava o comando no estado verificado (4) e devolve ao navegador (5) um *token* de confirmação ligado ao que o usuário viu, com validade de cento e vinte segundos. O *token* é assinado, e a parte aleatória dele fica no Redis. Repetir o pedido com a mesma chave de idempotência devolve o mesmo *token*, o que impede a criação de dois comandos por um duplo clique.

Na confirmação, o operador revisa o resumo e confirma, e o navegador envia o *token* à API (6). Só o usuário que emitiu o comando pode confirmá-lo. A API confere a parte aleatória do *token* no Redis (7) e relê a permissão do usuário diretamente do banco, e não do *cache*, de modo que uma permissão retirada nos últimos instantes já impede a confirmação (8). Então o comando passa ao estado confirmado, e a resposta chega ao diálogo (9). Dois índices únicos no banco garantem que haja no máximo um comando em andamento por tag e um rascunho de cada pessoa por tag. Um varredor que roda a cada dez segundos fecha os comandos cujo prazo venceu.

Figura 19: Fluxo da fase de execução do comando



Fonte: Autor.

Na execução, a API reverifica tudo mais uma vez, incluindo o dispositivo, a tag, o valor e a permissão de quem emitiu (1), e toma no Redis a trava da tag, com validade de trinta e cinco segundos (2). Em seguida, chama diretamente a ação de escrita da aquisição pelo Moleculer (3), sem fila e sem nova tentativa automática. A aquisição toma no Redis a trava do dispositivo (4). O comando de um operador tem prioridade sobre as escritas automáticas e sobre a leitura periódica do mesmo dispositivo, e nenhum byte é enviado depois de vinte e cinco segundos da chegada do pedido, para que o quadro nunca alcance o equipamento depois de a API ter desistido. A aquisição envia a escrita ao CLP (5) em um dos códigos de função do protocolo, conforme o tipo da tag. O sucesso é a resposta normal do equipamento, que espelha a requisição enviada (6), conferida contra o endereço e o valor pedidos. Não há releitura posterior do ponto. Com o eco válido, a aquisição publica o valor escrito como valor atual da tag, pelo mesmo caminho de uma leitura, e a próxima leitura periódica o corrige se o equipamento não o reteve. Em seguida, devolve o resultado à API, por chamada e por evento (7), e deixa no Redis uma marca com o resultado, que vale por uma hora. A API fecha o comando no estado executado e grava no banco as transições e a auditoria (8), e o diálogo mostra ao operador que o equipamento confirmou a escrita (9).

O registro de auditoria guarda quem emitiu o comando, qual era o valor, quando ocorreu cada transição e o resultado, com uma cópia do nome da pessoa, da tag, da planta e do dispositivo. Gatilhos do banco impedem apagar, truncar ou alterar o autor e o conteúdo de um comando já registrado. A auditoria do sistema registra apenas as escritas em variável, entre elas o comando, e o reconhecimento de alarme.

O comando de uma pessoa não é a única origem de escrita. O sistema reconhece três origens, o usuário, o componente de conteúdo de uma tela e o script, e as duas últimas só chegam à escrita por caminhos próprios. O componente passa por uma porta de conferência que verifica

se ele está publicado, se a escrita está ligada àquela tag e se a tela é acessível ao usuário, e não pode pular a confirmação. A escrita em hora marcada, que não existe como comando agendado de pessoa, é feita por um evento agendado cujo tratador é um script, que escreve na tag com a origem registrada como script. Essa escrita usa o mesmo executor da fase de execução, tem a prioridade de escrita automática, que cede ao comando do operador, e não tem limite de frequência próprio, de modo que evitar um laço de escrita é responsabilidade de quem escreve o script, e desabilitar o script ou o evento o interrompe. A tag interna, que não tem equipamento, recebe o valor diretamente do sistema e termina no mesmo estado de sucesso, sem passar pela aquisição.

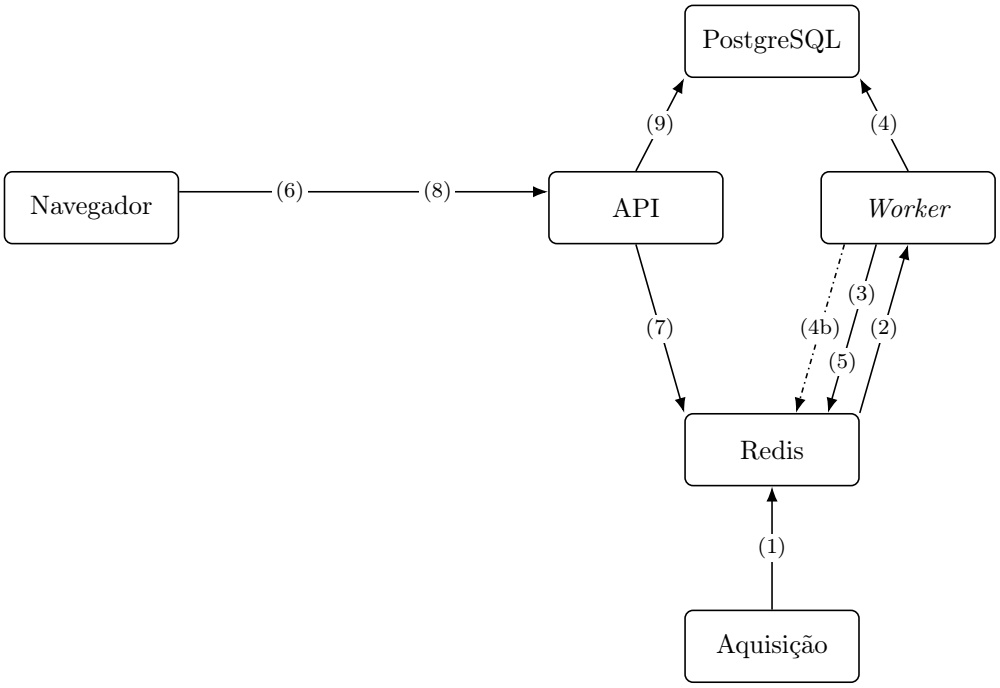
Há dois desfechos alternativos. Se o equipamento responde com exceção, com um eco que não confere ou não responde no prazo (6b), a escrita é tratada como falha, nunca como sucesso e nunca como estado desconhecido, e a mensagem pede que o operador confira o valor do ponto. O comando que falha não é repetido de forma automática. A falha também envia um sinal ao evento de falha de escrita do dispositivo, que só abre alarme se o integrador tiver ligado um alarme a ele. O segundo desfecho ocorre quando é o próprio sistema que perde o rastro, por exemplo, se a resposta da aquisição não chega à API. Nesse caso (7b), a API consulta a marca de resultado no Redis. Se a marca não existe, grava nela, de forma exclusiva, que a escrita foi abandonada, para que um pedido atrasado que chegue depois à aquisição não escreva. O comando é fechado como sem confirmação do equipamento, e a tag permanece bloqueada por trinta e cinco segundos a partir do fechamento, com mensagem própria para quem tentar nesse intervalo. Um resultado que a aquisição concluir mais tarde fica no registro do comando, sem mudar o estado nem a tela.

Quando uma peça falha, o sistema prefere recusar a arriscar. Sem o Redis, não há trava nem *token*, e o comando nunca executa. Com o Agendador ou a aquisição indisponíveis, a verificação falha de modo fechado. Se a API cair no meio da execução, o varredor, aos trinta e dois segundos, e a marca de resultado no Redis permitem fechar o comando com o estado real. Para clientes suspensos, um trabalho único enfileirado na suspensão faz esse fechamento. A reemissão do comando só é segura porque a marca de resultado distingue o que foi escrito do que nunca saiu da API.

4.6 Alarme

O alarme transforma uma condição do processo em um aviso que exige a atenção de uma pessoa. No sistema, a condição é descrita por um evento da tag, formado por um único comparador: a tag, um operador, o valor de referência e uma banda morta opcional. Os operadores são maior, maior ou igual, menor, menor ou igual, igual, diferente, dentro da faixa e fora da faixa. A banda morta evita que um valor oscilando em torno do limite abra e feche o alarme a cada leitura. Cada evento tem ainda atrasos de ativação e de retorno, de zero a uma hora. O alarme liga um ou mais eventos a um dos quatro níveis fixos, Crítico, Alto, Médio e Baixo. A Figura 20 mostra o fluxo.

Figura 20: Fluxo da detecção, da sinalização e do reconhecimento de um alarme



Fonte: Autor.

A leitura chega pelo caminho já descrito, publicada pela aquisição na fila de ingestão (1) e consumida pelo *worker* (2). Depois de gravar o valor atual, o *worker* executa os passos seguintes da ingestão, em ordem fixa, e o primeiro deles avalia os eventos da tag. A avaliação é feita por um *script* atômico no Redis, que guarda o estado de cada detector (3), compara o valor com o operador e o limite, aplica a banda morta e os atrasos, e usa o carimbo e o número de sequência da leitura para decidir se houve uma transição. Como a decisão depende da ordem da leitura, a mesma leitura processada por duas réplicas não gera dois alarmes. Quando há transição, o *worker* a registra antes numa fila de pendências no Redis e a aplica no banco do cliente (4). Na mesma transação em que grava o estado do detector, ele abre, atualiza ou fecha a ocorrência do alarme, que é o episódio do disparo ao fechamento, e grava a linha de transição, que é apenas inserida e nunca alterada. A ocorrência tem quatro estados, que combinam estar ativa ou normalizada com estar reconhecida ou não, e há no máximo uma ocorrência aberta por alarme. Um alarme com vários eventos mantém uma só ocorrência e passa a indicar o número de origens. Depois da confirmação da transação, o *worker* soma um à versão do sumário de alarmes do cliente no Redis (5), o que invalida o sumário guardado.

A sinalização aos operadores usa o mesmo *polling* das demais telas. O casco da aplicação, presente em todas as páginas, pergunta periodicamente o sumário de alarmes (6), e a API o lê no Redis (7). Se a versão do sumário mudou ou o sumário expirou, a API o recalcula a partir do banco e o grava de volta, apenas se a versão que leu ainda é a corrente. O sumário traz os contadores por nível e estado e os primeiros itens da lista de ativos, e o indicador exibe o nível mais grave ainda não reconhecido e a quantidade de ocorrências. Todos os operadores do cliente compartilham a mesma entrada, e, durante uma avalanche de alarmes, a versão sobe em média uma vez por segundo, para que centenas de ocorrências abertas de uma vez não gerem centenas de recálculos. O operador abre a página de alarmes, que lista as ocorrências ativas e o histórico, e reconhece uma ocorrência (8). O pedido leva a ocorrência e a revisão que a pessoa viu na tela, para que um redisparo não seja reconhecido por um clique dado sobre a

tela antiga. A API relê as permissões do usuário no banco, confere o estado e a revisão da ocorrência e grava o reconhecimento, com o nome da pessoa e um comentário opcional (9). Se outro evento do mesmo alarme se ativa depois do reconhecimento, a ocorrência volta ao estado não reconhecido.

Há outras entradas no mesmo caminho. O evento composto combina outros eventos com os operadores E, OU e NÃO, e o evento agendado dispara em horários definidos. Ambos usam o mesmo estado no Redis e a mesma fila de pendências, de modo que um alarme pode ser ligado a qualquer um deles. Os eventos de sistema do dispositivo, falha de comunicação, falha de leitura e falha de escrita, nascem sozinhos com o dispositivo Modbus, e cada ativação e cada retorno ao normal deles ficam gravados em uma tabela de registro que alimenta a linha do tempo. O alarme também aparece em outros pontos da interface. A tag mostra um selo com o contador das ocorrências abertas, e a planta no mapa mostra um pino com a cor da pior ocorrência aberta, que a API obtém por uma única consulta agregada. A ocorrência é guardada pelo prazo de retenção definido para os alarmes do cliente, e uma rotina noturna apaga o que passou dele antes de apagar as leituras.

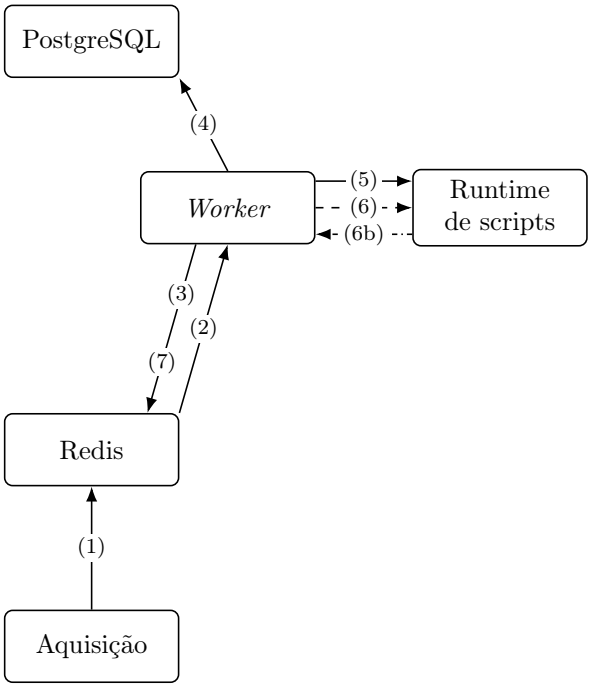
Se o processo cai entre o registro da transição na fila de pendências e a gravação no banco (4b), a transição não se perde: um trabalho periódico de sessenta segundos aplica o que está pendente há mais de um minuto e confere o evento no banco. Uma varredura de cinco minutos fecha as ocorrências cujo evento, tag, dispositivo ou alarme foi desabilitado ou excluído. Desabilitar um evento zera o seu estado, e ao reabilitá-lo a avaliação recomeça do zero, com o carimbo da leitura real.

Quando uma peça falha, o efeito é previsível. Com o Redis fora, a avaliação dos eventos para e o trabalho de ingestão é repetido, porque o valor atual também depende dele. Se o Redis volta vazio, a rotina de reconstrução repovoa o estado dos detectores a partir do banco. Se o banco do cliente fica indisponível, as transições ficam pendentes e são aplicadas quando ele volta. Com o *worker* parado, nenhum alarme novo é detectado, e a fila preserva as leituras para o momento em que ele retornar. A falha de comunicação do dispositivo e a falha de escrita são tratadas como eventos do próprio dispositivo, e o integrador pode ligar a eles um alarme comum. O sistema não cria alarme de sistema automaticamente.

4.7 Variável Meta

A variável meta é uma tag cujo valor não vem do equipamento, e sim de um cálculo escrito pelo administrador do cliente sobre outras tags da mesma planta, como uma diferença de pressões, um volume por hora ou um balanço. Ela se comporta como qualquer outra tag, com histórico, tela e alarme, mas depende de código de terceiros, o que exige isolamento. O código é escrito em JavaScript e executado por um motor pequeno, o QuickJS, compilado para WebAssembly e executado sob o Wasmtime, em um processo que não tem banco, rede, arquivo nem segredo (Bellard; Gordon, 2026; Bytecode Alliance, 2026; WebAssembly Community Group, 2026). A Figura 21 mostra o fluxo.

Figura 21: Fluxo do cálculo de uma variável meta



Fonte: Autor.

A variável meta recalcula a cada leitura nova de uma das tags que o código lê, ou em horários definidos, por minuto, hora, dia ou expressão de agenda, e também ao ser criada, editada ou habilitada. No gatilho por leitura, a aquisição publica a leitura na fila de ingestão (1), e o *worker* do sistema a consome (2). Depois de gravar o valor atual, um dos passos seguintes da ingestão consulta o grafo de dependências, mantido em memória, que informa quais variáveis meta leem aquela tag, e enfileira uma onda de cálculo na fila própria dos cálculos (3). Um lote de leituras gera uma onda só, e a ordem respeita as dependências, de modo que uma meta que depende de outra espera o valor novo dela. No gatilho por tempo, uma varredura por minuto cria a onda, e a varredura só abre o banco de um cliente que tenha alguma meta de tempo.

A onda é consumida pelo *worker* da família de cálculos internos, que roda no mesmo contêiner do *worker* do sistema. Como o ambiente de execução dos scripts não tem acesso a banco, o *worker* busca antes todos os dados de que o código precisa: o valor atual das tags lidas, no Redis, e os últimos pontos, de até quinhentos, nas tags consultadas por série, no banco do cliente (4). Esses dados seguem junto com o código em um único lote, enviado ao *runtime* de scripts por uma chamada do Moleculer a um *broker* dedicado, com regras de acesso restritas a essa ação (5). Para cada cálculo, o *runtime* cria uma instância nova do motor a partir de uma foto pré-aquecida, sem rede, sem arquivo e sem segredo, e com limites de tempo de processamento, memória, recursão e saída. Os clientes entram numa fila justa, e há um limite de avaliações simultâneas por cliente. O resultado de cada meta do lote volta com o valor ou o erro (6).

Ao receber o resultado, o *worker* valida o que veio, porque o servidor nunca confia em dado devolvido pelo ambiente isolado, e o publica como uma leitura da tag interna correspondente (7). Para isso, atribui o número de sequência da tag e a publica pelo mesmo caminho das leituras vindas do equipamento, e por isso a variável meta passa por cache, histórico, resumos e alarmes como qualquer tag. A qualidade da meta é a pior qualidade entre as tags que ela lê. Uma entrada desatualizada, com falha ou excluída impede o cálculo e dá qualidade ruim ao resultado, e o valor nunca é zero por omissão. Quando o administrador salva o código, a API

o envia ao *runtime* para uma análise que apenas lê a árvore do programa, sem executá-lo, e descobre quais tags e dispositivos o código usa. O cadastro dessas dependências é preenchido por essa análise, e o salvamento é recusado se o ambiente isolado estiver indisponível.

O mesmo ambiente isolado atende outras funções além da variável meta. A tag interna gerada, como o valor aleatório, a rampa e o passeio aleatório usados em testes e simulações, é um pequeno programa fornecido pela Movi e executado no mesmo motor, e seu valor é publicado pelo mesmo caminho a cada ciclo do dispositivo interno, sob uma trava de curta duração que impede dois ciclos simultâneos. O script de dispositivo e de evento, que reage a uma transição ou a um horário e pode escrever em tags, e o componente de conteúdo de uma tela do sinótico, que mostra texto e gráfico declarativo calculados pelo cliente, também rodam nele, cada um com limites próprios de tempo, memória e saída. O componente não é recalculado a todo instante, e sim em um ritmo configurado, com o resultado guardado no Redis por um tempo curto. Em todos os casos vale a mesma regra: o ambiente isolado recebe os dados no pedido e devolve um resultado, e quem escreve em banco, em fila ou no equipamento é sempre o código da API ou do *worker*, depois de validar o que voltou.

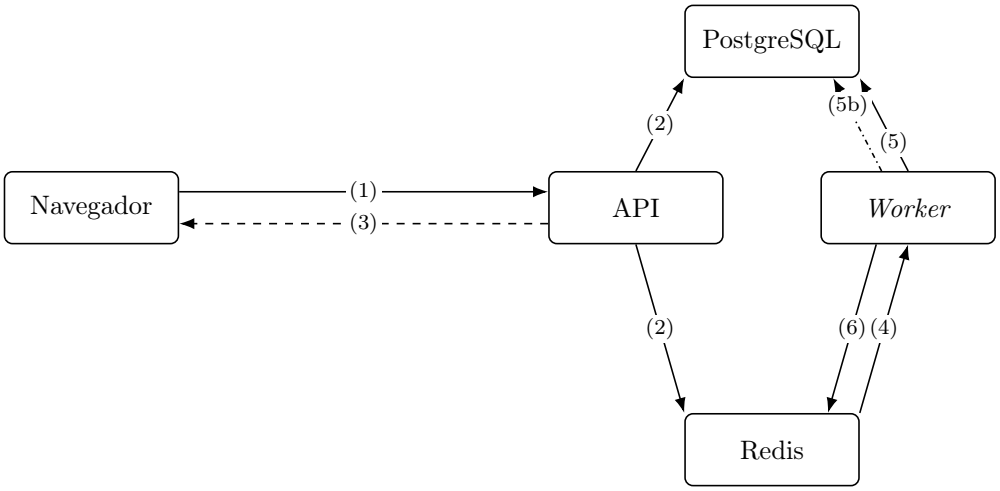
O caminho alternativo ocorre quando o código estoura o tempo, a memória ou a recursão, lança um erro ou divide por zero (6b). O *runtime* devolve um resultado parcial, em que apenas aquela meta recebe erro e as demais do lote seguem, e o *worker* publica a qualidade ruim em vez de um valor. Se o *runtime* está ocupado, o cálculo espera na fila do cliente, e a espera não é tratada como falha.

Quando uma peça falha, o efeito é restrito. Se o *runtime* cai, nenhuma meta é calculada, a onda não é concluída e a fila a repete, e a variável meta fica com o último valor, que a tela passa a mostrar como desatualizado. As demais tags, os alarmes sobre tags de equipamento e os comandos não dependem dele. Uma falha no motor ou na máquina virtual, no pior caso, compromete apenas esse serviço, que não possui segredo nem acesso ao banco, e a API valida tudo o que recebe dele. Esse risco foi aceito explicitamente, e as medidas de mitigação são a ausência de credenciais, o processo hospedeiro reciclado e a validação do resultado pelo servidor.

4.8 Criação de um Cliente

Cada cliente do sistema tem um banco de dados próprio, criado no momento do cadastro. Esse isolamento físico simplifica a segurança e a retenção, e em troca exige que a criação de um cliente seja um procedimento automatizado, completo e capaz de se desfazer. Apenas o administrador da Movi, o *system admin*, pode cadastrar clientes. A Figura 22 apresenta o fluxo.

Figura 22: Fluxo da criação de um cliente



Fonte: Autor.

O administrador da Movi preenche o formulário de novo cliente, com nome, dados cadastrais, os parâmetros do contrato, como o teto de retenção do histórico, e o primeiro administrador do cliente, e o navegador envia o pedido à API (1). A API confere o cargo, valida os campos e grava no banco central o cliente, o administrador inicial e o registro que liga o cliente ao seu banco, ainda marcado como não pronto (2). O sistema gera uma senha de uso único para o primeiro administrador, composta por três grupos de quatro caracteres e válida por vinte e quatro horas, que só serve para entrar e trocar a senha. Ela é mostrada uma única vez na tela, e nenhum e-mail é enviado. Ainda no mesmo passo, a API enfileira no Redis o trabalho de provisionamento e responde ao navegador (3), que passa a mostrar o cliente com a situação de banco em criação.

O trabalho de provisionamento é consumido pelo subprocesso provisionador (4), o único em todo o sistema que possui a credencial capaz de criar e apagar bancos na instância do PostgreSQL, e que executa uma tarefa por vez. Esse subprocesso roda no mesmo contêiner do *worker*, mas com um ambiente separado, e apenas ele recebe a credencial privilegiada. Em um único trabalho retomável, ele cria o banco e o usuário de banco do cliente, revoga o acesso dos demais usuários ao novo banco, instala a extensão de busca por texto e ensaia a migração dentro de uma transação que termina em desfazimento, para só então aplicar as migrações de verdade e inserir os dados iniciais, como os quatro níveis de alarme (5). O banco criado pertence ao cliente e só o usuário dele se conecta a ele, e todo o esquema vem de migrações geradas e versionadas. No fim, o trabalho registra o estado do cliente no Redis, marca o cliente como pronto no banco central e grava a marca que indica que o *cache* do cliente está pronto (6). A tela do administrador acompanha o progresso por *polling* e passa de banco em criação para ativo.

O caminho alternativo cobre as falhas. Se a primeira criação falha por um erro que não adianta repetir, como a recusa do ensaio da migração, o trabalho desfaz tudo (5b): apaga o banco e o usuário de banco que ele próprio reservou e remove o cadastro, de modo que o cliente simplesmente deixa de existir. Uma falha passageira, como uma queda curta do Redis ou do banco, é repetida, e somente uma criação parada há trinta minutos é desfeita. Um cliente que já esteve pronto nunca é desfeito por esse mecanismo. De hora em hora, uma varredura procura bancos ou usuários de banco sem cliente correspondente e avisa o administrador da Movi, mas nunca apaga nada por conta própria.

O banco do cliente pode também ficar em um servidor que a Movi não controla, mantido pelo próprio cliente. Nesse caso, o administrador da Movi informa servidor, porta, banco, usuário e senha, escolhe a segurança da conexão, que pode ser com certificado conferido, o padrão, sem conferência do certificado ou sem criptografia, por conta e risco do cliente, e aciona um teste antes de criar o cliente. O teste confere, em uma única transação somente de leitura, o modo de segurança escolhido, a autenticação do tipo SCRAM, a versão mínima do servidor, que o usuário não é superusuário, os privilégios, a extensão de busca, a codificação, a ordenação em português e que o banco está vazio, e apresenta todos os problemas de uma vez para que o responsável pelo banco os corrija. A senha é cifrada com uma chave de dados própria do cliente, embrulhada por uma chave mestra, e depois de migrado o banco o endereço, o nome e o usuário deixam de poder ser alterados. Como a Movi não cria o banco nem tira cópia de segurança dele, o ensaio da migração e o desfazimento são ainda mais importantes. Com esse banco fora do ar, tudo daquele cliente para, inclusive as telas, e excluir o cliente apenas apaga o cadastro na Movi, e os dados permanecem no servidor dele.

Desabilitar um cliente é a medida intermediária entre mantê-lo ativo e excluí-lo. Ao desabilitar, o login do cliente é recusado, as sessões são derrubadas e a aquisição deixa de ler os dispositivos dele na hora. O fechamento dos alarmes e dos comandos em andamento acontece em segundo plano e termina sozinho mesmo que a API caia no meio, graças a uma marca de efeitos pendentes que uma rotina de cinco minutos refaz. Esse é o estado exigido antes de excluir.

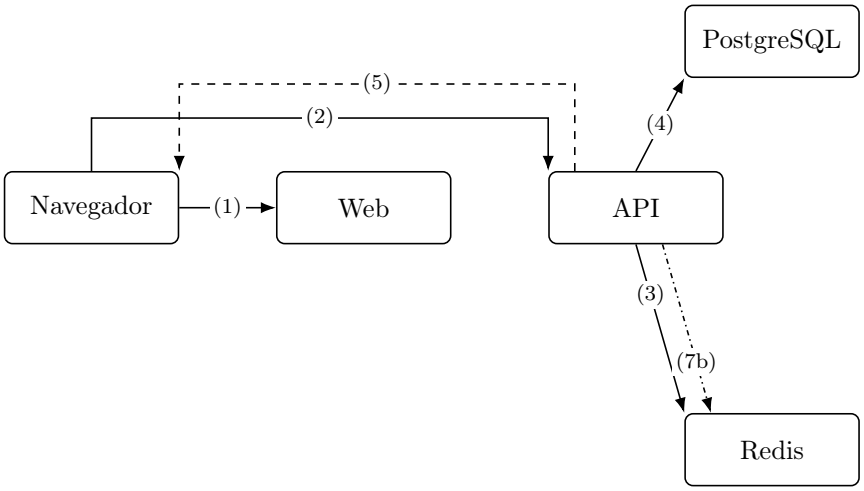
A exclusão de um cliente segue o caminho inverso, com mais cuidado, porque não tem volta. Só um cliente desabilitado pode ser excluído, e o administrador digita o identificador do cliente para confirmar. O trabalho espera a aquisição parar os ciclos do cliente e os trabalhos em andamento terminarem, apaga as chaves do Redis, os arquivos, os usuários e os vínculos, e por último o banco, o usuário de banco e o cadastro. Se cair no meio, recomeça sem prejuízo.

Cada falha tem um efeito claro. Sem o Redis, não é possível enfileirar o trabalho, e a API recusa o cadastro. Se o *worker* está parado, o cliente permanece em criação até que o provisionador volte a consumir a fila. Se o banco central ou a instância do PostgreSQL está fora, o trabalho é repetido e, no limite, desfeito. O limite declarado é que o provisionador compartilha o contêiner e o disco com os demais subprocessos, de modo que o isolamento da credencial privilegiada é por processo e por ambiente, e não por contêiner.

4.9 Login e Sessão

O login estabelece quem o usuário é e a qual cliente ele pertence, e todas as demais rotas da API dependem dessa informação. O cliente vem apenas da sessão, nunca da URL, o que impede um usuário de pedir dados de outro cliente. A Figura 23 apresenta o fluxo.

Figura 23: Fluxo do login e da renovação da sessão



Fonte: Autor.

O usuário abre a página de login, entregue pela aplicação Web (1), e informa e-mail e senha. O navegador os envia à API (2). A API aplica primeiro os limites de tentativas no Redis, por e-mail, por endereço de origem e global, e consulta o contador de falhas daquele e-mail (3). Em seguida, lê o usuário no banco central e verifica a senha comparando com o *hash* guardado, calculado com o algoritmo Argon2id (4), projetado para tornar o ataque por força bruta caro (Biryukov *et al.*, 2021). A resposta é a mesma para um e-mail que não existe e para uma senha errada, com tempo semelhante, para que o sistema não revele quais e-mails têm conta. Em caso de sucesso, a API cria a sessão no banco central, guardando apenas o *hash* do segredo de renovação, e devolve ao navegador o *token* de acesso, assinado no formato *JSON Web Token* (Jones; Bradley; Sakimura, 2015), com validade de quinze minutos, e um *cookie* de renovação (5). O *token* de acesso fica apenas na memória do navegador. O *cookie* de renovação é inacessível ao código da página, trafega só por conexão segura, só é enviado em requisições do próprio site e só vale para as rotas de autenticação.

A partir daí, cada requisição à API leva o *token* de acesso. A API verifica a assinatura, confere no Redis se a sessão não foi revogada e consulta as permissões do usuário, com o *cache* descrito anteriormente. A sessão dura no máximo doze horas e expira com oito horas sem uso, e dez minutos antes do fim a tela avisa e pede a senha de novo. Se a sessão acaba, a tela exibe um bloqueio informando que os alarmes não estão sendo monitorados e emite um som, sem apagar o conteúdo, para que o operador não deixe de perceber a perda da supervisão. Quando o *token* de acesso vence, o navegador chama a rota de renovação com o *cookie* (6), e a API troca o segredo por um novo, condicionado ao *hash* anterior, o que impede que duas renovações sobrepostas criem duas sessões. Para suportar respostas perdidas em redes ruins, uma renovação repetida dentro de trinta segundos devolve o mesmo *cookie* derivado do anterior.

O primeiro acesso, ou qualquer acesso com uma senha de uso único, obriga a troca de senha, e o servidor impõe essa obrigação em todas as rotas, e não só a tela. O usuário também pode sair a qualquer momento, e a saída funciona mesmo com o *token* vencido ou com o Redis fora do ar, para que o próximo operador de um mesmo terminal não entre como o anterior.

Depois do login, o que o usuário pode fazer depende do cargo e dos grupos. Há quatro cargos fixos: o administrador da Movi, que atua sobre todos os clientes, o administrador do cliente, que configura o sistema, o supervisor, que faz o que o operador faz e gerencia operadores, e o operador. Dentro do cliente, o administrador cria grupos, e cada grupo recebe permissões de

um catálogo fixo de dez, como ler tags, reconhecer alarmes e emitir comandos. Quem pertence a vários grupos soma as permissões. A permissão vale para o cliente inteiro, e a única fronteira rígida do sistema é o cliente. Como o *token* de acesso não carrega permissões, uma mudança de cargo ou de grupo vale na requisição seguinte: a alteração grava uma linha em uma fila de mudanças na mesma transação, e um processo apaga o *cache* de permissões do usuário no Redis, com uma varredura a cada trinta segundos que repara o que o Redis não aceitou. O supervisor só coloca um operador em um grupo cujas permissões ele mesmo possui, e nenhum grupo concede as ações reservadas aos administradores, como configurar alarmes e escrever código de cliente.

O administrador da Movi pode entrar em um cliente para prestar suporte. A entrada fica registrada apenas na sessão dele, que passa a apontar para aquele cliente, e não deixa rastro visível para os usuários do cliente. Dentro do cliente, ele faz tudo o que o administrador faz, inclusive comandar, e os comandos e os reconhecimentos dele ficam registrados com a cópia do nome. A sessão volta ao estado fora do cliente quando ele sai ou quando o cliente é excluído.

Há caminhos alternativos. Cinco senhas erradas bloqueiam o e-mail por quinze minutos, e o bloqueio vale também para um e-mail inexistente. Se um segredo de renovação já trocado é apresentado de novo fora da janela de trinta segundos (7b), o sistema entende que o segredo pode ter sido copiado e derruba a sessão inteira, marcando-a como revogada no Redis. Ao ultrapassar o número máximo de sessões ativas de um usuário, o login revoga a mais antiga e segue, sem recusar o acesso. Quando o Redis está fora do ar, as rotas autenticadas, o login e a renovação respondem que o serviço está indisponível, porque sem ele o sistema não consegue confirmar que a sessão não foi revogada. Esse é o comportamento desejado, e o sistema prefere negar o acesso a aceitar uma sessão que pode ter sido encerrada. Se o banco central fica indisponível, ninguém consegue entrar, e as sessões já abertas só continuam válidas enquanto as permissões estiverem no *cache*.

4.10 Garantias Comuns aos Fluxos

Alguns cuidados se repetem em mais de um fluxo e formam, juntos, o comportamento previsível do sistema. O primeiro é o isolamento entre clientes: cada cliente tem o próprio banco, o cliente de cada requisição vem apenas da sessão, e toda chave do Redis, todo identificador de tarefa e todo prefixo de armazenamento de arquivo cujo identificador nasce no banco do cliente leva também o identificador do cliente, de modo que dois clientes com o mesmo número de dispositivo nunca se confundem. O segundo é a ordem das leituras, decidida pelo número de sequência, e não pelo relógio de quem processa. O terceiro é a idempotência: a chave de idempotência do comando, o identificador de cada trabalho de leitura, a marca de resultado da escrita e a decisão atômica do histórico fazem com que uma repetição, por falha de rede ou por reentrega da fila, não produza um segundo efeito.

O quarto cuidado é a falha fechada, aplicada onde o erro seria mais caro. O comando é recusado quando falta o Redis ou a confirmação de que o equipamento se comunica, as rotas autenticadas respondem indisponibilidade quando o Redis cai, e um valor que não pôde ser renovado nunca é apresentado como bom. O quinto é a durabilidade do trabalho, obtida pelas filas duráveis, pelo *outbox* do cadastro, pela fila de pendências do alarme e pelas novas tentativas com espera. O sexto é a releitura da permissão onde a decisão é irreversível, no banco e não no *cache*, o que vale para a confirmação do comando e para o reconhecimento do alarme. A Tabela 5 associa cada garantia ao mecanismo que a sustenta e aos fluxos em que ela aparece.

Tabela 5: Garantias comuns aos fluxos do sistema.

Garantia	**Mecanismo**	**Fluxos**
Isolamento entre clientes	Banco próprio por cliente, cliente obtido apenas da sessão, identificador do cliente nas chaves do Redis e nas tarefas	Todos
Ordem das leituras	Número de sequência formado por época e contador, e não o relógio	Do campo ao banco, tempo real, alarme
Idempotência	Chave de idempotência do comando, identificador por leitura, marca de resultado da escrita, decisão atômica do histórico	Comando, do campo ao banco
Falha fechada	Recusa do comando sem Redis ou sem estado de comunicação, indisponibilidade das rotas autenticadas sem Redis	Comando, login, tempo real
Durabilidade do trabalho	Filas duráveis, *outbox*, pendências do alarme, novas tentativas com espera	Cadastro, do campo ao banco, alarme, criação de cliente
Dado velho não vira dado bom	Qualidade desatualizada, marca de falha de comunicação, qualidade ruim na variável meta	Tempo real, do campo ao banco, variável meta
Permissão relida onde importa	*Cache* nas leituras, banco na confirmação do comando e no reconhecimento do alarme	Tempo real, comando, alarme, login
Registro imutável	Registro de comando e de reconhecimento somente com inserção, protegido por gatilhos	Comando, alarme

Fonte: Autor.

Os parágrafos de falha de cada fluxo mostram que o sistema degrada de forma localizada, e não de uma vez. A Tabela 6 reúne o efeito da falha de cada bloco, de modo que se possa ver, em um só lugar, o que continua funcionando quando uma peça para.

Tabela 6: Efeito da falha de cada bloco nos fluxos do sistema.

Bloco que falha	**O que deixa de funcionar e o que continua**
Web	Páginas novas deixam de ser entregues. As telas já abertas continuam a consultar a API, e a aquisição, o histórico e os alarmes não são afetados.
API	Cadastro, login, comando e consultas deixam de responder. A aquisição continua a ler com a última configuração, e o *worker* continua a gravar o valor atual, o histórico e a avaliar os alarmes.
Worker	O valor atual, o histórico, os alarmes e as variáveis meta deixam de avançar, e as leituras se acumulam nas filas. As telas continuam a responder com o último valor, que passa a desatualizado.
Aquisição	Não há leitura nem escrita no equipamento, e os valores passam a desatualizados. O cadastro, as telas e os alarmes já abertos continuam disponíveis, e o comando é recusado.
Runtime de scripts	Variáveis meta, scripts e componentes de conteúdo param, e a variável meta fica com o último valor. As tags de equipamento, os alarmes sobre elas e os comandos seguem normalmente.

Bloco que falha	O que deixa de funcionar e o que continua
Redis	As rotas autenticadas e o login respondem indisponibilidade, a aquisição não publica e o comando não executa. Ao voltar vazio, a rotina de reconstrução refaz o valor atual, os estados e a lista de sessões revogadas.
PostgreSQL de um cliente	Apenas aquele cliente é afetado. O cadastro, o histórico e os alarmes dele param, e a tela de valores informa indisponibilidade, enquanto os demais clientes seguem normalmente.
PostgreSQL central	Login, gestão de usuários e de clientes deixam de funcionar. As sessões abertas continuam enquanto as permissões estiverem em *cache*.
Campo	O último valor é mantido e passa a desatualizado, o evento de falha de comunicação do dispositivo é ativado e o comando é recusado. Os demais dispositivos não são afetados.

Fonte: Autor.

4.11 Resumo dos Fluxos

A Tabela 7 reúne os nove fluxos, os blocos que cada um percorre e a fila ou a ação de serviço que transporta o trabalho entre eles. A coluna de transporte mostra que a comunicação entre os serviços de aplicação usa dois mecanismos. O primeiro é a chamada de ação do Moleculer, usada quando o solicitante precisa da resposta, como o comando e a consulta de configuração. O segundo é a fila durável do BullMQ, usada quando o trabalho pode esperar e não pode ser perdido, como a ingestão, o histórico e o provisionamento. Entre o navegador e a API, o transporte é o HTTP com dados em JSON.

Tabela 7: Resumo dos fluxos do sistema.

Fluxo	Blocos envolvidos	Fila ou ação usada
Cadastro de uma tag	Navegador, Web, API, Redis, PostgreSQL, Aquisição	HTTP, evento de mudança do cadastro e ações de registro do Moleculer
Do campo ao banco	Aquisição, Campo, Redis, *Worker*, PostgreSQL	Ação de leitura do Moleculer, filas de ingestão e de histórico, fila geral de tarefas periódicas
Valor atual em tempo real	Navegador, API, Redis, PostgreSQL	HTTP com *polling*, leitura direta do Redis
Histórico e tendência	Navegador, API, Redis, PostgreSQL	HTTP, *cache* de tendência no Redis, consulta somente leitura
Comando em quatro fases	Navegador, API, Redis, PostgreSQL, Aquisição, Campo	HTTP, ação de escrita do Moleculer, evento de escrita concluída
Alarme	Aquisição, Redis, *Worker*, PostgreSQL, API, Navegador	Fila de ingestão, fila geral de tarefas, sumário no Redis

Fluxo	**Blocos envolvidos**	**Fila ou ação usada**
Variável meta	Aquisição, Redis, *Worker*, Runtime de scripts, PostgreSQL	Fila de ingestão, fila de ondas de cálculo, ação de avaliação do Moleculer
Criação de um cliente	Navegador, API, Redis, *Worker*, PostgreSQL	Fila de provisionamento
Login e sessão	Navegador, Web, API, Redis, PostgreSQL	HTTP, lista de sessões revogadas no Redis

Fonte: Autor.

Os fluxos mostram um padrão que se repete. O que precisa de resposta imediata e de contexto do usuário passa pela API, que sempre confere a sessão e a permissão antes de qualquer outra ação. O que precisa de durabilidade passa por uma fila no Redis e é consumido por um *worker* idempotente. O valor atual vive no Redis, e o histórico no PostgreSQL, de modo que a tela em tempo real nunca consulta o banco. Em cada fluxo, a falha de uma peça é tratada de modo explícito, ora com a recusa do pedido, ora com a repetição do trabalho, ora com a marcação do valor como desatualizado, mas sem produzir um dado falso como se fosse verdadeiro. O capítulo seguinte apresenta a interface com que o operador percorre esses fluxos.

5 Interface do Sistema

Conforme a estrutura apresentada na introdução, este capítulo descreve a interface do sistema como o operador e o administrador a utilizam no dia a dia. Primeiro, são apresentados os princípios de projeto da interface homem-máquina (IHM) que orientaram as telas, em especial os da norma ISA-101. Em seguida, percorre-se a estrutura geral da aplicação e cada grupo de telas, do acesso e da administração da plataforma ao cadastro das instalações, da supervisão em tempo real à análise de alarmes, comandos e histórico, ligando cada tela ao requisito que ela atende. Por fim, uma tabela resume quais cargos acessam cada tela. Todos os nomes de menu, campos e ações citados são os que aparecem na interface, em português do Brasil.

5.1 Princípios de interface adotados

A interface foi projetada a partir das recomendações para sistemas de supervisão de alto desempenho, que pedem telas sóbrias, nas quais a atenção do operador seja conduzida ao que está fora do normal e não disputada por ornamentos (International Society of Automation, 2015; Elipse Software, 2026). Três princípios dessa linha foram adotados de forma explícita e se repetem em todas as telas apresentadas neste capítulo.

O primeiro princípio é o fundo neutro. O sistema usa um único tema claro, com a barra superior em grafite, o conteúdo sobre cinza claro e os blocos de informação em cartões brancos. Valores normais, equipamentos em operação e rótulos de apoio aparecem em tons de cinza, sem marca de cor, de modo que uma tela em regime normal parece quieta e uma tela com problema chama a atenção por contraste (International Society of Automation, 2015).

O segundo princípio é o uso da cor apenas para a condição anormal. Cor aparece somente em quatro situações: alarme, aviso, dado desatualizado e seleção ou foco do usuário. Os quatro níveis de alarme do sistema (Crítico, Alto, Médio e Baixo) são fixos e cada um tem cor, ícone e nome escrito, de maneira que a cor nunca é o único sinal de uma condição (International Society of Automation, 2015). Valores com leitura falha ou velha são distinguidos por ícone, texto e itálico, e nunca por substituição silenciosa por zero.

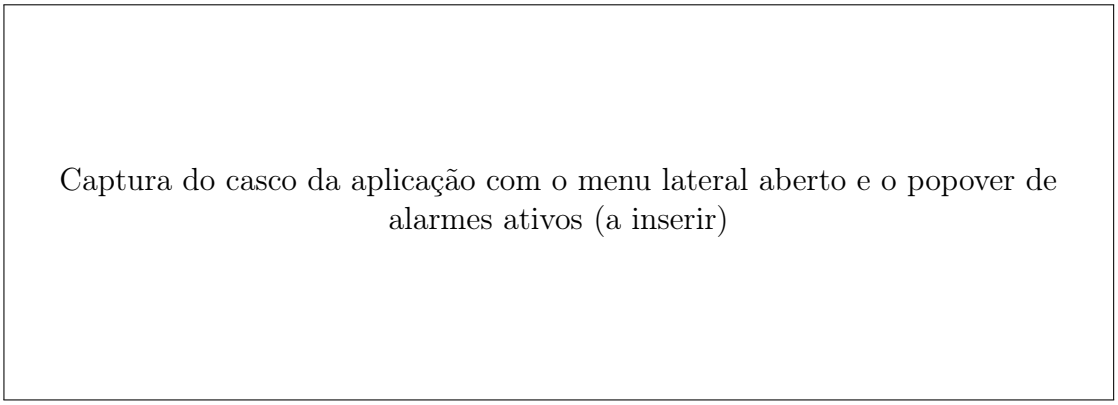
O terceiro princípio é a hierarquia de telas, que organiza a navegação do geral para o particular. No primeiro nível está a visão geral, formada pelo Mapa das plantas e pelo indicador de alarmes, sempre visível no topo da aplicação. No segundo nível estão as telas de área, que são o Dashboard e o Sinótico de cada planta. No terceiro nível estão as telas de detalhe, como a Lista de acompanhamento, o diálogo de comando, o gráfico de tendência de uma tag e o detalhe de uma ocorrência de alarme. No quarto nível, reservado a quem configura o sistema, ficam o cadastro, os eventos e os scripts. O operador desce da visão geral ao detalhe em poucos cliques e o caminho inverso é sempre oferecido por um comando de voltar (International Society of Automation, 2015).

Além dos três princípios, a consistência entre telas é garantida por um conjunto único de componentes visuais reutilizáveis, desenvolvidos sobre a biblioteca React e o *framework* Next.js (Meta Platforms, Inc., 2026; Vercel, Inc., 2026). Toda tela de lista, formulário ou detalhe nasce do mesmo modelo, com cabeçalho, barra de filtros e estados de carregamento, vazio e erro idênticos. Todas as funções podem ser executadas apenas com o teclado, o foco é sempre visível e o texto da interface segue um vocabulário único, em que, por exemplo, “Excluir” apaga um registro, “Cancelar” muda um estado e “Remover” desfaz um vínculo.

5.2 Estrutura geral da aplicação

Toda tela depois do acesso é montada dentro de um casco comum, mostrado na Figura 24. À esquerda fica o menu lateral, que lista apenas os itens que o cargo e as permissões do usuário permitem abrir. Na barra superior ficam o logotipo do cliente, o indicador de alarmes e o menu do usuário, com as opções "Meu perfil", "Sair" e "Sair de todas as sessões". Quando a Movi entra em um cliente para dar suporte, aparece também a opção "Sair do cliente". Esse desenho atende ao requisito de acesso por navegador, sem instalação local (RNF13), e ao controle de acesso por cargo e grupo (RF06), pois o menu é filtrado antes mesmo de qualquer tela ser requisitada à API.

Figura 24: Casco da aplicação com menu lateral e indicador de alarmes



Fonte: Autor.

Inserir captura: tela do Mapa (ou Início) logado como administrador do cliente, menu lateral completo visível, popover do indicador de alarmes aberto com ao menos um alarme Alto não reconhecido (por exemplo, nível baixo do tanque da bancada) e o nome do usuário com o cargo no canto superior.

O menu lateral segue a ordem da hierarquia de telas. Aparecem primeiro Mapa, Início, Dashboard, Sinótico, Lista de acompanhamento, Comandos, Alarmes e Linha do tempo, que são os itens de supervisão e dependem das permissões de ver telas, ver valores das tags, ver comandos e ver alarmes. Em seguida aparecem os itens de configuração, exclusivos do administrador do cliente: Plantas, Usuários, Grupos, Imagens, Telas, Pastas de telas, Configuração de alarmes, Eventos e Scripts. O item Usuários é visível também ao supervisor, que gere somente operadores. O item Clientes é exclusivo do administrador da Movi. O item "Meu perfil" é comum a todos.

O indicador de alarmes, no topo, mostra um sino com o nível mais grave entre os alarmes ainda não reconhecidos e o número deles. O sino pulsa apenas quando existe alarme Crítico ou Alto não reconhecido, e esse é praticamente o único elemento animado da tela fora dos desenhos de processo. Ao clicar, abre-se um painel com os alarmes ativos mais graves e um atalho para a tela de Alarmes. Se os dados do indicador estiverem velhos por falha de rede, um relógio é exibido junto ao sino, em vez de manter uma contagem que pareça atual. Esse comportamento cumpre o requisito de sinalização de alarmes (RF18).

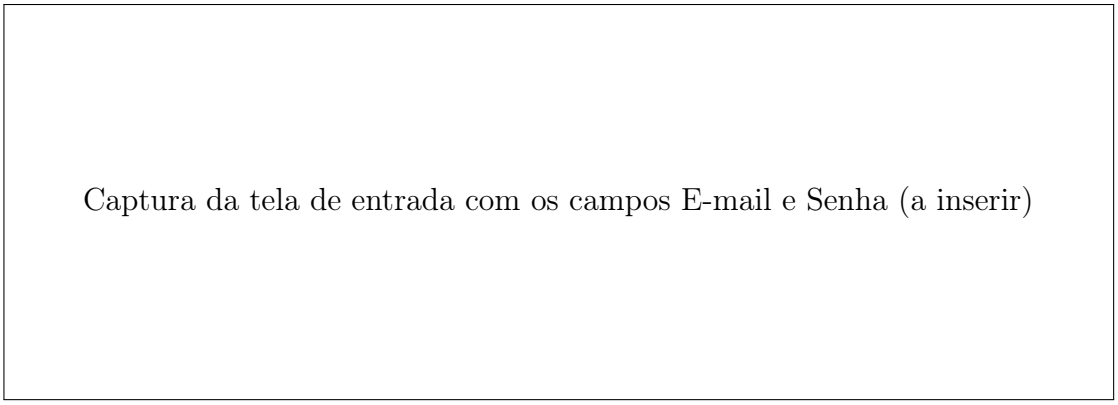
Todas as telas que mostram valores ao vivo exibem, acima do conteúdo, uma faixa de estado da conexão. Quando a rede oscila, a faixa informa "Conexão instável. Os valores podem estar desatualizados." e, quando o serviço não responde, informa "Serviço indisponível. Os valores mostrados estão desatualizados." Em ambos os casos o último valor continua na tela, marcado como desatualizado, para que o operador nunca confunda um dado antigo com um

dado corrente. Os valores são atualizados por *polling* curto ao valor mantido em memória no servidor, com um período único por cliente, entre 1 e 10 s, o que atende ao requisito de baixa latência na tela (RNF01).

5.3 Entrar e trocar a senha

A Figura 25 apresenta a tela de entrada, a única acessível sem sessão. Ela pede apenas o e-mail e a senha, e o botão "Entrar" envia as credenciais. Em caso de erro, a mensagem é a mesma para e-mail inexistente e para senha errada, para não revelar quais e-mails têm conta. Após cinco erros seguidos o acesso é bloqueado por 15 minutos. A sessão resultante usa um token de acesso de curta duração mantido apenas em memória e um *cookie* de renovação protegido, o que atende ao requisito de autenticação e gestão de sessões (RF05) (Jones; Bradley; Sakimura, 2015).

Figura 25: Tela de entrada



Fonte: Autor.

Inserir captura: rota /login, com o formulário vazio e o título "Entrar". Se possível, uma segunda captura com a mensagem de credenciais inválidas.

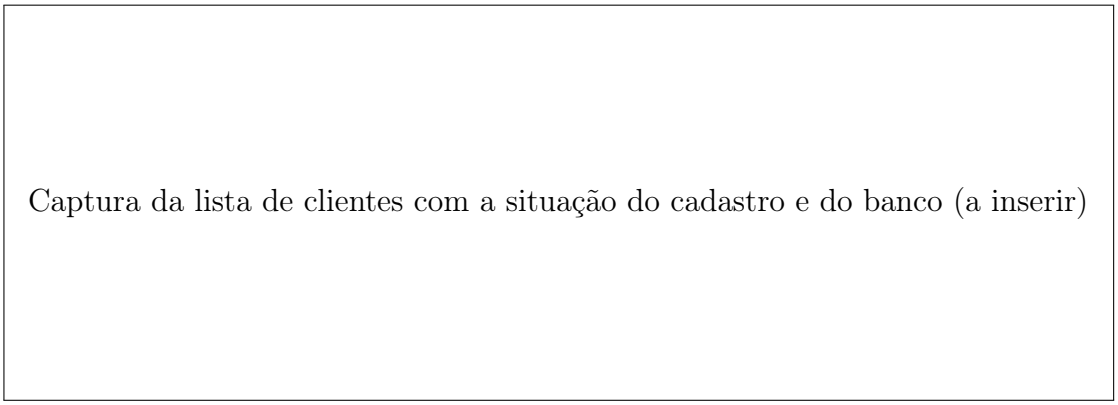
O sistema não envia e-mail, portanto não há convite nem "esqueci minha senha". O administrador do cliente cria o usuário e o sistema gera uma senha de uso único, mostrada uma só vez a quem criou, que vale por 24 horas e serve apenas para entrar. No primeiro acesso, a tela "Trocar senha" (rota `/trocar-senha`), com os campos "Senha atual", "Nova senha" e "Confirmar a nova senha", é a única que o servidor permite abrir até a troca ser concluída. A senha escolhida pela pessoa deve ter no mínimo oito caracteres. A mesma troca está disponível depois, na tela "Meu perfil", que também permite alterar os dados pessoais e o e-mail. Alterar a senha ou o e-mail encerra as demais sessões da pessoa.

5.4 Administração da Movi: clientes

O administrador da Movi, cargo que opera a plataforma e não um cliente em particular, enxerga apenas o item Clientes, que lista todos os clientes cadastrados com nome, situação e data de criação, como mostra a Figura 26. A busca aceita nome, identificador curto (*slug*) ou CNPJ. Cada linha oferece as ações de ver o cliente, entrar nele e atualizar seu banco. As situações possíveis do cadastro são "Cadastro incompleto", "Criando o banco", "Ativo", "Desabilitado" e "Excluindo", e a situação do banco vai de "Atualizado" a "Banco desatualizado" e

“Falha ao atualizar”. Esse conjunto de telas materializa o isolamento por cliente, com banco de dados próprio para cada um (RF12 e RNF05).

Figura 26: Lista de clientes da administração da Movi

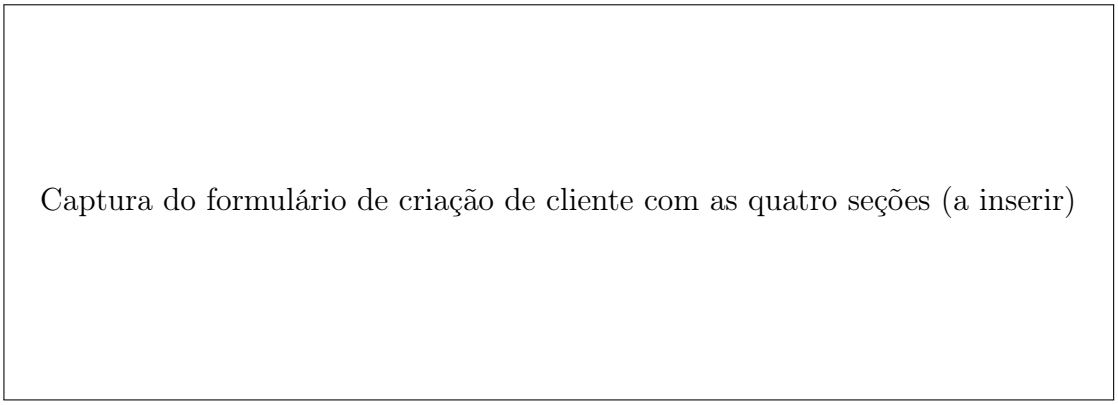


Fonte: Autor.

Inserir captura: rota /admin/clientes logada como administrador da Movi, com pelo menos dois clientes de exemplo (um Ativo e outro Desabilitado) e a busca visível.

A criação de um cliente, rota `/admin/clientes/novo` e mostrada na Figura 27, é um único formulário dividido em seções. Em “Dados do cliente” informam-se nome, *slug* e CNPJ, sendo o *slug* sugerido a partir do nome e imutável depois da criação. Em “Parâmetros do contrato” definem-se o período de atualização da tela, o teto de retenção do histórico, o limite de linhas de histórico por dia e a retenção do histórico de alarmes, valores que só a Movi altera. Em “Primeiro administrador” informam-se o nome e o e-mail de quem vai administrar o cliente. Em “Banco de dados” escolhe-se onde fica o banco, na “Instância da Movi” ou no “Servidor do cliente”. Nesse segundo caso aparecem os campos de conexão (servidor, porta, banco, usuário, senha e segurança da conexão) e o teste de conexão, que verifica o acesso e as permissões antes de gravar. O botão “Criar cliente” aciona o provisionamento do banco do cliente em segundo plano, o que é acompanhado pela situação “Criando o banco” na lista.

Figura 27: Formulário de criação de cliente



Fonte: Autor.

Inserir captura: rota /admin/clientes/novo preenchida com um cliente de exemplo (por exemplo, “Saneamento Demo”), banco na Instância da Movi e o administrador de exemplo.

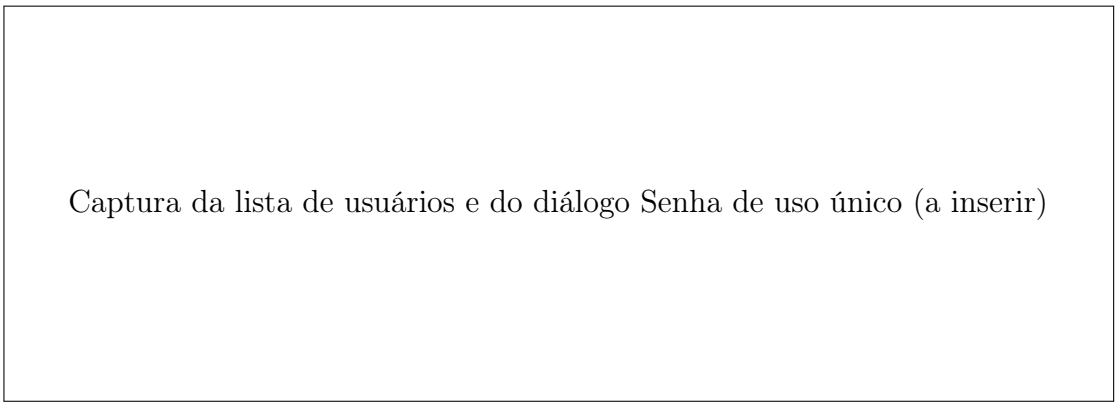
A tela de detalhe do cliente (rota `/admin/clientes/ver`) reúne dados, contrato, estado do banco, logotipo e ações. Entre as ações estão "Entrar no cliente", "Desabilitar" e "Habilitar", "Atualizar bancos pendentes" e "Excluir cliente", esta última protegida por uma confirmação em que se digita o *slug* do cliente. Ao entrar em um cliente, o administrador da Movi passa a ver as telas dele com os poderes de administrador do cliente, e o menu do usuário mostra "Sair do cliente" para voltar.

5.5 Usuários, cargos e grupos

O controle de acesso combina quatro cargos fixos com grupos de permissões criados pelo próprio cliente. Os cargos são o administrador da Movi, o administrador do cliente, o supervisor e o operador. As permissões vêm de um catálogo fixo de dez itens: ver telas, ver valores das tags, ver histórico, ver alarmes, reconhecer alarme, reconhecer todos os alarmes, ver comandos, comandar estado digital, comandar valor de referência e cancelar comandos. Quem pertence a vários grupos soma as permissões, e o administrador do cliente tem todas pelo cargo (RF06 e RNF05).

A tela "Usuários", na Figura 28, lista nome, e-mail, cargo e situação ("Ativo", "Desabilitado", "Senha de uso único" ou "Senha de uso único vencida"). O administrador cria usuários de qualquer cargo e o supervisor cria, edita, desabilita e exclui apenas operadores. No formulário de usuário informam-se nome, e-mail, telefone, cargo e grupos. Ao salvar um novo usuário, ou ao escolher a ação de gerar nova senha, um diálogo "Senha de uso único" mostra a senha uma única vez, com a advertência "Anote agora: ela não aparece de novo." e um botão para copiá-la.

Figura 28: Tela de usuários com cargos e situações

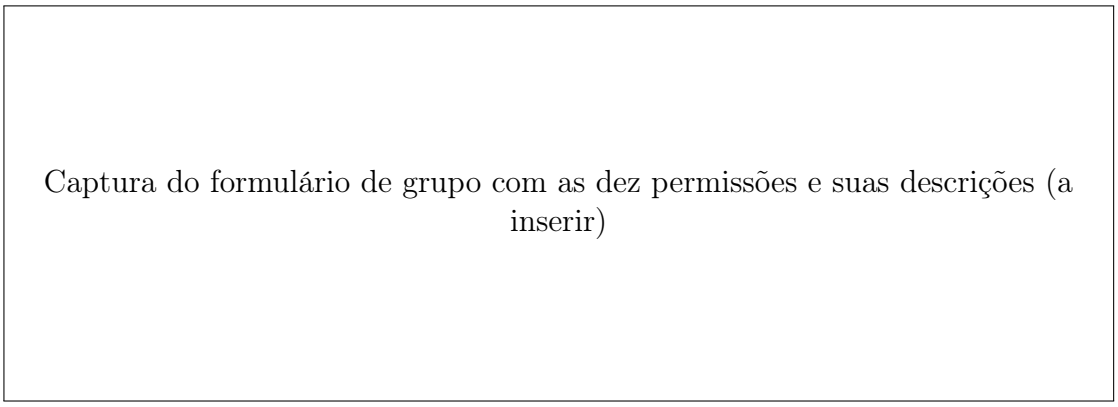


Fonte: Autor.

Inserir captura: rota /configuracao/usuarios com um administrador, um supervisor e dois operadores, e, sobreposto ou em figura ao lado, o diálogo "Senha de uso único" com uma senha de exemplo no formato de três grupos de quatro caracteres.

A tela "Grupos", na Figura 29, lista os grupos do cliente com nome e identificador. O formulário de grupo pede um nome e apresenta o catálogo de dez permissões como caixas de seleção, cada uma com o rótulo e uma frase explicando o que ela libera, por exemplo "Ver telas", que abre o mapa, o dashboard e o sinótico. Como a permissão de um grupo vale para o cliente inteiro, não há filtro por planta ou por área, o que simplifica o modelo e mantém a única fronteira rígida no cliente.

Figura 29: Formulário de grupo com o catálogo de permissões



Fonte: Autor.

Inserir captura: rota /configuracao/grupos/novo (ou editar) com um grupo de exemplo "Operação" que marca Ver telas, Ver valores das tags, Ver alarmes, Reconhecer alarme e Comandar estado digital.

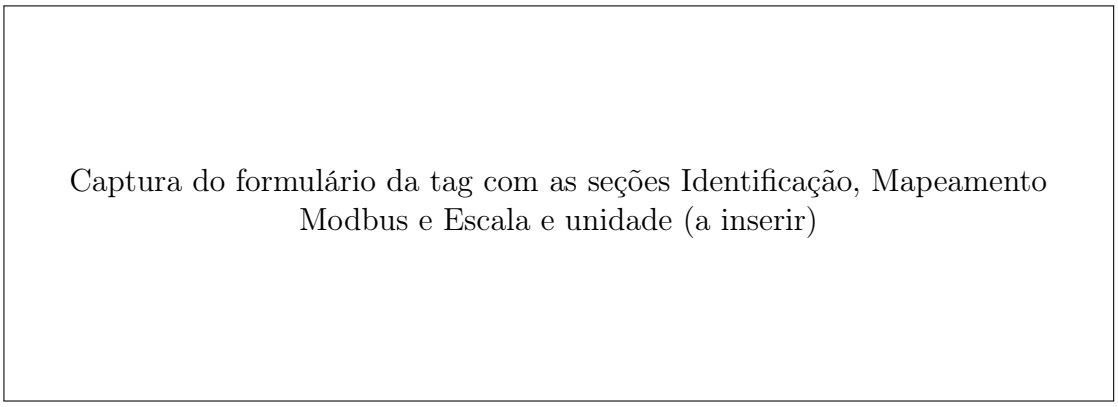
5.6 Cadastro: plantas, dispositivos de E/S, tags e pastas

O cadastro segue a hierarquia cliente, planta, dispositivo de E/S e tag, com uma árvore de pastas separada, apenas de tags, em cada planta (RF11). O administrador do cliente é o único que a edita, e toda alteração vale com o sistema em operação, pois a aquisição recarrega somente o que mudou (RF10). A tela "Plantas" lista, para cada instalação, nome, *slug*, coordenadas, quantidade de dispositivos e situação. O formulário de planta pede nome, *slug*, latitude e longitude, e a exclusão exige digitar o *slug* para confirmar, já que apaga tudo o que a planta contém. A planta só organiza os dispositivos e posiciona o pino no mapa, e não controla o acesso.

De uma planta chega-se à lista de "Dispositivos de E/S", que mostra nome, protocolo (Modbus TCP ou interno), endereço, situação da comunicação e situação de cadastro. O formulário do dispositivo Modbus reúne, em seções, a identificação, a "Conexão" (endereço, porta e a opção de encapsulamento, que permite usar o protocolo RTU sobre TCP), os "Tempos e tentativas", os "Limites do pedido" e a "Escrita" (Modbus Organization, 2012a). Como o mapeamento é feito por configuração, equipamentos de fabricantes diferentes são incluídos sem alteração de código (RNF09). O dispositivo interno não tem conexão, pois seus valores são gerados pelo próprio sistema.

A tela de uma tag, mostrada na Figura 30, é dividida em seções que aparecem conforme o protocolo. A seção "Identificação" recebe nome, *slug*, descrição, pasta, tipo (analógica, digital, texto ou multiestado), acesso (leitura, escrita ou leitura e escrita) e a chave "Habilitada". A seção "Mapeamento Modbus" recebe o tipo de registrador (bobina, entrada discreta, registrador de entrada ou registrador de retenção), o endereço, o tipo bruto e a ordem de bytes. A seção "Escala e unidade" define a conversão do valor bruto para a grandeza de engenharia, com os limites bruto e de engenharia, a unidade e as casas decimais, que é a linearização apresentada na fundamentação. As seções "Exibição", "Varredura", "Histórico", "Eventos" e "Comando" definem, respectivamente, o formato mostrado, o tempo de varredura com banda morta, o modo de gravação do histórico e sua retenção, os eventos de alarme da tag e a permissão e os limites do comando. Esse formulário atende às necessidades de aquisição periódica por *polling* (RF01) e de registro do histórico conforme o tipo de dado (RF02).

Figura 30: Formulário de uma tag analógica Modbus

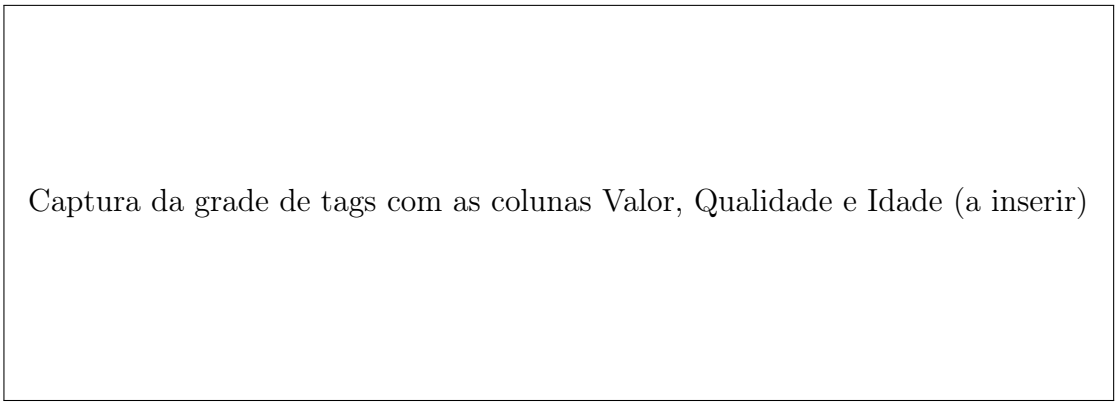


Fonte: Autor.

Inserir captura: rota /cadastro/tags/editar de uma tag analógica real da bancada (por exemplo, o nível do tanque), com registrador de retenção, tipo bruto inteiro de 16 bits sem sinal, escala de 0 a 4095 em bruto para 0 a 100 em engenharia e a unidade em porcentagem. Ajustar os números aos da bancada real.

Para cadastrar dezenas ou centenas de pontos de uma vez, a lista de tags de cada dispositivo oferece a ação "Editar em grade", ilustrada na Figura 31. A grade apresenta todas as tags do dispositivo em uma tabela semelhante a uma planilha, ordenadas por tipo de registrador e endereço, e aceita colar blocos copiados do Excel, editar célula a célula, duplicar uma linha com o número do nome e o endereço avançando automaticamente e desfazer com o teclado. Células com erro ficam em vermelho, como endereço fora da faixa ou combinação de registrador impossível, e células com aviso ficam em amarelo, como endereços sobrepostos. O salvamento é tudo ou nada, em uma única gravação seguida de uma única recarga da aquisição. Três colunas finais, somente de leitura, mostram o valor ao vivo já formatado com a unidade, a qualidade ("Bom", "Desatualizado", "Falha de comunicação" ou "Sem dado") e a idade da leitura, o que permite conferir o mapeamento Modbus enquanto ele é digitado. Quando não há valor, a célula exibe um traço, nunca zero.

Figura 31: Grade de tags com valor ao vivo



Fonte: Autor.

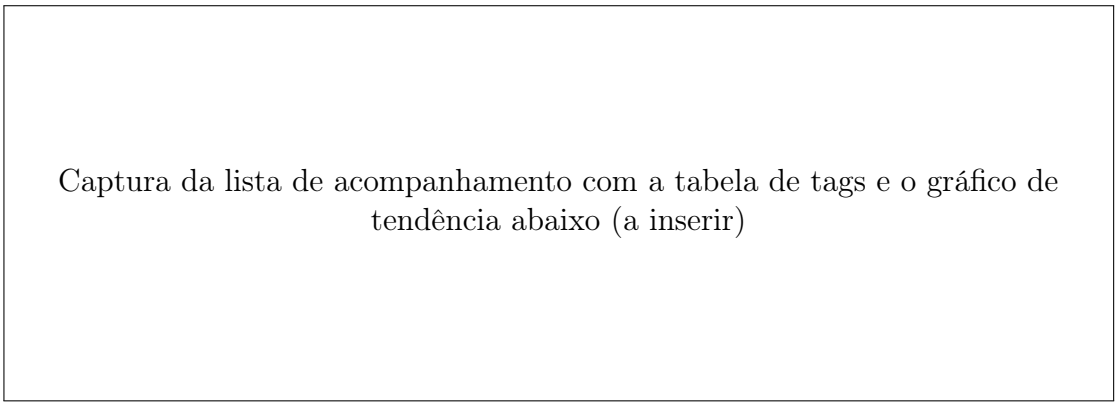
Inserir captura: rota /cadastro/dispositivos/tags/grade do dispositivo da bancada com ao menos 8 tags (nível, vazão, bombas, alarmes), as colunas Valor, Qualidade e Idade visíveis com valores reais e uma célula marcada com erro e outra com aviso, para ilustrar a validação.

A tela "Pastas" (rota `/cadastro/pastas`) mostra a árvore de pastas de tags da planta escolhida. As pastas têm apenas nome, não têm limite de níveis e podem reunir tags de dispositivos diferentes da mesma planta, o que reproduz a organização por equipamento sem criar um objeto de equipamento. As ações "Nova pasta", mover e excluir operam diretamente na árvore e a tag aparece na raiz ou em uma pasta apenas. As pastas organizam também a escolha de tags nos seletores de outras telas.

5.7 Lista de acompanhamento

A Lista de acompanhamento é a tela de detalhe mais usada pelo operador para acompanhar poucos pontos de forma contínua, de maneira semelhante às listas de observação dos sistemas supervisórios clássicos. Uma lista é um conjunto nomeado de até 50 tags, de até 10 plantas, criado e mantido pelo administrador do cliente e liberado a um ou mais grupos. O operador vê apenas as listas dos grupos a que pertence e não pode alterá-las, o que a tela informa com a mensagem "Só o administrador altera esta lista." A Figura 32 mostra a tela.

Figura 32: Lista de acompanhamento com valores ao vivo e gráfico



Fonte: Autor.

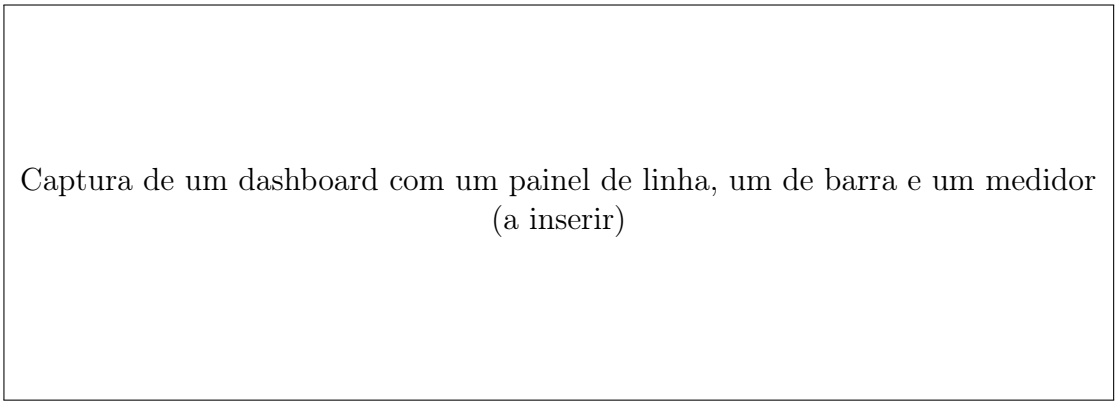
Inserir captura: rota /lista-de-acompanhamento com uma lista "Bancada" de 6 a 8 tags (nível, vazão, estados das bombas, uma tag em falha de comunicação), valores reais com unidade, qualidade e o gráfico de tendência das tags marcadas logo abaixo da tabela.

Cada linha mostra a tag, a planta, o valor atual formatado, a qualidade e a idade do dado. Um valor desatualizado aparece em cinza e itálico com o instante desde o qual está velho, e uma falha de comunicação aparece com o último valor conhecido acompanhado de um triângulo de alerta, nunca com um zero fictício. Tags que o usuário pode comandar exibem um botão de comando, que abre o diálogo de quatro fases descrito adiante. Abaixo da tabela, um gráfico mostra a tendência das tags que o administrador marcou, e cada linha oferece "Ver gráfico", que abre a tendência daquela tag em um diálogo. A tela atende à exibição de valores em tempo real (RF15 e RNF01) e ao comando a partir da supervisão (RF07).

5.8 Dashboard

O Dashboard é a tela de área em que o administrador monta, na própria interface, painéis de tendência e de situação com gráficos, e na qual o operador apenas observa. Os gráficos são desenhados com a biblioteca ECharts (Apache Software Foundation, 2026). A Figura 33 mostra um dashboard em execução com os três tipos de painel disponíveis.

Figura 33: Dashboard em execução com painéis de linha, barra e medidor



Fonte: Autor.

Inserir captura: rota /dashboard abrindo o dashboard “Bancada” com um painel de linha (nível do tanque nas últimas horas), um de barra (volume ou média por intervalo) e um medidor (nível atual dentro da faixa). Em regime normal, tudo em cinza, e um dos painéis com a cor de alarme para ilustrar o princípio da cor apenas para o anormal.

O painel de linha mostra, na janela escolhida, a evolução de até 12 tags. Em janelas de até uma hora cada ponto é a leitura bruta, e em janelas maiores o gráfico usa médias de 1 min, 10 min, 30 min ou 6 h, acompanhadas da faixa de mínimo e máximo, para que um pico curto não desapareça na média. A linha é interrompida onde falta dado, para que a lacuna seja visível, e a janela pode chegar a dez anos. O painel de barra mostra, por intervalo, a média, o mínimo, o máximo, o volume ou o tempo ligado, e o medidor mostra o valor atual dentro de uma faixa. Quando o equipamento deixa de responder, o medidor exibe o último valor com um triângulo de alerta e o instante da falha, ou apenas um traço, conforme a escolha do autor do painel. Cada painel tem uma tabela alternativa aos dados, para quem não usa o gráfico. Depois da primeira carga, o painel pede ao servidor apenas o trecho novo e o junta ao que já tem, o que limita o tráfego e a carga sobre o banco.

A montagem é feita na tela “Montar dashboard” (rota `/telas/dashboard`), aberta pela ação “Montar” na lista de telas. O administrador cria seções e painéis, escolhe o tipo (“Linha”, “Barra” ou “Medidor”) e as tags, e ajusta título, período, atualização, faixas de cor e linhas de meta. Arrastar e redimensionar os painéis tem alternativa por clique e por teclado, e o resultado pode ser pré-visualizado antes de publicado. Cada tela tem um rascunho e uma versão publicada, e o operador nunca vê o rascunho. Se outra pessoa salvou antes, a tela avisa e oferece recarregar sem apagar o trabalho. Essa montagem sem alterar o código cumpre o requisito de incluir telas com o sistema em produção (RF10).

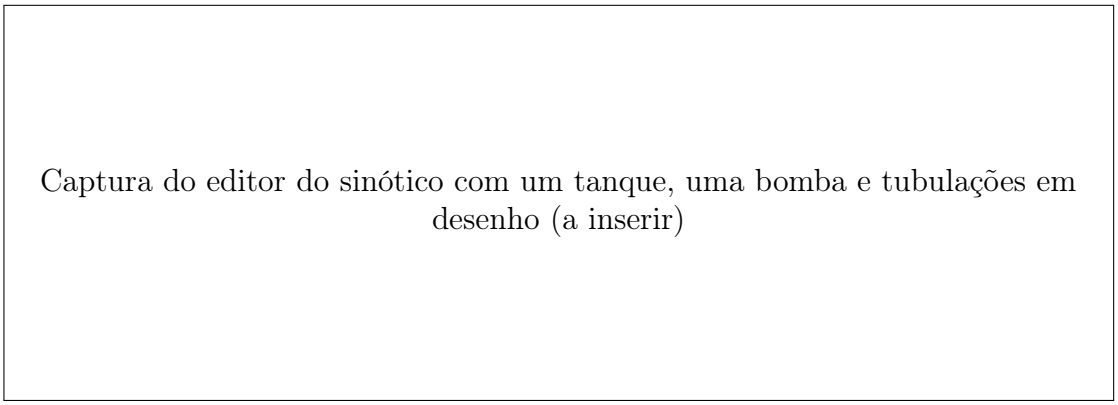
5.9 Sinótico

O Sinótico é a tela de área que representa o processo por meio de um desenho vetorial animado pelos dados em tempo real, e é o equivalente moderno do esquema sinótico das salas

de controle. Como o Dashboard, ele tem uma face de execução, usada pelo operador, e uma face de edição, usada pelo administrador do cliente. Ambas atendem ao requisito de exibição das telas de supervisão (RF16).

Na edição, aberta pela ação "Editar tela" em "Telas" (rota `/telas/editor`), o administrador desenha o processo no navegador, como mostra a Figura 34. À esquerda está a "Paleta" de objetos, agrupados em formas, texto, imagens, tubulações, valores, controles e outros, em um total de dezessete tipos, como retângulos, textos, tubulações com fluxo animado, cartões de valor, barras de nível, indicadores luminosos e botões. Ao centro fica a área de desenho, com grade, ímã, réguas, guias e zoom. À direita está o inspetor do objeto selecionado, com as abas "Aparência", "Dados" e "Animações", e as seções "Camadas" e "Tags" da lateral. É possível agrupar, alinhar, distribuir, desfazer e refazer, e todas as ações também funcionam só com o teclado. A biblioteca de imagens do cliente, gerida em "Imagens", fornece figuras de equipamentos.

Figura 34: Editor do sinótico com paleta, área de desenho e inspetor



Fonte: Autor.

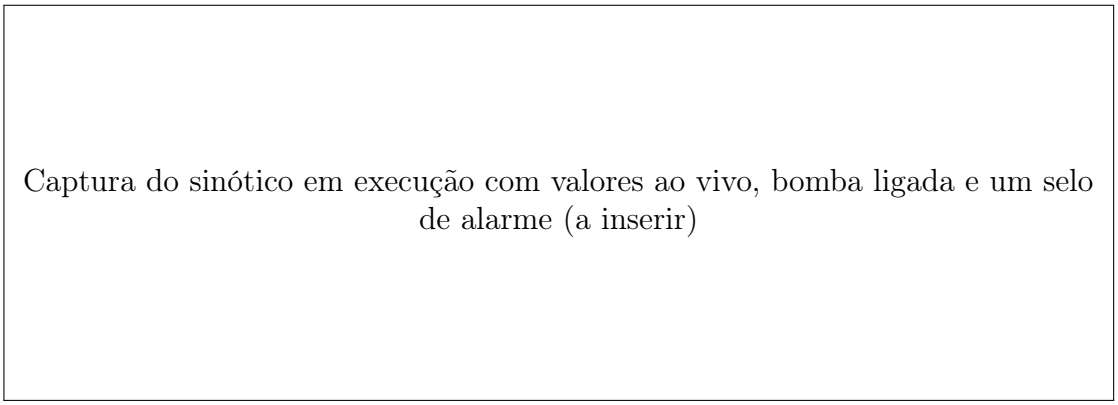
Inserir captura: rota /telas/editor com o sinótico da bancada em edição (tanque com barra de nível, bomba, tubulação, valor de nível e botão de comando), um objeto selecionado e a aba Dados do inspetor aberta mostrando o vínculo da propriedade com a tag.

O vínculo entre desenho e dado é feito na aba "Dados", na qual uma propriedade do objeto é ligada a uma tag. O vínculo pode ser simples, em tabela (valor ou faixa convertidos em texto ou cor) ou de comando, que apenas abre o diálogo de comando. A aba "Animações" permite cor por tabela, visibilidade, preenchimento proporcional, rotação, translação e piscar. Arrastar uma tag do painel "Tags" até o desenho cria o objeto de valor já vinculado. Salvar é uma ação manual e publicar pede confirmação, e uma pré-visualização mostra exatamente o que o operador verá. Um objeto com defeito não derruba o desenho, pois aparece como "Objeto inválido" enquanto o restante da tela continua funcionando.

Na execução, o operador abre o sinótico pelo item Sinótico do menu, que lista as telas por pasta, como mostra a Figura 35. A tela já abre com os valores atuais. Cada objeto de valor exibe o dado formatado conforme a tag e uma marca de qualidade: desatualizado em cinza e itálico com o instante de origem, falha de comunicação com o último valor e um triângulo de alerta, "Sem dado" e "Sem acesso". Um selo de alarme aparece sobre o objeto da tag em alarme. Se o sistema não recebe valor para um objeto em 10 s, o objeto mostra um traço e a indicação "Sem dado", e o valor desenhado no editor nunca é mostrado no lugar do real. A barra de controles do desenho oferece "Ampliar", "Reduzir", "Ajustar" e "Pausar animações", com zoom de 1 a 4 vezes, e a lista "Valores desta tela" apresenta, em forma de tabela, os objetos, valores,

situações e ações da tela, o que serve de caminho alternativo para quem não usa o desenho. Botões de comando abrem o diálogo de quatro fases e botões de navegação levam a outras telas, formando a hierarquia de telas.

Figura 35: Sinótico em execução com valores ao vivo e objeto em alarme



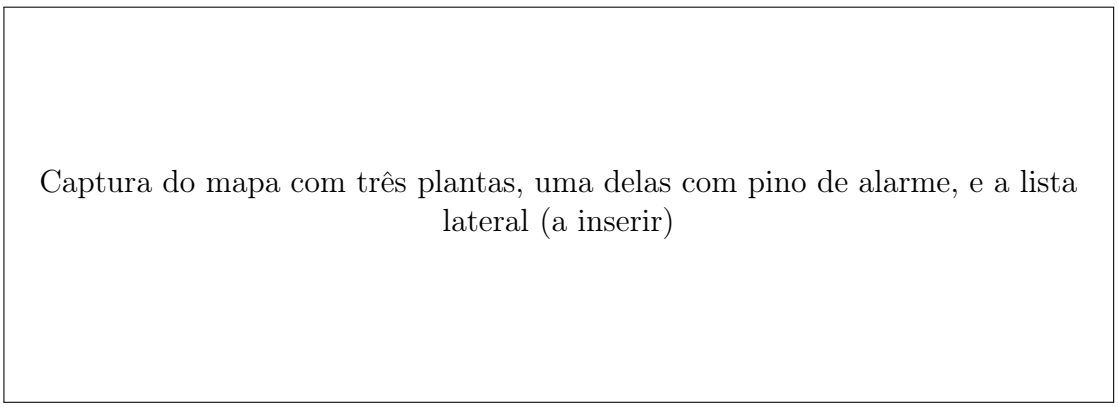
Fonte: Autor.

Inserir captura: rota /telas/ver (aberta a partir de /sinotico) do mesmo desenho da bancada em execução, valores reais, bomba ligada animada, o selo de alarme sobre o tanque com nível Alto e, ao lado, a lista "Valores desta tela" aberta.

5.10 Mapa das plantas

O Mapa, primeiro item do menu, é a visão geral do cliente e o topo da hierarquia de telas. A Figura 36 mostra um pino por planta com coordenada. O pino de uma planta sem alarme aberto é neutro, e o de uma planta com alarme assume a cor do alarme mais grave dela e traz o nome do nível escrito (Crítico, Alto, Médio ou Baixo), de modo que a cor nunca é o único sinal. A planta desabilitada aparece indicando que não é monitorada. Uma lista lateral repete todas as plantas, inclusive as sem coordenada, agrupadas em "Sem coordenada", e é o caminho completo para quem usa o teclado. Ao clicar em um pino ou em uma linha da lista, abre-se um resumo com nome, coordenadas, nível e quantidade de alarmes, e o atalho "Ver alarmes desta planta".

Figura 36: Mapa das plantas com o estado de alarme de cada uma



Fonte: Autor.

Inserir captura: rota /mapa com ao menos três plantas de exemplo em locais distintos, uma delas com pino colorido e o nível escrito, o resumo da planta aberto e a lista lateral com uma planta em "Sem coordenada".

O mapa de fundo vem do projeto OpenStreetMap, por meio da biblioteca Leaflet (Leaflet, 2026; OpenStreetMap Foundation, 2026). Como as imagens do mapa são buscadas de um serviço de terceiros, a tela informa, em uma nota fixa, que a região visível, o endereço de rede do usuário e o endereço do site são enviados a esse serviço, e há uma configuração que abre o mapa sem fundo, sem nenhuma chamada externa. Quem tem permissão de ver o mapa e não a de ver alarmes enxerga pinos neutros e a indicação "Alarmes indisponíveis", e a resposta do servidor sequer leva dados de alarme. A tela atende à exibição das telas de supervisão (RF17).

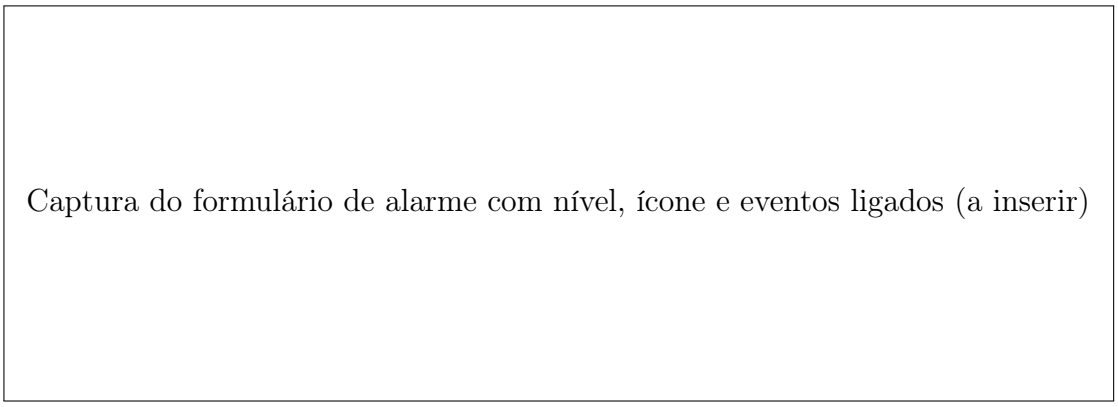
5.11 Alarmes e eventos

O tratamento de alarmes envolve duas famílias de telas, a de configuração, usada pelo administrador do cliente, e a de operação, usada por quem acompanha o processo. Os níveis de alarme seguem a prática da norma de gestão de alarmes da ISA, com quatro níveis fixos (International Society of Automation, 2016).

Na configuração, o ponto de partida é a tela "Eventos" (rota `/configuracao/eventos`), que lista os eventos do cliente com nome, tipo, situação e atrasos. Um evento é uma condição e pode ser de tag, composto ou agendado. O evento de tag compara a tag com um valor por um operador (maior, maior ou igual, menor, menor ou igual, igual, diferente, dentro ou fora de uma faixa), com banda morta e atrasos de ativação e de retorno de 0 a 3600 s. O evento composto combina outros eventos com E, OU e NÃO, e o evento agendado dispara em horários definidos, com uma prévia das próximas cinco execuções. Cada dispositivo Modbus traz ainda três eventos de sistema, de falha de comunicação, de falha de leitura e de falha de escrita. Um evento, por si só, não gera registro nem aviso. A seção de tratadores permite ligar a um evento um script, descrito adiante.

O alarme é um objeto separado, configurado na tela "Configuração de alarmes" (rota `/configuracao/alarmes`), que a Figura 37 apresenta. O formulário tem as seções "Identificação", com uma descrição de até 200 caracteres, "Nível e ícone", em que se escolhe um dos quatro níveis fixos e um ícone, "Planta" e "Eventos", em que se ligam de um a vinte eventos que, ao ficarem ativos, abrem o alarme. Essa separação permite que vários eventos acionem o mesmo alarme e que um evento acione vários alarmes. O sistema não cria alarmes sozinho, e quem quiser ver uma falha de comunicação como alarme cria um alarme comum e liga a ele o evento de sistema correspondente (RF18).

Figura 37: Formulário de configuração de um alarme

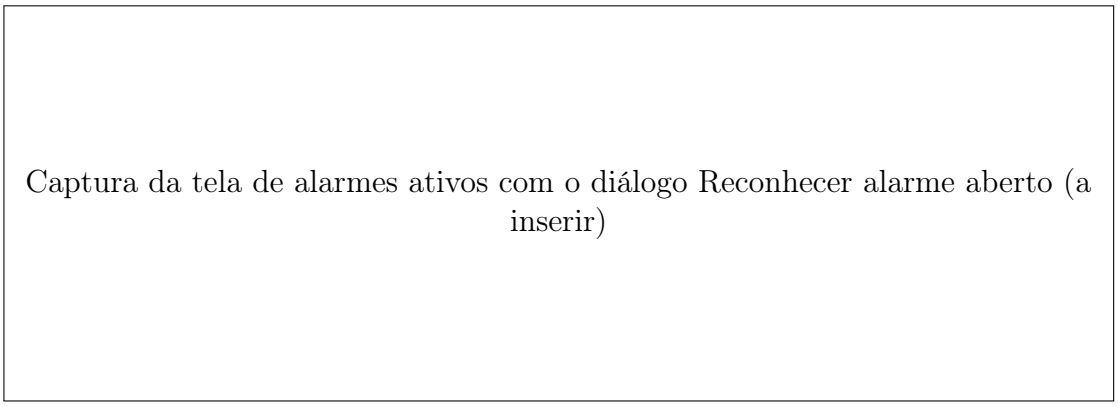


Fonte: Autor.

Inserir captura: rota /configuracao/alarmes/editar com o alarme “Nível alto do tanque” (nível Alto, ícone de triângulo), planta da bancada e dois eventos ligados (nível acima do limite e falha de comunicação do dispositivo).

Na operação, a tela “Alarmes” (rota `/alarmes`), mostrada na Figura 38, tem duas abas, “Ativos” e “Histórico”. A aba de ativos lista, para cada ocorrência, nível, descrição, planta, origens, instante de ativação (“Desde”), estado e quem reconheceu. Os estados são “Ativo, não reconhecido”, “Normalizado, não reconhecido”, “Reconhecido” e “Reconhecido depois de normalizar”, entre outros, cada um com cor e texto próprios. A aba de histórico traz as ocorrências já fechadas, com duração e motivo do fechamento, e os filtros “De”, “Até”, “Planta”, “Dispositivo”, “Tag”, “Situação” e “Faixa”. A ação “Reconhecer” abre um diálogo “Reconhecer alarme” com um campo “Comentário” opcional, e “Reconhecer todos” atua sobre os alarmes que a tela mostra, após uma confirmação. O reconhecimento é sempre referido à ocorrência que a pessoa viu, de modo que um novo disparo nunca é reconhecido por um clique dado em uma tela velha, e o registro guarda quem reconheceu, quando e com qual comentário (RNF06).

Figura 38: Tela de alarmes ativos com reconhecimento



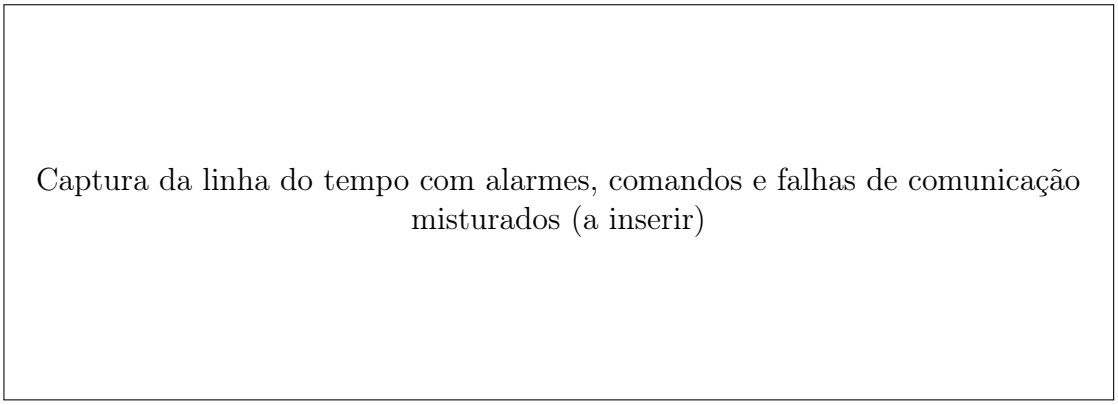
Fonte: Autor.

Inserir captura: rota /alarmes, aba Ativos, com três ocorrências de níveis diferentes (uma Crítica não reconhecida, uma Alta reconhecida, uma Média normalizada não reconhecida) e o diálogo “Reconhecer alarme” aberto com um comentário digitado, por exemplo, “Verificada a bomba em campo”.

5.12 Linha do tempo

A Linha do tempo, apresentada na Figura 39, é uma lista única, da mais nova para a mais velha, que reúne o que aconteceu com alarmes (disparo, reconhecimento, normalização e fechamento), com comandos (confirmado, executado, falho ou com resultado tardio) e com os dispositivos (início e fim de falha de comunicação, de leitura e de escrita). Cada linha traz data e hora, planta, origem, evento e o ator, que pode ser uma pessoa ou um script identificado pelo nome. Os filtros são "Período" (1 h, 24 h, 7 dias, 31 dias ou personalizado, até 31 dias), "Filtrar por planta", "Tags" e "Filtrar por origem", e a lista é carregada aos poucos, sem repetir nem perder linhas.

Figura 39: Linha do tempo de alarmes, comandos e eventos de dispositivo



Fonte: Autor.

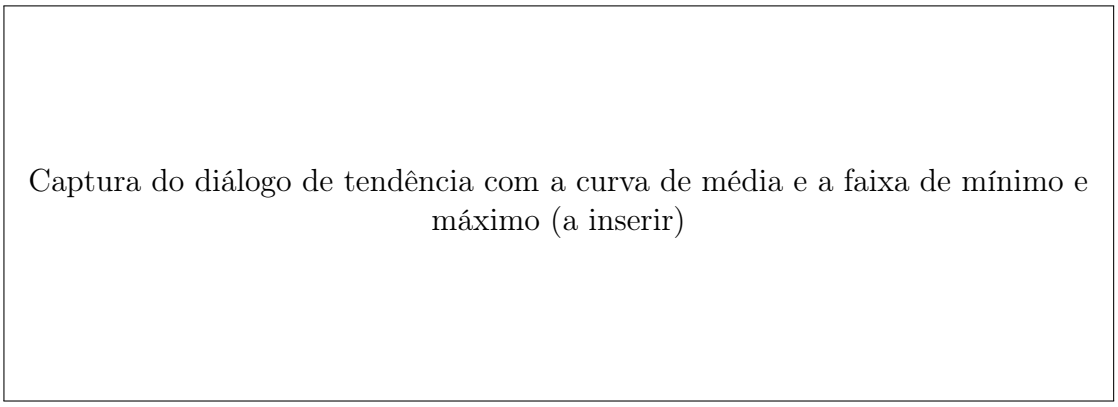
Inserir captura: rota /linha-do-tempo com o período de 24 h e linhas de origens distintas (um alarme disparado e reconhecido, um comando "Executado e confirmado pelo equipamento" e uma falha de comunicação do dispositivo e a volta ao normal), com o aviso fixo sobre a ordem do registro visível.

Cada origem só aparece para quem tem a permissão correspondente, e a tela avisa o que foi escondido. Um aviso permanente informa que a ordem exibida é a do registro do sistema e não a do relógio do equipamento, pois a linha do tempo é uma sequência cronológica de eventos e não promete a resolução de uma sequência de eventos com carimbo do equipamento. Esse é o recurso de consulta cronológica ao histórico (RF19), e a tela de comandos, descrita a seguir, completa a rastreabilidade das ações do operador (RNF06).

5.13 Histórico e tendência

O sistema não tem uma tela isolada de histórico. A consulta ao histórico de uma tag está incorporada às telas em que ela é útil, a Lista de acompanhamento e o Dashboard, e ambas usam o mesmo gráfico e a mesma regra de detalhe, o que garante coerência. A Figura 40 mostra o gráfico de tendência de uma tag, aberto pela ação "Ver gráfico" da lista de acompanhamento.

Figura 40: Tendência de uma tag em janela de sete dias



Fonte: Autor.

Inserir captura: diálogo “Gráfico da lista” (aberto em /lista-de-acompanhamento) com a janela de 7 dias para o nível do tanque, mostrando a curva de média, a faixa de mínimo e máximo e uma lacuna na curva, onde faltou dado.

O detalhe do gráfico é definido pela janela escolhida. Até uma hora, cada ponto é a leitura gravada. Até um dia, usa-se a média por minuto, até sete dias a média de dez minutos, até 31 dias a de trinta minutos e, acima disso, a média de seis horas, sempre acompanhada do mínimo e do máximo do intervalo para que um pico breve não desapareça. A curva cobre exatamente o período pedido e, onde falta dado, a linha é interrompida, de modo que a lacuna do histórico é visível e é, ela própria, a evidência de uma falha de leitura. Como cada tag grava o histórico de um modo (não gravar, gravar tudo ou gravar quando muda mais o registro periódico), o gráfico reflete o que foi realmente guardado (RF02 e RF17). Os valores guardados são sempre os de engenharia, e o que está além da retenção do cliente é tratado como inexistente.

5.14 Variável meta e scripts

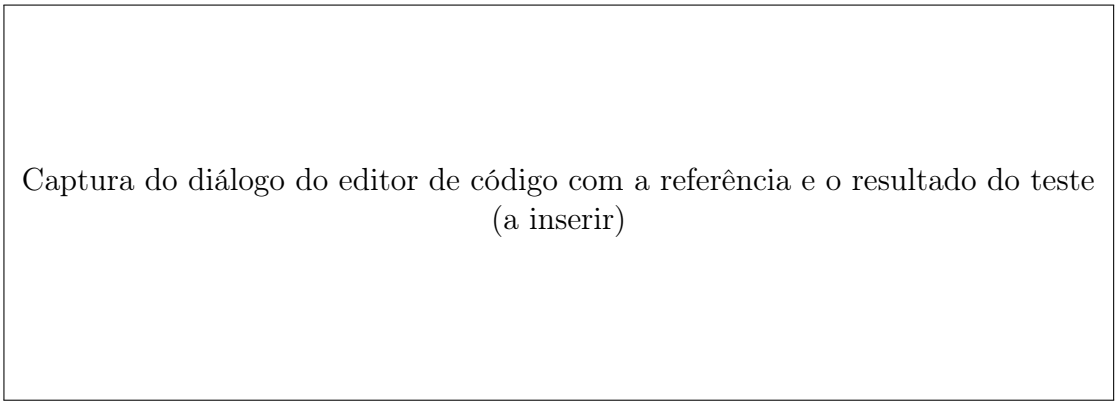
O sistema permite ao administrador do cliente escrever pequenos programas em JavaScript que calculam valores e reagem a eventos. Esse código é executado em um ambiente isolado, sem acesso à rede, a arquivos ou a segredos e com tetos de tempo e de memória, formado pelo interpretador QuickJS compilado para Wasm e executado sob o Wasmtime (Bellard; Gordon, 2026; Bytecode Alliance, 2026). Para o usuário, a diferença para um programa comum é pequena, pois o editor é o mesmo nos dois casos de uso, que são a variável meta e o script.

A variável meta é uma tag de um dispositivo interno cujo valor é calculado por um código que lê o valor atual de outras tags da mesma planta, como uma diferença de pressões ou um volume por hora. Na tela da tag, a seção “Variável meta” mostra o gatilho (a cada leitura nova de uma tag lida, ou por tempo) e a ação “Editar código”, que abre o diálogo “Código da variável meta”. O script, por sua vez, é configurado na tela “Scripts” (rota `/configuracao/scripts`), que lista nome, situação, eventos ligados e a última execução. O script é ligado a um ou mais eventos, para rodar “na ativação”, “no retorno” ou “nos dois”, e pode ler tags, escrever em tags, habilitar e desabilitar tags e dispositivos. A escrita em hora marcada se obtém ligando o script a um evento agendado. Todo comando escrito por um script é registrado com a origem “script” seguida do nome dele.

O editor de código, mostrado na Figura 41, abre em um diálogo e só é carregado quando necessário, para não pesar nas demais telas. Ele tem realce de sintaxe de JavaScript, numeração de linhas, a posição do cursor e um painel “Referência” com a lista das funções disponíveis e

uma frase sobre cada uma, por exemplo, “Último valor da tag” e “Escreve no dispositivo”. Pelo teclado, a tecla Tab indenta e as teclas Esc ou Ctrl+M liberam o foco para sair do editor, o que preserva a navegação por teclado. O botão “Testar” analisa o código sem executá-lo e mostra quais tags ele lê, e, no script, a área “Simular” mostra o que ele faria sem escrever nada. Ao salvar, o sistema confere se cada tag citada existe e se o autor pode usá-la. A tela de scripts exibe a “Última execução” e a “Última falha”, esta última mantida mesmo depois de uma execução bem-sucedida, para que um defeito intermitente não passe despercebido.

Figura 41: Editor de código da variável meta e do script



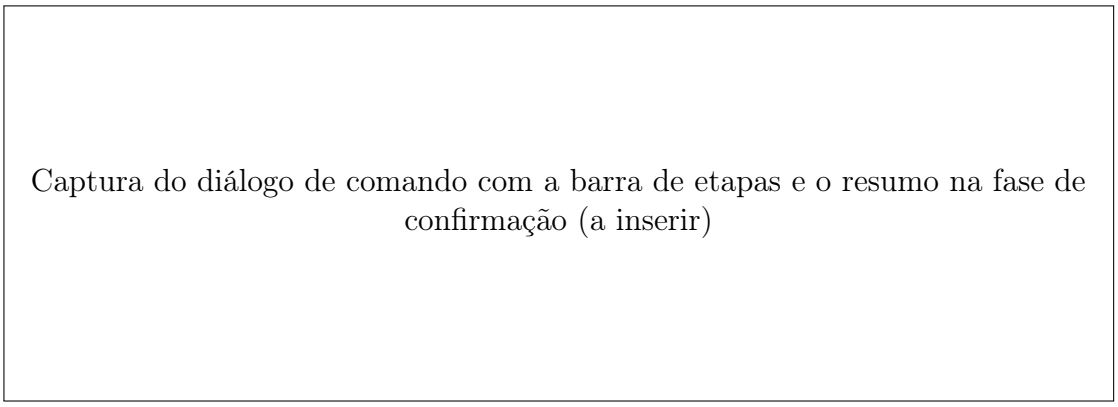
Fonte: Autor.

Inserir captura: diálogo “Código da variável meta” (a partir de uma tag interna em /cadastro/tags/editar) com um código curto de exemplo, por exemplo, a diferença entre duas pressões da bancada, o painel “Referência” aberto e o resultado do botão “Testar” listando as tags lidas.

5.15 Diálogo de comando de quatro fases

O comando é a única ação da interface que escreve no equipamento, e por isso passa por um diálogo próprio, reutilizado por todas as telas que oferecem comando, a Lista de acompanhamento e o Sinótico. O diálogo conduz o operador pelas quatro fases do comando, indicadas por uma barra de etapas, como mostra a Figura 42. Na “Seleção”, o operador escolhe a ação, “Ligada (1)” ou “Desligada (0)” para uma tag digital, ou digita o “Novo valor” para uma tag analógica, com o valor atual e o dispositivo visíveis. Na “Verificação”, acionada por “Verificar”, o servidor confere se a tag existe e aceita escrita, se o dispositivo está habilitado e comunicando, se o valor está dentro da faixa e se o usuário tem a permissão do tipo de comando. Aprovado, o comando vale um código de 120 s ligado exatamente ao que o operador viu. Na “Confirmação”, o operador revê o resumo e aciona “Confirmar”, e o servidor confere a permissão de novo. Na “Execução”, o comando é enviado ao equipamento e o diálogo mostra o “Resultado do comando”. Esse fluxo cumpre os requisitos de comando instantâneo e de condução pelas quatro fases (RF07 e RF09).

Figura 42: Diálogo de comando nas fases de confirmação e resultado



Fonte: Autor.

Inserir captura: diálogo de comando a partir do sinótico ou da lista, acionando a bomba da bancada, na etapa "Confirmação" (resumo com tag, planta, dispositivo, valor atual e novo valor), e, em segunda captura, o "Resultado do comando" com o desfecho de sucesso.

O resultado nunca é otimista. O sucesso significa que o equipamento respondeu à escrita com o eco que confere com o pedido. Se o equipamento não responde ou responde com erro, o diálogo informa a falha e o motivo, por exemplo, "O comando não foi executado. Nada foi escrito no equipamento." Se o sistema perde o rastro da escrita, a mensagem é "Sem confirmação do estado do comando. Confira a lista de comandos antes de repetir." e a tag fica bloqueada por 35 s para evitar uma segunda escrita sobreposta. Outro comando em andamento sobre a mesma tag também é recusado com uma mensagem própria. Esses cuidados evitam que um comando seja repetido por engano e protegem o equipamento, o que corresponde às travas e à auditoria do requisito de segurança (RNF06).

A tela "Comandos" (rota `/comandos`) é o histórico dessas ações. A tabela mostra data, planta, valor, origem, estado e quem emitiu cada comando, com filtros por estado, planta e período, e permite cancelar um comando ainda não confirmado. O registro guarda uma cópia do nome da pessoa, da tag e da planta, de modo que ele continua legível mesmo que o cadastro mude depois. O comando em hora marcada não é um comando de pessoa e sim um evento agendado ligado a um script que escreve.

5.16 Síntese: telas, cargos e requisitos

A Tabela 8 reúne as telas descritas neste capítulo, o cargo ou a permissão que dá acesso a cada uma e os requisitos que elas atendem. As permissões das telas de supervisão são concedidas por grupo e, por isso, são indicadas pelo nome da permissão, enquanto as telas de configuração dependem do cargo. O administrador da Movi, ao entrar em um cliente, acessa o que o administrador do cliente acessa.

Tabela 8: Telas, cargos que acessam e requisitos atendidos.

Tela	**Rota**	**Quem acessa**	**Requisitos**
Entrar e trocar senha	/login, /trocar-senha	Qualquer usuário	RF05, RNF05
Meu perfil	/meu-perfil	Qualquer usuário autenticado	RF05

Tela	Rota	Quem acessa	Requisitos
Clientes e criação de cliente	/admin/clientes	Administrador da Movi	RF06, RF12, RNF05
Usuários	/configuracao/usuarios	Administrador do cliente e supervisor (apenas operadores)	RF05, RF06
Grupos	/configuracao/grupos	Administrador do cliente	RF06, RNF05
Plantas, dispositivos, tags e pastas	/cadastro/...	Administrador do cliente	RF01, RF02, RF10, RF11, RNF09
Lista de acompanhamento	/lista-de-acompanhamento	Permissão de ver valores das tags	RF03, RF07, RF15, RNF01
Dashboard (execução)	/dashboard	Permissão de ver telas	RF15, RNF01
Montagem de dashboard e do sinótico, telas, pastas de telas e imagens	/telas/..., /configuracao/telas	Administrador do cliente	RF10, RF15, RF16
Sinótico (execução)	/sinotico, /telas/ver	Permissão de ver telas	RF03, RF07, RF16, RNF01
Mapa	/mapa	Permissão de ver telas	RF17, RF18
Alarmes (ativos e histórico)	/alarmes	Permissão de ver alarmes (reconhecer exige permissão própria)	RF18, RNF06
Configuração de alarmes e eventos	/configuracao/alarmes, /configuracao/eventos	Administrador do cliente	RF18
Linha do tempo	/linha-do-tempo	Permissão de ver alarmes ou comandos	RF19, RNF06
Scripts e variável meta	/configuracao/scripts	Administrador do cliente	RF10, RF13, RF14
Comandos (histórico) e diálogo de comando	/comandos	Permissões de ver ou emitir comandos	RF07, RF09, RNF06

Fonte: Autor.

O conjunto mostra que a mesma interface atende a quatro perfis de uso. O administrador da Movi administra clientes. O administrador do cliente configura o sistema e monta as telas. O supervisor gerencia operadores, e o operador acompanha o processo e emite comandos conforme as permissões do seu grupo. Essa separação, somada aos princípios de fundo neutro, de cor apenas para o anormal e de hierarquia de telas, mantém a interface sóbria para quem opera e completa para quem configura, tudo acessível por navegador e com atualização em tempo real a partir dos dados do processo.

6 Bancada de Teste

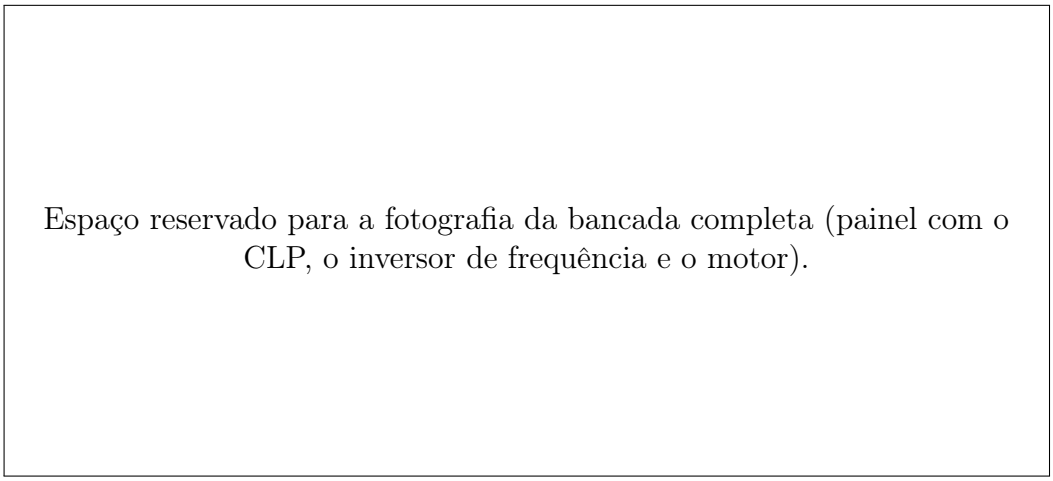
Este capítulo descreve a bancada de testes montada para validar o sistema supervisório em operação com equipamentos reais de campo. Primeiro, apresentam-se o objetivo da bancada e os equipamentos que a compõem. Em seguida, detalha-se a arquitetura de comunicação, que liga o supervisório ao controlador e o controlador ao inversor de frequência, e descreve-se a configuração do inversor. Depois, expõem-se a lógica programada no controlador e o modelo matemático do tanque simulado, que dá sentido físico ao processo. Por fim, mostra-se como a bancada é cadastrada no supervisório e define-se o roteiro dos ensaios, cujos resultados são relatados no capítulo seguinte.

6.1 Objetivo e Visão Geral

A bancada tem por finalidade submeter o sistema desenvolvido a um cenário de supervisão realista, com um controlador lógico programável (CLP) industrial, um inversor de frequência e um motor elétrico em funcionamento. O processo escolhido é o de uma bomba de água que enche um reservatório. Como a montagem de um tanque físico, com sensor de nível e tubulações, exigiria uma infraestrutura que foge ao escopo do trabalho, o nível do reservatório é simulado por software, calculado a cada ciclo dentro do próprio CLP. O motor, por outro lado, é real. Ele é acionado pelo inversor e fornece grandezas elétricas verdadeiras, como frequência de saída e corrente, de modo que a parte simulada convive com a parte física e as duas são tratadas de forma idêntica pelo supervisório.

O papel do supervisório nessa bancada é o de qualquer sistema SCADA. Ele lê periodicamente as informações do processo, exibe o nível do tanque, a frequência e a corrente do motor e o estado da bomba em telas e gráficos, sinaliza alarmes quando o nível sai da faixa desejada e permite que o operador ligue e desligue a bomba e altere a referência de frequência. A Figura 43 reserva o espaço para o registro fotográfico da bancada montada.

Figura 43: Bancada de teste montada.



Fonte: Autor.

Inserir a fotografia da bancada montada em Imagens/Desenvolvimento/Cap5/ e trocar o espaço reservado por `\includegraphics`. Informar o local (laboratório do instituto) e a data da montagem.

6.2 Equipamentos Utilizados

A bancada reúne um computador servidor, que executa o sistema supervisório em contêineres, uma rede local Ethernet, um controlador programável, um inversor de frequência e um motor de indução trifásico. A Tabela 9 resume os itens e a função de cada um.

Tabela 9: Equipamentos da bancada de teste.

Equipamento	**Modelo**	**Função na bancada**
Servidor	A definir	Executa os contêineres do supervisório (telas, *API*, aquisição, banco e cache)
Rede local	Switch Ethernet	Interliga o servidor, o CLP e o computador de programação
CLP	Schneider Modicon TM221CE40T	Executa a lógica da bancada, simula o tanque e atua como servidor Modbus TCP e mestre Modbus RTU
Inversor de frequência	Schneider Altivar ATV320	Aciona o motor e disponibiliza, por comunicação serial, o comando e as medições
Motor	Motor de indução trifásico do instituto	Representa a bomba de água

Fonte: Autor.

Preencher na Tabela 9 o modelo do servidor e os dados da placa do motor (potência, tensão, corrente, frequência e velocidade nominais).

O Modicon TM221CE40T é um controlador compacto da família M221, programado no ambiente EcoStruxure Machine Expert Basic. Esse modelo dispõe de entradas e saídas digitais embutidas, de uma porta Ethernet e de uma porta serial, identificada como SL1, que pode ser configurada para Modbus como mestre ou escravo (Schneider Electric, 2026d,c). A porta Ethernet permite que o controlador responda a requisições Modbus TCP na porta 502, comportando-se como servidor, enquanto a porta serial permite que ele consulte o inversor como mestre de uma rede Modbus RTU.

Confirmar no guia de hardware do TM221CE40T a quantidade de entradas e saídas digitais embutidas, o tipo das saídas (transistor), a existência da porta Ethernet e as portas seriais disponíveis (SL1 e eventual SL2). Citar a página.

O Altivar ATV320 é um inversor de frequência para aplicações de máquinas, que converte a tensão de alimentação em uma tensão trifásica de frequência e amplitude variáveis, permitindo controlar a velocidade do motor. Além de entradas e saídas próprias, ele oferece uma porta de comunicação Modbus serial em conector RJ45, por meio da qual é possível enviar comandos e referências e ler o estado e as grandezas medidas (Schneider Electric, 2026a,b). O motor, por sua vez, é o elemento que torna o ensaio tangível. Sua frequência de alimentação, ajustada pelo inversor, determina a velocidade de rotação, e é essa mesma frequência que, no modelo do tanque, determina a vazão de entrada de água.

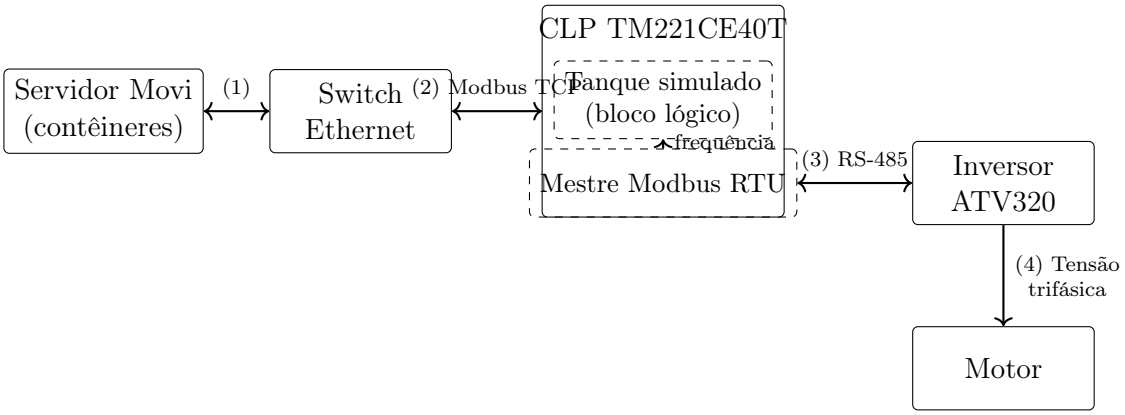
6.3 Arquitetura de Comunicação

A comunicação da bancada ocorre em dois segmentos independentes, cada um com o seu meio físico e o seu protocolo. No primeiro, o serviço de aquisição do supervisório consulta o

CLP por Modbus TCP, pela rede Ethernet (Modbus Organization, 2012a). No segundo, o CLP consulta o inversor por Modbus RTU, sobre um enlace serial RS-485 (Modbus Organization, 2012b). O CLP funciona, portanto, como um intermediário, que é servidor para o supervisório e mestre para o inversor. Essa organização é a mesma encontrada em instalações reais, nas quais um controlador concentra os equipamentos de campo e expõe seus dados a um único ponto de acesso.

A Figura 44 apresenta a topologia. O servidor do supervisório envia requisições Modbus TCP ao CLP por meio do switch da rede local (1 e 2). O CLP responde com os valores do processo, mantidos em palavras e bits de memória. Em paralelo, o CLP consulta e comanda o inversor por Modbus RTU em RS-485 (3), e o inversor alimenta o motor (4). O tanque, por ser simulado, não aparece como equipamento, mas como um bloco lógico dentro do CLP, que recebe a frequência lida do inversor e calcula o nível.

Figura 44: Topologia de comunicação da bancada.



Fonte: Autor.

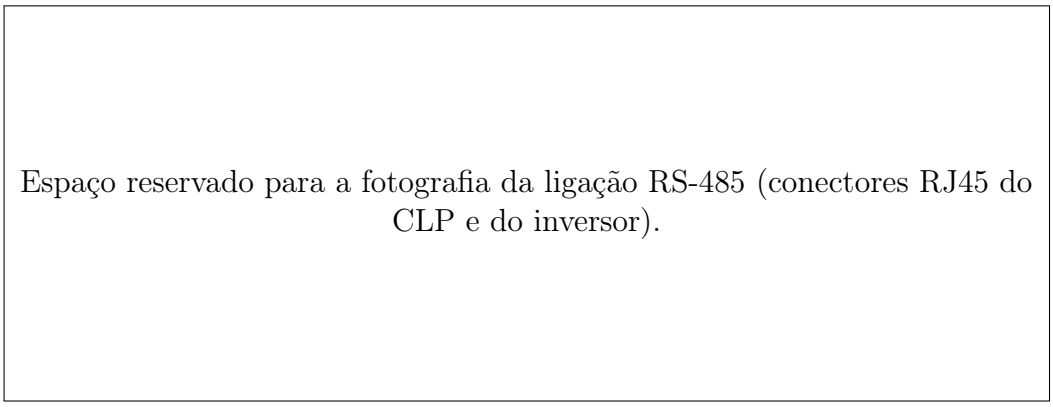
No segmento Ethernet, o CLP atende o servidor como um escravo Modbus TCP. As palavras de memória (`%MW`) e os bits de memória (`%M`) do controlador são acessíveis ao supervisório pelos mesmos endereços numéricos, contados a partir de zero, de modo que a palavra `%MW0` corresponde ao registrador de retenção de endereço 0 e o bit `%M0` à bobina de endereço 0 (Schneider Electric, 2026c). O supervisório usa as funções de leitura de registradores e bobinas para obter os valores e as funções de escrita para enviar comandos.

Confirmar no guia de programação do Machine Expert Basic a correspondência entre `%MW`/`%M` e os endereços Modbus (base zero), as funções Modbus suportadas pelo servidor do TM221 (leitura e escrita de registradores e bobinas) e o identificador de unidade aceito.

No segmento serial, o CLP utiliza os blocos de comunicação do Machine Expert Basic, `READ_VAR` e `WRITE_VAR`, para ler e escrever registradores do inversor. A porta SL1 é configurada como Modbus mestre, com a taxa de transmissão, a paridade e o número de bits de parada compatíveis com os do inversor. A ligação física é feita por cabo de par trançado entre o conector RJ45 do CLP e o conector RJ45 de comunicação do inversor, ligando os terminais de dados diferenciais e a referência comum. A Figura 45 reserva o espaço para o registro dessa ligação.

Confirmar nos guias de hardware do TM221 e do ATV320 a pinagem dos conectores RJ45 (D1/D+, D0/D- e comum) e a necessidade de resistores de terminação de 120 ohms e de polarização. Descrever o cabo efetivamente utilizado.

Figura 45: Ligação serial RS-485 entre o CLP e o inversor.



Fonte: Autor.

6.4 Configuração do Inversor ATV320

Para que o CLP comande o inversor, é preciso configurá-lo para aceitar o comando e a referência de frequência pela comunicação Modbus, em vez dos terminais físicos ou do painel. Isso envolve dois grupos de parâmetros. O primeiro define os parâmetros da comunicação serial, como o endereço do escravo, a taxa de transmissão e o formato do quadro. O segundo define os canais de comando e de referência, e ajusta os dados do motor e as rampas de aceleração e desaceleração. A Tabela 10 lista os parâmetros previstos, com os valores propostos para a bancada.

Tabela 10: Parâmetros propostos para o ATV320.

Parâmetro	**Valor proposto**	**Finalidade**
Endereço Modbus	2	Identifica o inversor na rede serial
Taxa de transmissão Modbus	19200 bit/s	Velocidade do enlace RS-485
Formato Modbus	8 bits, paridade par, 1 bit de parada (8E1)	Formato padrão do Modbus serial (Modbus Organization, 2012b)
Tempo limite Modbus	10 s	Falha de comunicação se o mestre deixar de consultar
Canal de comando 1	Modbus	O comando de marcha e parada vem da rede
Canal de referência 1	Modbus	A referência de frequência vem da rede
Rampas de aceleração e de desaceleração	5 s	Suaviza a partida e a parada do motor
Frequência mínima e máxima	0 Hz e 50 Hz	Faixa de operação da bomba
Dados da placa do motor	Conforme a placa	Tensão, frequência, corrente e velocidade nominais

Fonte: Autor.

Confirmar no manual de programação do ATV320 os nomes exatos e os códigos dos parâmetros da Tabela 10 (menu de comunicação: endereço, taxa, formato e tempo limite do Modbus, canal de comando e canal de referência, perfil de comando), os valores padrão de fábrica e o procedimento de configuração do motor (autoajuste, se utilizado). Indicar se o perfil de comando adotado é o de entradas e saídas ou o de acionamento CiA402, e definir os valores das rampas e dos dados do motor.

Com o inversor configurado, o CLP passa a interagir com ele por um pequeno conjunto de registradores, designados no manual pelos nomes `CMD` (palavra de controle), `LFR` (referência de frequência enviada pela rede), `RFR` (frequência de saída), `LCR` (corrente do motor) e `ETA` (palavra de estado) (Schneider Electric, 2026a). A palavra de controle liga e desliga o motor e reconhece falhas, a referência define a frequência desejada, e as demais fornecem as medições e o estado. A Tabela 11 apresenta o mapa previsto. Para reduzir o número de transações no enlace serial, a leitura dos quatro registradores consecutivos que contêm estado, frequência e corrente é feita em uma única consulta, e a escrita da palavra de controle e da referência em uma única requisição.

Tabela 11: Registradores Modbus do ATV320 utilizados (endereços a confirmar).

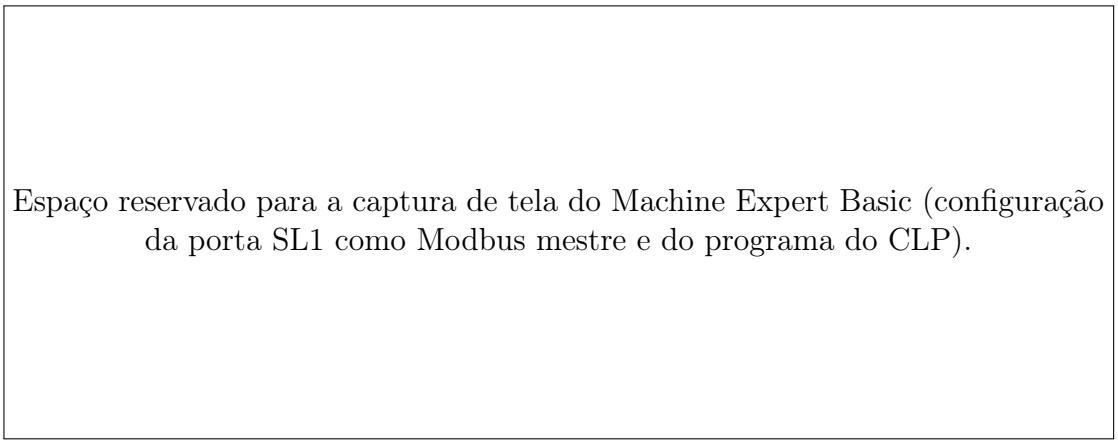
Nome	**Endereço proposto**	**Sentido**	**Descrição e unidade**
`CMD`	8501	Escrita	Palavra de controle (marcha, parada e reconhecimento de falha)
`LFR`	8502	Escrita	Referência de frequência pela rede, em décimos de hertz
`ETA`	3201	Leitura	Palavra de estado (operação habilitada, falha)
`RFR`	3202	Leitura	Frequência de saída, em décimos de hertz
`LCR`	3204	Leitura	Corrente do motor, em décimos de ampere

Fonte: Autor.

Confirmar no manual do Modbus do ATV320 os endereços, as unidades e a escala de `CMD`, `LFR`, `ETA`, `RFR` e `LCR`, o significado de cada bit de `CMD` e de `ETA`, a sequência de valores de `CMD` para partir e parar o motor e o registrador que traz o código da última falha. Verificar também se os endereços do manual são de base zero ou de base um e se `LFR` é o registrador correto para a referência pela rede.

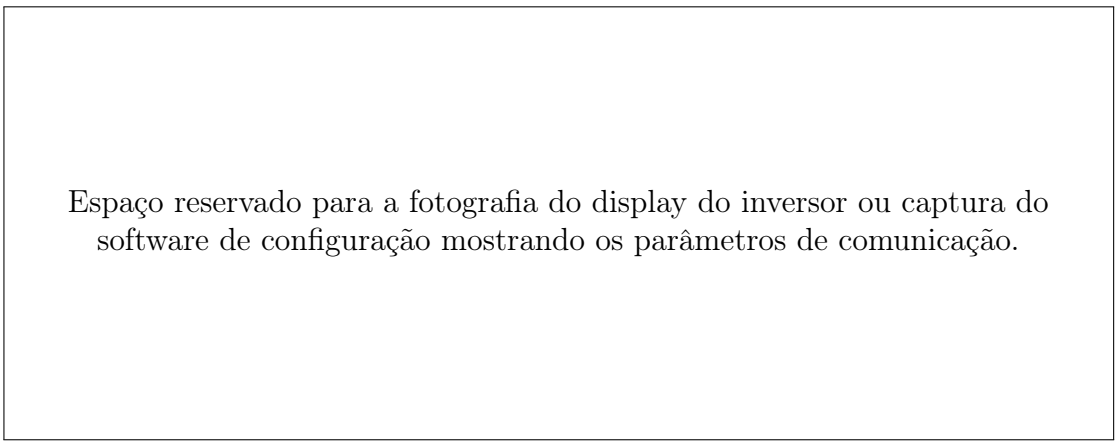
A parametrização do inversor e o ambiente de programação do controlador são os dois pontos em que a bancada depende de configuração manual. As Figuras 46 e 47 reservam o espaço para o registro dessas telas.

Figura 46: Configuração da comunicação serial no EcoStruxure Machine Expert Basic.



Fonte: Autor.

Figura 47: Parametrização do inversor ATV320.

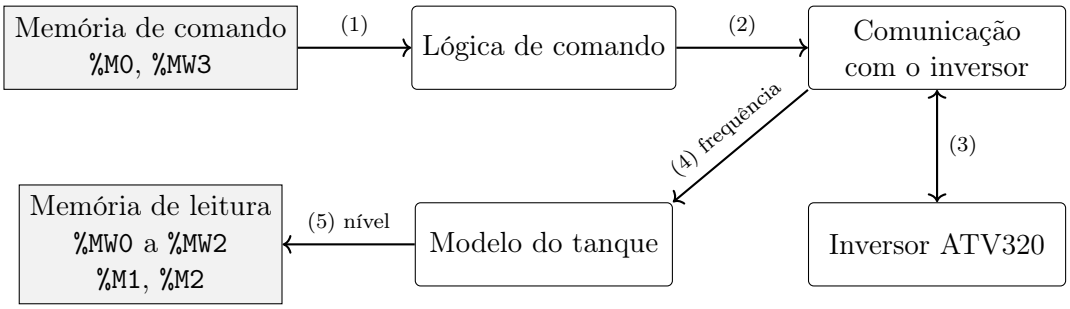


Fonte: Autor.

6.5 Programa do Controlador

O programa do CLP executa, a cada ciclo, quatro funções encadeadas. A primeira é a comunicação com o inversor, que lê estado, frequência e corrente e escreve a palavra de controle e a referência. A segunda é a lógica de comando, que traduz as solicitações do supervisório em ações sobre o inversor. A terceira é o modelo do tanque, que calcula o nível a partir da frequência lida. A quarta é a publicação dos resultados nas posições de memória que o supervisório consulta. A Figura 48 mostra o fluxo de informações entre esses blocos e as áreas de memória acessíveis pelo Modbus TCP.

Figura 48: Blocos funcionais do programa do CLP.



Fonte: Autor.

O supervisório escreve o comando liga/desliga e a referência de frequência na memória do controlador (1). A lógica de comando lê essas posições e decide o que enviar ao inversor (2). O bloco de comunicação transfere a palavra de controle e a referência ao inversor e traz de volta o estado, a frequência e a corrente (3). A frequência medida alimenta o modelo do tanque (4), que calcula o nível e o publica, junto com as demais grandezas, nas posições de leitura (5). O supervisório, por fim, consulta essas posições em seu ciclo de aquisição.

A lógica de comando segue uma regra de prioridade conservadora. A partida só é efetivada quando não há falha ativa no inversor e quando a comunicação serial está íntegra. Se o inversor sinaliza falha, o bit de falha é ativado e o comando de marcha é ignorado até que a falha seja reconhecida. Se o bloco de comunicação deixa de obter resposta do inversor por mais de um número de ciclos configurável, o CLP ativa um bit de falha de comunicação serial e envia o comando de parada. Esse comportamento evita que um erro de comunicação deixe o motor funcionando sem supervisão.

A Tabela 12 reúne o mapa de memória proposto, com os endereços acessíveis ao supervisório e as áreas internas de troca com o inversor. Os valores analógicos são mantidos como inteiros em décimos de unidade, o que evita o uso de números de ponto flutuante e simplifica a leitura pelo supervisório, que reconstrói o valor de engenharia por uma escala linear.

Tabela 12: Mapa de memória proposto para o CLP.

Endereço	**Tipo**	**Conteúdo**
%M0	Bit	Comando liga/desliga escrito pelo supervisório
%M1	Bit	Estado da bomba (1 quando o motor está em marcha)
%M2	Bit	Falha do inversor
%M3	Bit	Falha de comunicação serial com o inversor
%MW0	Palavra	Nível do tanque, em décimos de por cento
%MW1	Palavra	Frequência de saída, em décimos de hertz
%MW2	Palavra	Corrente do motor, em décimos de ampere
%MW3	Palavra	Referência de frequência escrita pelo supervisório, em décimos de hertz
%MW10 a %MW13	Palavras	Área de leitura do inversor (estado, frequência, frequência interna e corrente)
%MW20 e %MW21	Palavras	Área de escrita no inversor (palavra de controle e referência)

Endereço	Tipo	Conteúdo

Fonte: Autor.

Confirmar a viabilidade do mapa de memória no Machine Expert Basic (limites de `%MW` e `%M`, endereços retentivos e necessidade de declarar as áreas de memória como acessíveis à rede). Informar a configuração dos blocos `READ_VAR` e `WRITE_VAR` (endereço do escravo, tipo e quantidade de objetos, tempo limite, gerenciamento da mensagem) efetivamente usada e o tempo de ciclo medido.

6.6 Modelo do Tanque Simulado

O nível do reservatório é calculado por um integrador discreto, executado uma vez a cada período de amostragem. Em cada período, o nível aumenta em proporção à vazão de entrada, que depende da frequência de saída do inversor, e diminui em proporção à vazão de saída, tomada como constante. A frequência é a medida real, lida do inversor, de modo que o modelo reage ao comando de partida, à referência e à parada exatamente como reagiria uma bomba real. Esse tratamento representa uma bomba centrífuga em primeira aproximação, em que a vazão cresce com a rotação do motor, e se justifica pela finalidade da bancada, que é exercitar o supervisório e não reproduzir com fidelidade a hidráulica de um sistema específico.

Seja $h[k]$ o nível no instante k, em porcentagem da capacidade do tanque, T_s o período de amostragem, em segundos, $f[k]$ a frequência de saída do inversor, em hertz, e $b[k]$ o estado da bomba, igual a 1 quando o motor está em marcha e a 0 quando está parado. A vazão de entrada, expressa em porcentagem da capacidade por segundo, é proporcional à frequência.

$$q_\mathrm{e}[k] = k_\mathrm{q} \cdot f[k] \cdot b[k] \quad [\%/\mathrm{s}] \tag{10}$$

Nessa expressão, k_q é o coeficiente de vazão, em porcentagem por segundo e por hertz. A vazão de saída q_s é uma constante, que representa o consumo do reservatório. O nível no instante seguinte é o nível atual somado à variação líquida no período, limitado entre 0 e 100 por cento, que correspondem ao tanque vazio e ao tanque cheio.

$$h[k+1] = \min\left\{100,\ \max\left\{0,\ h[k] + T_\mathrm{s} \cdot (q_\mathrm{e}[k] - q_\mathrm{s})\right\}\right\} \quad [\%] \tag{11}$$

O limite inferior impede que o nível se torne negativo quando a bomba está parada e o tanque esvazia, e o limite superior representa o extravasamento, em que o excesso de água é perdido. Em regime, com a bomba ligada a uma frequência constante, o nível varia linearmente com o tempo. O tempo necessário para o nível passar de um valor inicial h_1 a um valor final h_2 é dado pela razão entre a variação de nível e a vazão líquida.

$$t = \frac{h_2 - h_1}{k_\mathrm{q} \cdot f - q_\mathrm{s}} \quad [\mathrm{s}] \tag{12}$$

Para ilustrar o cálculo, adotam-se parâmetros que fazem o enchimento durar alguns minutos, tempo adequado para observação na tela. O período de amostragem é de 1 segundo, o coeficiente é $k_\mathrm{q} = 0{,}025\,\%/(\mathrm{s}\cdot\mathrm{Hz})$ e a vazão de saída é $q_\mathrm{s} = 0{,}4\,\%/\mathrm{s}$. O nível inicial é de 20 por cento e a bomba é ligada com a frequência de 40 hertz. Pela Equação 10, a vazão de entrada vale:

$$q_{\mathrm{e}} = 0{,}025 \cdot 40 \cdot 1 = 1{,}0\,\%/\mathrm{s}$$

A vazão líquida é a diferença entre as vazões, igual a $1{,}0 - 0{,}4 = 0{,}6\,\%/\mathrm{s}$, de modo que, em cada período de 1 segundo, o nível sobe 0,6 ponto percentual. Aplicando a Equação 11 nos três primeiros ciclos, obtém-se:

$$\begin{aligned} h[1] &= 20{,}0 + 1 \cdot (1{,}0 - 0{,}4) = 20{,}6\,\% \\ h[2] &= 20{,}6 + 1 \cdot (1{,}0 - 0{,}4) = 21{,}2\,\% \\ h[3] &= 21{,}2 + 1 \cdot (1{,}0 - 0{,}4) = 21{,}8\,\% \end{aligned}$$

Suponha que a bomba seja desligada neste instante, de modo que $f[3] = 0$ e $b[3] = 0$ e a vazão de entrada se anule. O nível passa a decrescer com a vazão de saída de $0{,}4\,\%/\mathrm{s}$:

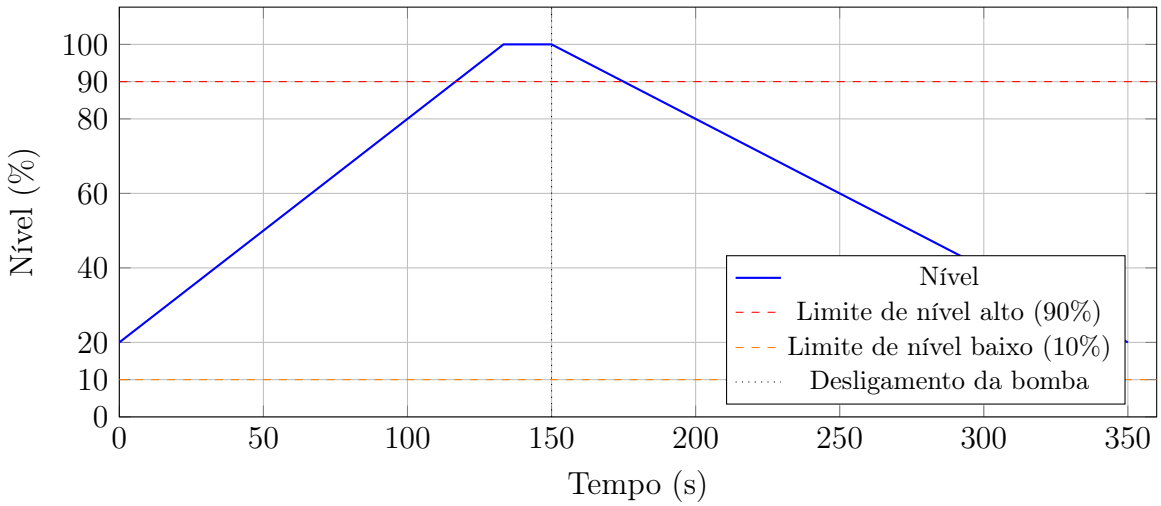
$$\begin{aligned} h[4] &= 21{,}8 + 1 \cdot (0 - 0{,}4) = 21{,}4\,\% \\ h[5] &= 21{,}4 + 1 \cdot (0 - 0{,}4) = 21{,}0\,\% \end{aligned}$$

Os tempos característicos do ciclo completo seguem da Equação 12. Com a bomba a 40 hertz, o nível leva $(100-20)/0{,}6 \approx 133{,}3$ segundos para ir de 20 a 100 por cento e $(90-20)/0{,}6 \approx 116{,}7$ segundos para alcançar 90 por cento, que é o limite do alarme de nível alto adotado a seguir. Com a bomba parada, o esvaziamento de 100 a 20 por cento leva $(100-20)/0{,}4 = 200$ segundos. O efeito do limite superior também é visível. Se o nível for de 99,8 por cento e a bomba continuar ligada a 40 hertz, o cálculo bruto resultaria em $99{,}8 + 0{,}6 = 100{,}4$, e o limitador mantém o nível em 100 por cento.

Como o controlador opera com inteiros, a implementação trabalha com o nível em décimos de por cento e com a frequência em décimos de hertz. Com esses valores, a vazão de entrada, em décimos de por cento por segundo, é igual à frequência em décimos de hertz dividida por 40, e a vazão de saída vale 4. Assim, com $F = 400$ (40 hertz), a vazão de entrada vale 10 e a variação líquida por ciclo vale 6 décimos, o que reproduz o exemplo.

A Figura 49 mostra o nível calculado pela própria equação em um ciclo completo de liga e desliga. A bomba é ligada em $t = 0$ com o tanque em 20 por cento, o nível sobe até saturar em 100 por cento em $t \approx 133{,}3$ segundos, a bomba é desligada em $t = 150$ segundos e o tanque esvazia até retornar a 20 por cento em $t = 350$ segundos. As linhas tracejadas indicam os limites dos alarmes.

Figura 49: Nível simulado em um ciclo de liga e desliga da bomba a 40 Hz.



Fonte: Autor.

Os parâmetros k_q e q_s podem ser alterados no programa do controlador sem qualquer modificação no supervisório, o que permite acelerar ou retardar o enchimento conforme a necessidade de cada ensaio.

Confirmar com o autor os valores definitivos de k_q, q_s, o período de amostragem de 1 segundo (tarefa periódica do CLP) e a frequência de operação dos ensaios. Se o programa final usar valores diferentes, refazer o exemplo numérico e o gráfico.

6.7 Configuração no Movi

O cadastro da bancada no supervisório segue a hierarquia do sistema: cliente, planta, dispositivo de entrada e saída e tags. Todo o cadastro é feito pela interface web, por um usuário com o cargo de administrador do cliente, sem alteração de código, o que valida o requisito de inclusão de novos pontos com o sistema em operação. O cliente representa a organização, neste caso o instituto, e é criado por um administrador da plataforma, que também provisiona o banco de dados próprio do cliente.

A planta, que organiza os dispositivos e é o pino do mapa, recebe o nome da bancada e as coordenadas de latitude e longitude do local. O dispositivo de entrada e saída corresponde ao CLP e é do tipo Modbus TCP, com endereço IP do controlador na rede local, porta 502, identificador de unidade (*unit ID*) igual ao configurado no controlador e os tempos limite e o número de tentativas padrão do sistema. A Tabela 13 resume esse cadastro.

Tabela 13: Cadastro da bancada no Movi.

Item	**Valor proposto**
Cliente	Nome do instituto (a definir)
Planta	Bancada de bombeamento
Coordenadas da planta	Latitude e longitude do instituto (a conferir)
Dispositivo de E/S	CLP TM221, protocolo Modbus TCP

Item	Valor proposto
Endereço IP e porta	Endereço do CLP na rede local (a definir), porta 502
Identificador de unidade	Valor aceito pelo TM221, por exemplo 1 (a confirmar)
Tempo limite e tentativas	Padrões do sistema (1000 ms e 2 tentativas)

Fonte: Autor.

Preencher a Tabela 13: nome do cliente, latitude e longitude do instituto (conferir no mapa), endereço IP do CLP e identificador de unidade efetivamente usado.

As tags representam as variáveis monitoradas e comandadas. Cada uma carrega o mapeamento Modbus (tipo de registrador e endereço), o tipo bruto do dado, a escala de conversão para a unidade de engenharia e o período de varredura. Como os valores analógicos do controlador são inteiros em décimos, a escala linear leva, por exemplo, o intervalo bruto de 0 a 1000 ao intervalo de engenharia de 0 a 100 por cento, no caso do nível. A Tabela 14 apresenta as sete tags previstas.

Tabela 14: Tags cadastradas no Movi para a bancada (endereços propostos).

Nome	Endereço Modbus	Tipo bruto	Unidade	Formato e escala	Período
Nível do tanque	Registrador de retenção 0 (`%MW0`)	Inteiro 16 bits sem sinal	%	Analógico, 0 a 1000 em 0 a 100	1 s
Frequência de saída	Registrador de retenção 1 (`%MW1`)	Inteiro 16 bits sem sinal	Hz	Analógico, 0 a 500 em 0 a 50	1 s
Corrente do motor	Registrador de retenção 2 (`%MW2`)	Inteiro 16 bits sem sinal	A	Analógico, 0 a 200 em 0 a 20 (faixa a confirmar)	1 s
Estado da bomba	Bobina 1 (`%M1`)	Digital	n/a	Simples, 0 desligada e 1 ligada	1 s
Comando liga/desliga	Bobina 0 (`%M0`)	Digital, leitura e escrita	n/a	Simples	1 s
Referência de frequência	Registrador de retenção 3 (`%MW3`)	Inteiro 16 bits sem sinal, leitura e escrita	Hz	Analógico, 0 a 500 em 0 a 50	5 s
Falha do inversor	Bobina 2 (`%M2`)	Digital	n/a	Simples, 0 normal e 1 em falha	1 s

Fonte: Autor.

Confirmar os endereços da Tabela 14 depois de fechado o mapa de memória do CLP e a faixa de corrente conforme o motor. Conferir o período mínimo de varredura aceito pelo sistema (500 ms) e se a tag de comando admite período diferente de 1 s.

A tag de comando liga/desliga e a de referência de frequência têm acesso de leitura e escrita, e é por elas que o operador atua sobre a bancada. A escrita passa pelo fluxo de comando em quatro fases do sistema, composto por seleção, verificação, confirmação e execução, o que protege o processo de acionamentos acidentais. Como o supervisório considera o comando bem-sucedido quando recebe a resposta do controlador à escrita, sem releitura, a confirmação do efeito real é feita pela leitura posterior do estado da bomba e da frequência, que reflete o que o inversor de fato executou.

Dois alarmes são configurados sobre a tag do nível. O alarme de nível alto é disparado quando o nível ultrapassa 90 por cento, com o nível de severidade Crítico, pois o risco é o de transbordamento do reservatório. O alarme de nível baixo é disparado quando o nível cai abaixo de 10 por cento, com o nível de severidade Alto, pois indica a aproximação de uma condição de falta de água. Cada alarme é construído sobre um evento de comparação de tag, com o operador e o valor de disparo, e uma banda morta para evitar disparos repetidos por pequenas oscilações em torno do limite. Quando disparados, os alarmes aparecem no indicador de alarmes do cabeçalho e na lista de alarmes, onde podem ser reconhecidos pelo operador.

Confirmar a associação entre severidades e limites (nível alto Crítico e nível baixo Alto, conforme proposto) e os valores da banda morta. Registrar o nome exato dos eventos e dos alarmes criados.

As telas de supervisão completam a configuração. O sinótico representa o tanque como um reservatório cujo preenchimento acompanha o valor do nível, a bomba por um símbolo cuja cor indica o estado de marcha, e a frequência e a corrente por indicadores numéricos. O símbolo da bomba é o ponto de comando, a partir do qual se abre o diálogo de quatro fases para ligar e desligar. O *dashboard* reúne um gráfico de linha com o nível e a frequência ao longo do tempo, um medidor do nível atual e o indicador de estado da bomba. A lista de acompanhamento mostra, em uma única tabela, as sete tags com seus valores e a qualidade da leitura. A Figura 50 reserva o espaço para as capturas dessas telas.

Figura 50: Telas de supervisão da bancada no Movi.

Espaço reservado para as capturas do sinótico do tanque com a bomba e do *dashboard* com o gráfico de nível e frequência.

Fonte: Autor.

Montar o sinótico, o dashboard e a lista de acompanhamento no Movi e inserir as capturas em Imagens/Desenvolvimento/Cap5/. Descrever o desenho do sinótico efetivamente criado (objetos, vínculos de animação e cores).

6.8 Roteiro de Ensaios

Os ensaios foram planejados para exercitar, em sequência, as funções centrais do supervisório sobre a bancada: leitura contínua, comando, acompanhamento de um processo dinâmico, sinalização e reconhecimento de alarmes, tratamento de falha de comunicação e alteração de parâmetro em operação. A Tabela 15 resume o conjunto. Cada ensaio é detalhado a seguir, com o objetivo, o procedimento, o comportamento esperado na tela e um espaço reservado para a evidência e o resultado, que são relatados no capítulo seguinte.

Tabela 15: Resumo dos ensaios da bancada.

Ensaio	**Descrição**	**Funcionalidade exercitada**
E1	Leitura contínua com a bomba desligada	Aquisição periódica e exibição de valores
E2	Partida pelo supervisório e enchimento	Comando em quatro fases e acompanhamento em gráfico
E3	Parada e esvaziamento	Comando de parada e histórico
E4	Alarme de nível alto e reconhecimento	Detecção de evento, alarme e reconhecimento
E5	Perda de comunicação e retorno	Detecção de falha de comunicação e recuperação
E6	Troca de referência de frequência	Escrita de valor analógico e reação do processo

Fonte: Autor.

6.8.1 E1: Leitura Contínua com a Bomba Desligada

O objetivo é verificar que o sistema adquire periodicamente as variáveis da bancada e as exibe corretamente quando o processo está em repouso. Com o CLP ligado, o inversor energizado e a bomba desligada, o ensaio consiste em abrir a lista de acompanhamento e o sinótico e observar os valores durante alguns minutos. Espera-se ver o nível estável ou em lento decréscimo, conforme a vazão de saída, a frequência e a corrente nulas e o estado da bomba como desligada, todos com qualidade boa e atualização no período de varredura de 1 segundo. O histórico deve registrar os pontos no mesmo ritmo.

Espaço reservado para a captura da lista de acompanhamento e do sinótico no ensaio E1.

E1: inserir a captura de tela, o horário, a duração da observação e o resultado (valores lidos, taxa de atualização observada e eventuais falhas).

6.8.2 E2: Partida pelo Supervisório e Enchimento

O objetivo é validar o comando de partida e o acompanhamento de um processo dinâmico. O operador define a referência de frequência em 40 hertz e liga a bomba pelo diálogo de comando, que percorre as quatro fases e exige confirmação. O CLP repassa o comando ao inversor, o motor parte seguindo a rampa de aceleração e o nível do tanque passa a subir. Espera-se que o símbolo da bomba mude de cor ao indicar a marcha, que a frequência e a corrente subam até o valor de regime, que o nível suba com a inclinação prevista pela Equação 11 (cerca de 0,6 ponto percentual por segundo) e que o gráfico do *dashboard* mostre o enchimento. Mede-se o intervalo entre a confirmação do comando e a mudança de estado na tela.

Espaço reservado para a captura do diálogo de comando e da tela durante o enchimento no ensaio E2.

E2: inserir as capturas, o intervalo medido entre o comando e a resposta na tela, a inclinação do nível observada (comparar com 0,6 ponto percentual por segundo) e a corrente em regime.

6.8.3 E3: Parada e Esvaziamento

O objetivo é validar o comando de parada e o comportamento do processo após ele. Com o tanque em nível intermediário e a bomba ligada, o operador a desliga pelo supervisório. O motor desacelera pela rampa até parar, a frequência e a corrente caem a zero e o nível passa a decrescer com a vazão de saída, de 0,4 %/s. Espera-se que a tela reflita o estado de bomba desligada, que o gráfico mostre a mudança de inclinação do nível no instante da parada e que o histórico preserve toda a trajetória, que pode ser consultada posteriormente na tela de tendência.

Espaço reservado para a captura do gráfico de tendência com a curva de enchimento e esvaziamento no ensaio E3.

E3: inserir a captura do gráfico de tendência, a taxa de queda do nível observada e o tempo de parada do motor.

6.8.4 E4: Alarme de Nível Alto e Reconhecimento

O objetivo é verificar a detecção da condição de nível alto, a sinalização do alarme e o seu reconhecimento. O ensaio parte de um nível próximo de 80 por cento com a bomba ligada. O operador mantém a bomba em marcha até que o nível ultrapasse 90 por cento. Espera-se que, ao cruzar o limite, o alarme de nível alto seja gerado com a severidade Crítico, apareça no indicador de alarmes do cabeçalho e na lista de alarmes, e que, depois de o operador desligar a bomba e o nível voltar abaixo do limite descontada a banda morta, o alarme retorne ao estado normal. Registra-se também o reconhecimento do alarme pelo operador, com a identificação de quem o realizou e o horário.

Espaço reservado para a captura da lista de alarmes com o alarme de nível alto ativo e reconhecido no ensaio E4.

E4: inserir as capturas do alarme ativo e reconhecido, o instante do disparo, o valor de nível no disparo e o tempo entre o cruzamento do limite e a sinalização na tela.

6.8.5 E5: Perda de Comunicação e Retorno

O objetivo é verificar como o sistema trata a perda do enlace com o equipamento de campo e a sua recuperação. Com a bancada em operação normal, desconecta-se o cabo Ethernet entre o switch e o CLP. Espera-se que o supervisório detecte a falha de comunicação depois das tentativas configuradas, que o dispositivo passe a ser exibido como sem comunicação, que os valores deixem de ser atualizados e sejam sinalizados como desatualizados, e que o evento de falha de comunicação do dispositivo seja registrado. Ao reconectar o cabo, espera-se que a aquisição retome sozinha, que o estado volte a comunicando e que os valores voltem a ser atualizados, sem intervenção do operador. Observa-se também que, durante a falha, o CLP continua executando o seu programa e o nível simulado continua evoluindo, o que se evidencia ao comparar o valor ao retorno com o último valor exibido.

Espaço reservado para a captura da tela com o dispositivo sem comunicação e, depois, com a comunicação restabelecida no ensaio E5.

E5: inserir as capturas, o tempo até a detecção da falha, o tempo até a retomada após a reconexão e o comportamento do histórico durante a lacuna.

6.8.6 E6: Troca de Referência de Frequência

O objetivo é validar a escrita de um valor analógico em operação e a reação do processo. Com a bomba ligada a 40 hertz, o operador altera a referência para 30 hertz, pelo mesmo diálogo de quatro fases. O inversor ajusta a velocidade do motor segundo a rampa e a vazão de entrada do modelo diminui. Com $f = 30$ hertz, a vazão de entrada passa a $0{,}025 \cdot 30 = 0{,}75\,\%/\mathrm{s}$ e a vazão líquida a $0{,}75 - 0{,}4 = 0{,}35\,\%/\mathrm{s}$. Espera-se que a frequência e a corrente na tela acompanhem a nova referência e que a inclinação da curva do nível diminua de 0,6 para 0,35 ponto percentual por segundo, o que confirma que a escrita chegou ao inversor e produziu o efeito esperado.

Espaço reservado para a captura do gráfico de nível e frequência antes e depois da troca de referência no ensaio E6.

E6: inserir a captura, os valores de frequência antes e depois, a corrente em cada regime e a inclinação do nível medida (comparar com 0,35 ponto percentual por segundo).

Com o roteiro definido e a bancada configurada, o capítulo seguinte apresenta os resultados obtidos em cada ensaio e a discussão a respeito do desempenho do sistema.

7 Resultados

Este capítulo apresenta os resultados obtidos com o sistema desenvolvido e discute até que ponto eles confirmam os requisitos especificados. Conforme a estrutura apresentada na introdução, a avaliação combina três fontes de evidência: as medidas de desempenho feitas com o sistema completo em contêineres, o ensaio de aquisição contra um servidor Modbus TCP de teste e os ensaios na bancada com o inversor de frequência. Primeiro, descreve-se o ambiente em que as medidas foram feitas e o método comum a todas elas. Em seguida, são apresentadas as medidas já realizadas, que cobrem o cadastro de clientes, a autenticação, o tempo de resposta das telas, o motor de scripts isolado e a gravação do histórico. Depois, relata-se a capacidade de aquisição, medida com simuladores, e o ensaio ponta a ponta contra o servidor de teste. Na sequência, reservam-se os espaços para os seis ensaios da bancada, com o comportamento esperado em cada um. Por fim, os requisitos são confrontados com as evidências, resume-se a qualidade de software e discutem-se as limitações observadas e a relação dos resultados com a literatura.

7.1 Ambiente de ensaio

As medidas foram obtidas em um único computador, o do próprio autor, que fez o papel de servidor. Todos os processos do sistema executaram em contêineres *Docker* (Docker, Inc., 2026), cada um com limites de processamento e de memória definidos para reproduzir os recursos planejados para a implantação, de modo que o teto medido seja o de um contêiner pequeno e não o da máquina inteira. O *PostgreSQL* (PostgreSQL Global Development Group, 2026) na versão 18 e o *Redis* (Redis Ltd., 2026) na versão 8 também executaram em contêineres. A Tabela 16 resume o equipamento e as versões, e a Tabela 17 lista os limites aplicados a cada contêiner.

Tabela 16: Ambiente de ensaio.

Item	**Descrição**
Computador (servidor)	Intel Core i5-13420H com 12 *threads*, cerca de 32 GB de memória, Windows 11 Pro
Virtualização	Docker Desktop com máquina virtual de 4 processadores lógicos e 6 GB de memória durante a campanha de capacidade
Banco de dados	PostgreSQL 18.6 em contêiner
Cache, filas e transporte	Redis 8.10 em contêiner, com política *noeviction*
Aplicação	Node.js 24, imagens de produção construídas em estágios, uma por serviço
Dispositivo de campo simulado	Simuladores Modbus TCP, um do próprio repositório, com milhares de CLP virtuais, e o ProtoProbe do autor (porta 502, escravo 1)
Arquitetura do processador	amd64 no ensaio, enquanto a implantação prevê processadores ARM

Fonte: Autor.

Tabela 17: Limites de recursos dos contêineres.

Contêiner	**Processamento**	**Memória**	**Observação**
API (Fastify)	0,25 *vCPU*	512 MB	Processo HTTP, sem *workers*
Worker	0,25 *vCPU*	512 MB	Um supervisor e um subprocesso por família de fila
Aquisição	0,25 *vCPU*	512 MB	Agendador e leitura Modbus, sem banco
Scripts isolados	0,25 *vCPU*	512 MB	Hospedeiro do *WebAssembly*, raiz somente leitura
Web (Next.js)	0,25 *vCPU*	512 MB	Somente telas
Redis	0,25 *vCPU*	1 GB	*maxmemory* de 512 MB
PostgreSQL	2 *vCPU*	1 GB	*shared_buffers* de 256 MB

Fonte: Autor.

A soma dos limites é de 3,5 *vCPU* e 4,5 GiB, que cabe na máquina virtual usada. Os limites de processamento são aplicados por cota de tempo, o mesmo mecanismo usado pelos orquestradores de contêineres, o que faz um contêiner de 0,25 *vCPU* parar por alguns milissegundos ao esgotar a cota em cada janela de tempo. Esse comportamento aparece nos resultados, por exemplo no percentil 95 da latência do motor de scripts, e é uma característica do ambiente e não do código.

O método dos ensaios de capacidade foi o mesmo em todos os tipos de carga. Cada degrau de carga roda dois ciclos de aquecimento, descartados, e doze ciclos medidos. Um degrau é considerado sustentado quando pelo menos 99 % das leituras esperadas foram atendidas, o atraso no percentil 95 ficou abaixo de 20 % do período de varredura, a profundidade das filas não cresceu na segunda metade da janela, nenhum contêiner reiniciou e o Redis ficou abaixo de 70 % da memória máxima configurada. O teto de um tipo de carga é o maior degrau sustentado antes do primeiro reprovado, e um degrau reprovado que passa ao ser repetido é tratado como ruído de medição. Esse critério acompanha a prática das avaliações experimentais da literatura de contêineres, que mede latência, vazão e consumo de processador e de memória sob carga crescente (Netto, H. *et al.*, 2019; Netto, Hylson V. *et al.*, 2016).

Há ressalvas que valem para todos os resultados. A carga, o simulador de CLP, o navegador e os contêineres dividem o mesmo computador, que também executava outras atividades do autor. Não há latência de rede entre os contêineres, nem balanceador, nem criptografia de transporte no Redis. O PostgreSQL local tem disco rápido e aceita muito mais conexões que a instância de 1 GiB prevista. Os números, portanto, são de ordem de grandeza e valem como teto do servidor com equipamento que responde de imediato, e não como previsão para uma planta com modems lentos.

7.2 Medidas de desempenho já realizadas

7.2.1 Cadastro de clientes e custo da autenticação

O sistema é multi-tenant com banco próprio por cliente, e por isso a criação de um cliente envolve criar o banco, aplicar o esquema e cadastrar os dados iniciais, tudo num trabalho assíncrono retomável. Mediu-se o tempo entre o pedido de criação e o banco pronto para uso. Com o sistema aquecido, o processo levou 1,7 s e, na primeira criação de um cliente, que inclui o ensaio da migração com reversão, levou 3,4 s. O aplicador de migrações sozinho levou de 0,36 a 0,54 s. Esses valores sustentam o requisito de banco próprio por cliente, criado de forma

automática (RF12), pois a entrada de um novo cliente não exige intervenção manual nem espera perceptível.

A exclusão de um cliente é a operação inversa e mais cara. O comando que remove o banco com desconexão forçada levou de 5 a 18 s na pilha local anterior, tempo dominado pelo ponto de verificação do banco. A limpeza das chaves do cliente no Redis usa uma varredura incremental cujo custo cresce com o total de chaves da instância e não com as chaves do cliente, com cerca de 15 ms para 500 chaves, de 0,08 a 0,37 s para 20 mil e de 0,56 a 1,4 s para 100 mil. Como a exclusão roda em segundo plano e não bloqueia as telas, esses tempos são aceitáveis, mas indicam que a limpeza deve ser repensada se o Redis for compartilhado por muitos clientes. Na campanha de capacidade, a exclusão de clientes de teste ficou em execução por horas sem terminar, achado registrado e ainda sem causa comprovada, o que a seção de limitações retoma.

Conferir se o achado da campanha de 13 e 14/10 (exclusão de cliente que não termina na pilha de capacidade) foi corrigido depois do relatório. O texto o trata como pendente.

A autenticação usa o *Argon2id*, função de derivação de chave projetada para dificultar ataques por força bruta com *hardware* paralelo (Biryukov *et al.*, 2021). Com 19.456 KiB de memória, duas iterações e um nível de paralelismo, medido em 30 amostras no processador do ensaio, o cálculo do *hash* teve mediana de 24,2 ms e percentil 95 de 26,2 ms, e a verificação teve mediana de 23,9 ms e percentil 95 de 26,8 ms. O custo é imperceptível para quem entra no sistema e, ao mesmo tempo, caro o bastante para tornar impraticável a tentativa em massa de senhas contra um banco vazado, o que atende ao requisito de segurança de acesso (RNF05). A Tabela 18 reúne essas medidas.

Tabela 18: Medidas de criação de cliente e de autenticação.

Medida	**Resultado**	**Data**
Criação de cliente, pedido até banco pronto	1,7 s (quente) e 3,4 s (frio)	08/10/2026
Aplicação das migrações do cliente	0,36 a 0,54 s	08/10/2026
Remoção do banco do cliente (*DROP DATABASE* forçado)	5 a 18 s	08/10/2026
Limpeza de chaves do Redis, 500 e 100 mil chaves	15 ms e de 0,56 a 1,4 s	08/10/2026
Hash de senha Argon2id, mediana e percentil 95	24,2 ms e 26,2 ms	08/10/2026
Verificação de senha Argon2id, mediana e percentil 95	23,9 ms e 26,8 ms	08/10/2026

Fonte: Autor.

7.2.2 Desempenho das telas

O desempenho das telas foi medido com o *Lighthouse* em modo de celular, com limitação simulada de rede e de processador, sobre a versão de produção da interface e uma instância isolada do sistema, com a sessão de um usuário de teste. Foram feitas três execuções em cada uma das três telas principais, o sinótico em modo de execução, a lista de alarmes e o dashboard, e a Tabela 19 apresenta a mediana, junto com o piso adotado como limite de regressão para os próximos lotes. O *Largest Contentful Paint*, que mede o tempo até o maior elemento aparecer, ficou entre 1,25 e 1,51 s, abaixo do alvo de 2,5 s, e o deslocamento de *layout* acumulado ficou muito abaixo do limite de 0,1. O tempo total de bloqueio da *thread* principal, que serve de aproximação da responsividade, foi maior no dashboard por causa do desenho dos gráficos.

Tabela 19: Resultados do *Lighthouse* (celular), mediana de três execuções. Des.: desempenho, Ace.: acessibilidade, BP: boas práticas, LCP e TBT em ms.

Tela	**Des.**	**Ace.**	**BP**	**SEO**	**LCP**	**CLS**	**TBT**
Sinótico (execução)	99	100	100	54	1.260	0	121
Alarmes	99	100	100	54	1.253	0,008	141
Dashboard	94	100	100	54	1.513	0,001	282
Piso adotado	94 / 94 / 89	95	95	49	-	-	-

Fonte: Autor.

A pontuação de acessibilidade em 100 indica que o teste automático não encontrou violação das diretrizes WCAG, o que não substitui a conferência com leitor de tela e com a navegação por teclado, que ficou como risco aceito. A pontuação de *SEO* em 54 é esperada e não representa defeito, porque a área autenticada é marcada como não indexável por decisão de projeto. A medição foi feita com outras atividades pesadas no mesmo computador, de modo que os valores tendem a ser conservadores. As medidas dão suporte ao requisito de acesso por navegador sem instalação (RNF13) e ao de exibição das telas de supervisão (RF15 e RF16).

As medianas vêm do registro de 15/10/2026 (madrugada) e o piso consta no CLAUDE.md do código com data de 14/10/2026. Confirmar qual data deve aparecer no texto e se vale citar o INP, que o Lighthouse não mede em modo de navegação.

Além da pontuação geral, mediu-se o custo de atualizar uma tela de sinótico com muitos objetos. Uma tela de teste com 200 objetos, 120 vínculos entre objetos e tags e 506 nós gráficos foi mantida aberta por 60 s com dez tags que mudavam a cada ciclo. Nenhuma tarefa longa, isto é, nenhuma tarefa de mais de 50 ms na *thread* principal, foi registrada. O tempo para aplicar um ciclo de valores, medido do fim da resposta até a última alteração na árvore da página, teve mediana de 6 ms e percentil 95 de 8 ms, abaixo do orçamento de 16 ms, que corresponde a um quadro de uma tela de 60 quadros por segundo. Esse resultado mostra que o ritmo de atualização por *polling* curto não degrada a interface e sustenta, do lado do cliente, o requisito de telas de tempo real com baixa latência (RNF01) e o de execução do sinótico (RF16).

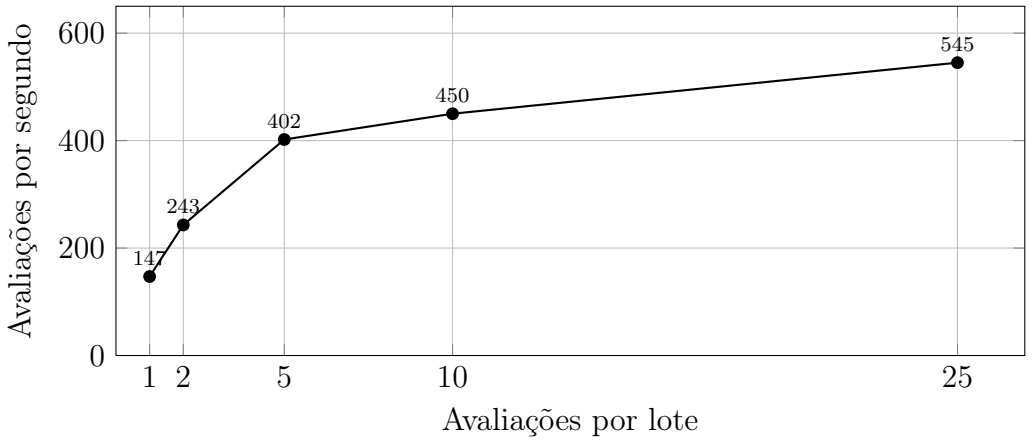
7.2.3 Motor de scripts isolado

O cálculo de variáveis derivadas, de scripts de eventos e de componentes de conteúdo escritos pelos clientes executa em um motor isolado, com o interpretador *QuickJS* (Bellard; Gordon, 2026) compilado para *WebAssembly* (WebAssembly Community Group, 2026) e executado sob o *Wasmtime* (Bytecode Alliance, 2026), sem acesso a rede, arquivo, banco nem segredo. O isolamento, exigido pelo requisito de execução segura do código do usuário (RNF07), tem um custo, e as medidas procuram dimensioná-lo. Com um hospedeiro já aquecido em um contêiner de 0,25 *vCPU*, o custo fixo de uma avaliação foi de 3,1 ms na mediana para a variável derivada, 3,5 ms para o script e 3,3 ms para o componente. O percentil 95 ficou entre 46 e 56 ms e o 99 entre 58 e 60 ms. Esses valores altos nas caudas correspondem à pausa da cota de processador, e não ao motor. O motor anterior, em *Python*, custava cerca de 34 ms por avaliação nas mesmas condições, o que justificou a troca.

A vazão foi medida com um programa de comparação que executa avaliações em sequência. Um hospedeiro de 0,25 *vCPU* sustentou de 144 a 149 avaliações por segundo, e quatro hospedeiros chegaram a 353. Por *vCPU*, isso equivale a uma faixa de 380 a 590 avaliações por

segundo, com ruído de cerca de 50 % entre execuções, porque o computador do ensaio executava outras cargas. O agrupamento de avaliações em lote aumentou a vazão, como mostra a Figura 51. Com lote de 25 avaliações, o mesmo hospedeiro chegou, em média, a 545 avaliações por segundo, cerca de 3,7 vezes a de uma avaliação por vez.

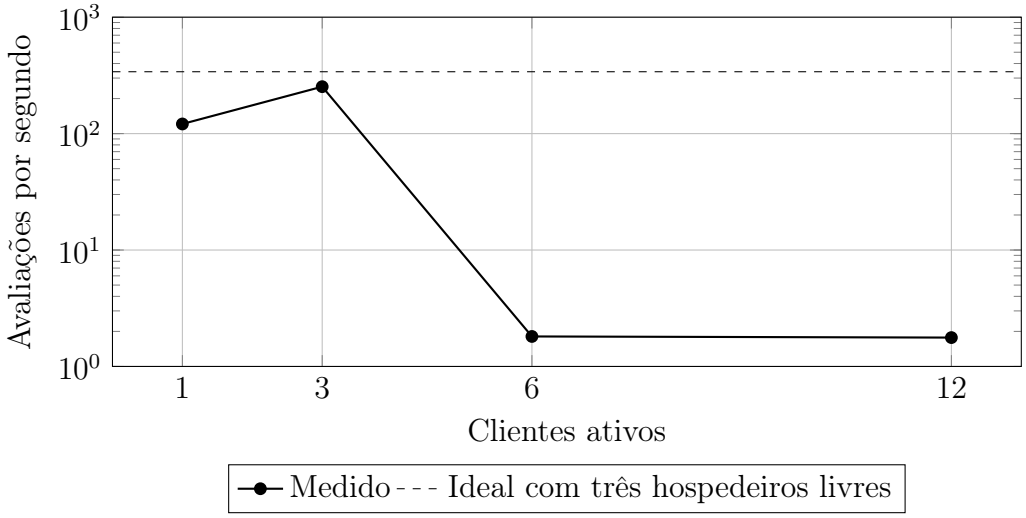
Figura 51: Vazão do motor de scripts em função do tamanho do lote, um hospedeiro de 0,25 *vCPU*.



Fonte: Autor.

O ponto de maior interesse é o comportamento com muitos clientes ativos. Cada troca de cliente em um hospedeiro obriga a reciclar o ambiente de execução, para que nada de um cliente vaze ao seguinte, e essa reciclagem custa cerca de 0,55 s a 0,25 *vCPU*, quase 180 vezes o custo de uma avaliação. Com três hospedeiros, um cliente ativo usou 121 avaliações por segundo, três clientes ativos chegaram a 253, que é 74 % do ideal de 340, e seis ou mais clientes derrubaram a vazão para menos de duas avaliações por segundo, como mostra a Figura 52. A conclusão de projeto é que o número de clientes ativos por contêiner não deve passar o número de hospedeiros, e que a escala se faz por réplicas do contêiner. O resultado ilustra o compromisso entre isolamento forte e desempenho, e só se torna problema quando muitos clientes avaliam scripts simultaneamente, o que o agrupamento por cliente atenua na prática. Essas medidas confirmam o requisito de cálculo de valores derivados e de execução de scripts (RF13 e RF14) e mostram o seu custo em recursos.

Figura 52: Vazão do motor de scripts com três hospedeiros em função do número de clientes ativos.



Fonte: Autor.

Quanto à memória, o hospedeiro do motor usou 17 MiB de memória residente no teste de vazão, e o contêiner dedicado aos scripts atingiu o pico de 166 MiB dos 512 MiB permitidos. No contêiner de *workers*, que concentra a ingestão e o histórico, o pico foi de 435 MiB, ou 85 % do limite, o mais apertado de todos, e a API chegou a 340 MiB. Esses números mostram que o isolamento dos scripts é barato em memória e que o limite que merece atenção é o do *worker*.

A Tabela 20 traz o teto sustentado de variáveis derivadas, chamadas de metas, e de tags internas geradas, comparado com o piso planejado. As variáveis derivadas usadas no ensaio liam duas tags e foram avaliadas de 1,2 a 1,85 vezes por ciclo. As tags internas foram geradas em lotes de 25 por dispositivo. Em todos os casos, o primeiro recurso a saturar foi o contêiner de *workers*, nas filas de ingestão e de histórico, e não o motor de scripts, que consome 3 ms por avaliação. As metas ficaram abaixo do piso planejado a 5 e a 30 s e no piso a 10 s, e as tags internas ficaram abaixo do piso nas três faixas, o que levou à abertura de uma especificação dedicada à capacidade dessas duas filas.

Tabela 20: Teto sustentado de variáveis derivadas e de tags internas geradas por contêiner de 0,25 *vCPU*.

Carga	**Período**	**Teto medido**	**Piso planejado**	**Alvo**
Variável derivada (meta)	5 s	50	150	450
Variável derivada (meta)	10 s	250	250	750
Variável derivada (meta)	30 s	150	1.000	3.000
Tag interna gerada	5 s	100	350	1.050
Tag interna gerada	10 s	350	525	1.575
Tag interna gerada	30 s	250	2.800	8.400

Fonte: Autor.

7.2.4 Gravação do histórico e isolamento entre clientes

O histórico de leituras é gravado por uma fila dedicada, em lotes, com decisão atômica de gravar e gravação idempotente, numa tabela particionada por mês. O teste de carga dessa fila mediu, para um cliente, de 186 a 235 trabalhos por segundo com 40 linhas cada, o que equivale a 7.430 a 9.400 linhas por segundo, a um custo de 1,8 a 2,7 ms de processador por trabalho. Com quatro clientes simultâneos, a taxa ficou entre 128 e 277 trabalhos por segundo. A decisão de gravar, feita por um *script* no Redis, custou 16,4 µs por chamada e 2,3 % do processador do Redis a 1.000 leituras por segundo. A retirada de 200.000 linhas pelo expurgo levou 1,6 s, ou 125 mil linhas por segundo. Esses resultados sustentam o registro contínuo do histórico (RF02) com folga em relação à carga de um cliente típico.

O mesmo teste verifica se um cliente que gera muito trabalho prejudica os demais. Nos dois cenários de sobrecarga de um cliente lento, o de pressão alta e o de controle com pressão baixa, a latência máxima de gravação do cliente vizinho ficou, depois das correções, entre 274 e 426 ms no primeiro e entre 303 e 313 ms no segundo, contra o limite de 15 s do critério. Nas primeiras execuções, porém, o limite foi ultrapassado, com 20,2 s no cenário de pressão alta e com 54,9 e 77,3 s no de controle. A investigação mostrou que o cenário anterior deixava milhares de trabalhos na fila e contaminava o seguinte, o que foi corrigido no teste, e que o resultado é muito sensível à disputa de processador na máquina, pois com oito laços de processamento ocupando o computador a latência do vizinho foi de 91,8 s. O resultado confirma o enfileiramento justo entre clientes (RNF03) em condições normais e mostra uma fragilidade sob escassez de processador, hoje registrada como risco do projeto.

Outro teste de carga manteve, por 10 minutos, seis clientes com leituras contínuas e conferiu que nenhuma tag ficava desatualizada na leitura final. Depois de uma correção no próprio teste, que passou a considerar o catálogo já carregado, três execuções seguidas passaram, em 639, 652 e 636 s. Isso confirma que o valor atual das tags permanece disponível e atualizado no Redis sob carga contínua (RF03), e que o orçamento de conexões do banco impõe espera, e não perda, quando o número de clientes simultâneos excede o de conexões reservadas.

7.3 Aquisição e ensaio com o simulador

7.3.1 Capacidade de leitura Modbus

A capacidade de aquisição foi medida com um simulador Modbus TCP do próprio repositório, que emula milhares de CLP virtuais em portas distintas e registra as requisições recebidas. Cada CLP virtual tinha 25 tags, lidas em cinco requisições por ciclo, número que o planejador do sistema gera para blocos de registradores afastados. A Tabela 21 resume o teto de uma única réplica do serviço de aquisição, com 0,25 *vCPU*, em três períodos de varredura. O que limitou a carga foi a soma do processador da aquisição e do Redis, ambos próximos de 100 % do limite, e não o banco de dados, que tinha dois processadores. O teto em tags por segundo foi praticamente o mesmo em 10 e em 30 s, entre 250 e 325 tags por segundo, o que confirma que o limite é de capacidade de processamento e não do ciclo de varredura.

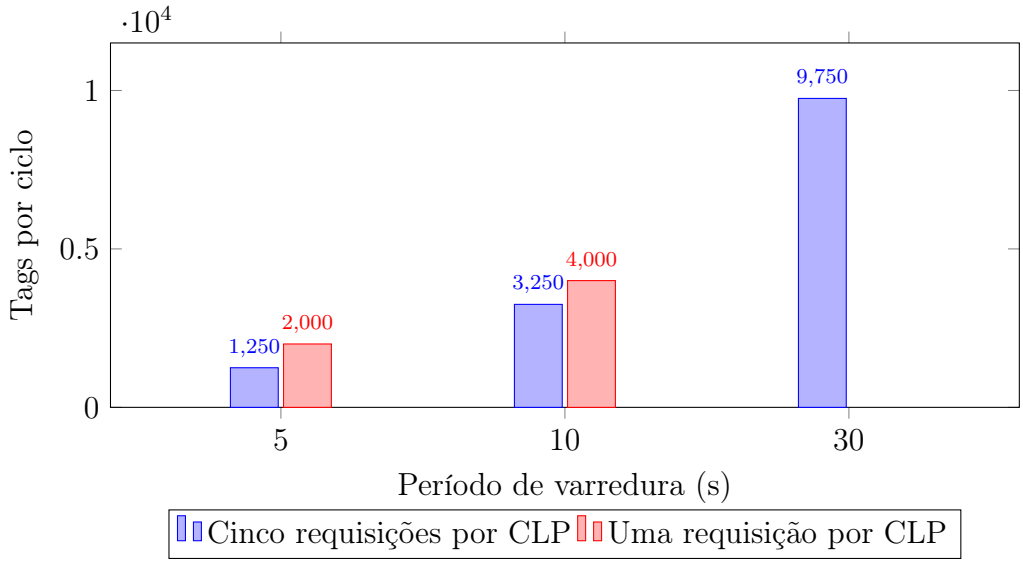
Tabela 21: Teto de leitura Modbus por réplica de aquisição (0,25 *vCPU*, 25 tags por CLP, cinco requisições por ciclo).

Período	Reprovou em	Teto (tags)	Tags por segundo
5 s	1.500 tags, atraso no percentil 95 de 1.052 ms (limite de 1.000 ms)	1.250	250
10 s	4.000 tags, atraso no percentil 95 de 4.882 ms e fila de despacho crescendo	3.250	325
30 s	12.200 tags, atraso no percentil 95 de 8.306 ms (limite de 6.000 ms)	9.750	325

Fonte: Autor.

A Figura 53 compara esses tetos com os obtidos em rodadas anteriores com uma única requisição por CLP e ciclo. O custo da requisição é significativo, porque cada ida e volta Modbus tem um custo fixo de enquadramento, resposta e despacho independente do número de registradores. Passar de uma para cinco requisições reduziu o teto de 2.000 para 1.250 tags a 5 s, queda de 37 %, e de 4.000 para 3.250 a 10 s, queda de 19 %. Em equipamentos reais, o tempo de resposta do *modem* e da rede soma-se a cada requisição, de modo que o teto prático será menor. A medida dimensiona uma réplica, mas o efeito de uma segunda réplica não foi medido, de modo que a replicação horizontal (RNF02) é sustentada pela arquitetura. O requisito de aquisição periódica (RF01) fica confirmado em carga. Como a fila reparte o trabalho e o Moleculer leva cada dispositivo sempre à mesma réplica, o total deve crescer com o número de réplicas até o Redis passar a ser o limite, o que também chegou perto de 100 % no ensaio.

Figura 53: Teto de tags por réplica de aquisição, com uma e com cinco requisições por CLP.



Fonte: Autor.

7.3.2 Ensaio ponta a ponta com o servidor de teste

Para validar a cadeia completa contra um servidor que se comporta como um CLP comum, o autor usou o ProtoProbe, um servidor Modbus TCP de teste que escuta na porta 502 com

identificador de escravo 1 e expõe 2.500 registradores de retenção de 16 bits sem sinal. O ensaio cadastrou, pela interface do sistema, uma planta, um dispositivo Modbus apontando para o servidor e uma tag no endereço 7, lida de 1 em 1 s com função de leitura de registradores de retenção. Também se cadastrou um dispositivo interno com uma tag gravável, um evento que compara a tag lida com um limite e um *script* que copia o valor lido para a tag interna. Nenhuma função de escrita foi usada nessa prova.

O resultado foi positivo em toda a cadeia. O valor 12.314, que o servidor de teste mantém no endereço 7, chegou ao valor atual da tag com qualidade boa. O evento disparou, o *script* executou no contêiner isolado em 13 a 87 ms e gravou a tag interna com o mesmo valor, sem falha registrada. Como os registradores do servidor de teste são estáticos, o limite do evento foi movido entre dois valores para provocar a desativação e a ativação, e o *script* executou nas duas transições. A prova mostra que a leitura Modbus, o valor atual no Redis, o detector de eventos, o motor de scripts e a escrita em tag interna funcionam em sequência e atende ao requisito de comunicação por configuração, sem código específico do equipamento (RNF09).

O ensaio de carga de leitura contra o ProtoProbe consiste em cadastrar um número crescente de tags sobre os 2.500 registradores, com períodos de 1 e de 5 s, manter a leitura por um intervalo fixo e conferir três coisas: se o número de leituras gravadas no histórico coincide com o esperado, se o valor atual no Redis acompanha as mudanças e se a tela de lista de acompanhamento exibe o valor com o atraso previsto. Como os registradores do servidor de teste são estáticos, o ensaio de carga usa tags internas geradas para variar o valor, ou o próprio autor altera registradores durante a medição. A Tabela 22 reserva o espaço para os resultados.

Tabela 22: Ensaio de carga de leitura contra o ProtoProbe.

Tags	Período	Duração	Leituras gravadas	Atraso até o Redis	Atraso até a tela
10	1 s	a medir	a medir	a medir	a medir
50	1 s	a medir	a medir	a medir	a medir
100	5 s	a medir	a medir	a medir	a medir
500	5 s	a medir	a medir	a medir	a medir

Fonte: Autor.

Medir: ensaio de carga de leitura contra o ProtoProbe (porta 502, escravo 1, 2.500 registradores). Não há número de vazão contra o ProtoProbe nos documentos do código, só a prova funcional da cadeia (specs/049-scripts-em-javascript/prova-protoprobe.md) e a campanha de capacidade com o simulador do repositório. Escolher o número de tags e os períodos da tabela, a duração (sugestão: 10 minutos por linha), e como medir o atraso até a tela (carimbo da leitura contra o instante em que o valor aparece).

7.4 Resultados da bancada

A bancada reúne um CLP que simula o nível do reservatório e atua como servidor Modbus TCP para o sistema, um inversor de frequência que aciona um motor real e é comandado pelo CLP por rede serial (Schneider Electric, 2026a), e o servidor que executa o sistema. Os ensaios E1 a E6, listados na Tabela 23, percorrem o comportamento esperado de uma estação de bombeamento, da leitura contínua com a bomba parada até a troca da referência de frequência com a bomba em operação. Em todos, interessam duas medidas de tempo. A primeira é o tempo entre a confirmação do comando e a resposta do equipamento, primeiro a do CLP à escrita e

depois a do inversor, que aparece como a mudança da frequência na tela, e que mede a latência do caminho de escrita. A segunda é o atraso do supervisório, o tempo entre uma mudança no equipamento e o instante em que ela aparece na tela do operador. Pela arquitetura de *polling*, esse atraso é limitado pela soma do período de varredura da tag, do período de atualização da tela e de uma margem de processamento, e é essa soma que os ensaios procuram confirmar (RNF01).

Os ensaios E1 a E6 seguem o roteiro do Capítulo 6 (bancada: CLP TM221 simulando o tanque e inversor ATV320). Conferir os parâmetros citados (referência de 40 e 30 Hz, inclinações de 0,6, 0,4 e 0,35 ponto percentual por segundo, limite de 90 %) se o Capítulo 6 mudar, e preencher as tabelas e os gráficos com as medições.

Tabela 23: Ensaios da bancada.

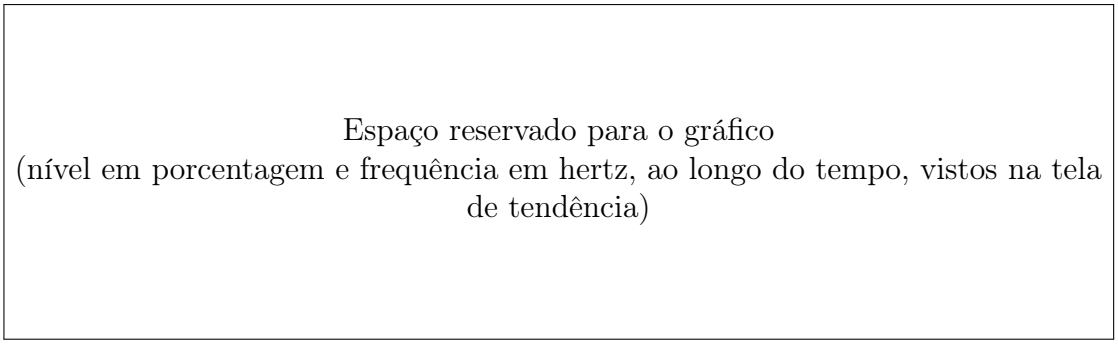
Id	**Ensaio**	**Requisitos verificados**
E1	Leitura contínua com a bomba desligada	RF01, RF02, RF03, RF04, RNF01
E2	Partida pelo supervisório e enchimento	RF07, RF09, RNF01
E3	Parada e esvaziamento	RF07, RF09, RF19, RNF01
E4	Alarme de nível alto e reconhecimento	RF18, RF19, RNF06
E5	Perda de comunicação e retorno	RF01, RF03, RF18
E6	Troca da referência de frequência	RF07, RF09, RNF06

Fonte: Autor.

7.4.1 E1: leitura contínua com a bomba desligada

Com a bomba parada, a frequência do inversor é zero e o nível simulado varia pouco, o que isola o desempenho da aquisição de qualquer efeito do processo. O ensaio mantém as tags de nível, de frequência de saída, de estado do inversor e de corrente em leitura contínua por um intervalo longo e confere três propriedades. A primeira é que o histórico não perde amostras, isto é, o número de leituras gravadas é o esperado para o período de varredura. A segunda é que o carimbo de tempo de cada leitura é monotônico e espaçado pelo período configurado, com variação de poucos milissegundos. A terceira é que a tela de lista de acompanhamento mostra o valor com qualidade boa e atrasa em relação ao equipamento no máximo o período de varredura mais o período de atualização da tela. Espera-se uma taxa de entrega próxima de 100 % e nenhum alarme, e que o ruído do sensor de nível apareça como variação pequena em torno de um valor estável.

Figura 54: Nível simulado e frequência do inversor no ensaio E1.



Fonte: Autor.

Tabela 24: Tempos do ensaio E1.

Medida	**Esperado**	**Medido**
Taxa de leituras gravadas em relação às esperadas	acima de 99 %	a medir
Variação do intervalo entre carimbos consecutivos	poucos ms	a medir
Atraso do supervisório (equipamento até a tela)	varredura mais atualização da tela	a medir

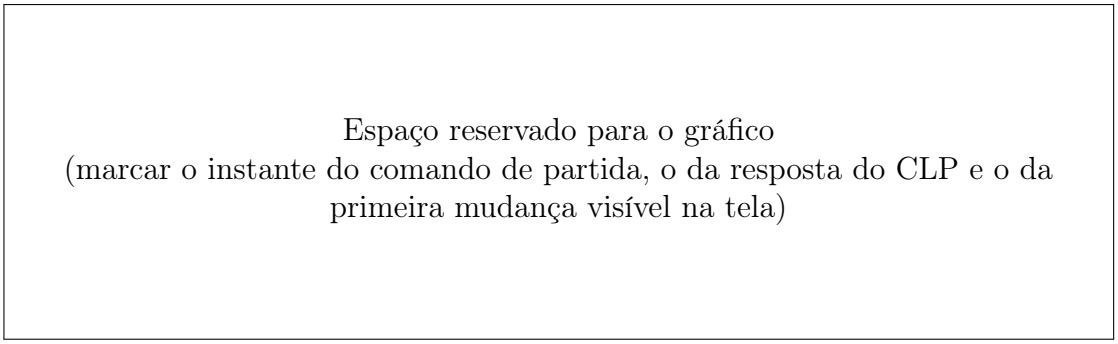
Fonte: Autor.

Medir e inserir: gráfico da tela do E1 (captura da tendência) e a tabela com taxa de entrega, variação do intervalo e atraso do supervisório. Informar o período de varredura e o período de atualização da tela usados.

7.4.2 E2: partida da bomba e enchimento

O operador define a referência de 40 hertz e envia o comando de partida, que percorre as quatro fases do comando: a seleção da tag, a verificação das condições e da permissão, a confirmação e a execução. Na execução, o sistema escreve a ordem em um registrador do CLP, que a repassa ao inversor pela rede serial. O ensaio mede o tempo entre a confirmação do operador e a resposta do CLP à escrita, que é o instante em que o comando é dado como concluído, pois o sucesso do comando é a resposta do equipamento. Mede também o tempo até a frequência do inversor começar a subir na tela. Depois, a frequência sobe segundo a rampa de aceleração do inversor e o nível simulado passa a subir cerca de 0,6 ponto percentual por segundo. Espera-se que a resposta do CLP chegue em dezenas de milissegundos em rede local, bem abaixo do limite de 30 s do comando, e que a subida da frequência e do nível apareça na tela com o atraso do supervisório já medido no E1. Espera-se também que a trava por tag impeça um segundo comando simultâneo sobre a mesma bomba e que o comando fique registrado com o nome do operador, a tag, o valor e o instante.

Figura 55: Nível simulado e frequência do inversor no ensaio E2.



Fonte: Autor.

Tabela 25: Tempos do ensaio E2.

Medida	**Esperado**	**Medido**
Confirmação do operador até a resposta do CLP à escrita	dezenas de ms	a medir
Resposta do CLP até a primeira mudança de frequência na tela	varredura mais atualização da tela	a medir
Duração da rampa de aceleração observada	valor configurado no inversor	a medir
Inclinação do nível	cerca de 0,6 ponto percentual por segundo	a medir

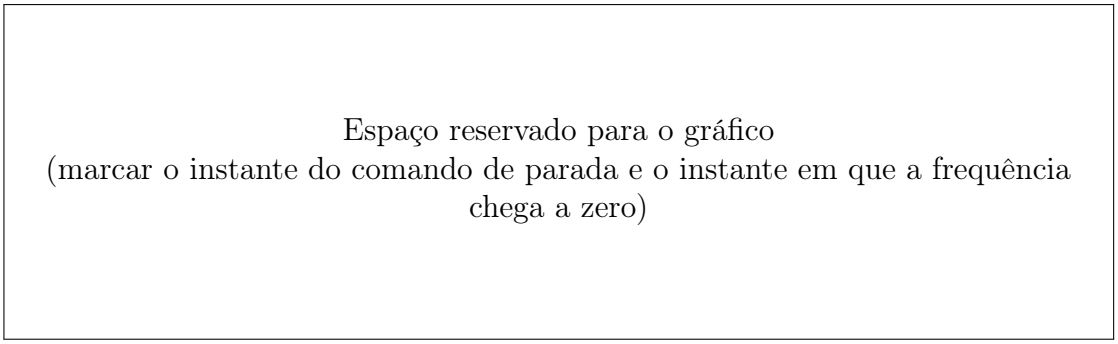
Fonte: Autor.

Medir e inserir: gráfico da tela do E2 e a tabela. Informar a rampa de aceleração do inversor e confirmar a referência de 40 Hz, e anexar a linha do registro de comandos que mostra a auditoria do comando.

7.4.3 E3: parada da bomba e esvaziamento

O ensaio de parada repete o caminho do comando com a ordem de parada. A frequência desce segundo a rampa de desaceleração e o nível simulado passa a cair com a vazão de saída, de 0,4 ponto percentual por segundo. Espera-se o mesmo tempo de comando do E2, já que o caminho de escrita é o mesmo, e que a tela acompanhe a descida da frequência até zero e a mudança do estado do inversor para parado. O ensaio também verifica que a leitura continua sem perda durante a transição e que nenhum alarme de comunicação aparece, o que separa o efeito do processo do efeito do sistema.

Figura 56: Nível simulado e frequência do inversor no ensaio E3.



Fonte: Autor.

Tabela 26: Tempos do ensaio E3.

Medida	**Esperado**	**Medido**
Confirmação do operador até a resposta do CLP à escrita	dezenas de ms	a medir
Resposta do CLP até a frequência zero na tela	rampa de desaceleração mais atraso do supervisório	a medir
Inclinação do nível após a parada	cerca de 0,4 ponto percentual por segundo (queda)	a medir
Leituras perdidas durante a transição	nenhuma	a medir

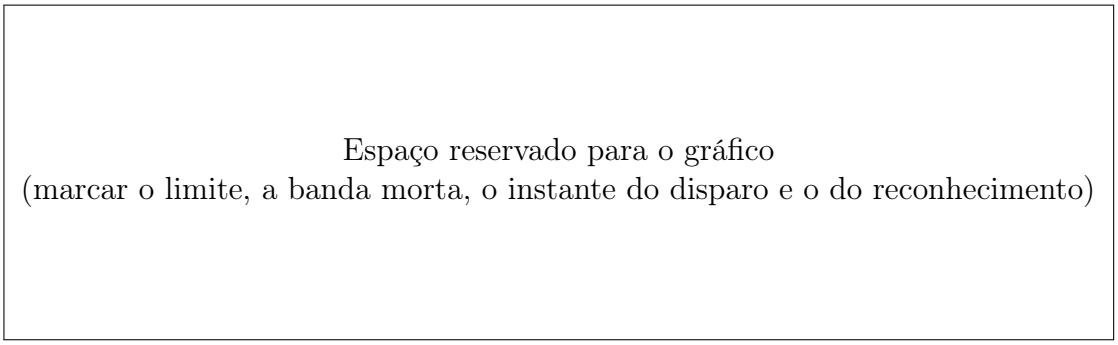
Fonte: Autor.

Medir e inserir: gráfico da tela do E3 e a tabela. Informar a rampa de desaceleração e o tempo de parada do motor.

7.4.4 E4: alarme de nível alto e reconhecimento

Com a bomba em operação a partir de um nível próximo de 80 por cento, o nível sobe até cruzar o limite de 90 por cento do alarme de nível alto. O detector compara cada leitura com o limite, usa uma banda morta para evitar oscilação em torno do valor e, ao ativar, cria uma ocorrência de alarme com a severidade Crítico e a descrição fixa no instante do disparo. O ensaio mede o tempo entre o instante em que a leitura cruza o limite e o instante em que o indicador de alarmes da tela mostra a ocorrência, que deve ser da ordem do atraso do supervisório. Em seguida, o operador reconhece o alarme, com comentário opcional, e o ensaio confere que a ocorrência passa ao estado reconhecido e que o registro guarda o nome do operador e o instante. Por fim, a bomba é parada, o nível cai abaixo do limite, descontada a banda morta, e o alarme retorna ao normal. Espera-se que o alarme apareça uma única vez por cruzamento, sem repetição por ruído do sensor, e que a linha do tempo de alarmes e comandos reconstrua a sequência na ordem correta.

Figura 57: Nível simulado, limite do alarme e instantes de alarme, reconhecimento e retorno ao normal no ensaio E4.



Fonte: Autor.

Tabela 27: Tempos do ensaio E4.

Medida	**Esperado**	**Medido**
Cruzamento do limite até o alarme na tela	varredura mais atualização da tela	a medir
Ocorrências geradas por cruzamento	uma	a medir
Reconhecimento até o registro com nome e instante	imediato (próxima atualização da tela)	a medir

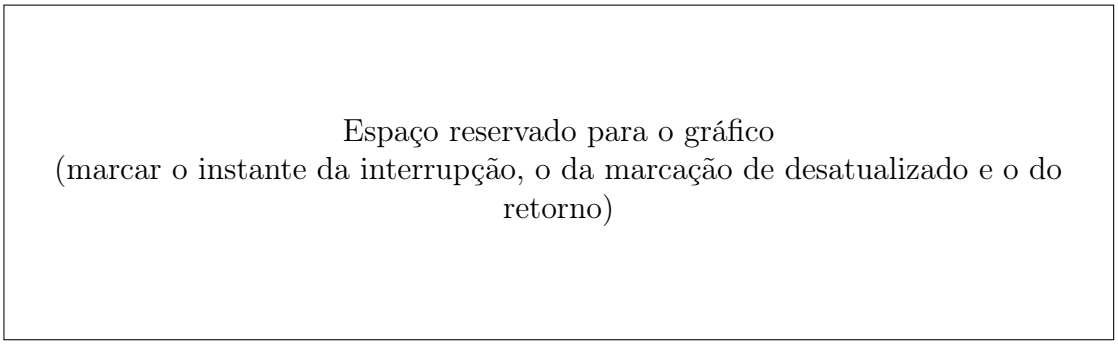
Fonte: Autor.

Medir e inserir: gráfico da tela do E4 (nível, limite e alarme), a tabela e uma captura da linha do tempo com o disparo e o reconhecimento. Informar a banda morta configurada e confirmar o limite de 90 por cento e a severidade Crítico.

7.4.5 E5: perda e retorno de comunicação

Neste ensaio, a comunicação com o equipamento de campo é interrompida de forma controlada, desconectando o cabo Ethernet entre o comutador e o CLP, e depois restabelecida. Durante a falha o CLP continua executando o programa e o nível simulado continua evoluindo. O sistema marca o valor de cada tag como desatualizado depois de três períodos de varredura sem leitura nova, mantendo o último valor e o carimbo original, e o estado de comunicação do dispositivo passa a degradado depois de três ciclos de leitura falhos. Esses dois mecanismos têm o mesmo prazo nominal, e o evento de falha de comunicação do dispositivo pode ser ligado a um alarme comum. Ao restabelecer o cabo, o primeiro ciclo bem sucedido fecha o estado degradado e o valor volta a ter qualidade boa, sem rajada de leituras acumuladas. O ensaio mede o tempo entre a interrupção e a marcação de desatualizado, o tempo entre o retorno do cabo e a volta à qualidade boa, e confere que não há lacuna artificial preenchida no histórico, pois a falha de leitura não gera linha e a lacuna é a evidência da falha.

Figura 58: Valor, qualidade e estado de comunicação durante a perda e o retorno de comunicação no ensaio E5.



Fonte: Autor.

Tabela 28: Tempos do ensaio E5.

Medida	**Esperado**	**Medido**
Interrupção até a marcação de desatualizado	três períodos de varredura	a medir
Interrupção até o estado de comunicação degradado	três ciclos falhos	a medir
Retorno do cabo até a qualidade boa na tela	um ciclo mais atualização da tela	a medir
Linhas gravadas no histórico durante a falha	nenhuma	a medir

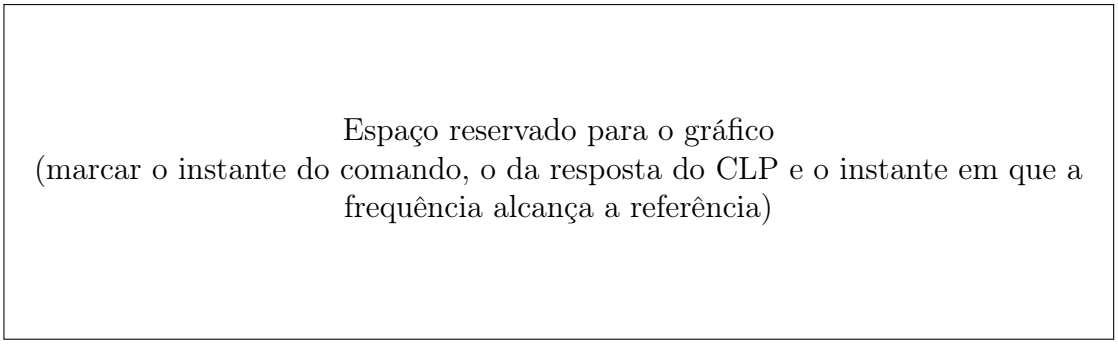
Fonte: Autor.

Medir e inserir: gráfico da tela do E5 e a tabela. Informar o período de varredura (1 s no roteiro). Confirmar se o alarme de falha de comunicação foi ligado nesse ensaio.

7.4.6 E6: troca da referência de frequência

Com a bomba ligada a 40 hertz, o operador altera a frequência de referência para 30 hertz pelo mesmo comando de quatro fases, agora com um valor numérico em vez de uma ordem de liga e desliga. O sistema escreve o novo valor no CLP, que o repassa ao inversor, e a frequência de saída migra para o novo valor seguindo a rampa. A inclinação do nível simulado deve passar de 0,6 para 0,35 ponto percentual por segundo. O ensaio mede o tempo do comando até a resposta do CLP e o tempo para a frequência medida atingir a referência, e confere duas propriedades de segurança. A primeira é que um segundo comando enviado imediatamente à mesma tag é recusado pela trava por tag, em vez de gerar duas escritas concorrentes. A segunda é que o reenvio do mesmo comando, com a mesma chave de idempotência, não cria uma segunda escrita. Espera-se que a resposta do CLP chegue em dezenas de milissegundos, que a frequência alcance a referência no tempo da rampa do inversor e que o registro de comandos guarde o valor anterior, o novo valor, o operador e a origem.

Figura 59: Frequência de referência e frequência medida no ensaio E6.



Fonte: Autor.

Tabela 29: Tempos do ensaio E6.

Medida	**Esperado**	**Medido**
Confirmação do operador até a resposta do CLP à escrita	dezenas de ms	a medir
Resposta do CLP até a frequência medida igual à referência	rampa do inversor mais atraso do supervisório	a medir
Inclinação do nível depois da troca	cerca de 0,35 ponto percentual por segundo	a medir
Segundo comando imediato na mesma tag	recusado pela trava	a medir
Reenvio com a mesma chave de idempotência	uma única escrita	a medir

Fonte: Autor.

Medir e inserir: gráfico da tela do E6 e a tabela. Informar o registrador da referência, a rampa configurada e a corrente em cada regime.

7.5 Validação dos requisitos

A Tabela 30 confronta cada requisito funcional e não funcional com a evidência disponível e com a situação resultante. Considera-se atendido o requisito com medida realizada ou com teste automatizado que exercita o comportamento, e parcialmente atendido o que depende da bancada ou de uma medida que ainda não foi feita. Os requisitos de comando, de alarmes e de latência são os que mais dependem dos ensaios E1 a E6 e por isso aparecem como parciais até que as tabelas anteriores sejam preenchidas. Nenhum requisito ficou sem evidência. A redundância entre zonas de disponibilidade fica como trabalho futuro, e a arquitetura em contêineres é o que permite levá-la a um orquestrador (Netto, Hylson Vescovi *et al.*, 2017).

Tabela 30: Validação dos requisitos.

Id	Evidência	Situação
RF01	Testes do driver, ensaio contra o ProtoProbe, campanha de capacidade até 9.750 tags a 30 s, E1	Atendido
RF02	Testes de gravação e de partição, teste de carga do histórico (até 9.400 linhas por segundo), E1	Atendido
RF03	Teste de carga de valores atuais de 10 minutos com seis clientes, E1	Atendido
RF04	Testes automatizados do carimbo por ciclo, E1	Atendido
RF05	Testes de login, renovação, revogação e troca de senha, medida do Argon2id	Atendido
RF06	Testes de cargos, grupos e permissões, verificação do isolamento entre clientes	Atendido
RF07	Testes da cadeia de comando, prova do comando contra o ProtoProbe, E2, E3 e E6	Parcial
RF08	Testes e conferência visual do evento agendado com *script* que escreve	Atendido
RF09	Testes da trava por tag, da idempotência e do registro de comandos, E2 e E6	Parcial
RF10	Testes de reconciliação e recarga do cadastro sem reinício da aquisição	Atendido
RF11	Testes do cadastro, do esquema e das pastas de tags	Atendido
RF12	Criação de cliente em 1,7 a 3,4 s, testes de provisionamento e de exclusão	Atendido
RF13	Testes das variáveis derivadas e das tags internas, teto medido por contêiner	Atendido
RF14	Medidas do motor isolado, testes de isolamento, prova do *script* contra o ProtoProbe	Atendido
RF15	*Lighthouse* da tela de dashboard, testes, E1	Atendido
RF16	Medida de 8 ms para aplicar um ciclo no sinótico, *Lighthouse*, conferência visual	Atendido
RF17	Testes do mapa e das consultas de tendência ao histórico, E3	Atendido
RF18	Testes dos detectores e das ocorrências, E4 e E5	Parcial
RF19	Testes da linha do tempo, do registro de comandos e de alarmes, E4	Atendido
RNF01	Medida do lado do cliente, teste de carga de valores atuais, atraso do supervisório em E1 a E6	Parcial
RNF02	Medida do teto por réplica, afinidade por dispositivo, sem medida com duas réplicas	Parcial
RNF03	Teste de carga do histórico com cliente lento, sensível à disputa de processador	Parcial
RNF04	Testes de reconstrução do Redis e de filas duráveis, sem ensaio de queda de réplica	Parcial
RNF05	Argon2id, isolamento por banco, testes de permissão, teste de segurança de fim de lote	Atendido
RNF06	Testes de limite de requisições, de trava por tag e de registro de comando e de reconhecimento	Atendido
RNF07	Testes de isolamento do motor de scripts, medidas de custo e de memória	Atendido
RNF08	Testes da cifra de segredos com AES-256-GCM e chave por cliente	Atendido
RNF09	Mesma configuração Modbus com o ProtoProbe, os simuladores e o CLP da bancada	Atendido

Id	Evidência	Situação
RNF10	Rotas versionadas sob `/v1` com contratos JSON verificados por teste de forma	Atendido
RNF11	Serviços independentes ligados por Moleculer e filas, guardas de arquitetura	Atendido
RNF12	Pilha completa de contêineres com limites de recursos, usada em todas as medidas	Atendido
RNF13	Acesso por navegador, *Lighthouse* em celular, interface em português	Atendido

Fonte: Autor.

7.6 Qualidade de software

A qualidade do código foi tratada como parte do resultado, porque um sistema supervisório que mexe em equipamentos precisa ser verificável. O repositório reúne cerca de 3.600 arquivos de teste, dos quais cerca de 3.000 são testes unitários, mais de 600 são testes de integração, que exercitam o banco e o Redis reais, e 17 são testes de carga como os descritos neste capítulo. Cada arquivo de código nasce com o seu teste, e o código das camadas de domínio e de aplicação tem cobertura exigida de 100 % de linhas e de ramos. Os testes unitários usam repositórios em memória e um relógio injetado, o que permite reproduzir cenários de tempo sem esperar, e os de integração criam bancos descartáveis para não tocar nos dados do desenvolvimento.

Um conjunto de guardas de arquitetura roda sobre o repositório inteiro e reprova violações de regras de projeto. O guarda de importações confere que uma camada não depende de outra fora da ordem permitida, que os serviços não importam o que não devem e que a criptografia fica confinada aos adaptadores. O guarda de testes reprova arquivo sem teste e teste que não afirma nada, o guarda de comentários e o de travessões mantêm o padrão do código, e o guarda do kit de componentes impede o uso de elementos de interface fora da biblioteca comum. A verificação do fim de cada lote de trabalho executa a análise estática, a verificação de tipos, os testes unitários, os guardas e a auditoria de dependências, e ainda um teste de segurança do tipo caixa preta e uma revisão adversarial do código escrita por um revisor independente. Esse conjunto não é uma integração contínua em servidor, que ainda não existe, e sim um roteiro executado localmente antes de cada publicação.

Os números de arquivos de teste são contagens aproximadas dos arquivos versionados no repositório do código (sem rodar os testes). Se quiser números de testes executados e de cobertura, é preciso rodar a suíte e inserir o resultado. Confirmar também se vale citar a revisão adversarial no texto.

7.7 Limitações e comparação com a literatura

Os resultados têm limitações que precisam ser lidas junto dos números. A primeira é o ambiente, pois todas as medidas saem de um computador que dividia a máquina com a carga e com outras atividades, em arquitetura diferente da prevista e sem rede entre os contêineres, de modo que os tetos medidos são de ordem de grandeza e variaram até 50 % entre execuções. A segunda é a ausência de equipamento real de grande porte, pois a carga de leitura foi gerada por simuladores que respondem em 5 ms, e o tempo de resposta de modems e de redes industriais soma-se a cada requisição. A terceira é o efeito de réplicas, que não foi medido, de modo que a escalabilidade horizontal é argumentada pela arquitetura e confirmada apenas no teto de uma réplica. A quarta é a redundância entre zonas, que é descrita e não implantada. A quinta é a capacidade do contêiner de *workers*, que ficou abaixo do piso planejado para variáveis derivadas

e tags internas, e que concentra o próximo trabalho de otimização. Por fim, o isolamento entre clientes na fila de histórico é sensível à disputa de processador, e a exclusão de clientes em segundo plano teve um defeito ainda por investigar.

Esses resultados podem ser comparados com a literatura sobre sistemas supervisórios de baixo custo e baseados na web. O sistema SCADA de baixo custo desenvolvido em C# com um CLP comercial como unidade remota e testado em uma planta de processamento de alimentos valida o funcionamento em campo e argumenta o ganho econômico, mas não relata latência nem vazão do software (Phuyal *et al.*, 2020). O presente trabalho segue a mesma motivação de reduzir o custo para pequenas e médias empresas e acrescenta uma avaliação quantitativa dos componentes de software, ainda que em ambiente de laboratório. Já a dissertação sobre um supervisório baseado na web compara a abordagem proposta com outra em tempo de resposta, carga de processador e tráfego de rede, o que aproxima o seu método da avaliação feita aqui para a latência e o consumo de recursos (Abbas, 2011). A diferença é a arquitetura, pois o presente sistema separa a aquisição, o cache e o histórico em serviços de contêineres independentes, o que torna cada gargalo mensurável isoladamente.

As avaliações experimentais dos trabalhos sobre replicação em contêineres, que servem de referência de método, medem latência percebida pelo cliente, vazão, consumo de processador, de memória e de rede, variam a carga e comparam a execução em contêineres com a execução direta em máquinas físicas (Netto, H. *et al.*, 2019; Netto, Hylson V. *et al.*, 2016; Netto, Hylson Vescovi *et al.*, 2017). Este capítulo adota as mesmas métricas, com latência do caminho de comando e de aquisição, vazão em leituras por segundo, memória e processador por contêiner e comportamento sob perda de comunicação no ensaio E5. Faltam duas comparações dessa literatura. Uma é a comparação com a execução fora de contêiner, que não foi feita porque o foco é a viabilidade em recursos pequenos e não o custo da virtualização. A outra é a variação do número de réplicas, que fica como trabalho futuro, junto com a implantação em um orquestrador, para o qual a separação em serviços já prepara o sistema.

8 Conclusão

Este trabalho partiu de um problema concreto. Os sistemas supervisórios convencionais exigem licenças proprietárias, equipamentos dedicados e instalação local especializada, o que os afasta de empresas de pequeno e médio porte, embora a necessidade de acompanhar, comandar e registrar um processo a distância seja a mesma de uma grande planta (Zanghi, 2019; Phuyal *et al.*, 2020). A resposta proposta foi um sistema supervisório web, multi-cliente e orientado a serviços, executado em contêineres, que o usuário acessa pelo navegador sem instalar nada. O objetivo geral, desenvolver e validar esse sistema, foi atendido na medida descrita nas seções seguintes: o sistema foi construído por inteiro, executa como um conjunto de contêineres em um único servidor e foi exercitado por testes automatizados, por campanhas de carga e por ensaios com equipamentos reais.

8.1 Retomada dos objetivos

O primeiro objetivo específico era revisar os conceitos que embasam a solução. O Capítulo 2 reuniu a fundamentação sobre sistemas supervisórios e seus módulos clássicos, a Indústria 4.0 e as tecnologias correlatas, a computação em nuvem, os contêineres e sua orquestração, os bancos de dados e o cache, as filas de trabalho, o protocolo Modbus em suas variantes TCP e RTU, a autenticação por tokens e a execução isolada de código. Essa base sustentou cada decisão tomada nos capítulos seguintes e permitiu ler o problema com o vocabulário da literatura (Bangemann *et al.*, 2014; Lasi *et al.*, 2014).

O segundo objetivo era especificar os requisitos e projetar a arquitetura. O Capítulo 3 fixou dezenove requisitos funcionais e treze não funcionais e mostrou como cada módulo de um supervisório é atendido por componentes concretos: a interface web, a API, os *workers* por família de fila, o serviço de aquisição, o motor de scripts isolado, o banco PostgreSQL, com um banco central e um banco próprio para cada cliente, e o Redis. A separação em serviços de implantação independente, ligados por mensagens e por filas duráveis, é a característica que torna o conjunto escalável e substituível por partes, e o fato de todos os serviços executarem em contêineres é o que o torna portátil entre servidores.

O terceiro objetivo era implementar a aquisição via Modbus TCP, com o agendamento das leituras e escritas e o registro do valor atual e do histórico. O Capítulo 4 descreveu o caminho do dado do campo ao banco e do banco à tela. O serviço de aquisição lê os dispositivos por afinidade, o valor corrente de cada tag fica no Redis, o histórico é gravado em lotes numa tabela particionada por mês e as médias por minuto e por hora atendem às consultas longas. No Capítulo 7, o ensaio de capacidade contra um servidor Modbus de teste sustentou até 9.750 tags a 30 s por réplica de aquisição, e a gravação do histórico chegou a cerca de 9.400 linhas por segundo para um cliente, com folga sobre a carga de um cliente típico. O isolamento entre clientes, com banco próprio e criação automática em poucos segundos, foi medido e atendido.

O quarto objetivo era a interface de supervisão. O Capítulo 5 apresentou as telas de cadastro, de acompanhamento em tempo real, o dashboard, o sinótico com editor gráfico, o mapa das plantas, os alarmes, a linha do tempo e o histórico com tendência, todas montadas sobre um conjunto único de componentes e guiadas pelos princípios de interfaces para automação (International Society of Automation, 2015; Elipse Software, 2026). O desempenho do lado do cliente, medido em celular, e o tempo de 8 ms para aplicar um ciclo de valores no sinótico indicam que a interface acompanha o processo sem sobrecarregar o navegador.

O quinto objetivo era o comando de equipamentos em quatro fases. O fluxo de seleção, verificação, confirmação e execução foi implementado com trava por tag, idempotência e registro

de auditoria, e o sucesso do comando é definido pela resposta do controlador à escrita, sem releitura posterior do ponto. Os testes automatizados e a prova contra o servidor de teste exercitam toda a cadeia. A validação com o equipamento real, porém, depende dos ensaios de bancada descritos adiante, e por isso o requisito aparece como parcialmente atendido.

O sexto objetivo era o controle de acesso por cargo e por grupo. O sistema separa quatro cargos fixos e permite que cada cliente monte grupos de permissões a partir de um catálogo fechado, com a fronteira rígida no cliente. As senhas são guardadas com Argon2id, com custo de cerca de 24 ms por verificação, e os segredos que o sistema precisa ler de volta são cifrados. Esse conjunto foi coberto por testes de cargos, de grupos, de isolamento entre clientes e por um teste de segurança de caixa preta no fim de cada lote de desenvolvimento (Biryukov *et al.*, 2021; OWASP Foundation, 2026).

O sétimo objetivo era validar o sistema em uma bancada com um CLP e um inversor de frequência e medir o desempenho frente ao volume esperado. O Capítulo 6 descreveu a bancada, formada por um CLP TM221 que simula o nível de um reservatório e por um inversor ATV320 que aciona um motor, e fixou o roteiro de seis ensaios. O Capítulo 7 apresentou o ambiente, as medidas de desempenho e a validação dos requisitos, e o conjunto dos resultados permite afirmar que a arquitetura cumpre o que se propôs na escala medida. Os ensaios de bancada propriamente ditos ainda dependem da execução pelo autor, como se explica a seguir.

8.2 Contribuições

A primeira contribuição é um sistema supervisório completo e funcional, de código próprio, que cobre o cadastro hierárquico, a aquisição, o histórico, os alarmes, os comandos e as telas de supervisão, e que pode ser executado com um conjunto de contêineres em um computador comum. A segunda é a organização multi-cliente com banco de dados próprio por cliente, criado e removido de forma automática, que oferece um isolamento mais forte que o de tabelas compartilhadas (Chong; Carraro, 2006). A terceira é o comando em quatro fases com garantias explícitas, cada uma associada a um mecanismo verificável por teste.

A quarta contribuição é a execução isolada do código escrito pelo usuário, em variáveis derivadas, eventos e componentes de conteúdo. O código roda em um interpretador compilado para WebAssembly, sem rede, sem arquivos e sem segredos, com limites de tempo e de memória (Haas *et al.*, 2017; Bytecode Alliance, 2026). Isso dá flexibilidade ao integrador sem entregar a ele o servidor. A quinta é a documentação quantitativa do comportamento do sistema em recursos modestos, com tetos de capacidade por réplica e análise dos gargalos, que serve de base para dimensionar uma implantação e para comparações futuras.

8.3 Limitações

Os resultados devem ser lidos com as limitações que o próprio Capítulo 7 registrou. Todas as medidas foram obtidas em um único computador, que dividia os recursos com a carga e com outras atividades, e variaram até 50 % entre execuções, de modo que os valores são ordens de grandeza e não garantias. A carga de leitura veio de simuladores de resposta rápida, e o tempo de modems e de redes industriais reais reduz o teto prático. O efeito de réplicas adicionais não foi medido, e a redundância entre servidores é descrita e não implantada.

Entre os requisitos, sete aparecem como parcialmente atendidos. Os requisitos de comando em quatro fases e de sua conclusão pela resposta do equipamento (RF07 e RF09), de geração de alarmes (RF18) e de baixa latência das telas (RNF01) dependem dos ensaios com a bancada. Os de replicação horizontal (RNF02), de enfileiramento justo entre clientes (RNF03) e de

disponibilidade (RNF04) dependem de medidas com mais de uma réplica e de ensaio de queda de réplica, que não foram feitas.

A capacidade do contêiner de *workers*, que ficou abaixo do piso planejado para variáveis derivadas e tags internas, e a sensibilidade do isolamento entre clientes na fila de histórico à disputa de processador também permanecem como pontos a melhorar. Por fim, parte da avaliação de qualidade apoia-se em contagens de arquivos de teste e não em cobertura medida de ponta a ponta.

Fechar esta seção depois dos ensaios E1 a E6 da bancada: informar quais requisitos passam de parcial para atendido, os tempos de resposta do comando e o atraso do supervisório medidos, e ajustar a frase sobre o objetivo de validação em bancada. Se o Capítulo 7 for atualizado, conferir também a lista de requisitos parciais.

8.4 Trabalhos futuros

O primeiro caminho de evolução é levar o sistema a um provedor de nuvem e a um orquestrador de contêineres. Como todos os serviços já executam em contêineres, a mudança não exige alterar o código, e o orquestrador oferece distribuição automática de réplicas, recuperação de falhas e atualização gradual (Netto, Hylson Vescovi *et al.*, 2017; The Kubernetes Authors, 2026). Essa implantação permitiria medir o efeito do número de réplicas sobre a vazão e comparar a execução em contêineres com a execução direta, a mesma comparação feita pelos trabalhos do orientador sobre contêineres gerenciados pelo Kubernetes (Netto, Hylson V. *et al.*, 2016; Netto, H. *et al.*, 2019).

O segundo caminho é a replicação do banco de dados e dos serviços de estado por meio de replicação de máquinas de estados e de um algoritmo de consenso como o Raft, o que tornaria o sistema tolerante à queda do servidor que hoje guarda o banco e o Redis (Ongaro; Ousterhout, 2014; Netto, H. *et al.*, 2019). A avaliação desse ganho, com injeção de falhas e medida da latência sob falha, seguiria o método das referências citadas.

Do lado dos protocolos, a arquitetura do serviço de aquisição prevê a inclusão de novas famílias de fila, o que abre espaço para o OPC UA e o MQTT ao lado do Modbus TCP (Carballo *et al.*, 2024). Do lado das funções, ficam como evolução a emissão de relatórios, as notificações por canais externos como o correio eletrônico e mensageiros, a restrição de acesso por planta, que hoje não existe porque a fronteira de acesso é o cliente, e um aplicativo móvel que explore a notificação ativa de alarmes. Também fica como trabalho futuro a execução dos ensaios de bancada com cargas reais mais variadas, inclusive com mais de um equipamento e com redes industriais de campo, para ampliar a confiança nos tetos medidos.

Em síntese, o trabalho entrega uma base funcional e verificável para um supervisório web acessível a empresas que não podem arcar com a solução tradicional. As limitações apontadas definem o próximo ciclo de trabalho, e a arquitetura em contêineres, escolhida desde o início, é o que permite que esse ciclo ocorra sem reescrever o que já foi construído.

Referências

ABBAS, Hosny Ahmed. **Efficient Web-Based SCADA System**. 2011. Dissertação (Mestrado em Engenharia Elétrica) – Faculty of Engineering, South Valley University, Aswan.

APACHE SOFTWARE FOUNDATION. **Apache ECharts Handbook**. Apache Software Foundation. Disponível em: `https://echarts.apache.org/handbook/en/get-started/`. Acesso em: 16 out. 2026.

BANGEMANN, Thomas *et al.* State of the Art in Industrial Automation. *In:* COLOMBO, Armando W. *et al.* (ed.). **Industrial Cloud-Based Cyber-Physical Systems**: The IMC-AESOP Approach. Cham: Springer, 2014. p. 23–47. ISBN 978-3-319-05623-4. DOI: `10.1007/978-3-319-05624-1_2`.

BELLARD, Fabrice; GORDON, Charlie. **QuickJS Javascript Engine**. Disponível em: `https://bellard.org/quickjs/`. Acesso em: 16 out. 2026.

BIGHETI, Jeferson André. **Arquitetura de Automação e Controle Orientada a Microserviços para a Indústria 4.0**. 2020. Tese (Doutorado em Engenharia Elétrica) – Faculdade de Engenharia, Universidade Estadual Paulista "Júlio de Mesquita Filho", Bauru.

BIRYUKOV, Alex *et al.* **Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications**. RFC. [*S. l.*], set. 2021. Disponível em: `https://www.rfc-editor.org/rfc/rfc9106`. Acesso em: 16 out. 2026.

BYTECODE ALLIANCE. **Wasmtime Documentation**. Bytecode Alliance. Disponível em: `https://docs.wasmtime.dev/`. Acesso em: 16 out. 2026.

CARBALLO, Jose Antonio *et al.* Modern SCADA for CSP Systems Based on OPC UA, Wi-Fi Mesh Networks, and Open-Source Software. **Energies**, v. 17, n. 24, p. 6284, 2024. DOI: `10.3390/en17246284`.

CHONG, Frederick; CARRARO, Gianpaolo. **Architecture Strategies for Catching the Long Tail**. Microsoft Corporation. Abr. 2006. Acesso em: 16 out. 2026.

DOCKER, INC. **Docker Documentation**. Docker, Inc. Disponível em: `https://docs.docker.com/`. Acesso em: 16 out. 2026.

ELIPSE SOFTWARE. **Metodologia para Desenvolvimento de IHMs de Alta Performance Visual**. Elipse Software. Disponível em: `https://kb.elipse.com.br/metodologia-para-desenvolvimento-de-ihms-de-alta-performance-visual/`. Acesso em: 15 jun. 2026.

HAAS, Andreas *et al.* Bringing the Web up to Speed with WebAssembly. *In:* PROCEEDINGS of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation. [*S. l.: s. n.*], 2017. p. 185–200. DOI: `10.1145/3062341.3062363`.

HENRIQUES DA SILVA, Fabrício Rodrigues. **Um estudo sobre os benefícios e os riscos de segurança na utilização de Cloud Computing**. 2010. Trabalho de Conclusão de

Curso (Graduação em Ciência da Computação) – Centro Universitário Augusto Motta, Rio de Janeiro.

INTERNATIONAL SOCIETY OF AUTOMATION. **Enterprise-Control System Integration. Part 1: Models and Terminology**. Research Triangle Park, NC, 2000. ISBN 1-55617-727-5.

INTERNATIONAL SOCIETY OF AUTOMATION. **Human Machine Interfaces for Process Automation Systems**. Research Triangle Park, NC, 2015.

INTERNATIONAL SOCIETY OF AUTOMATION. **Management of Alarm Systems for the Process Industries**. Research Triangle Park, NC, 2016.

JONES, Michael; BRADLEY, John; SAKIMURA, Nat. **JSON Web Token (JWT)**. RFC. [*S. l.*], maio 2015. Disponível em: `https://www.rfc-editor.org/rfc/rfc7519`. Acesso em: 16 out. 2026.

KHALID, Wajahat *et al.* Open-Source Internet of Things-Based Supervisory Control and Data Acquisition System for Photovoltaic Monitoring and Control Using HTTP and TCP/IP Protocols. **Energies**, v. 17, n. 16, p. 4083, 2024. DOI: `10.3390/en17164083`.

LASI, Heiner *et al.* Industry 4.0. **Business & Information Systems Engineering**, v. 6, n. 4, p. 239–242, 2014. DOI: `10.1007/s12599-014-0334-4`.

LEAFLET. **Leaflet: an open-source JavaScript library for interactive maps**. Leaflet. Disponível em: `https://leafletjs.com/reference.html`. Acesso em: 16 out. 2026.

META PLATFORMS, INC. **React Documentation**. Meta Platforms, Inc. Disponível em: `https://react.dev/`. Acesso em: 16 out. 2026.

MICROSOFT. **TypeScript Documentation**. Microsoft. Disponível em: `https://www.typescriptlang.org/docs/`. Acesso em: 16 out. 2026.

MODBUS ORGANIZATION. **Modbus Messaging on TCP/IP Implementation Guide**. Versão 1.0b. Hopkinton, MA, 2012. Disponível em: `https://www.modbus.org/docs/Modbus_Messaging_Implementation_Guide_V1_0b.pdf`. Acesso em: 15 maio 2026.

MODBUS ORGANIZATION. **Modbus over Serial Line Specification and Implementation Guide**. Versão 1.02. Hopkinton, MA, 2012. Disponível em: `https://www.modbus.org/docs/Modbus_over_serial_line_V1_02.pdf`. Acesso em: 15 maio 2026.

MOHAMED, Mohamed A.; ALTRAFI, Obay G.; ISMAIL, Mohammed O. Relational vs. NoSQL Databases: A Survey. **International Journal of Computer and Information Technology**, v. 3, n. 3, p. 598–601, 2014.

MOLECULERJS. **Moleculer Documentation**. MoleculerJS. Disponível em: https://moleculer.services/docs/0.14/. Acesso em: 14 jun. 2026.

MONOSTORI, L. *et al.* Cyber-physical systems in manufacturing. **CIRP Annals**, v. 65, n. 2, p. 621–641, 2016. DOI: 10.1016/j.cirp.2016.06.005.

NETTO, Hylson *et al.* Incorporating the Raft consensus protocol in containers managed by Kubernetes: an evaluation. **International Journal of Parallel, Emergent and Distributed Systems**, 2019. DOI: 10.1080/17445760.2019.1608989.

NETTO, Hylson V. *et al.* State machine replication in containers managed by Kubernetes. **Journal of Systems Architecture**, 2016. DOI: 10.1016/j.sysarc.2016.12.007.

NETTO, Hylson Vescovi *et al.* Coordenação de Containers no Kubernetes: Uma Abordagem Baseada em Serviço. *In:* SIMPÓSIO Brasileiro de Redes de Computadores e Sistemas Distribuídos (SBRC). [*S. l.*]: Sociedade Brasileira de Computação, 2017.

ONGARO, Diego; OUSTERHOUT, John. In Search of an Understandable Consensus Algorithm. *In:* PROCEEDINGS of the 2014 USENIX Annual Technical Conference. [*S. l.*]: USENIX Association, 2014. p. 305–319.

OPENJS FOUNDATION. **Fastify Documentation**. Fastify. Disponível em: https://fastify.dev/docs/latest/. Acesso em: 16 out. 2026.

OPENJS FOUNDATION. **Node.js Documentation**. Node.js. Disponível em: https://nodejs.org/docs/latest/api/. Acesso em: 16 out. 2026.

OPENSTREETMAP FOUNDATION. **OpenStreetMap**. OpenStreetMap Foundation. Disponível em: https://www.openstreetmap.org/about. Acesso em: 16 out. 2026.

OWASP FOUNDATION. **OWASP Cheat Sheet Series**. OWASP Foundation. Disponível em: https://cheatsheetseries.owasp.org/. Acesso em: 16 out. 2026.

PEDROSA, Paulo Henrique C.; NOGUEIRA, Tiago. Computação em Nuvem. Trabalho acadêmico (disciplina G04). [*S. l.*], 2011.

PHUYAL, Sudip *et al.* Design and Implementation of Cost Efficient SCADA System for Industrial Automation. **International Journal of Engineering and Manufacturing**, v. 10, n. 2, p. 15–28, 2020. DOI: 10.5815/ijem.2020.02.02.

PISCHING, Marcos André. **Arquitetura para descoberta de equipamentos em processos de manufatura com foco na indústria 4.0**. 2017. Tese (Doutorado em Engenharia de Controle e Automação Mecânica) – Escola Politécnica, Universidade de São Paulo, São Paulo.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL Documentation**. PostgreSQL Global Development Group. Disponível em: https://www.postgresql.org/docs/current/. Acesso em: 14 jun. 2026.

REDIS LTD. **Redis Documentation**. Redis Ltd. Disponível em: https://redis.io/docs/latest/. Acesso em: 14 jun. 2026.

SCHNEIDER ELECTRIC. **Altivar Machine ATV320: Modbus Serial Link Manual**. [*S. l.*]. Disponível em: https://www.se.com/br/pt/. Acesso em: 16 out. 2026.

SCHNEIDER ELECTRIC. **Altivar Machine ATV320: Programming Manual**. [*S. l.*]. Disponível em: https://www.se.com/br/pt/. Acesso em: 16 out. 2026.

SCHNEIDER ELECTRIC. **EcoStruxure Machine Expert - Basic: Modicon M221 Programming Guide**. [*S. l.*]. Disponível em: https://www.se.com/br/pt/. Acesso em: 16 out. 2026.

SCHNEIDER ELECTRIC. **Modicon M221 Logic Controller: Hardware Guide**. [*S. l.*]. Disponível em: https://www.se.com/br/pt/. Acesso em: 16 out. 2026.

STANKOVIC, John A. Research Directions for the Internet of Things. **IEEE Internet of Things Journal**, v. 1, n. 1, p. 3–9, 2014. DOI: 10.1109/JIOT.2014.2312291.

STOUFFER, Keith *et al.* **Guide to Industrial Control Systems (ICS) Security**. Gaithersburg, MD, maio 2015. DOI: 10.6028/NIST.SP.800-82r2. Disponível em: https://nvlpubs.nist.gov/nistpubs/specialpublications/nist.sp.800-82r2.pdf. Acesso em: 13 jun. 2026.

TASKFORCE.SH INC. **BullMQ Documentation**. Taskforce.sh Inc. Disponível em: https://docs.bullmq.io/. Acesso em: 16 out. 2026.

THE KUBERNETES AUTHORS. **Kubernetes Documentation**. The Linux Foundation. Disponível em: https://kubernetes.io/docs/home/. Acesso em: 16 out. 2026.

VERCEL, INC. **Next.js Documentation**. Vercel, Inc. Disponível em: https://nextjs.org/docs. Acesso em: 14 jun. 2026.

WALI, Arwa; ALSHEHRY, Fatimah. A Survey of Security Challenges in Cloud-Based SCADA Systems. **Computers**, v. 13, n. 4, p. 97, 2024. DOI: 10.3390/computers13040097.

WEBASSEMBLY COMMUNITY GROUP. **WebAssembly Specification**. World Wide Web Consortium. Disponível em: https://webassembly.github.io/spec/core/. Acesso em: 16 out. 2026.

YADAV, Geeta; PAUL, Kolin. **Architecture and Security of SCADA Systems: A Review**. arXiv:2001.02925 [cs.CR]. arXiv. 9 jan. 2020. Disponível em: https://arxiv.org/abs/2001.02925. Acesso em: 13 jun. 2026.

ZANGHI, Eric. **Sistemas SCADA: Conceitos**. Porto, Portugal, 2019. Série Proteção e Comunicação de Sistemas Elétricos de Potência (PROTCOM).

ZOD. **Zod Documentation**. Disponível em: https://zod.dev/. Acesso em: 16 out. 2026.